

14 - Data Scientist:

Machine Learning/AI Engineer

<https://www.codecademy.com/enrolled/paths/machine-learning-engineer>

Syllabus

1. Introduction to Machine Learning Engineer Career Path

Introduction to the Machine Learning Engineer Career Path

1 Informational, 1 Lesson, 1 Article

2. Machine Learning Fundamentals

Welcome to Machine Learning Fundamentals

1 Informational

Introduction to Machine Learning

Feature Engineering I: Data Transformations

Supervised Learning I : Regressors, Classifiers and Trees

Feature Engineering II : Feature Selection Methods

Unsupervised Learning Algorithms I

Assessment: Machine Learning Fundamentals

[Take assessment](#)

3. Software Engineering for Machine Learning/AI Engineers

Welcome to Software Engineering for Machine Learning/AI Engineers

1 Informational

18% progress

18%

Software Engineering in Python I

4 Lessons, 4 Quizzes, 4 Projects, 1 Video, 3 Articles

5% progress

5%

Learn the Command Line

3 Articles, 4 Lessons, 4 Quizzes, 5 Projects, 1 Informational

Learn Git: Introduction to Version Control

1 Informational, 3 Videos, 3 Lessons, 4 Projects, 3 Quizzes, 1 External resource, 4 Articles

Software Engineering in Python II

4 Lessons, 3 Quizzes, 2 Projects, 2 Articles

Bash Scripting

1 Lesson, 1 Quiz, 1 Project

Learn Git II: Git for Deployment

2 Informationals, 1 Video, 4 Articles, 1 External resource, 2 Lessons, 5 Projects, 2 Quizzes

Assessment: Software Engineering for Machine Learning/AI Engineers

[Take assessment](#)

4. Intermediate Machine Learning

Welcome!

1 Informational

Supervised Learning II: Advanced Regressors and Classifiers

Regularization and Hyperparameter Tuning

75% progress

75%

Ensemble Methods in Machine Learning

2 Articles, 2 Lessons, 2 Quizzes, 2 Projects

Assessment: Intermediate Machine Learning

[Take assessment](#)

5. Building Machine Learning Pipelines

Machine Learning Pipelines: Bringing Machine Learning and Engineering together

1 Informational

Build Machine Learning Pipelines

1 Article, 1 Lesson, 1 Project

6. Machine Learning/AI Engineer: Final Portfolio

Machine Learning Engineer Final Portfolio

1 Kanban project

7. Machine Learning Career Path: Next Steps

Next Steps

1 Informational

Welcome to the Machine Learning Engineer Career Path

Your Machine Learning Engineering journey begins here!

Hi! If you're completely new to data analysis, Python, or pandas, start with [ML/AI Engineering Foundations Skill Path](#) then join us here!

Algorithmic decision-making is to be found everywhere today. From detecting fraudulent transactions to recommending movies to figuring out the nearest car for a ride, so many of the applications we use every day rely on this. What powers these systems? Machine Learning Engineering!

About the Machine Learning Engineer Career Path

Machine learning engineering concerns itself with building robust systems that use machine learning at scale to speed up data-driven decision-making. The people who build these systems, Machine Learning Engineers, are thus one of the most highly sought after roles within tech today. This career path is designed to take the learner from zero to Machine Learning Engineer!

The career path begins with a no-code introductory module which is followed by alternating tracks on machine learning and engineering. Eventually it combines the two to teach you how to build end-to-end machine learning pipelines.

The career path currently progresses through the following five tracks:

Overview and Pre-requisites

If at this moment, you are wondering about any of the following questions, start with the Introduction module that follows right after this article:

The introductory module aims to answer all of the above and provide clarity on how to make the most out of this career path as well as the Codecademy platform.

If you already have some foundational knowledge of data science and programming, the second track, Machine Learning Fundamentals would be the place to start. If not, do not worry! The [Machine Learning/AI Engineering Foundations Skill Path](#) is designed to take you through exactly all that you need to know to get started with Machine Learning fundamentals. If you're not sure, take a look at the syllabus of the Foundations Skill Path and you can decide for yourself!

We would like to stress to the learner that this career path is a Work in Progress (WIP)! :) We're in the process of building more advanced machine learning content as well as more content relating to building ML pipelines with PySpark, cloud deployment of ML models, MLOps, etc. So it is by no means exhaustive at the moment and we're excited to bring you more advanced content soon! In the meantime, we look forward to receiving your feedback on our newest career path so we can keep improving this experience for you.

Structure

Throughout this Path, you will see lessons, quizzes, articles, and projects. Lessons and articles introduce new tools and concepts, quizzes give you a chance to quickly check your understanding, and projects give you a chance to apply your new skills.

There are a lot of projects, but pay special attention to Portfolio Projects and Cumulative Projects because they give you a chance to apply your skills and build up your portfolio.

The content in this Career Path is cumulative, and we recommend that you take the courses in order. However, if you are familiar with a given technology or idea, feel free to jump ahead. Just be aware that skipping modules will affect your completion percentage.

Finally, we are constantly updating our content to be sure it is up to date and of the highest quality. Keep an eye on the notification bells, which we will use to alert you to new content. Sometimes these updates might affect your completion percentage, and we'll always do our best to let you know with a notification bell when that might happen.

The Machine Learning Engineer Career Path

Welcome!

Welcome to the

[Machine Learning](#)

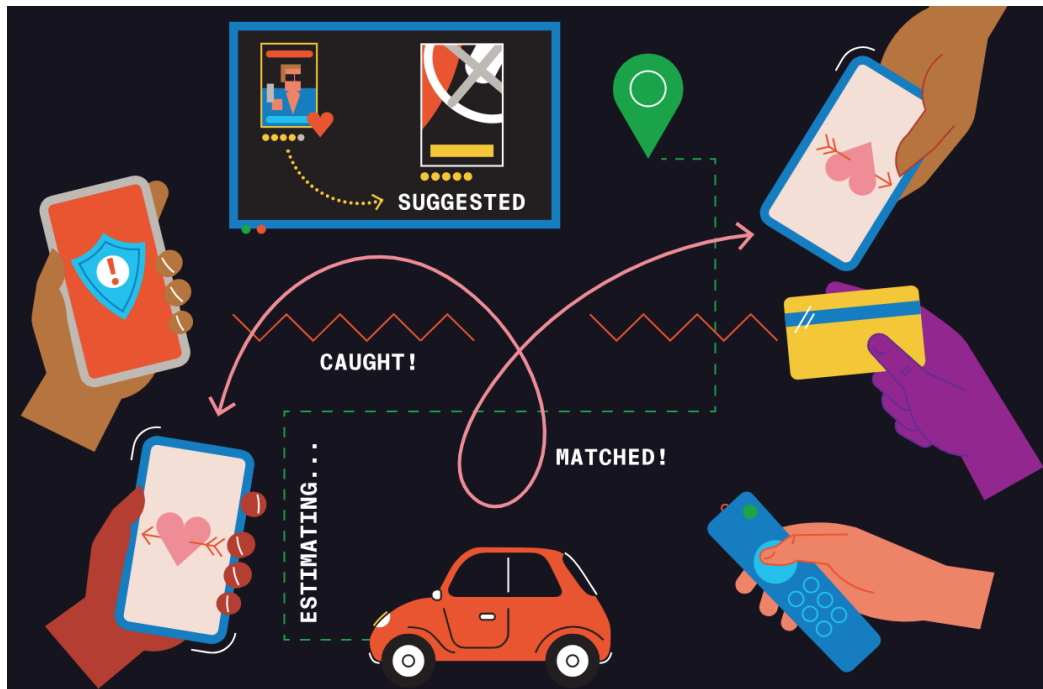
Preview: Docs Loading link description

Engineer (MLE) Career Path! No matter what device you're reading this in, odds are there's machine learning engineering behind many of the applications on your device. Here are some examples of applications you may know of that use machine learning engineering:

- Netflix's movie recommendation system
- Twitter's feed curation
- Capital One's credit card fraud alert system
- Snapchat's filters
- Lyft/Ola's predictive systems on time of arrival

... and many, many more! The decision-making systems in our world are increasingly reliant on algorithmic thinking. Machine learning engineering is key to implementing these algorithms and automating the prediction process.

This career path is designed to take you through the journey of becoming a machine learning engineer from start to finish. In this lesson, you'll learn about what machine learning engineering is, what a MLE does, how the role relates to other roles within Tech, what skills are needed to become a MLE and how to make the most of this career path!



The Machine Learning Engineer Career Path

What is Machine Learning?

Machine learning is an umbrella term for a set of algorithms that analyze and find patterns from known historical information (“*training data*”) to make predictions on unknown/new information. What goes into these algorithms?

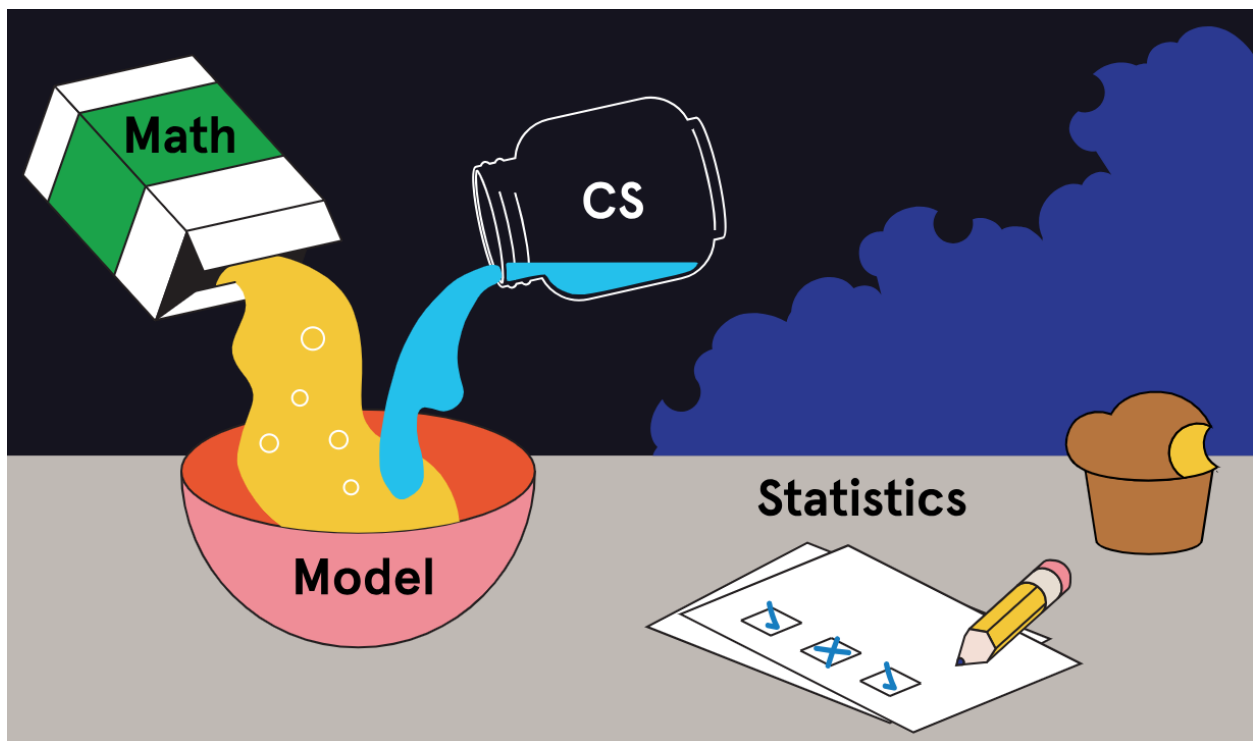
Mathematics: Mathematical functions form the basis of the modeling process. Specifically, the fields of Probability, Linear Algebra and Calculus are crucial to building mathematical systems capable of modeling real-world data.

Computer Science: Programming languages implement these mathematical models by translating them into a series of executable tasks that a computing machine can implement.

Statistics: Statistical inference and evaluation techniques are at the heart of making sure the model reflects the data as much as possible. They help us answer: How do we know our model works?

The parameters in the mathematical models (mathematics) are “learned” by implementing them using programming (computer science) and upon evaluating their performance (statistics), they’re altered if unsatisfactory... And implemented and evaluated again to see if they perform better and so on and so forth.

The [machine learning](#) process is thus not a one-and-done process. Rather, it’s *iterative*. Not unlike the way we humans learn! :) This capacity of machine learning to simulate human cognition makes it a subfield of Artificial Intelligence(AI).



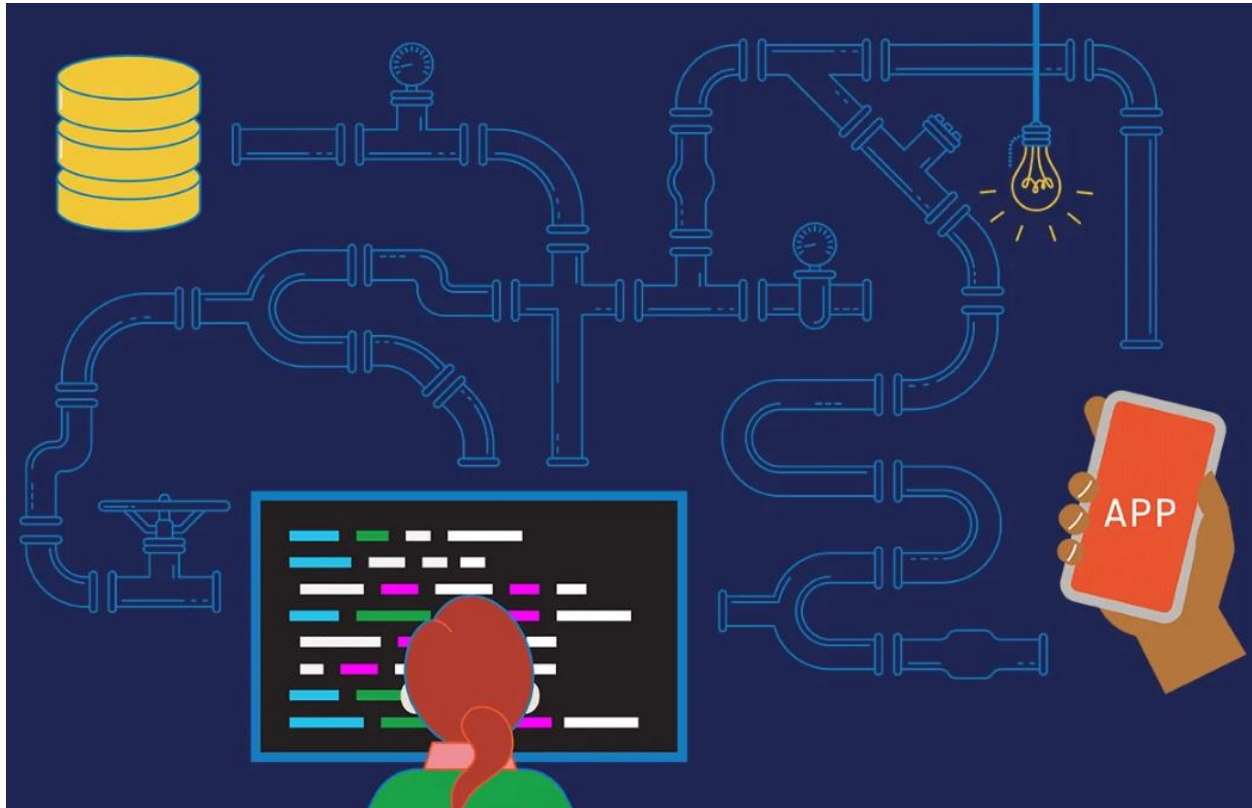
The Machine Learning Engineer Career Path

What is Machine Learning Engineering?

What makes [machine learning](#) so ubiquitous today? After all, the practice of collecting and analyzing data isn't new in human history! The answer is quite simple: the advances in hardware and computing technologies are what make machine learning possible!

The more training data we have, the better an [Algorithm](#) can learn its patterns. However, storing and accessing vast amounts of data requires robust hardware and pipelines. And running a machine learning algorithm requires lots of computing power and an infrastructure that allows for a seamless flow of data between the different steps of the algorithm. Here's where the "engineering" in Machine Learning Engineering comes in!

Machine Learning Engineering concerns itself with designing, building, maintaining and fine-tuning computational systems that can execute sophisticated machine learning algorithms on large amounts of data.



The Machine Learning Engineer Career Path

MLE, Data Scientist, Data Engineer... how do these roles differ?

If you're scratching your head at this point as to what the differences are between a [Machine Learning](#) Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

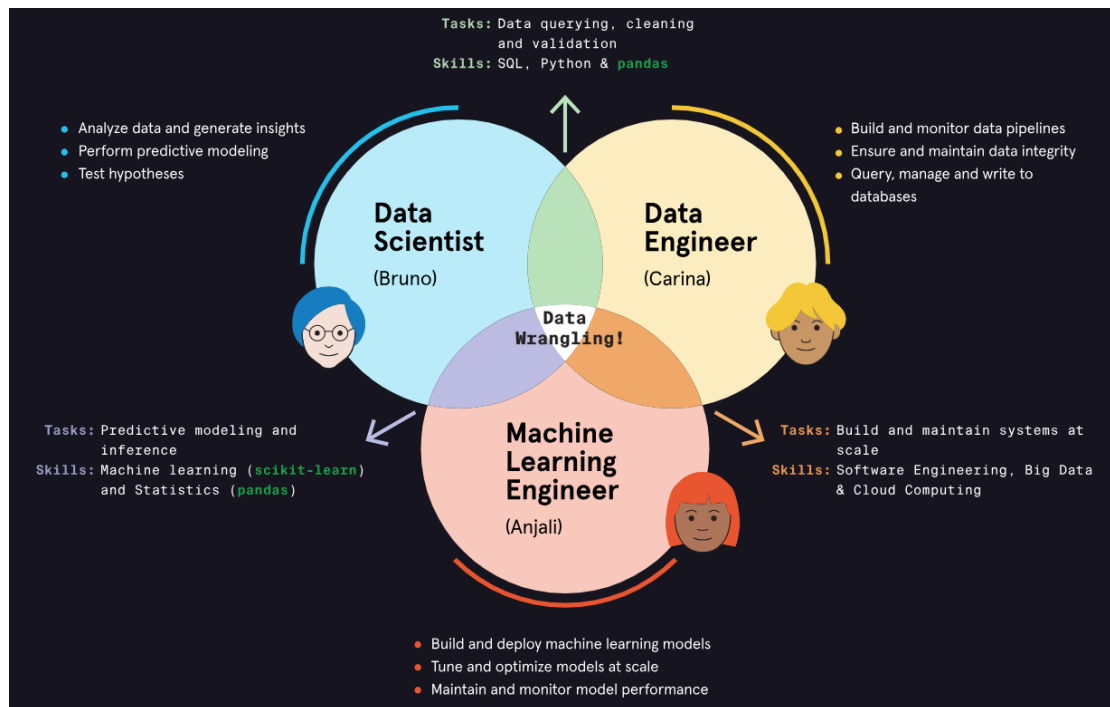
Engineer (MLE), Data Scientist (DS) and Data Engineer (DE) ... you're not alone — because it is confusing! The boundaries between these fields are fuzzy and depending on the company, the expectations of these roles can vary a lot.

However, on average, there are a pretty well-defined set of skills and tasks that define these roles. Often knowing the skills of one is immensely useful in another role and transitions from one to another are not uncommon.

We're going to distinguish between these three roles - Anjali, Bruno and Carina are a Machine Learning Engineer (MLE), Data Scientist (DS) and Data Engineer (DE), respectively. The Venn diagram to the right has three circles representing each role and we've listed

The primary tasks they perform
Where the skills overlap between the roles

We're now ready to build a tech startup with Anjali (MLE), Bruno (DS) and Carina (DE)!



The Machine Learning Engineer Career Path

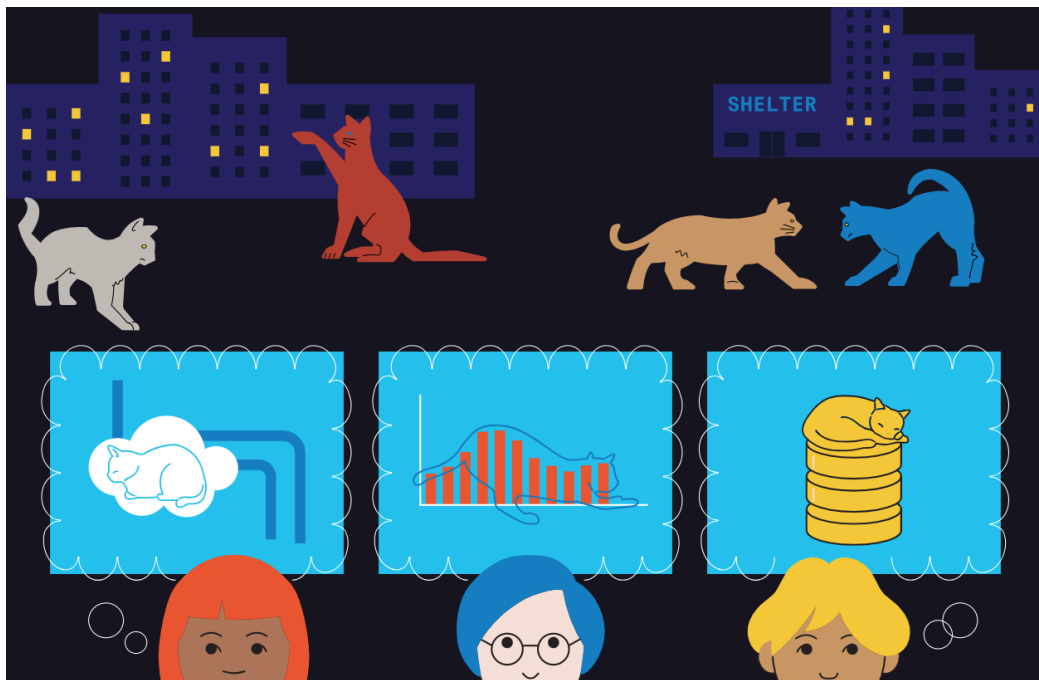
PurrspectiveAI: Machine Learning Engineering in action!

Anjali (MLE), Bruno (DS) and Carina (DE) are huge cat lovers! They want to build an app that solves the problem of abandoned street cats in metropolitan cities as efficiently as possible. (It's been shown that the lifespan of cats increases significantly when they're domesticated!) So they ideate and come up with PurrspectiveAI, a quick and efficient way to match a cat that's been found with either the closest shelter or a permanent home.

To put this app together, Bruno (DS) and Carina (DE) work together on the data acquisition. They scrape data on shelters in New York City for starters. Bruno works on finding a [machine learning](#) model that identifies the closest shelter to a geographic location within NYC. Carina builds a data engineering system that takes in three major inputs

in real time: alerts from folks who find abandoned cats; locations of shelters across NYC; pet adopters and their preferences. Carina's system is designed to output a suitable candidate for cat adoption or a shelter to temporarily host the cat.

Meanwhile, Anjali (MLE) scopes out the design for a machine learning system that can run Bruno's (DS) model on scale. After all, the point is to not just solve this problem in NYC but in all cities across the world! Anjali decides on using `Digital Ocean` as the cloud provider that she would deploy the model on. She begins to build a machine learning pipeline using `PySpark` intended to incorporate Bruno's machine learning model that will sit within Carina's larger data engineering design.



The Machine Learning Engineer Career Path

PurrspectiveAI: MLE for the win!

The [Machine Learning](#) Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

Engineer often works in collaboration with a Data Scientist/Analyst and a Data/DevOps Engineer. Here's how they would collaborate in PurrspectiveAI:

Anjali (MLE) and Bruno (DS) collaborate to find the right model **at scale**. For instance, Bruno's model might be trained exclusively on cats in Brooklyn and might fail to scale to other demographics. Anjali needs to be proficient at machine learning herself to spot this and go back to Bruno with her diagnostics so he may prototype a different model and so on. Additionally, there may be changes to the model parameters or hyperparameters that *can only become apparent at scale*.

Anjali (MLE) and Carina's (DE) system designs need to be **in sync**. The data flow and cadence from Carina's databases need to fit in Anjali's machine learning pipelines. Where the model might break due to an error in data entry, Anjali would have to go to Carina to make sure something systemic isn't happening. Additionally,

Anjali automates the handing over of the model results (i.e., cat and cat owner matched!) to Carina's system so it can automatically 'ping' an expectant cat owner.

Outside of these collaborations, Anjali's task involves making sure that the model is as efficient and tuned as possible. Computational resources and cloud space are all cost intensive, so her knowledge of ML system design and operations is immensely useful here.

This team effort makes sure that PurrerspectiveAI is a success, not just in Brooklyn, but across the globe! Feline felicitations to the team! And it would've definitely not been possible without our MLE.

The Machine Learning Engineer Career Path

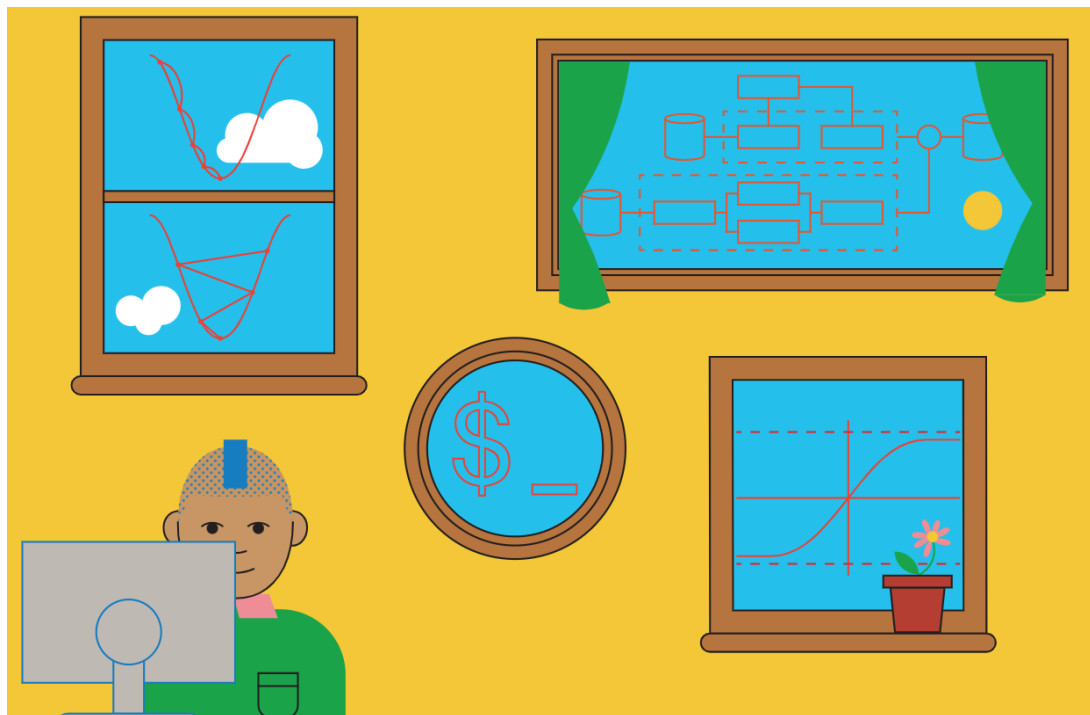
What will you learn in this Career Path?

As we've seen, [Machine Learning](#) Engineering requires a combination of ML and [software engineering](#). Software engineering is a field of study that involves using engineering principles to design, build, test, and maintain software systems.

skills as well as a familiarity with ML systems design. This makes the MLE a uniquely sought-after role within Tech! And this career path has been designed to teach you all of the above from scratch. You will learn:

- Basics of machine learning and software engineering
- Implementation of machine learning algorithms from beginner to advanced
- How to build ML pipelines
- How to build an end-to-end ML app
- How to deploy and maintain ML apps
- AutoML and MLOps techniques

We hope you're excited for this journey. If you're completely new to data analysis, Python, or pandas, start with [ML/AI Engineering Foundations Skill Path](#) before getting started with the next track, Machine Learning Fundamentals. If you already have some familiarity with these topics, here we go!



Make the Most of Your Codecademy Membership

Learn about the different features Codecademy has to offer!

Codecademy has helped [tens of millions of people](#) to learn to code, everything from `hello world` to getting new tech jobs, but when it comes down to it, you get out what you put in. If you want to be best set up for success, this guide is for you.

Valuable Resources

There are hundreds of hours of free content on Codecademy, and thousands of hours of Pro material; but no matter your membership level, the key to success is learning intelligently. If you build the habits to use our features, it will be easier for you to master technical skills.

Mobile App

Keep practicing and stay sharp as you go with our mobile app on [iOS](#) and [Android](#).

Workspaces

[Workspaces](#) allow learners to work in their own integrated development environment (IDE) right inside Codecademy. Build or try whatever you want with these unguided sandboxes, share your code, or keep it private. Not sure what to make? Grab code from your coursework and use it as a template.

Cheatsheets

Coding is less about memorization and more about understanding principles; cheatsheets help you quickly refresh your memory and get back to work. You can find them linked in lessons, or head to their [homepage](#) to find them all, broken down by the course section in which you learned them.

Docs

Cheatsheets are a handy companion for consuming coursework, but when coding in the real world, programmers need documentation. With [Docs](#), you have an entirely open-source reference material to cover far more than a course can.

Code Challenges

With technical interviews (and coding in general), practice makes perfect. Now, you can practice real [code challenges](#) from actual interviews to see how your skills stack up, or just solve them for fun. If you get stuck, we'll point you to what you still need to learn.

Learn What to Learn

Not certain what language to pick up or what career path to choose? Flick through the [Learn What to Learn course](#), where we've consolidated the best resources to help you make a choice with confidence. An hour of planning now could save you days of wasted effort later.

There is also an excellent [forum post](#) by Galina, one of our curriculum developers, that discusses how to best get started when beginning your coding journey.

Community

Learning to code is challenging, but it's easier if you can team up. Ask questions, stay connected, and find motivation with the Codecademy Community!:

[Community](#)

[Events](#)

[Facebook group](#)

Video Library

Need a break from the IDE but still want to be productive? Watch hundreds of educational and motivational video content on everything from basic coding concepts to career advice, all on-topic and [on-platform](#).

Pro Features

If you want to supercharge your learning, Pro's the thing for you - here's how it can help.

Paths

[Skill Paths](#) and [Career Paths](#) are step-by-step roadmaps that take you through what you need to learn a specific skill or transition to a specific career in tech. They'll tell you what to learn and in what order, cutting out the guesswork and to the chase. Our Career Paths cover everything that you'll find in a \$15,000 boot camp curriculum, plus more, and you can team up to tackle them with a [cohort group](#).

Projects, Challenge Projects, and Portfolio Projects

As you learn, you'll work on guided projects to put the things you're learning to use. You'll find a variety of project types. Some offer step-by-step guidance while others offer more of a challenge by only describing the final outcome of the project. If you take a Path, you'll build your own [Portfolio Projects](#) as well, designed to simulate professional coding tasks. The solutions to these portfolio projects will be unique to you! Check out all the projects available to you via the [Projects Library](#).

Project Tasks

Keep track of your progress by dragging each task from "To Do" to "In Progress" to "Done" as you work on them. You can also click on a task to see more information about it.

To Do	In Progress	Done 0/6 done
... Create your files On your computer...		
... Set up version control Set up Git tracking in...		
... Build the home page The home page will be...		
... Build the contact page The contact page incl...		
... Style the content Now that you have your...		
...		

Interview prep

If you're taking one of our Career Paths, you'll find interview prep essentials built-in. Learn interview techniques to prepare for technical interviews, including algorithm and data structure practice. Go even further with [dedicated skill paths](#) on acing the interview across a range of languages and disciplines.

Practice & Review

With your Pro membership, you'll find it easier to brush up on what you've learned. Choose for yourself what to practice with links from each module of your coursework. Or let us take the guesswork out with smart practice. Click the practice session link from syllabi and AI will prioritize concepts for you by using spaced repetition — a [scientifically proven](#) way of helping you memorize new ideas.

Welcome to Machine Learning Fundamentals!

Preview what you will learn in this first Machine Learning unit.

Hi! If you're completely new to data analysis, Python, or pandas, start with [ML/AI Engineering Foundations Skill Path](#) then join us here!

Welcome to the Machine Learning Fundamentals Unit! This unit will introduce you to what machine learning is, the data transformations needed to prepare your data for machine learning modeling, and a host of fundamental machine learning algorithms and feature engineering techniques.

Why did we build this?

Our world is increasingly reliant on algorithms powered by machine learning for decision-making. Knowing how machine learning algorithms work and how to implement them is empowering to anyone interested in Tech! If you're new to data science and have always wanted to delve deeper into machine learning, this skill path is for you. If you're already familiar with some of the following material but would like to systematically review them, this skill path is also for you as well! The prerequisite for this unit is familiarity with `Python` and specifically, the `pandas` and `matplotlib` libraries.

What will you learn?

- Supervised and Unsupervised Learning Basics

- Feature Engineering techniques

- Implementing Supervised Learning algorithms like Regressors, Classifiers and Trees

- Evaluating the performance of algorithms

- Implementing Unsupervised Learning algorithms such as PCA and K-Means

Why Use Machine Learning?

Begin Your Journey

DING DING DING!

It was but a dream and there's still time. Understanding [Machine Learning](#) will help you change this bleak future, or be ready for it.

As you begin your journey through our Machine Learning curriculum, you will learn the different applications of this powerful field. You will build your own models, test them, and try to improve them. You will analyze bigger, and more complex data and deliver faster, more accurate results — even on a very large scale using:

Supervised Learning
 Regression
 ClassificationAaaaa
Unsupervised Learning

Throughout your exploration of ML, we hope you gain an understanding of how to harness predictive models to do good, and enhance human life rather than replace it!

If you're completely new to data analysis, Python, or pandas, start with [ML/AI Engineering Foundations Skill Path](#) then join us here!

```
class Student:
    def __init__(self, name, year):
        self.name = name
        self.year = year
        self.grades = [] # Step 5

    def add_grade(self, grade): # Step 6
        if isinstance(grade, Grade):
            self.grades.append(grade)

    # Optional extra method
    # def get_average(self):
    #     if not self.grades:
    #         return 0
    #     return sum(grade.score for grade in self.grades) / len(self.grades)

class Grade:
    minimum_passing = 65 # Step 3

    def __init__(self, score): # Step 4
        self.score = score
```

```
# Optional extra method
# def is_passing(self):
#     return self.score >= self.minimum passing

# Step 2: Create students
roger = Student("Roger van der Weyden", 10)
sandro = Student("Sandro Botticelli", 12)
pieter = Student("Pieter Bruegel the Elder", 8)

# Step 7: Add a grade
pieter.add_grade(Grade(100))

# You can check results like this:
# print(pieter.grades[0].score) # Should output 100
# print(pieter.get_average())   # If you enable the optional method
```

Object-Oriented Programming

Introduction to Object-Oriented Programming

In programming, most languages offer various features that give us different ways to tackle technical problems. With so many different languages out there, with their own unique set of features, it became necessary to create a classification system to help distinguish those sets of features. This ultimately led to the creation of the term *programming paradigm* - a way to classify different programming languages and the unique features that they offered.

As we explore Python deeper, our code might fall into multiple paradigm categories at once. This is because most modern-day languages offer more than one specific paradigm we can program in. While we could spend all day exploring all the paradigms Python offers, we will instead dive into one of the most popular paradigms, and one we have already been using (maybe unknowingly), called

[Object-Oriented Programming](#)

Preview: Docs Object-oriented programming (OOP) is a software development paradigm which encourages sculpting desired entities with properties and methods in named classes to create applications. (OOP).

At the forefront of any language classified as an OOP language, there must exist the ability to create programs around classes and objects. We have already started working with these concepts earlier as we built our own custom classes, class methods, and instance objects. To recap, let's take a look at an example:

```
class Dog:
    sound = "Woof"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(Dog.sound)
```

Copy to Clipboard

In the above example, we are representing a real-world entity (a dog) as a class with properties (name and age) and methods (bark). These features make up the core of the OOP paradigm and ultimately allow us to build more intricate programs. This however only scratches the surface of what we can accomplish. To explore the paradigm further, we will examine the four core pillars of OOP:

- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Each of these pillars will allow us to expand our skills so that we can take full advantage of the power of object-oriented programming in Python! We will begin exploring these pillars in later exercises but for now, let's refresh ourselves on OOP fundamentals.

Instructions

Checkpoint 1 Enabled

1.

To start our exploration into OOP, create a class that will represent an employee of a company.

In **script.py**:

Define a class called `Employee`

Define a class variable `new_id` and set it equal to `1`

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Each `Employee` instance will need its own unique ID.

Inside the `Employee` class:

Define an `__init__()` method

Inside `__init__()`, define `self.id` and set it equal to the class variable `new_id`

Lastly, increment `new_id` by 1

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now create a function to output the instance id.

Inside the `Employee` class:

Define a `say_id()` method

Inside `say_id()`, output the string "My id is " and then the instance id.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Lastly, create 2 employees and have them give their ids.

Outside of the `Employee` class:

Define the variable `e1` and set it to an instance of `Employee`

Define the variable `e2` and set it to an instance of `Employee`

Have both `e1` and `e2` output their ids

```
# Write your code below
class Employee():
    new_id = 1
    def __init__(self):
        self.id = Employee.new_id
        Employee.new_id += 1

    def say_id(self):
        print("My id is {}".format(self.id))

e1 = Employee()
e2 = Employee()
e1.say_id()
e2.say_id()
```

Object-Oriented Programming

OOP Pillar: Inheritance

When we hear the word "inheritance", code may not be the first thing that springs to mind; we're probably more likely to think of inheriting genetic traits, like the eye color from a mother or dimples from a grandfather. In the world of

[Object-Oriented Programming](#)

Preview: Docs Object-oriented programming (OOP) is a software development paradigm which encourages sculpting desired entities with properties and methods in named classes to create applications.

[Inheritance](#)

Preview: Docs Loading link description

is actually one of the core pillars for creating intricate structures with our classes. To dive into this concept, let's examine a `Dog` and `Cat` class:

```
class Dog:

    def bark(self):
        print('Woof!')

class Cat:
```

```
def meow(self):  
    print('Meow!')
```

Copy to Clipboard

These two classes define two distinct animals with their own methods of communication. Now, what if we wanted to give both of these classes the ability to eat by calling a

method

Preview: Docs Loading link description

called `eat()`. We could write the method twice in both classes but then we would be repeating code! We also may need to write it inside every specific animal class we ever create. Instead, we can utilize the power of inheritance.

Since both `Cat` and `Dog` fall under the classification of `Animal` we can create a *parent class* to represent properties and methods they can both share! Here is what it might look like:

```
class Animal:  
    def eat(self):  
        print("Nom Nom Nom...eating food!")
```

Copy to Clipboard

Great, we have an `Animal` class with a `eat()` method, but how do we actually get the `Dog` and `Cat` class to **inherit** this method so it can be shared with both classes? Well here is what the base structure will look like:

```
class ParentClass:  
    #class methods/properties...  
  
class ChildClass(ParentClass):  
    #class methods/properties...
```

Copy to Clipboard

If we apply this structure to our example, our code looks like this:

```
class Dog(Animal):  
    def bark(self):  
        print('Bark!')  
  
class Cat(Animal):  
    def meow(self):  
        print('Meow!')
```

Copy to Clipboard

Now, let's see inheritance in action:

```
fluffy = Dog()  
zoomie = Cat()  
  
fluffy.eat() # Nom Nom Nom...eating food!  
zoomie.eat() # Nom Nom Nom...eating food!
```

Copy to Clipboard

As we can see, there are some clear advantages of utilizing inheritance. Not only are we able to reuse methods across multiple classes using our *parent class*, but we are also able to create parent-child relationships between entities!

Instructions

Checkpoint 1 Enabled

1.

Now that there is an `Employee` class we want to make a more specific type of employee, `Admin`.

In **script.py**:

Create an `Admin` class that inherits from the `Employee` class

Inside the body of the class insert the `pass` statement

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now it's time to test out your inheritance implementation.

At the bottom of **script.py**:

Define a variable `e3` and set it to an instance of the `Admin` class

Now if you call the `.say_id()` method of the `Admin` instance in `e3`, you will get output with the instance's id.

```
class Employee():
    new_id = 1
    def __init__(self):
        self.id = Employee.new_id
        Employee.new_id += 1

    def say_id(self):
        print("My id is {}".format(self.id))

# Write your code below
class Admin(Employee):
    pass

e1 = Employee()
e2 = Employee()
e3 = Admin()
e3.say_id()
```

Overriding Methods

When implementing

[inheritance](#)

Preview: Docs Loading link description

, a child class may want to change the behavior of a

[method](#)

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

from its parent class. In Python, all we have to do is *override* a method definition. An overriding method in a subclass is one that has the same definition as the parent class but contains different behavior.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def make_noise(self):
        print("{} says, Grrrr".format(self.name))

pet1 = Animal("Rex")
pet1.make_noise() # Rex says, Grrrr
```

Copy to Clipboard

The `Animal` class above has one attribute, `self.name` and one method, `.make_noise()`. The `.make_noise()` method outputs a somewhat generic animal sound, "Rex says, Grrrr". If we define a subclass of `Animal` we may want to make a different sound.

```
class Cat(Animal):

    def make_noise(self):
        print("{} says, Meow!".format(self.name))

pet2 = Cat("Maisy")
pet2.make_noise() # Maisy says, Meow!
```

Copy to Clipboard

Now we've made a class for a more specific type of animal, `Cat`. It has all the attributes and methods of `Animal`. However, if you call the `.make_noise()` method on this instance of `Cat` it will say "Maisy says, Meow!".

Instructions

Checkpoint 1 Enabled

1.

As an admin, you feel it is not important to give your ID, but just let others know they're talking to an admin.

Inside the `Admin` class:

Define a method `say_id()`

Inside the method, output "I am an Admin"

Now when you call `.say_id()` with `e3` you should see the `.say_id()` method output from `Admin`.

```
class Employee():
    new_id = 1
    def __init__(self):
        self.id = Employee.new_id
        Employee.new_id += 1

    def say_id(self):
        print("My id is {}".format(self.id))

class Admin(Employee):
    # Write your code below
    def say_id(self):
        print('I am an Admin')

e1 = Employee()
e2 = Employee()
e3 = Admin()
e3.say_id()
```

super()

When overriding methods we sometimes want to still access the behavior of the parent

[method](#)

Preview: Docs Loading link description

. In order to do that we need a way to call the method of the parent class. Python gives us a way to do that using `super()`.

`super()` gives us a *proxy object*. With this proxy object, we can invoke the method of an object's parent class (also called its superclass). We call the required function as a method on `super()`:

```
class Animal:
    def __init__(self, name, sound="Grrrr"):
        self.name = name
        self.sound = sound

    def make_noise(self):
        print("{} says, {}".format(self.name, self.sound))

class Cat(Animal):
    def __init__(self, name):
        super().__init__(name, "Meow!")

pet_cat = Cat("Rachel")
pet_cat.make_noise() # Rachel says, Meow!
```

Copy to Clipboard

In the above example, we have the class `Animal` and the subclass `Cat`. `Animal` has 2 attributes, `name` and `sound` and one method, `.make_noise()`. The `.make_noise()` method outputs the `name` and `sound` of an instance.

The `Cat` subclass has an `__init__()` method which means the `__init__()` method of its superclass, `Animal` will not be called when creating an instance of `Cat`. The `__init__()` method from the subclass is overriding the one from the superclass.

To still invoke the `__init__()` method of `Animal`, `super().__init__(name, "Meow!")` is called inside the subclass `__init__()` method. This additional logic allows us to add the "Meow" sound from within the `Cat` class, but still use the `__init__()` method of the `Animal` class.

`super()` is used in subclasses to invoke a needed behavior from the superclass alongside the behavior of a subclass method.

Instructions

Checkpoint 1 Enabled

1.

Once the managers found out that the admins were walking around just telling people they are admins, the managers stepped in and made them also say their ID.

Inside the `Admin` class:

Add a line that also calls the `Employee` class `.say_id()` method

Now the output should be the admin's ID and that they are an admin.

Use `super()` to call the `Employee` class `.say_id()` method. Use the following syntax:

```
super().method()
```

```
class Employee():
    new_id = 1
    def __init__(self):
        self.id = Employee.new_id
        Employee.new_id += 1

    def say_id(self):
        print("My id is {}".format(self.id))

class Admin(Employee):
    def say_id(self):
        super().say_id()
        print("I am an admin.")

e1 = Employee()
e2 = Employee()
e3 = Admin()
e3.say_id()
```

Multiple Inheritance: Part 1

Let's now look at a feature allowed by Python called *multiple inheritance*. As you may have guessed from the name, this is when a subclass inherits from more than one superclass. One form of multiple

[inheritance](#)

Preview: Docs Loading link description

is when there are multiple levels of inheritance. This means a class inherits members from its superclass and its super-superclass.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def say_hi(self):
        print("{} says, Hi!".format(self.name))

class Cat(Animal):
    pass
```

```
class Angry_Cat(Cat):
    pass

my_pet = Angry_Cat("Mr. Cranky")
my_pet.say_hi() # Mr. Cranky says, Hi!
```

Copy to Clipboard

In the above example, `Angry_Cat` inherits from `Cat` and `Cat` inherits from `Animal`. Both `Angry_Cat` and `Cat` have access to the `Animal` class `name` attribute and `.say_hi()`

[method](#)

Preview: Docs Loading link description

. Any feature added to `Cat`, `Angry_Cat` will also have access to.

Instructions

Checkpoint 1 Enabled

1.

Managers decide to start walking around more to let people know who is in charge.

Inside **script.py**:

Define a `Manager` class and have it inherit from the `Admin` class

Inside the `Manager` class, define a method `say_id()` that outputs that they are in charge.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

The Managers want to set a good example so they also let people know their ID and that they are an admin.

Inside the `.say_id()` method of `Manager`:

Call the `Admin` class `.say_id()` method

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now test it out.

At the bottom of **script.py**:

Define a variable `e4` and set it to an instance of the `Manager` class

Call the `.say_id()` method of the instance in `e4`

Now you will get output from all 3 classes.

```
class Employee():
    new_id = 1
    def __init__(self):
        self.id = Employee.new_id
        Employee.new_id += 1

    def say_id(self):
        print("My id is {}".format(self.id))

class Admin(Employee):
    def say_id(self):
        super().say_id()
        print("I am an admin.")

# Write your code below
class Manager(Admin):
    def say_id(self):
        print("I am in charge!")
        super().say_id()

e1 = Employee()
e2 = Employee()
e3 = Admin()
e4 = Manager()
e4.say_id()
```

Multiple Inheritance: Part 2

Another form of multiple

[inheritance](#)

Preview: Docs Loading link description

involves a subclass that inherits directly from two classes and can use the attributes and methods of both.

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def action(self):
        print("{} wags tail. Awww".format(self.name))

class Wolf(Animal):
    def action(self):
        print("{} bites. OUCH!".format(self.name))

class Hybrid(Dog, Wolf):
    def action(self):
        super().action()
        Wolf.action(self)

my_pet = Hybrid("Fluffy")
my_pet.action() # Fluffy wags tail. Awww
               # Fluffy bites. OUCH!
```

Copy to Clipboard

The above example shows the class `Hybrid` is a subclass of both `Dog` and `Wolf` which are also both subclasses of `Animal`. All 3 subclasses can use the features in `Animal` and `Hybrid` can use the features of `Dog` and `Wolf`. But, `Dog` and `Wolf` can not use each other's features.

This form of multiple inheritance can be useful by adding functionality from a class that does not fit in with the current design scheme of the current classes.

Care must be taken when creating an inheritance structure like this, especially when using the `super()`

. In the above example, calling `super().action()` inside the `Hybrid` class invokes the `.action()` method of the `Dog` class. This is due to it being listed before `Wolf` in the `Hybrid(Dog, Wolf)` definition.

The line `Wolf.action(self)` calls the `Wolf` class `.action()` method. The important thing to note here is that `self` is passed as an

[argument](#)

Preview: Docs Loading link description

. This ensures that the `.action()` method in `Wolf` receives the `Hybrid` class instance to output the correct `name`.

Instructions

Checkpoint 1 Enabled

1.

Admins in the company need access to the consumer-facing website. This means that admins must also be users of the site.

The class `User` has been added and has the attributes `username` and `role` and the `.say_user_info()` method.

To get the admins the user access they need:

Have the `Admin` class inherit from the `User` class alongside the `Employee` class. Be sure to have the `Employee` class listed first in the `Admin` class definition.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now let's make sure the admins get their user data set up.

Inside the `.__init__()` method of the `Admin` class:

Call the `.__init__()` method of the `User` class

Pass the `Admin` class instance, `id` and the string `"Admin"` as arguments to the `.__init__()` method call

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Confirm the user data is set up correctly.

Call the `.say_user_info()` method using the `Admin` instance in `e3`

```
class Employee():
    new_id = 1
    def __init__(self):
        self.id = Employee.new_id
        Employee.new_id += 1

    def say_id(self):
        print("My id is {}".format(self.id))

class User:
    def __init__(self, username, role="Customer"):
        self.username = username
        self.role = role

    def say_user_info(self):
        print("My username is {}".format(self.username))
        print("My role is {}".format(self.role))

# Write your code below
class Admin(Employee, User):
    def __init__(self):
        super().__init__()
        User.__init__(self, self.id, "Admin")

    def say_id(self):
        super().say_id()
        print("I am an admin.")

e1 = Employee()
e2 = Employee()
e3 = Admin()
e3.say_user_info()
```

OOP Pillar: Polymorphism

In computer programming,

[polymorphism](#)

Preview: Docs Loading link description

is the ability to apply an identical operation onto different types of objects. This can be useful when an object type may not be known at the program

[runtime](#)

Preview: Docs Loading link description

. Polymorphism can be applied using Python in multiple ways. We have already experienced a form of it when exploring

[inheritance](#)

Preview: Docs Inheritance is one of the most important principles of object-oriented programming (OOP). It allows for one class (child class) to inherit the fields and methods of another class (parent class). An inherited class is also called a subclass of its superclass (parent class). Inheritance is considered one of the four pillars of OOP, along with abstraction, and encapsulation, and polymorphism. java // Parent Class class Machine { // Machine class member

```
class Animal:
    def __init__(self, name):
        self.name = name

    def make_noise(self):
        print("{} says, Grrrr".format(self.name))
```

```
class Cat(Animal):

    def make_noise(self):
        print("{} says, Meow!".format(self.name))

class Robot:

    def make_noise(self):
        print("beep.boop...BEEEEEP!!!")
```

Copy to Clipboard

The example above shows an `Animal` class, its subclass `Cat`, and another standalone class `Robot`. Each class has a

[method](#)

Preview: Docs Loading link description

`.make_noise()` with different outputs. The identical method name with different behaviors is a form of polymorphism.

```
an_animal = Animal("Bear")
my_pet = Cat("Maisy")
my_vacuum = Robot()
objects = [an_animal, my_pet, my_vacuum]
for o in objects:
    o.make_noise()

# OUTPUT
# "Bear says, Grrrrr"
# "Maisy says, Meow!"
# "beep.boop...BEEEEEP!!!"
```

Copy to Clipboard

With the classes instantiated and added to a list, we are able to iterate through the list and call `.make_noise()`. This is done without needing to know what type of class `.make_noise()` belongs to.

Instructions

Checkpoint 1 Enabled

1.

A meeting needs to be scheduled with at least one employee, one admin and one manager.
Define a variable `meeting` and set it equal to a list that contains an instance of each class, `Employee()`, `Admin()` and `Manager()`

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

With the different types of employees in the meeting, have them all say their ID.
Using a for loop iterate through the list `meeting`
Using your defined loop variable, call the `.say_id()` method on each instance in the list

```
class Employee():
    new_id = 1
    def __init__(self):
        self.id = Employee.new_id
        Employee.new_id += 1

    def say_id(self):
        print("My id is {}".format(self.id))

class Admin(Employee):
    def say_id(self):
        super().say_id()
        print("I am an admin.")

class Manager(Admin):
    def say_id(self):
        print("I am in charge!")
        super().say_id()

# Write your code below
meeting = [Employee(), Admin(), Manager()]
for member in meeting:
```

```
member.say_id()
```

Output:

```
My id is 1.  
My id is 2.  
I am an admin.  
I am in charge!  
My id is 3.  
I am an admin.
```

Dunder Methods

The code below shows that when working with different object types like, `int`, `str` or `list`, the `+` operator performs different functions. This is known as *operator overloading* and is another form of

[polymorphism](#)

Preview: Docs Polymorphism is one of the four principles of object-oriented programming (OOP). It promotes dynamic inheritance of child classes from their parent class in order to make use of these features themselves in code. With polymorphism, child classes are able to inherit methods from their parent class for use. This ability to inherit functionality from parent classes not only speeds up the development process, but also helps us reason and understand our code more intuitively as we can see the correlation between objects for easier understanding and manipulation of source code. py from math import pi class Shape: def init(self, name): self.name = name

```
# For an int and an int, + returns an int  
2 + 4 == 6  
  
# For a string and a string, + returns a string  
"Is this " + "addition?" == "Is this addition?"  
  
# For a list and a list, + returns a list  
[1, 2] + [3, 4] == [1, 2, 3, 4]
```

Copy to Clipboard

To implement this behavior, we must first discuss *dunder methods*. Every defined class in Python has access to a group of these special methods. We've explored a few already, the constructor `__init__()` and the

[string](#)

Preview: Docs Loading link description
representation

[method](#)

Preview: Docs Loading link description

`__repr__()`. The name dunder method is derived from the **Double UNDER**scores that surround the name of each method. Recall that the `__repr__()` method takes only one

[parameter](#)

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called tripleThis(): js function tripleThis(x) { return x * 3; } tripleThis(6);
, `self`, and must return a string value. The returned value should be a *string representation* of the class, which can be seen by using `print()` on an instance of the class. Review the sample code below for an example of how this method is used. Defining a class's dunder methods is a way to perform operator overloading.

```
class Animal:  
    def __init__(self, name):  
        self.name = name  
  
    def repr (self):  
        return self.name  
  
    def add (self, another_animal):
```

```

        return Animal(self.name + another_animal.name)

a1 = Animal("Horse")
a2 = Animal("Penguin")
a3 = a1 + a2
print(a1) # Prints "Horse"
print(a2) # Prints "Penguin"
print(a3) # Prints "HorsePenguin"

```

Copy to Clipboard

The above code has the class `Animal` with a dunder method, `__add__()`. This defines the `+` operator behavior when used on objects of this class type. The method returns a new `Animal` object with the names of the operand objects concatenated. In this example, we have created a `"HorsePenguin"`!

The line of code `a3 = a1 + a2` invokes the `__add__()` method of the left operand, `a1`, with the right operand `a2` passed as an [argument](#)

Preview: Docs Loading link description

. The `name` attributes of `a1` and `a2` are concatenated using the `__add__()` parameters, `self` and `another_animal`. The resulting string is used as the name of a new `Animal` object which is returned to become the value of `a3`.

Instructions

Checkpoint 1 Enabled

1.

There is now a `Meeting` class with an `attendees` list attribute and an `__add__()` dunder method that adds `Employee` instances to the `attendees` list. Before we try and add employees to a meeting, we want to make sure we can know how many employees are in a meeting.

Inside the `Meeting` class:

Overload the `len()` operation by defining a `__len__()` dunder method

Inside the `__len__()` definition, return the length of the attribute `attendees`

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now add three employees to a meeting:

Using the `Meeting` instance `m1`, add each of the employee instances `e1`, `e2`, and `e3`. Use one line for each employee instance.

Output the length of meeting instance `m1`

You should see the output from each employee being added and then the length of the meeting, `3`.

```

class Employee():
    new_id = 1
    def __init__(self):
        self.id = Employee.new_id
        Employee.new_id += 1

class Meeting:
    def __init__(self):
        self.attendees = []

    def __add__(self, employee):
        print("ID {} added.".format(employee.id))
        self.attendees.append(employee)

    # Write your code
    def __len__(self):
        return len(self.attendees)

e1 = Employee()
e2 = Employee()
e3 = Employee()
m1 = Meeting()
m1 + e1
m1 + e2
m1 + e3

print(len(m1))

```

OOP Pillar: Abstraction

When a program starts to get big, classes might start to share functionality or we may lose sight of the purpose of a class's

[inheritance](#)

Preview: Docs Loading link description

structure. In order to alleviate issues like this, we can use the concept of *abstraction*.

Abstraction helps with the design of code by defining necessary behaviors to be implemented within a class structure. By doing so, abstraction also helps avoid leaving out or overlapping class functionality as class hierarchies get larger.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def make_noise(self):
        pass

class Cat(Animal):
    def make_noise(self):
        print("{} says, Meow!".format(self.name))

class Dog(Animal):
    def make_noise(self):
        print("{} says, Woof!".format(self.name))

kitty = Cat("Maisy")
doggy = Dog("Amber")
kitty.make_noise() # "Maisy says, Meow!"
doggy.make_noise() # "Amber says, Woof!"
```

Copy to Clipboard

Above we have `Cat` and `Dog` classes that inherit from `Animal`. The `Animal` class now inherits from an imported class `ABC`, which stands for Abstract Base Class.

This is the first step to making `Animal` an abstract class that cannot be instantiated. The second step is using the imported decorator `@abstractmethod` on the empty

[method](#)

Preview: Docs Loading link description

`.make_noise()`.

The below line of code would throw an error.

```
an animal = Animal("Scruffy")

# TypeError: Can't instantiate abstract class Animal with abstract method make_noise
```

Copy to Clipboard

The abstraction process defines what an `Animal` is but does not allow the creation of one. The `__init__()` method still requires a name, since we feel all animals deserve a name.

The `.make_noise()` method exists since all animals make some form of noise, but the method is not implemented since each animal makes a different noise. Each subclass of `Animal` is now required to define their own `.make_noise()` method or an error will occur.

These are some of the ways abstraction supports the design of an organized class structure.

Instructions

Checkpoint 1 Enabled

1.

Take a look at the code in `script.py`. The abstract class `AbstractEmployee` is defined. It has all the logic that has previously existed in the `Employee` class, except that the `.say_id()` method is not implemented and has the `@abstractmethod` decorator.

The `Employee` class currently has no implementation.

Run the code and observe that `e1.say_id()` is causing an `AttributeError` since the `Employee` class has no implementation.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Let's fix this error:

Make the `Employee` class inherit from the `AbstractEmployee` class

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Nice work! But wait, there's still an error!

```
TypeError: Can't instantiate abstract class Employee with abstract methods say_id
```

Copy to Clipboard

The `.say_id()` method in the `AbstractEmployee` class uses the `@abstractmethod` decorator. This means any class that inherits from `AbstractEmployee` must implement a `.say_id()` method.

Inside the `Employee` class replace the `pass` statement, then:

Define a `say_id()` method that outputs a message with `self.id`

When complete you should see the output in the console.

```
from abc import ABC, abstractmethod

class AbstractEmployee(ABC):
    new_id = 1
    def __init__(self):
        self.id = AbstractEmployee.new_id
        AbstractEmployee.new_id += 1

    @abstractmethod
    def say_id(self):
        pass

# Write your code below
class Employee():
    pass

e1 = Employee()
e1.say_id()
```

```
from abc import ABC, abstractmethod

class AbstractEmployee(ABC):
    new_id = 1
    def __init__(self):
        self.id = AbstractEmployee.new_id
        AbstractEmployee.new_id += 1

    @abstractmethod
    def say_id(self):
        pass

# Write your code below
class Employee(AbstractEmployee):
    def say_id(self):
        print("My id is {}".format(self.id))

e1 = Employee()
e1.say_id()
```

OOP Pillar: Encapsulation

[Encapsulation](#) is one of the four principles of object-oriented programming (OOP), and it advances the benefits of modularity and hiding away of complexities in order to better maintain and reason about code.

is the process of making methods and data hidden inside the object they relate to. Languages accomplish this with what are called access modifiers like:

Public
Protected
Private

In general, public members can be accessed from anywhere, protected members can only be accessed from code within the same module and private members can only be accessed from code within the class that these members are defined.

Python doesn't have any inbuilt mechanism to prevent access from any member (i.e. all members are public in Python). However, there is a common convention amongst developers to use a single underscore `self._x` to indicate that a member is protected. Accessing a protected member outside of the module will not cause an error, it is added by developers to inform other developers that they should be careful when accessing this member in such a manner.

Similarly, we can declare a member as private with two leading underscores `self.__x`. This is more than just a convention in Python because of a mechanism called *name mangling*. Members that are preceded with two underscores have their names modified in the background to `obj._Classname__x`. While they can still be publicly accessed, the purpose of this mechanism is to prevent clashing member names of any inheriting classes that might define a member of the same name.

Note that this is different from the dunder methods we discussed earlier. A dunder [method](#)

Preview: Docs Loading link description

has two leading and two trailing underscores and is treated differently than a private member. One important difference is that dunder method names are not mangled.

Instructions

Checkpoint 1 Enabled

1.

The `Employee` class contains one attribute `id`. An instance variable `e` is defined and then passed to the function `dir()` which is output to the console.

`dir()` is a built-in Python function that returns a list of all class members, including dunder methods.

When you run the code, you will see a list of class members with `id` as the last element.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now add an attribute that uses the single underscore naming convention.

Inside the `Employee` class `__init__()` method:

Define the single underscore variable `_id` and set it equal to whatever you want

When you run the code you can see `_id` as the second to last element in the output list.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now define a variable using the double-underscore.

Inside the `Employee` class `__init__()` method:

Define the double underscore variable `__id` and set it equal to whatever you want

When you run the code you can see a new variable `_Employee__id` as the first element in the output list. This is the result of name-mangling the variable `self.__id`.

```
class Employee():
    def __init__(self):
        self.id = None
        # Write your code below
        self._id = None
        self.__id = None

e = Employee()
print(dir(e))
```

Output-only Terminal

Output:

```
['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'format', 'ge', 'getattribute', 'gt', 'hash', 'init', 'init_subclass', 'le', 'lt', '_module_', '_ne_', '_new_', '_reduce_', '_reduce_ex_', '_repr_', '_setattr_', 'sizeof', 'str', 'subclasshook', 'weakref', 'id']
```

Getters, Setters and Deleters

Using *getter*, *setter*, and *deleter* functions are one way to implement [Encapsulation](#) within Python where the state of class attributes can be handled within the class. These functions are useful in making sure that the data being handled is appropriate for the defined class functionality.

```
class Animal:
    def __init__(self, name):
        self._name = name
        self._age = None

    def get_age(self):
        return self._age

    def set_age(self, new_age):
        if isinstance(new_age, int):
            self._age = new_age
        else:
            raise TypeError

    def delete_age(self):
        del self._age
        print("_age Deleted")
```

Copy to Clipboard

Looking at the `Animal` class above there is an `_age` attribute with a single underscore. This notates it is intended to be used only within the module. There are then 3 methods related to `age` each with a different purpose. These define the getter, setter, and deleter of the specific property.

The first [Method](#) related to `age` is a getter and returns `self._age`. The setter is implemented below that. It includes logic that ensures that the value passed to `new_age` is an integer. If so, `self._age = new_age`. If not, raise an error. This is useful and shows the power of using these functions for encapsulation.

The deleter is implemented below the setter. It outputs a confirmation message and uses the `del` keyword to delete the `self._age` attribute.

```
a = Animal("Rufus")
print(a.get_age()) # None

a.set_age(10)
print(a.get_age()) # 10

a.set_age("Ten") # Raises a TypeError

a.delete_age() # " _age Deleted"
print(a.get_age()) # Raises a AttributeError
```

Copy to Clipboard

Above we see `a.get_age()` gets the `_age` value, `a.set_age(10)` sets the value and `a.delete_age()` deletes the attribute entirely. A `TypeError` occurs with `a.set_age("Ten")` because the defined logic in the setter is looking only for an integer. An `AttributeError` occurs with `a.get_age()` after the attribute was deleted.

Instructions

Checkpoint 1 Enabled

1.

Using a getter, setter, and the single underscore naming convention, the employees of the company will now be able to use their names. The single underscore attribute `_name` has been added to the `Employee` class. Names default to `None` unless a string argument is passed during instantiation.

Inside the `Employee` class:

Define a getter method called `get_name()` that returns the class attribute `_name`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Add a setter method.

Inside the `Employee` class:

Define a setter method `set_name` that has an additional parameter for the name string and sets the class attribute `_name`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Lastly, Add a deleter method.

Inside the `Employee` class:

Define a deleter method `del_name` that deletes the attribute.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Great job defining the getter and setter. Uncomment the code at the bottom of **script.py** to test out the getter, setter, and deleter.

Move on to the next exercise to wrap up the lesson!

```
class Employee():
    new_id = 1
    def __init__(self, name=None):
        self.id = Employee.new_id
        Employee.new_id += 1
        self.name = name

    def get_name(self):
        return self.name

    def set_name(self, new_name):
        self._name = new_name

    def del_name(self):
        del self._name

e1 = Employee("Maisy")
e2 = Employee()
print(e1.get_name())

e2.set_name("Fluffy")
print(e2.get_name())

e2.del_name()
print(e2.get_name())
```

Review

Congratulations! You've completed the [object-oriented programming](#) with Python lesson. Let's do a quick recap of what you learned.

We discussed the four pillars of object-oriented programming as they apply to the Python programming language.

[Inheritance](#)

Python allows classes to inherit on multiple levels. Meaning a class can inherit from a base class as well as a derived class. Python also supports multiple inheritance, where one class can inherit from any number of other classes. This allows us to describe complex relationships between objects with minimal repeated code.

Polymorphism

Polymorphism is a concept that allows functions and objects to behave in different ways depending on context. There is the [polymorphism](#) of functions like `len()` or the addition operator `+`, which can act differently depending on the provided data.

Abstraction

Python supports the concept of abstraction by allowing objects with methods that have the same name, to be called in a general manner. Further, Python provides the Abstract Base Class (ABC) for us to create a more clearly defined interface.

Encapsulation

Python's approach to [encapsulation](#) is unique compared to most other object-oriented programming languages. In Python, all members of an object are publicly accessible but there are conventions to indicate to developers that a member is intended to be protected or private.

In this lesson, we learned more complicated relationships between classes. We learned:

- How to create a subclass of an existing class.
- How to redefine existing methods of a parent class in a subclass by overriding them.
- How to leverage a parent class's methods in the body of a subclass [method](#) using the `super()` function.
- How to write programs that are flexible using interfaces and polymorphism.
- How to write [data types](#) that look and feel like native data types with dunder methods.

These are really complicated concepts! It's a long journey to get to the state of comfortably being able to build class hierarchies that embody the concerns that your software will need to. Give yourself a pat on the back, you earned it!

Instructions

Take a look at the code in **script.py**. We've added a complete example that aligns with the four pillars of object-oriented programming. Can you identify each one within the class interactions?

The @property Decorator

Learn about the most pythonic way of using getters, setters, and deleters!

Introduction

In this article, we'll explore the *@property decorator* - a more pythonic way to use getters and setters in our object-oriented programs! Before we dive in, let's briefly review the concept of getters and setters.

Let's recall that getters and setters are useful tools for achieving encapsulation (a way to prevent data from direct modification). We can define a private attribute and then use getters and setters to expose a public means of reading and modifying a class attribute value. Further, getters and setter methods allow us to create complex behavior such as limiting access under certain conditions or imposing restrictions on allowable ranges of values for an attribute.

Let's start by looking at an example class called `Box` with one attribute called `weight`. In this case, `weight` will be a private attribute with a getter and a setter (`getWeight()` and `setWeight()`).

```
class Box:
    def __init__(self, weight):
        self.__weight = weight

    def getWeight(self):
        return self.__weight

    def setWeight(self, weight):
        if weight >= 0:
            self.__weight = weight
```

Notice two things:

We want to follow best practices by denoting `weight` as a private attribute using `__` (dunder) notation. This, however, does not make an attribute private, and we can still manipulate it directly.

We are also posing some restrictions on our setter so that the `weight` of an instance of the `Box` class can only be set to values greater than zero. We can see this if we try to manipulate an instance:

```
box = Box(10)
```

```
box.setWeight(-5)
print(box.getWeight())
```

```
box.setWeight(5)
print(box.getWeight())
```

That our `box` weight was unchanged on the first call:

```
10
5
```

Notice that we need to call the methods instead of directly interacting with the `weight` attribute. What if we could have the best of both worlds? That is, a way to directly interact with the `weight` attribute but still benefit from the complex behavior of methods such as the weight restriction. This is where the built-in function `property()` comes in.

The built-in property() function

The Python built-in `property()` function accepts four optional arguments: `fget`, `fset`, `fdel`, and `doc`. The first three represent getter, setter, and deleter methods, respectively, and the last one is a docstring for the attribute.

Let's take a look at the advantages by refactoring our `Box` class:

```
class Box:
    def __init__(self, weight):
        self.__weight = weight

    def getWeight(self):
        return self.__weight

    def setWeight(self, weight):
        if weight >= 0:
            self.__weight = weight

    def delWeight(self):
        del self.__weight

    weight = property(getWeight, setWeight, delWeight, "Docstring for the 'weight' property")
```

Notice we have added one additional method to our class called `delWeight()` to serve as a deleter for the property. While it is not strictly required, we will add it to show the full potential of the `property()` function. We then call the `property()` function and pass the getter, setter, and deleter in as arguments. This will immediately allow us to use the following syntax for our class:

```
box = Box(10)

print(box.weight) #this calls .getWeight()

box.weight = 5 #this called .setWeight()

del box.weight #this calls .delWeight()

box.weight = -5 #box.__weight is unchanged
```

With this change, our program gains some immediate benefits:

We can now use the `weight` attribute as if it was public. We no longer have to call the setters, getters, and deleter methods directly and

thus giving our program a simpler syntax.

Even though we no longer call the methods directly, we still can maintain constraints such as the weight limit in `setWeight()`. It's the best of both worlds!

If we had a huge codebase that used our methods multiple times in multiple places, a single change to the method name would seriously mess up our program since we would have to change it everywhere! We no longer have this issue using the `property()` method since we never call it directly.

While this is a big advantage for our programs to be more "pythonic", we can go even further by using the decorator pattern to replace the need to call the `property()` function altogether.

@property Decorator

The most pythonic way to define getters, setters, and deleters is by using the `@property` decorator. This decorator is syntactic sugar for using the `property()` function and helps our code look much cleaner. Let's take a look:

```
class Box:
    def __init__(self, weight):
        self.__weight = weight

    @property
    def weight(self):
        """Docstring for the 'weight' property"""
        return self.__weight
```

```

@weight.setter
def weight(self, weight):
    if weight >= 0:
        self.__weight = weight

@weight.deleter
def weight(self):
    del self.__weight

```

Let's break this down:

```

        weight()
    @property
    @weight.setter    @weight.deleter

```

This is the equivalent of doing:

```

weight = property(getWeight, setWeight, delWeight, "Docstring for the 'weight' property")

```

And thus giving us the same syntactical advantage as before:

```

box = Box(10)

box.weight = 5

del box.weight

```

Wrap up

To summarize, we learned:

- The limitations of using regular setter and getter methods.
- How to use the `property()` function to create cleaner getters, setters, and deleters.
- The `@property` decorator is the most "pythonic" way to use the `property()` function.

When using the decorator, remember three rules:

- All three methods must use the same member name (ex. `weight`).
- The first method must be the getter and is identified using `@property`.
- The decorators for the setter and deleter are defined by the name of the method `@property` is used with.

Keep the `@property` decorator in mind when approaching any object-oriented program! It will save time and keep code cleaner and more maintainable.

Functional Programming in Python

An intro to functional programming in Python

Introduction

In this article, we will explore the concept of [functional programming](#), including its differences from [object-oriented programming](#).

This article is divided into the following sections:

- [Introduction to functional programming](#)
- [Functional vs. object-oriented programming](#)
- [Declarative vs. Imperative programming](#)
- [Writing functions in functional programming](#)
- [Using recursion instead of loops](#)
- [Passing functions as arguments to other functions](#)
- [Conclusion](#)

Introduction to functional programming

Functional programming is a programming paradigm in which code is structured primarily in the form of functions. The origins of this programming style arise from a branch of mathematics known as *lambda calculus*, which is the study of functions and their mathematical properties. In contrast to the popular object-oriented and procedural approaches, functional programming offers a different way of thinking when solving a problem.

Let's explore how to program solutions in a functional style using [Python](#).

Functional vs. object-oriented programming

In **object-oriented programming (OOP)**, you structure your code based on the concept of objects. In commonly used OOP languages (such as Java or C#), we create a template for an object using the keyword `class`, and we instantiate an object by using the keyword `new`. These objects contain fields that store data and methods that manipulate data. In this style, an object can be passed as an argument in the function of another object.

In functional programming, you create a solution by structuring code mainly using functions. Functions are “first-class citizens” in this paradigm, meaning we can use them to store and manipulate data (with limitations). Similar to objects in object-oriented programming, in functional programming, functions can be passed in as arguments to other functions and can be returned by other functions.

Declarative vs. imperative programming

Before we learn of the advantages functional programming offers, we must first discuss the differences between an *imperative paradigm* and a *declarative paradigm*.

Imperative programming solves a problem by describing the step-by-step solution. Imperative programming is concerned with “how to solve a problem.”

In contrast, **declarative programming** relies on the underlying framework or programming language to solve a problem. The programmer’s only task is to describe what problem they would like solved. Declarative programming is concerned with “what problem to solve.”

As an analogy, consider the example of obtaining a cup of coffee. Imperative programming would be the equivalent of you making the cup of coffee yourself by performing the following steps:

Declarative programming is the equivalent of going to your local coffee shop and telling the barista, “I’d like a cup of coffee with one teaspoon of sugar, please.” The barista prepares your coffee, gives you the cup, and you drink it without concerning yourself with the details of how it was made.

For a programming analogy, consider the example of having to sort a list in Python. Imperatively sorting the list would be you implementing a sorting algorithm like this:

```
lst = [8, 4, 14, 9, 12, 5, 7, 1, 10, 2, 3]

# Sort using Selection Sort algorithm
for i in range(len(lst)):
    min_idx = i
    for j in range(i+1, len(lst)):
        min_idx = j if lst[j] < lst[min_idx] else min_idx
    temp = lst[i]
    lst[i] = lst[min_idx]
    lst[min_idx] = temp
```

If we were to solve the list declaratively in Python, we would call the `lst.sort()` function like so:

```
lst = [8, 4, 14, 9, 12, 5, 7, 1, 10, 2, 3]
```

```
list.sort(lst)
```

Which style a programmer follows might depend on their goals. If you are looking for concise code that gets you an answer efficiently, you might go with a more declarative approach. If you are working with other people and want to ensure that your code is easy to follow, you may choose a more imperative style.

OOP and procedural programming follow an imperative approach; functional programming follows a declarative approach.

Writing functions in functional programming.

As we said in the introduction to this article, the central concept in functional programming is that of a function; however, there are requirements that a function must adhere to when writing one. The first of these requirements is the function must be deterministic. That is, for any given input, the function must return the same output when provided with the same set of inputs. Let's illustrate this with the following example of an `add` function:

```
def add(x, y):  
    return x + y
```

This function will always return 7 if `add(5, 2)` is called.

Another of these requirements is that functions must be free of as many *side effects* as possible. A side effect is when a function alters some external variable. The goal is to minimize, not eliminate, side effects. It would be impossible to eliminate all side effects in a function because a program eventually must have an external effect to be useful.

A **pure function** is defined as a deterministic function with no side effect.

Consider this example:

```
ans = 0  
  
def add(x, y): # This function has side effects  
    ans = x + y
```

The `add(x, y)` function defined in this way has a side effect because it alters the external variable `ans`.

To make this a pure function, we should write this as:

```
ans = 0  
  
def add(x, y): # This function has no side effects  
    return x + y  
  
ans = add(x, y)
```

Notice how the `add(x, y)` function no longer alters the `ans` variable during its execution. Also, the variables `x` and `y` only exist within the scope of the function. Operating on them allows the function to have no side effects.

For reusability, a function should always have all relevant parameters passed in and should not rely on global variables.

Consider the example of a function (`app()`) that replaces the first and last element of a list with 0:

```
lst = [1,2,3,4,5]
```

```
def app():  
    ret = list(lst)  
    ret[0] = 0  
    ret[-1] = 0  
    return ret
```

```
zero_list = app()
```

Writing a function in this way makes it dependent on an external list variable called `lst`. The function is not reusable as using it elsewhere requires `lst` to be defined externally. While it is not reusable, `app()` is still considered free of side effects as it reads, but does not modify, the variable `lst`. A better way to write the function would be to pass in the required list variable as an argument like so:

```
lst = [1,2,3,4,5]
```

```
def app(l):  
    ret = list(l)  
    ret[0] = 0  
    ret[-1] = 0  
    return ret
```

```
zero_list = app(lst)
```

The `app(l)` function is free of side effects and is now also reusable.

We care about pure functions because as our programs get more complex, we want to avoid incorrect alterations of external variables. For example, if we call functions from different threads running concurrently, we need to construct our functions carefully to avoid side effects. Following the functional programming paradigm is a great way to keep code clean and efficient and simplify testing.

Using Recursion Instead of Loops

To execute a section of code repeatedly, object-oriented programmers use a `while` loop or a `for` loop. **Loops** do not adhere to the functional programming style because they maintain an external counter which is altered from inside the loop (side effect!). In functional programming, we use **recursion** to execute code repeatedly. Recursive functions are typically written in the following style:

```
def recursive_function(n):  
    # Base case to terminate recursion goes here  
    # Call to recursive_function with new parameter goes here
```

The classic example of a recursive function is the computation of the `n`th Fibonacci number done like so:

```
def fib(n):  
    if n <= 1:  
        return 1  
    return fib(n-1) + fib(n-2)
```

Passing functions as arguments to other functions

In traditional OOP, you can pass objects as arguments to a function. In functional programming, you can pass functions as arguments to other functions. This is known as treating functions as *first-class citizens*, a term you may often see in industry.

Let's take a look at an example:

```
def add(x, y):  
    return x + y  
  
def sub(x, y):  
    return x - y  
  
def times3(a, b, function):  
    return 3 * function(a, b)  
  
add_then_times3 = times3(2, 4, add) # 18  
sub_then_times3 = times3(2, 4, sub) # -6
```

This code defines a `times3()` function that multiplies the result of an `add` or subtract (`sub`) function by three and returns the value. The `times3()` function also allows the passing of any other function that accepts two parameters and returns an integer.

Functional programming promotes the concept of brevity when writing code. It is important to be concise and write functions in as few lines as possible as long as readability is maintained. We can also pass a function to another function by writing it in the argument using a [lambda](#).

We can shorten the previous code using a lambda like so:

```
def times3(a, b, function):  
    return 3 * function(a, b)  
  
add_then_times3 = times3(2, 4, lambda x, y: x + y) # 18  
sub_then_times3 = times3(2, 4, lambda x, y: x - y) # -6
```

This will output the same result as the verbose implementation from earlier. Lambdas are a great tool as they allow a programmer to write a function while maintaining the flow of ideas. A programmer is no longer required to stop, write a function elsewhere, and then resume work. A drawback of lambdas is that they are only suitable for implementing simple functions; it is not possible to write all functions as lambdas!

Conclusion

In summary, functional programming is a powerful paradigm that confers many advantages. In this paradigm, we structure solutions in the form of functions. Functional programming differs from object-oriented programming. It follows a declarative style as opposed to an imperative style leading to code that is shorter, easier to test, less prone to bugs, and well-suited for multithreading applications. Many languages allow for implementing the functional programming paradigm; one of the more popular ones is Python.

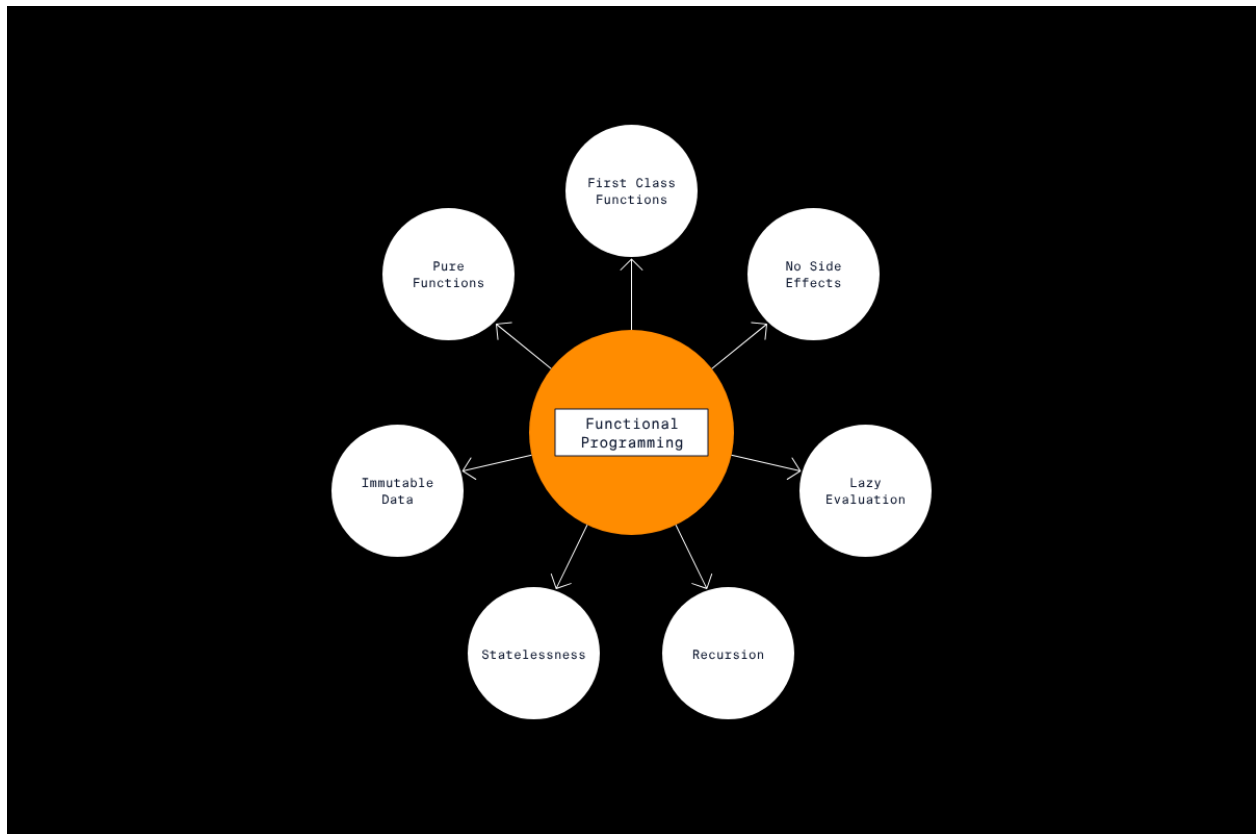
Functional Programming

Introduction

Get ready to become a master of functional programming! This content branches off some content you may have run into from our Intermediate Python. We will do a quick review of:

tuples
lambda functions
`map()`, `reduce()`, `filter()`
working with
[CSV](#)
Preview: Docs Comma-Separated Values (CSV) is a text file format where individual values are separated by commas.
and
[JSON](#)
Preview: Docs Loading link description
files

Then we will dive into more advanced problems using these tools!



Immutable Data types

Immutable [data types](#) are important to use in [functional programming](#).

Preview: Docs Loading link description

as they offer advantages, such as:

- thread-safe data manipulation
- preventing programmers from accidentally changing a value

We can create a [tuple](#) of tuples with a mix of datatypes, like the following:

```
student = (("Scott", 28, 'A'), ("Nicole", 26, 'B'), ("John", 29, 'D')) #  
mixed elements
```

Copy to Clipboard

This is a great way to store our mixed data. However, there is room for improvement. The student tuple contains records of students in a CS class where each tuple stores the student's name, age, and the grade they received. Defining a tuple in this manner is prone to errors as it requires the programmer to remember the position of each piece of data in the tuple.

Instead, we can use a `namedtuple` data type from the `collections` library like so:

```
from collections import namedtuple  
  
# Create a class called student  
student = namedtuple("student", ["name", "age", "grade"])  
  
# Create tuples for the three students  
  
scott = student("Scott", 28, 'A')  
nicole = student("Nicole", 26, 'B')  
john = student("John", 29, 'D')
```

Copy to Clipboard

We access the student information:

```
from collections import namedtuple  
  
# create a class called student  
student = namedtuple("student", ["name", "age", "grade"])  
  
# Create tuples for the three students  
scott = student("Scott", 28, 'A')  
nicole = student("Nicole", 26, 'B')  
john = student("John", 29, 'D')  
  
# Access Scott's information for example  
print(scott.name) # Output: Scott  
print(scott.age) # Output: 28  
print(scott.grade) # Output: 'A'
```

Copy to Clipboard

Take note that the name of the tuple and the variable that stores the tuple must be identical.

We can package three student tuples neatly into a tuple called `students` like so:

```
students = (scott, nicole, john)
```

Copy to Clipboard

The programmer is no longer required to remember the position of each piece of data as they can reference it using the property name.

Instructions

Checkpoint 1 Failed, try again

1.

Create a `namedtuple` called `country` to represent a country. It should contain fields to represent the name of a country, its capital, and the continent on which it is located.

The `country` tuple should contain `name`, `capital`, and `continent` as fields.

Checkpoint 2 Step instruction is unavailable until previous steps are completed
2.

Create three tuples that represent the following countries:

France: capital: Paris, continent: Europe

Japan: capital: Tokyo, continent: Asia

Senegal: capital: Dakar, continent: Africa

The country name should be used as the variable name. Note that your capitalization does not matter, so you can define the France variable as `France` or `france`, and you can define its `capital` as either `"Paris"` or `"paris"`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Pack all three countries into a tuple named `countries`.

```
load_file_in_context("exercise2.py")

try:
    # Attempt to access variable called senegal:
    if not (senegal.capital == "dakar" or senegal.capital == "Dakar"):
        fail_tests("Expected capital of Senegal to be Dakar")
    if not (senegal.name == "senegal" or senegal.name == "Senegal"):
        fail_tests("Expected name of Senegal to be `Senegal`")
    if not (senegal.continent == "africa" or senegal.continent == "Africa"):
        fail_tests("Expected continent of Senegal to be Africa")
except NameError:
    try:
        # If senegal does not exist, maybe it was spelled as 'Senegal':
        if not (Senegal.capital == "dakar" or Senegal.capital == "Dakar"):
            fail_tests("Expected capital of Senegal to be Dakar")
        if not (Senegal.name == "senegal" or Senegal.name == "Senegal"):
            fail_tests("Expected name of Senegal to be `Senegal`")
        if not (Senegal.continent == "africa" or Senegal.continent == "Africa"):
            fail_tests("Expected continent of Senegal to be Africa")
    except NameError:
        fail_tests("Expected `Senegal` to be defined.")

try:
    # Attempt to access variable called france:
    if not (france.capital == "paris" or france.capital == "Paris"):
        fail_tests("Expected capital of France to be Paris")
    if not (france.name == "france" or france.name == "France"):
        fail_tests("Expected name of France to be `France`")
    if not (france.continent == "europe" or france.continent == "Europe"):
        fail_tests("Expected continent of France to be Europe")
except NameError:
    try:
        # If france does not exist, maybe it was spelled as 'France':
        if not (France.capital == "paris" or France.capital == "Paris"):
            fail_tests("Expected capital of France to be Paris")
        if not (France.name == "france" or France.name == "France"):
            fail_tests("Expected name of France to be `France`")
        if not (France.continent == "europe" or France.continent == "Europe"):
            fail_tests("Expected continent of France to be Europe")
    except NameError:
        fail_tests("Expected `France` to be defined.")

try:
    # Attempt to access variable called japan:
    if not (japan.capital == "tokyo" or japan.capital == "Tokyo"):
        fail_tests("Expected capital of Japan to be Tokyo")
    if not (japan.name == "japan" or japan.name == "Japan"):
        fail_tests("Expected name of Japan to be `Japan`")
    if not (japan.continent == "asia" or japan.continent == "Asia"):
        fail_tests("Expected continent of Japan to be Asia")
except NameError:
    try:
        # If japan does not exist, maybe it was spelled as 'Jenegal':
        if not (Japan.capital == "tokyo" or Japan.capital == "Tokyo"):
            fail_tests("Expected capital of Japan to be Tokyo")
        if not (Japan.name == "japan" or Japan.name == "Japan"):
            fail_tests("Expected name of Japan to be `Japan`")
        if not (Japan.continent == "asia" or Japan.continent == "Asia"):
            fail_tests("Expected continent of Japan to be Asia")
    except NameError:
        fail_tests("Expected `Japan` to be defined.")

pass_tests()
```



```

from abc import ABC, abstractmethod

class AbstractEmployee(ABC):
    new_id = 1
    def __init__(self):
        self.id = AbstractEmployee.new_id
        AbstractEmployee.new_id += 1

    @abstractmethod
    def say_id(self):
        pass

class User:
    def __init__(self):
        self.username = None

    @property
    def username(self):
        return self._username

    @username.setter
    def username(self, new_name):
        self._username = new_name

class Meeting:
    def __init__(self):
        self.attendees = []

    def __add__(self, employee):
        print("{} added.".format(employee.username))
        self.attendees.append(employee.username)

    def __len__(self):
        return len(self.attendees)

class Employee(AbstractEmployee, User):
    def __init__(self, username):
        super().__init__()
        User.__init__(self)
        self.username = username

    def say_id(self):
        print("My id is {}".format(self.id))

    def say_username(self):
        print("My username is {}".format(self.username))

```

Review of Lambda Functions

Lambda functions are crucial in

[functional programming](#).

Preview: Docs Loading link description

as they allow the production of neat and concise functions that require other functions as an [argument](#)

Preview: Docs Loading link description

. You should be familiar with lambda functions from previous courses on Python, but if you are not (or would like a refresher), this exercise will serve as a brief review.

A lambda function is a short

[anonymous function](#)

Preview: Docs Defines a function without a name using the lambda keyword.

that can accept several parameters but only returns one value. Lambdas can be stored as a variable or defined inline in the accepting function.

Consider the following example:

We determine the cost of a two-dimensional sheet of metal by multiplying the price per square-meter times the area of the sheet. We can cut the sheet into either a rectangle or a triangle. To model this, let's write a general function in Python that calculates the price of a sheet of metal. The function will accept the following parameters:

price per square-meter
a
[tuple](#)
Preview: Docs Loading link description
of dimensions,
a function to compute area. The function will return the price of a sheet of metal.

Recall from geometry:

Area of a triangle: $0.5 * (\text{base} * \text{height})$
Area of a rectangle: $\text{base} * \text{height}$

Writing the function without lambdas, we have:

```
def rect(b, h):  
    return b * h  
  
def tri(b, h):  
    return 0.5 * (b * h)  
  
# ppsm: price per square meter  
# dim: dimensions tuple  
def total_cost(ppsm, dim, area):  
    return ppsm * area(dim[0], dim[1])  
  
print(total_cost(3, (5, 5), rect)) # Rectangular sheet costing 75 units  
print(total_cost(4, (6, 7), tri)) # Triangular sheet costing 84 units
```

Copy to Clipboard

To make the code shorter, we can replace the `rect(b,h)` and the `tri(b,h)` functions with lambdas like so:

```
# ppsm: price per square meter  
# dim: dimensions tuple  
def total_cost(ppsm, dim, area):  
    return ppsm * area(dim[0], dim[1])  
  
print(total_cost(3, (5, 5), lambda b, h: b*h)) # Rectangular sheet costing 75 units  
print(total_cost(4, (6, 7), lambda b, h: 0.5 * b*h)) # Triangular sheet costing 84 units
```

Lambda functions can be stored in a variable like so:

```
rect = lambda x, y: x * y  
tri = lambda x, y: 0.5 * x * y
```

Lambdas are a very powerful tool in Python. However, they have a drawback in that they are only suited for "short and fast" functions. Long, complicated functions cannot be written as lambdas.

Instructions

Checkpoint 1 Enabled

1. Complete the `odd_or_even()` function provided. The body of the function will return `even_function(n)` if `n` is even and return `odd_function(n)` if `n` is odd.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2. Convert the functions `squared(x)` and `cubed(x)` into lambdas stored in variables called `square` and `cube` respectively.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3. For this checkpoint, odd numbers are to be squared and even numbers are to be cubed! Create a variable called `test` and set it to `odd_or_even()` with `n` being 5 and pass in `square` and `cube` in the appropriate locations.

```

"""
def squared(x):
    return x * x

def cubed(x):
    return x*x*x
"""
def odd_or_even(n, even_function, odd_function):
    pass # Remove this statement for Checkpoint 1.

# Checkpoint 2 code goes here.

# Checkpoint 3 code goes here.

#print(test) # Uncomment the print function to see the results of Checkpoint 3.

```

```

"""
def squared(x):
    return x * x

def cubed(x):
    return x*x*x
"""
def odd_or_even(n, even_function, odd_function):
    if n % 2 == 0:
        return even_function(n)
    else:
        return odd_function(n)

square = lambda x: x*x
cube = lambda x: x*x*x

test = odd_or_even(5, cube, square)

print(test) # Uncomment the print function to see the results of Checkpoint 3.

```

Review of filter(), map(), and reduce()

In this exercise, we will review the `map()`, `filter()`, and `reduce()`, higher-order functions provided by Python. These functions accept an iterable and a processing function as arguments and return another iterable. Similar to the exercise on lambdas, this section should serve as a refresher. You may recognize these functions from intermediate Python content. If you need more than a quick refresher and feel stuck at all during this exercise, we recommend you review this [article](#) as an aid.

The first function we will look at is `filter()`, which converts the following imperative code:

```

nums = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

# filter_values is not a higher-order function
def filter_values(predicate, lst):

    # Mutable list required because this example is imperative, not declarative
    ret = []
    for i in lst:
        if predicate(i):
            ret.append(i)
    return ret

filtered_numbers = filter_values(lambda x: x % 2 == 0, nums)

print(filtered_numbers)

```

```
# This will output the list: [2, 4, 6, 8, 10]
```

Copy to Clipboard

into the following declarative code:

```
nums = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

filtered_numbers = filter(lambda x: x % 2 == 0, nums)

print(tuple(filtered_numbers))

# This will output the tuple: (2, 4, 6, 8, 10)
```

Copy to Clipboard

map() converts the following imperative code:

```
nums = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

def mapper(function, lst):
    ret = []
    for i in lst:
        ret.append(function(i))
    return ret

mapped_numbers = mapper(lambda x: x*x, nums)

print(tuple(mapped_numbers))

# This will output: (1, 4, 9, 16, 25, 36, 49, 64, 81, 100)
```

Copy to Clipboard

Into the following declarative code:

```
numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

mapped_numbers = map(lambda x: x*x, numbers)

print(tuple(mapped_numbers))

# This will also output: (1, 4, 9, 16, 25, 36, 49, 64, 81, 100)
```

Copy to Clipboard

The `reduce()` function accepts an iterable and a two-parameter function (no more than two) known as an “*accumulator*”. It uses the accumulator to recursively process the contents of the iterable and “reduce” them to one value. An example application of `reduce()` would be to compute the sum of all numbers in a list.

The sum of all elements in a

[tuple](#)

Preview: Docs Loading link description

written imperatively:

```
nums = (2, 6, 7, 9, 1, 4, 8)

sum = 0

for i in nums:
    sum += i

print(sum) # Output: 37
```

Copy to Clipboard

Using `reduce()` to find the sum declaratively:

```
"""
In Python 3, the `reduce()` function has been moved to the `functools` library, so we need to
import it before we can use it.
"""
from functools import reduce
```

```
nums = (2, 6, 7, 9, 1, 4, 8)

reduced_nums = reduce(lambda x, y: x + y, nums) # reduced_nums is a number

print(reduced_nums) # Output: 37
```

The lambda in `reduce()` requires exactly two parameters.

Higher-order functions are powerful tools that you can use to significantly shorten code, thereby leading to cleaner and more concise solutions. In later exercises, we will explore how to combine these functions.

Instructions

Checkpoint 1 Enabled

1. Use `filter()` to remove odd numbers from tuple `nums`. Print your result using the following line of code:

```
print(tuple(filtered_numbers))
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2. Using `map()` multiply all of the elements of `nums` by 3. Print your result with the following line of code:

```
print(tuple(mapped_numbers))
```

Copy to Clipboard

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3. Find the sum of all the elements in the tuple `nums` using the `reduce()` function. Print your result with the following line of code:

```
print(sum)
```

```
nums = (16, 2, 19, 22, 10, 23, 16, 2, 27, 29, 19, 26, 12, 20, 16, 29, 6, 2, 12, 20)

# Checkpoint 1 code goes here.

# Checkpoint 2 code goes here.

# Checkpoint 3 code goes here.
```

```
from functools import reduce
nums = (16, 2, 19, 22, 10, 23, 16, 2, 27, 29, 19, 26, 12, 20, 16, 29, 6, 2, 12, 20)

filtered_numbers = filter(lambda x: x % 2 == 0, nums)

print(tuple(filtered_numbers))

mapped_numbers = map(lambda x: x * 3, nums)

print(tuple(mapped_numbers))

sum = reduce(lambda x, y: x + y, nums)

print(sum)
```

Mapping a Filtered Collection

In this exercise, we will see how we can combine the `map()` and `filter()` functions.

Conceptually, if you're working with a collection of items and find yourself saying, "I need to map only values that have property x," you will likely need to use `map()` and `filter()` together. In English, "I need to map filtered values" translates into Python like this:

```
map(mapping_function, filter(predicate, iterable))
```

Let's take a look at an example.

We have a tuple representing the records of students in various math classes. The tuple will be structured in the following way: `student(name, grade, course)`. As an example, `student("Peter", 'B', 101)` will represent a student named Peter who received a grade of B in a Math 101 course.

We want to register students who receive a grade of B or better in their respective math courses for a special advanced math course: Math 201. To create the record for the course, we will filter all students with grade B or higher and map their math course to 201.

The initial grade in the new tuple will contain the letter 'X' to represent that it is not yet defined because the student has not yet finished Math 101. We can do this in Python like so:

First, we create a `namedtuple` to represent a student:

```
from collections import namedtuple

# Create a class called student
student = namedtuple("student", ["name", "grade", "course_number"])
```

Copy to Clipboard

We then create records for the sample data:

```
# Create the records for the students in the form of tuples

peter = student("Peter", 'B', 101)
amanda = student("Amanda", 'C', 101 )
sarah = student("Sarah", 'A', 102)
lisa = student("Lisa", 'D', 101)
alex = student("Alex", 'A', 102)
maria = student("Maria", 'B', 101)
andrew = student("Andrew", 'C', 102)

math_class = (peter, amanda, sarah, lisa, alex, maria, andrew)
```

Copy to Clipboard

Finally, we use `map()` to create a new record for a student consisting of:

- their name
- the initial grade X
- the math course Math 201 only IF the student's previous grade was B or above in their respective math course

```
math_201 = map(lambda s: student(s.name, 'X', 201), filter(lambda q: q.grade <= 'B', math_class))

print(tuple(math_201))
```

Copy to Clipboard

The line `q.grade <= 'B'` removes students with grades less than B. The `<=` appears because the character A is lower than B in the alphabet (how Python compares the characters).

The code "maps a filtered value". Remember, that because `map()` returns an iterable, we use the `tuple()` function to generate all the values we need.

The output will be all the students who are eligible for Math 201:

```
(student("Peter", 'X', 201), student("Sarah", 'X', 201), student("Alex", 'X', 201),
student("Maria", 'X', 201))
```

Copy to Clipboard

Instructions

Checkpoint 1 Failed, try again

1.

Given the tuple `nums` multiply values that are greater than 10 by two. Save your result as `greater than 10 doubled`. Print your answer using the following line of code:

```
print(tuple(greater than 10 doubled))
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Convert the following code from the imperative style to the declarative:

```
lst = []
for i in nums:
    if i % 3 == 0:
        lst.append(i)

for i in range(len(lst)):
    lst[i] = lst[i] * 3

tuple(lst)
```

Copy to Clipboard

Here, `nums` represents the tuple provided in the workspace.

Save your result in a variable called `functional_way`. Print your answer using the following line of code:

```
print(tuple(functional_way))
```

Copy to Clipboard

If you get stuck, look over the hint.

```
nums = (2, 12, 5, 8, 9, 3, 16, 7, 13, 19, 21, 1, 15, 4, 22, 20, 11)

# Checkpoint 1 code goes here.
greater_than_10_doubled = map(lambda x: x*2, filter(lambda y: y > 10, nums))

print(tuple(greater_than_10_doubled))

# Checkpoint 2 code goes here.
functional_way = map(lambda x: x * 3, filter(lambda y: y % 3 == 0, nums))

print(tuple(functional_way))
```

Reducing a Filtered Collection

LIBRARY - **FUNCTOOLS**

In this exercise, we will explore how to use `reduce()` and `filter()` together. Let's consider the following example. We have a tuple representing the menu items for a fictitious restaurant: The Codecademy Steakhouse. We wish to find the least expensive appetizer on the menu. We can express this process by the sentence: "filter the menu for appetizers and return the least expensive one." Let's do this imperatively first.

The starting code:

Click to see starting code

```
from collections import namedtuple
from functools import reduce

# Prices are in USD
menu_item = namedtuple("menu_item", ["name", "dish_type", "price"])

jsp = menu_item("Jumbo Shrimp Platter", "Appetizer", 29.95)
lc = menu_item("Lobster Cake", "Appetizer", 30.95)
scb = menu_item("Sizzling Canadian Bacon", "Appetizer", 9.95)
ccc = menu_item("Codecademy Crab Cake", "Appetizer", 32.95)
cs = menu_item("Caeser Salad", "Salad", 14.95)
mgs = menu_item("Mixed Green Salad", "Salad", 11.95)
cp = menu_item("Codecademy Potatoes", "Side", 34.95)
mp = menu_item("Mashed Potatoes", "Side", 14.95)
rs = menu_item("Ribeye Steak", "Entree", 75.95)
phs = menu_item("Porter House Steak", "Entree", 131.95)
```

```
menu = (jsp, lc, scb, ccc, cs, mgs, cp, mp, rs, phs)
```

Copy to Clipboard

Imperative solution:

```
# apps only contains items with dish type == "Appetizer"
apps = [i for i in menu if i.dish_type == "Appetizer"]
cheapest_app = apps[0]
for i in apps:
    if i.price < cheapest_app.price:
        cheapest_app = i

print(cheapest_app) # Output will be: menu_item("Sizzling Canadian Bacon", "Appetizer", 9.95)
```

We can eliminate the `for` loop and do this declaratively by using `filter()` and `reduce()` like this:

```
cheapest_app = reduce(lambda x, y: x if x.price < y.price else y, filter(lambda x: x.dish_type ==
"Appetizer", menu))

print(cheapest_app) # Output will be: menu_item("Sizzling Canadian Bacon", "Appetizer", 9.95)
```

In the `reduce()` function, the lambda `lambda x, y: x if x.price < y.price else y` returns the cheaper of the two `menu_items` compared by their price.

The `filter()` function returns an iterable with all `menu_items` that have "Appetizer" as `dish_type`. This iterable is then passed into `reduce()` to be processed by its lambda function. Combining these two functions converts the imperative code into a one-line solution!

Instructions

Checkpoint 1 Enabled

1.

Given the extended menu for The Codecademy Steakhouse, using `filter()` and `reduce()` find the most expensive dish that is an entree and assign it to a variable called `entree`. Make sure to print out your answer!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Using `filter()` and `reduce()`, find the least expensive dish that is either a side or a salad and assign it to a variable called `least_expensive`. Make sure to print out your answer!

```
from collections import namedtuple
from functools import reduce

# Prices are in USD
menu_item = namedtuple("menu item", ["name", "dish type", "price"])

jsp = menu_item("Jumbo Shrimp Platter", "Appetizer", 29.95)
lc = menu_item("Lobster Cake", "Appetizer", 30.95)
scb = menu_item("Sizzling Canadian Bacon", "Appetizer", 9.95)
ccc = menu_item("Codecademy Crab Cake", "Appetizer", 32.95)
cs = menu_item("Caeser Salad", "Salad", 14.95)
mgs = menu_item("Mixed Green Salad", "Salad", 10.95)
cp = menu_item("Codecademy Potatoes", "Side", 34.95)
mp = menu_item("Mashed Potatoes", "Side", 14.95)
a = menu_item("Asparagus", "Side", 15.95)
rs = menu_item("Ribeye Steak", "Entree", 75.95)
phs = menu_item("Porter House Steak", "Entree", 131.95)
grs = menu_item("Grilled Salmon", "Entree", 36.95)

menu = (jsp, lc, scb, ccc, cs, mgs, cp, mp, a, rs, phs, grs)
entree = 0
least_expensive = 0

entree = reduce(lambda x, y: x if x.price > y.price else y, filter(lambda x: x.dish_type ==
"Entree", menu))

print(entree)

least_expensive = reduce(lambda x, y: x if x.price < y.price else y, filter(lambda x: x.dish_type
== "Side" or x.dish_type == "Salad", menu))
```



```
print(least_expensive)
```

Reducing a Mapped Collection

In this exercise, we will be focussing on the benefits of using `reduce()` and `map()` together.

Consider the example of having a

[dictionary](#)

Preview: Docs Loading link description

representing a cost of an item sale called `costs` which maps an item name to a

[tuple](#)

Preview: Docs Loading link description

containing the total number of units sold and the price per unit. A dictionary entry would look like this:

```
"name": (total_number_of_item_sold, price_per_item)
```

Copy to Clipboard

We wish to find the total cost of all items sold. If this were a list or a tuple, we could simply apply `reduce()` on it to find the sum. This, however, would present a problem if we attempt this with a dictionary.

The lambda provided to `reduce()` requires that the two parameters and the returned value be of the same type. For example, in the lambda `lambda x, y: x*y`, the `x`, `y`, and return type are all integers. As you can see, you cannot directly reduce a dictionary to a number because they are not of the same type; we must process the data in the dictionary first.

We can use `map()` to iterate through the dictionary and compute the cost of every item sold. We can potentially store this in a tuple and then reduce that tuple to a single number, the total cost.

Note: when passing a dictionary as an iterable, the function will iterate through the list of the dictionaries keys.

Let's look at an example with the following program:

```
from functools import reduce

# Dictionary entry: {"name": (number_or_units_sold, price_per_unit_GBP)}
costs = {"shirt": (4, 13.00), "shoes": (2, 80.00), "pants": (3, 100.00), "socks": (5, 5.00)}

k = reduce(lambda x, y: x+y, map(lambda q: costs[q][0] * costs[q][1], costs))
print(k) # Output will be a total cost of: 537.0 GBP
```

Copy to Clipboard

This dictionary is passed into `map()` along with the lambda `lambda q: costs[q][0] * costs[q][1]`. The lambda function takes the price tuple and generates a `total_cost_per_item` by multiplying the `number_of_units_sold` (`costs[q][0]`) by the `price_per_unit_GBP` (`costs[q][1]`). The lambda in the `reduce()` function is now working strictly with integers to sum them up and returns a total cost of £537.

We could have done this exercise using `namedtuple`, but we excluded it for brevity. Using `map()` to process a dictionary is key when working with the

[JSON](#)

Preview: Docs Loading link description

format, as we will see in a later exercise.

Instructions

Checkpoint 1 Enabled

1.

The dictionary provided represents the number of a given fruit sold over three days - a dictionary entry is:

```
fruit_name:(amount sold on day 1, amount sold on day 2, amount sold on day 3)
```

Copy to Clipboard

Using `map()` and `reduce()`, find the total number of fruits sold. Store this answer in a variable called `total_fruits`. Make sure to print out your solution.

```

from functools import reduce

fruits = {"Grape":(4, 6, 2), "Lemon":(7, 3, 1), "Orange":(5, 8, 1), "Apple":(2, 8, 10), "Watermelon":(0, 9, 6)}

total_fruits = 0

total_fruits = reduce(lambda x, y: x + y, map(lambda q: fruits[q][0] + fruits[q][1] + fruits[q][2], fruits))

print(total_fruits)

```

Combining all Three Higher-Order Functions

Now that you've learned how to combine any two functions, let's see how (and why) we can combine all three! A reason for doing this would be when you need to "filter" a collection before you "map" it (or "map" then "filter") and then "reduce" it to a single number. To illustrate this in practice, let's revisit the inventory problem from earlier, but this time we are only interested in the total sum of prices of items that sold for less than a certain value.

In this Python exercise, we are given the record of items represented by a

[dictionary](#)

Preview: Docs A dictionary is an unordered set of (key, value) pairs. It provides a way to map pieces of data to each other, and allows for quick access to values associated to keys. The syntax of a dictionary is as follows: pseudo dictionary = { key1: value1, key2: value2, key3: value3 }

. We are interested in the total sum of prices of items that sold for less than £150. We do this by:

First "map" the items their individual total cost (*(number of units sold) * (price per unit)*).

Then eliminate ("filter" out) all items that cost more than £150.

Then "reduce" the individual costs to a single number that represents the total cost of the items.

We can do this with the following code:

```

from functools import reduce

# Dictionary entry: {"name": (number_or_units_sold, price_per_unit_GBP)}

costs = {"shirt": (4, 13.00), "shoes": (2, 80.00), "pants": (3, 100.00), "socks": (5, 5.00), "ties": (3, 14.00), "watch": (1, 145.00)}

k = reduce(lambda x, y: x + y, filter(lambda r: r <= 150.00, map(lambda q: costs[q][0] * costs[q][1], costs)))

print(k) # Output will be a total cost of: 264.0 GBP

```

Copy to Clipboard

The focus of the code is:

```

k = reduce(lambda x, y: x + y, filter(lambda r: r <= 150.00, map(lambda q: costs[q][0] * costs[q][1], costs)))

```

Copy to Clipboard

This first computes the total cost of each item using `map()`:

```

map(lambda q: costs[q][0]*costs[q][1], costs)

```

Copy to Clipboard

Then `filter()` is used to eliminate all items that cost more than £150:

```
filter(lambda r: r <= 150.00, map(lambda q: costs[q][0] * costs[q][1], costs))
```

Copy to Clipboard

Then `reduce()` is used to compute the total sum:

```
reduce(lambda x, y: x + y, filter(lambda r: r <= 150.00, map(lambda q: costs[q][0] * costs[q][1], costs)))
```

Copy to Clipboard

This example demonstrates how to use the three functions together.

Instructions

Checkpoint 1 Enabled

1.

Given the record of item sales `costs`, find the total cost of items that cost more than £150. Assign the answer to variable `total`. Make sure to print out your solution.

Use all three higher-order functions for this exercise.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Given the tuple `nums`, use `map()`, `filter()`, and `reduce()` to find all numbers less than 10, add five to them, and find their total product. Assign the answer to variable `product`. Make sure to print out your solution.

```
from functools import reduce

costs = {"shirt": (4, 13.00), "shoes": (2, 80.00), "pants": (3, 100.00), "socks": (5, 5.00),
        "ties": (3, 14.00), "watch": (1, 145.00)}

nums = (24, 6, 7, 16, 8, 2, 3, 11, 21, 20, 22, 23, 19, 12, 1, 4, 17, 9, 25, 15)

total = reduce(lambda x, y: x+y, filter(lambda r: r > 150.00, map(lambda q: costs[q][0] * costs[q][1], costs)))

print(total)

product = -1

product = reduce(lambda x, y: x * y, map(lambda z: z + 5, filter(lambda q: q < 10, nums)))

print(product)
```

Importing Data From a CSV File

Functional programming is widely applicable in the data science domain as higher-order functions can be used to process data files efficiently. One of the most common formats for a data file is a

[CSV](#)

Preview: Docs Comma-Separated Values (CSV) is a text file format where individual values are separated by commas.

file (comma-separated value). In this exercise and the next, we will work with the three higher-order functions to process data contained in a CSV file. For this exercise, we will use a CSV file containing housing data as an example.

The file `zillow.csv` contains housing data collected by the American real-estate company Zillow for the city of Tallahassee Florida. In this exercise we will use `map()` to import the data from the CSV file and represent it using a `namedtuple`.

The data in the file is represented by the following table (which shows the first five lines):

[Index](#)

Preview: Docs Loading link description	Square footage	Year	List price (USD)
1	2222	1981	250000

2	1628	2009	185000
3	3824	1954	399000
4	1137	1993	150000
5	3560	1973	315000

To work with this file, we must first open it and create a *reader* object (an iterator) that will read the file:

```
import csv

with open('zillow.csv', newline = '') as csvfile:
    reader = csv.reader(csvfile, delimiter=',', quotechar='|')
```

Copy to Clipboard

A CSV file often contains millions of lines of data. Importing the entire contents of a CSV file is impractical as this would occupy too much RAM resulting in poor program performance. To avoid importing all the data at once, `reader` is an iterator object that maintains a pointer to the file and iterates through the data when `next(reader)` is called.

We create a `namedtuple` to represent each record:

```
from collections import namedtuple

house = namedtuple("house", ["index", "square_footage", "year", "list_price"])
```

Copy to Clipboard

We use the `map()` function to read in a line of data and store it in a

[tuple](#)

Preview: Docs Loading link description

. Incoming data arrives as a list of strings regardless of their intended type; therefore we must cast each element to convert it to the proper type. The order of the data is dictated by the first line in a CSV file.

This is done like so:

```
# Read record into namedtuple: house(index, square_footage, year, list_price)
h = map(lambda x: house(int(x[0]), int(x[1]), int(x[2]), int(x[3])), reader)
```

Copy to Clipboard

Because the amount of data in the `zillow` file is small, it would be okay to create a tuple called `houses` and populate it with all individual house tuples generated like so:

```
print(tuple(h))
```

Copy to Clipboard

In the next exercise, we will see how we can use the higher-order functions to process data contained in the CSV file.

Instructions

Checkpoint 1 Enabled

1.

Create a `namedtuple` called `tree` intended to store tuples from the records of the provided CSV file `trees.csv`. The CSV file will store the following data for a tree entry:

```
index (inches)
width (inches)
height (feet)
volume (in ft3).
```

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create an iterator called `mapper` that will “map” the records to tuples of type `tree`. Note: the index and height are of type `int`; the width and volume are of type `float`. Uncomment the following lines after creating `mapper`:

```
#trees = tuple(mapper)
#print(trees)
```

```
import csv
from collections import namedtuple
from functools import reduce

# Code for Checkpoint 1 goes here.
tree = namedtuple("tree", ["index", "width", "height", "volume"])

with open('trees.csv', newline = '') as csvfile:
    reader = csv.reader(csvfile, delimiter=',', quotechar='|')
    next(reader) # Skip the first line in trees.csv that contains the data labels.
    # Code for Checkpoint 2 goes here.
    mapper = map(lambda x: tree(int(x[0]), float(x[1]), int(x[2]), float(x[3])), reader)
    trees = tuple(mapper)
    print(trees)
```

Processing Data From a CSV File

In this exercise, we will see how we can process the data that is stored in a

[CSV](#)

Preview: Docs Loading link description
file.

When working with a file that contains a large amount of data, generating every possible record using `tuple()` is inefficient. Recall from the previous exercise that `map()` returns an iterator that we can use to bring in data. We can apply the higher-order functions to this iterator to process the data and only bring in the relevant set of records. For example, let's say we only want to retain tuples for houses built after 1985. We can do this by applying `filter()` with the lambda `lambda q: q[2] > 1985` like so:

```
h = filter(lambda q: q[2] > 1985, map(lambda x: house(int(x[0]), int(x[1]), int(x[2]),
int(x[3])), reader))

houses = tuple(h)
print(houses) # Print out tuples of house built after 1985
```

Copy to Clipboard

Now let's say we want the most expensive house built after 1985. We can use `reduce()` to do this for us like so:

```
h = filter(lambda q: q[2] > 1985, map(lambda x: house(int(x[0]), int(x[1]), int(x[2]),
int(x[3])), reader))
houses = tuple(h)

from functools import reduce

most_expensive = reduce(lambda x, y: x if x.list price > y.list price else y, houses)
print(most_expensive) # Print out tuples of house built after 1985
```

Copy to Clipboard

When working with CSV files, it is important to ensure they are properly formatted; some entries might have leading or trailing whitespaces that we must remove, for example.

Instructions

Checkpoint 1 Enabled

1.

Create a tuple called `trees` and populate it with data of trees that have a height greater than 75. First, create an iterator called `t` where you filter out trees that don't meet the height requirement. Then create the `trees` tuple like so:

```
trees = tuple(t)
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a variable called `widest` and store in it the record of the widest tree in `trees`. You may want to print the answer to see which tree is the widest!

```
import csv
from collections import namedtuple
from functools import reduce

tree = namedtuple("tree", ["index", "width", "height", "volume"])

with open('trees.csv', newline = '') as csvfile:
    reader = csv.reader(csvfile, delimiter=',', quotechar='|')
    next(reader)
    mapper = map(lambda x: tree(int(x[0].strip()), float(x[1]), int(x[2]), float(x[3])), reader)

    t = filter(lambda x: x.height > 75, mapper)
    trees = tuple(t)
    widest = reduce(lambda x, y: x if x.width > y.width else y, trees)
    print(widest)
```

Processing Data From a JSON File

We may find ourselves in situations where we are working with JSON Data. The JSON format is the preferred way to structure data such that it may be transmitted over the internet. An API call usually responds with data formatted in JSON. We can process JSON data using the higher-order functions we have been working with.

However, unlike a CSV file, we cannot use an iterator when we read data from a JSON file. The file's entire contents are read and stored in a Python dictionary. If a JSON object contains another JSON object, the dictionary will contain another dictionary.

Unfortunately, the `json` library in Python does not provide a method to produce an iterator that we can use to read data similar to a CSV; we will have to work with the `dict` type.

To illustrate this in practice, we will process data from the sample JSON file: `cities.json`, which contains the following data:

Click to see JSON data

```
{
  "city": [
    {
      "name": "New York",
      "country": "United States of America",
      "coordinates": [40.7128, -74.0060],
      "continent": "North America"
    },
    {
      "name": "Los Angeles",
      "country": "United States of America",
      "coordinates": [34.0522, 118.2437],
      "continent": "North America"
    },
    {
      "name": "Montreal",
      "country": "Canada",
      "coordinates": [45.5017, -73.5673],
      "continent": "North America"
    },
    {
      "name": "Toronto",
      "country": "Canada",
      "coordinates": [43.6532, -79.3832],
      "continent": "North America"
    }
  ]
}
```

Copy to Clipboard

To work with the JSON file, we must first be open it and read it as we did for a CSV file:

```
import json
with open('cities.json') as json_file:
    cities = json.load(json_file)
```

Copy to Clipboard

In the JSON file, there is an object called `city` that consists of an array of JSON objects containing data relevant to a particular city. When the JSON file is read, it will produce a dictionary containing one entry: a map between the string "city" and a list of dictionaries that represent data relevant to each contained city. The dictionary structure will be:

```
{"city": [{"name": "name_of_city", "country": "country_of_city", "coordinates": [latitude, longitude], "continent": "continent_of_city"}...]}
```

Copy to Clipboard

The data formatted this way is hard to read and susceptible to errors when using it. It would be better to use `map()` to store this data as a `namedtuple`.

```
import json
from collections import namedtuple

city = namedtuple("city", ["name", "country", "coordinates", "continent"])

with open('cities.json') as json_file:
    data = json.load(json_file)
    # cities is a dictionary with one entry: {"cities": [{"name": name, "country": country, "coordinates": [lat, long], "continent": continent}...]}

# Extract the collection of cities from the dictionary - this is a list of dictionaries.
cities_list = data["city"]

cities = map(lambda x: city(x["name"], x["country"], x["coordinates"], x["continent"]),
             cities_list)
```

Copy to Clipboard

The `cities` iterable can now store the entries of the JSON file in a much more readable way.

The other higher-order functions can be used in the same way as `map()` with `cities` now being the supplied iterable.

Instructions

Checkpoint 1 Failed, try again

1.

Store all cities that are on the continent of Asia in the tuple called `asia`. Make sure to print out your result.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Find the western-most city in the `asia` tuple and store it in the `west` variable. Make sure to print out your result.

Note: the western-most country is the one where the longitude is the lowest number (minimum of

`city.coordinates[1]`)

```
import json
from collections import namedtuple
from functools import reduce

city = namedtuple("city", ["name", "country", "coordinates", "continent"])

with open('cities.json') as json_file:
    data = json.load(json_file)

cities = map(lambda x: city(x["name"], x["country"], x["coordinates"], x["continent"]),
             data["city"])

asia = tuple(filter(lambda q: q.continent == "Asia", cities))

print(asia)

west = None

west = reduce(lambda x, y: x if x.coordinates[1] < y.coordinates[1] else y, asia)
```

```
print(west)
```

Review

Congratulations on completing this module on

[functional programming](#)

Preview: Docs Loading link description

in Python. We've covered many topics on how to write neat and efficient solutions using functional programming. Let's briefly recap everything covered so far:

Functional programming follows the declarative approach to programming; the programmer describes what needs to be done as opposed to how it is done.

Functional programming relies heavily on immutable

[data structures](#)

Preview: Docs Loading link description

.

It is preferable to use a

[tuple](#)

Preview: Docs A tuple is a data structure consisting of an ordered collection with an arbitrary number of elements.

instead of a list when writing code because a tuple is immutable.

Using a `namedtuple` to store data leads to a more readable data structure.

The `namedtuple` collection resides in the `collections` library.

A `namedtuple` is created like so:

```
name = ("name", ["property1", "property2", ..., "property"])
```

Copy to Clipboard

Representing an object by a `namedtuple` is done like so:

```
obj = name(property1, property2, ..., property)
```

Copy to Clipboard

Referencing an element of a `namedtuple` is done like this:

```
property = name.property
```

Copy to Clipboard

A lambda function is an anonymous function.

A lambda function can be stored in a variable like any other data type.

A lambda function can be defined directly while passing it as an [Argument](#) of another function.

The three higher-order functions Python provides are:

```
map()
filter()
reduce()
```

These functions can be supplied as the iterable [parameter](#) to any of the higher-order functions. Ex: `h = map(lambda, filter(lambda, map(lambda, iterable)))`

Combining `map()` and `filter()` filters an iterable before a mapping function is applied.

Combining `reduce()` and `filter()` filters an iterable before it is reduced.

A

[CSV](#)

Preview: Docs Loading link description

file can be processed by supplying the CSV reader as the iterable to the higher-order functions.

A

[JSON](#)

Preview: Docs Loading link description

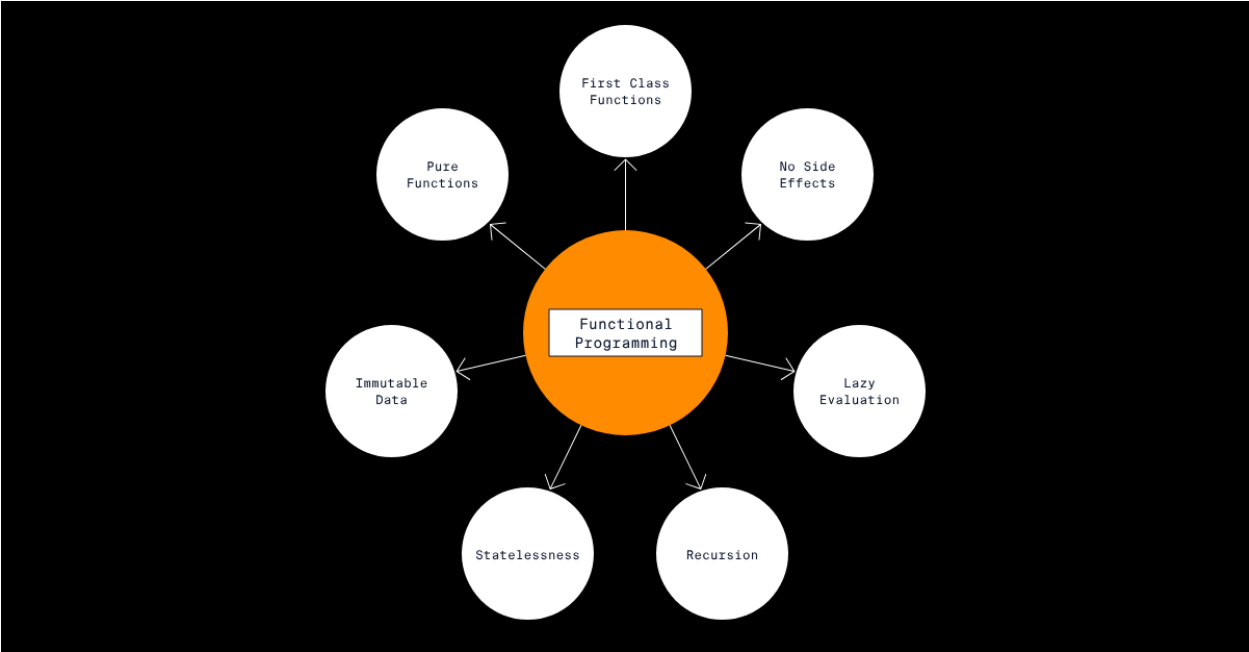
file can only be represented in Python by a

[dictionary](#).

An iterable is not available to read data stored in a JSON file.

A dictionary must be provided to the higher-order functions when processing a JSON file.

Happy Coding!



Logging in Python

Learn about the benefits and functionalities of using a logging module!

Introduction

In this article, we will introduce logging for programmers. We will learn about:

- Advantages of using logs instead of using print statements
- Briefly introduce the `logging` module to write logs in Python

Advantages of Logs

Imagine you are developing an application, and upon running tests, you find some unexpected behavior occurring. You decide to add some print statements throughout the code to see what could be causing your errors. Before long, your beautiful, clean code is littered with print statements! As your code becomes messier, tracking where your print statements exist in your program can become a massive headache. These print statements will also need to be removed before the application enters a final, production-ready state, which can be tedious. There is a potential risk of a print statement getting left behind.

To circumvent a myriad of untidy print statements scattered throughout the code, you can use a logging utility to provide programmers with logs that detail either error messages or general information and output from a program. We can keep our code neatly organized by utilizing a logging utility, making it easy for other programmers to view and understand what is happening and maintain the code. Organized code also makes the task of debugging significantly easier. Using a logging utility can help with debugging by providing detailed information on what caused an error, what time the error occurred, where inside the code the error occurred, or simply providing general information like the values of variables that you need to debug. Finally, by using a logging module, we can save time from needing to remove print statements manually.

Using Logs in Python

Now that we've covered some high-level examples of why we would want to use a logging utility, let's look at how to log specifically with Python. By importing Python's `logging` module, we can:

- Identify the date and time of a custom or error message
- Format logs to make debugging easier
- Set severity levels for the logs
- Output the logs to various streams

We'll briefly introduce these functionalities in this article. In the next lesson, we will go more in-depth on these concepts and practice with them in some interactive coding exercises!

To show how logging is used in a real-world example, imagine we are building an application that accepts username and password input from the user and is responsible for verifying that the entered information matches a registered user in the system. If it does not, it will raise an error. We can add logging statements to the function where this code occurs to log the values entered and log the error message if an exception or error is raised.

```
import logging
logger = logging.getLogger(__name__)
def attempt_login(username, password):
    logger.info("Attempting to validate the entered credentials!")
    if not verify_credentials(username, password):
        logger.error("Invalid credentials for username:
{username}!".format(username=username))
```

Dated Timestamps

It is almost always helpful to know the date and time when a logged message or error occurred for debugging or investigative purposes. The Python `logging` module provides a formatting option to include dated timestamps for logged messages. Take our login application, for example. Timestamped log messages can help us determine the exact time and date that login attempts occurred in our application. We can look at an example of such a log message below:

```
[2021-11-08 03:16:05,980] {ERROR} login:attempt_login - Invalid credentials for username: foo!
```

Additional Formatting Options

Debugging an application can be cumbersome, especially the more complex the application is and the more lines of code and modules it has. The `logging` module fortunately provides several options for formatting what information is included in the log message and how that information appears. We've just seen that there is a formatting option that allows us to include timestamps. We can also include information like the module name where the logged message occurred, or even more, the specific line number within the code file. These formatting options eliminate any need to figure out which module, function, or line number a debugging message originated from.

Severity Level

There are varying levels of severity for logging messages in Python: notset, debug, info, warning, error, and critical. These log levels are used in specific cases that range from providing general information to alerting of severe errors. Being able to set different severity levels for logged messages allows us to filter out logs of certain levels or limit easily. For our login application, once we are ready to move it into a production-ready state, we will not need to have any debug log messages included in our log file. The great advantage to using the logging module is that we do not have to go back and manually remove our logging debug statements from the code like we would have to for print statements. We can filter out any logging statements that have a debug severity level by setting a log level for the logger.

Log Files

Another advantage of using the logging module over console print statements is that the logs can be saved to a file, allowing the log messages to be accessed beyond code execution time. Additionally, the saved log file can then be indexed and made searchable through other software and applications.

Conclusion

In short, logging provides many benefits over using print statements, including:

- the ability to set a destination for the logged messages (e.g. console, file, sockets)
- formatting options for the log messages that include timestamps and line numbers, filtering of log messages based on log level severity
- much more!

This powerful module is key to making clean, readable, maintainable code that can easily be debugged and investigated by using the generated log output.

Logging in Python

Introduction to Logging

Python includes a useful logging module within its standard library that allows for each event within an application to be logged and tracked. By logging messages, we can easily debug our software and investigate events during the development, testing, and production phases.

We could print messages to the console, but this can quickly add clutter to our code. The more complex the application, the more difficult it can be to know where a print statement originated. There is also a chance that some print statements could be left behind and not removed once the application goes into a production-ready state. Using the logging module allows us to write clean and maintainable code that can be easily debugged, tracked, and supported by other developers down the road.

The logging module provides several formatting options that make logged messages very easy to read. While print statements can be formatted, it is not as simple and fast compared to using the logging module. Cleanly formatted log messages can help troubleshoot issues in the application and also help future developers with debugging who may work on the application.

Let's investigate the advantages of using the logging module with an example.

Instructions

Checkpoint 1 Enabled

1.

Imagine you frequently travel out of the country and develop an application to convert currencies between countries quickly.

View the file **with_printouts.py** and compare it to the file **with_logging.py**. Examine the differences in readability, clean lines of code, simplicity, and reduction of repeated code.

Now, execute the **with_printouts.py** code by clicking Run while you have the file open.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now, execute the **with_logging.py** code. Examine the differences in output in terms of readability of the printout/logged messages and the detailed information that is provided in the messages between this output and the previous **with_printouts.py** output.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Imagine you only want to view **ERROR** related printout/logged messages. Consider how this can be accomplished quickly without having to remove several **print** statements. This can be done easily using the **logging** module! Let's introduce how. Click Run to move onto the next exercise!

WITH PRINT(with_printout.py)

```
import logging
import sys

from datetime import datetime

exchange_rates = {
    "USD": 1.0,
    "EUR": 0.861775,
    "GBP": 0.726763,
    "INR": 75.054725,
    "AUD": 1.333679,
    "CAD": 1.237816,
    "SGD": 1.346851
}

def convert(from_country, to_country, amount):
    if to_country not in exchange_rates:
        timestamp = datetime.now()
        print(timestamp)
        print( name )
        print("ERROR: The TO country supplied is not a valid country.")
```

```

    sys.exit(0)

if from_country not in exchange_rates:
    timestamp = datetime.now()
    print(timestamp)
    print( __name__ )
    print("ERROR: The FROM country supplied is not a valid country.")
    sys.exit(0)

to_rate = exchange_rates[to_country]
from_rate = exchange_rates[from_country]

if from_country != "USD":
    converted_to_usd = amount / from_rate
    timestamp = datetime.now()
    print(timestamp)
    print( __name__ )
    print("INFO: Converting from {from_country} to USD:
{converted_to_usd}".format(from_country=from_country, converted_to_usd=converted_to_usd))

    converted_from_usd = converted_to_usd * to_rate
    timestamp = datetime.now()
    print(timestamp)
    print( __name__ )
    print("INFO: Converting from USD to {to_country}:
{converted_from_usd}".format(to_country=to_country, converted_from_usd=converted_from_usd))

    return converted_from_usd

else:
    converted_from_usd = amount * to_rate
    timestamp = datetime.now()
    print(timestamp)
    print( __name__ )
    print("INFO: Converting from USD to {to_country}:
{converted_from_usd}".format(to_country=to_country, converted_from_usd=converted_from_usd))
    return converted_from_usd

timestamp = datetime.now()
print(timestamp)
print( __name__ )
print("DEBUG: Current rates: {exchange_rates}".format(exchange_rates=exchange_rates))
currency = convert("EUR", "USD", 45)
print(currency)

```

Output-only Terminal

Output:

2025-08-22 19:07:42.521038

main

DEBUG: Current rates: {'USD': 1.0, 'EUR': 0.861775, 'GBP': 0.726763, 'INR': 75.054725, 'AUD': 1.333679, 'CAD': 1.237816, 'SGD': 1.346851}

2025-08-22 19:07:42.521075

main

INFO: Converting from EUR to USD: 52.217806271938734

2025-08-22 19:07:42.521082

main

INFO: Converting from USD to USD: 52.217806271938734

52.217806271938734

[WITH LOGGING \(with loggingout.py\)](#)

```

import logging
import sys

```

```

logger = logging.getLogger(__name__)
stream_handler = logging.StreamHandler(sys.stdout)
formatter = logging.Formatter('%(asctime)s %(levelname)s %(message)s')
stream_handler.setFormatter(formatter)
logger.addHandler(stream_handler)
logger.setLevel(logging.DEBUG)

exchange_rates = {
    "USD": 1.0,
    "EUR": 0.861775,
    "GBP": 0.726763,
    "INR": 75.054725,
    "AUD": 1.333679,
    "CAD": 1.237816,
    "SGD": 1.346851
}

def convert(from_country, to_country, amount):
    if to_country not in exchange_rates:
        logger.error("The TO country supplied is not a valid country.")
        sys.exit(0)

    if from_country not in exchange_rates:
        logger.error("ERROR: The FROM country supplied is not a valid country.")
        sys.exit(0)

    to_rate = exchange_rates[to_country]
    from_rate = exchange_rates[from_country]

    if from_country != "USD":
        converted_to_usd = amount / from_rate
        logger.info("Converting from {from_country} to USD:
{converted_to_usd}".format(from_country=from_country, converted_to_usd=converted_to_usd))

        converted_from_usd = converted_to_usd * to_rate
        logger.info("Converting from USD to {to_country}:
{converted_from_usd}".format(to_country=to_country, converted_from_usd=converted_from_usd))

        return converted_from_usd

    else:
        converted_from_usd = amount * to_rate
        logger.info("Converting from USD to {to_country}:
{converted_from_usd}".format(to_country=to_country, converted_from_usd=converted_from_usd))
        return converted_from_usd

logger.debug("Current rates: {exchange_rates}".format(exchange_rates=exchange_rates))
currency = convert("EUR", "USD", 45)
print(currency)

```

Output-only Terminal

Output:

```

[2025-08-22 19:08:12,629] DEBUG [Current rates: {'USD': 1.0, 'EUR': 0.861775, 'GBP': 0.726763,
'INR': 75.054725, 'AUD': 1.333679, 'CAD': 1.237816, 'SGD': 1.346851}]
[2025-08-22 19:08:12,629] INFO [Converting from EUR to USD: 52.217806271938734]
[2025-08-22 19:08:12,629] INFO [Converting from USD to USD: 52.217806271938734]
52.217806271938734

```

Creating a Logger

The first step to utilizing the `logging` module is to import it into our application. We can do this using the following line of code:

```
import logging
```

Copy to Clipboard

Next, we can obtain our logger object using the `getLogger(name)`

[method](#)

Preview: Docs Loading link description

. This method takes in an optional, single input

[parameter](#)

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis()`: `js function tripleThis(x) { return x * 3; } tripleThis(6);`

called `name` that represents the name of the logger. Calling `getLogger()` with the same `name` value returns the same logger object. If we give no `name`, the root logger is returned.

We recommend using the built-in Python variable `__name__` for this value since it will return the current module's name. This reduces the chance of accidentally reusing a logger name and retrieving the wrong logger object.

Here, we set our logger object to a variable called `logger` below. We will use it later on in this exercise.

```
logger = logging.getLogger(__name__)
```

Copy to Clipboard

Next, we need to inform the logger where we want our logged messages to go. To do this, we will use something called a *handler*. For Python, we will use the `logging` module's `StreamHandler` class. This class handles where our logged messages will output. The `StreamHandler` class takes in an optional, single input called `stream`. If we do not supply this value, the logs will direct to `sys.stderr`. Since we want our output to be direct to the console, we should supply `sys.stdout` as the `stream` value. Note that we must import the `sys` library to reference `stdout` as the `stream` value.

```
import sys
stream_handler = logging.StreamHandler(sys.stdout)
```

Copy to Clipboard

Finally, we can now add our stream handler object to the logger. The `logging` module provides a method called `addHandler(hdlr)` that adds a specific handler to the logger object. The `hdlr` input represents the handler object to add, which in our example is the `StreamHandler` object.

```
logger.addHandler(stream_handler)
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Import the `logging` module and the `sys` module into our `script.py` file.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a new logger by setting it to a variable called `logger`. The name of the logger should be the module's name.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Add a handler to the logger to direct the logged messages to the console. First, create a `StreamHandler` object called `stream_handler`. Then, add `stream_handler` to your `logger` object.

Note that no output will show in the console just yet until we actually log a message.

```
import logging
import sys
```

```
logger = logging.getLogger( name )

stream_handler = logging.StreamHandler(sys.stdout)
logger.addHandler(stream_handler)
print(logger.handlers)
```

Log Levels

When programming or debugging, we may want to focus on certain types of events. Lucky for us, there are defined logging levels that indicate specific levels of severity for a log message. Each logging level is a constant within the `logging` module with an associated numeric value. The higher this numeric value, the higher the severity of the log message. Each logging level is defined as:

Level	Value
NOTSET	0
DEBUG	10
INFO	20
WARNING	30
ERROR	40
CRITICAL	50

We should use `logging.DEBUG` to provide detailed information that is useful for debugging the application. It has a numeric value of 10.

We should use `logging.INFO` for general operations where expected information or output is logged. It has a numeric value of 20.

We should use `logging.WARNING` to alert us to a current or impending, unexpected issue or error. This logging level does mean that the software or application will continue to run despite the warning message. It has a numeric value of 30.

We should use `logging.ERROR` to indicate serious problems that cause functionality within the software or application to break. It has a numeric value of 40.

We should use `logging.CRITICAL` for the most severe of errors and issues. These errors indicate that the software or application may stop running altogether. It has a numeric value of 50.

Finally, the `logging.NOTSET` logging level searches for the first non-NOTSET ancestor logger and inherits its logging level. If there is none, our output threshold will be zero.

Instructions

Checkpoint 1 Enabled

1.

Print the numeric values of each logging level. Remember to import the logging module. Later in this lesson, we will show how we can use these levels to control which logging messages get outputted.

```
import logging
print(logging.DEBUG)
print(logging.INFO)
print(logging.WARNING)
print(logging.ERROR)
print(logging.CRITICAL)
print(logging.NOTSET)
```

Logging Errors and Messages

The `logging` module has several methods that we can use to log messages and errors with an assigned severity level. Those methods are:

```
debug(msg) which logs a message with level DEBUG
info(msg) which logs a message with level INFO
warning(msg) which logs a message with level WARNING
error(msg) which logs a message with level ERROR
critical(msg) which logs a message with level CRITICAL
```

Using our logger object, we can call these methods directly while inputting our desired log message as the

parameter

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis()`: `js function tripleThis(x) { return x * 3; } tripleThis(6);`, like the following:

```
logger.debug("This is a debug message!")
```

Copy to Clipboard

The input parameter `msg` is a

string

Preview: Docs Loading link description

value and can be a custom message or even the message contents of a caught error or exception.

The `logging` module also provides a

method

Preview: Docs Loading link description

`log(level, msg)` that allows us to log a specific log level and message. The `level` parameter represents the numeric log level to use, and the `msg` parameter is the string value that will be logged, same as above. We can use the `log(level, msg)` method like the following:

```
logger.log(logging.INFO, "This is an info message!")
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Inside the `division` function, add an `ERROR` log message if dividing by 0. The message should read:

Attempting to divide by 0!.

Inside the terminal output, type `python3 script.py` and press Enter. Type the following inputs, when prompted:

Dividend = 6

Divisor = 0

View the log statements that show in the terminal output. The newly added `ERROR` log message should appear. To check your answer, click **Check Work** once your program finishes running.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

You can use one of two ways to log an error:

```
logger.error('This is an error message.')
```

Copy to Clipboard

```
logger.info(logging.ERROR, 'This is an error message.')
```

2.

Use the `log(level, msg)` method to log a `CRITICAL` message inside the division function `except` block. The log message should read `No dividend or divisor value entered!`.

Inside the terminal output, execute `python3 script.py` with the following inputs:

Dividend = 6

Divisor = blank (no value entered)

View the log statements that show in the terminal output. The newly added `CRITICAL` log message should appear. To check your answer, click Check Work once your program finishes running.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Add an `if` statement check to see if `result` is equal to `None` at the bottom of `script.py`. If so, log a `WARNING` with the following message: `The result value is None!`.

Inside the terminal output, execute `python3 script.py` with the following inputs:

Dividend = 6

Divisor = 0

View the log statements that show in the terminal output. The newly added `WARNING` log message should appear. To check your answer, click Check Work once your program finishes running.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Locate each `input` statement within the code. After each of these statements, add `INFO` log messages that show what value was entered.

Inside the terminal output, execute `python3 script.py` with the following inputs:

Dividend = 6

Divisor = 3

Execute `python3 script.py` inside the terminal to run the code with the above inputs. View what log statements show (and do not show) in the terminal output. We will see that no `DEBUG` or `INFO` log messages show in our console. Continue to the next exercise to learn why.

INITIAL CODE:

```
import logging
import sys

logger = logging.getLogger(__name__)
stream_handler = logging.StreamHandler(sys.stdout)
logger.addHandler(stream_handler)

def division():
    try:
        dividend = float(input("Enter the dividend: "))
        divisor = float(input("Enter the divisor: "))
    except ValueError:
        return None
    if divisor == 0:
        return None
    else:
        return dividend/divisor
result = division()
```

SOLUTION WITH LOGGING

```
import logging
import sys

logger = logging.getLogger(__name__)
stream_handler = logging.StreamHandler(sys.stdout)
logger.addHandler(stream_handler)

def division():
    logger.debug("Starting Division! ")
    try:
        dividend = float(input("Enter the dividend: "))
        logger.info(dividend)
        divisor = float(input("Enter the divisor: "))
        logger.info(divisor)
    except ValueError:
```

```

    logger.log(logging.CRITICAL, "No dividend or divisor value entered!")
    return
if divisor == 0:
    logger.error("Attempting to divide by 0!")
    return
else:
    return dividend/divisor
result = division()
if result == None:
    logger.warning("The result value is None!")

```

Setting the Log Level

In the previous exercise checkpoint, you may have noticed that when we attempted to add `DEBUG` messages to our calculator application, they didn't appear in the console. What's going on here?

Well, each logger object has a default set of log levels that will output unless a different log level is explicitly set. When we set a log level for a logger, any log message that has this log level and higher will output. The default log level is `WARNING` in Python's `logging` module.

We used the default `WARNING` level in the previous exercise, so only log messages with `WARNING` level or higher were processed and logged to the console.

To change the default log level for a logger, we can use a

[method](#)

Preview: Docs Loading link description

called `setLevel(level)`. The `level`

[parameter](#)

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis()`:
 function `tripleThis(x) { return x * 3; }`
`tripleThis(6);`
 represents the numeric value of the log level to use. If we want the `DEBUG` message to show from our previous exercise checkpoint, we can use the `setLevel(level)` method like below:

```

logger.setLevel(logging.DEBUG)

```

Copy to Clipboard

This will set our log level to `DEBUG` and allow all log messages with a severity of `DEBUG` or higher to log to the console.

There are several cases where setting the log level can be helpful. If we are trying to troubleshoot a problem within our application that is causing it to crash, it would be optimal to see only the `CRITICAL` level log messages and filter out all other messages.

If we are still in a development phase of our application and have to debug often, setting the log level to `DEBUG` is useful for obtaining helpful debugging log messages. Once our application is ready for deployment, we can easily set the log level to `WARNING` to filter out developer-friendly information like `DEBUG` and `INFO` messages.

Instructions

Checkpoint 1 Passed

1.

In our calculator application, after the logger object and handlers, set the log level for `logger` to `DEBUG`.

Click Check Work to check your work.

If we wanted to set our level to `INFO`, we would use the following line of code:

```

logger.setLevel(logging.INFO)

```

Copy to Clipboard

Checkpoint 2 Passed

2.

Uncomment the last three lines in the program. In the terminal, execute `python3 script.py` with the following inputs:

Dividend = 6

Divisor = 3

View the log statements that show in the terminal output. All log messages, including `DEBUG` and `INFO` messages, now show.

INITIAL:

```

import logging
import sys

logger = logging.getLogger(__name__)
stream_handler = logging.StreamHandler(sys.stdout)
logger.addHandler(stream_handler)

def division():
    logger.debug("Starting Division!")
    try:
        dividend = float(input("Enter the dividend: "))
        logger.info(dividend)
        divisor = float(input("Enter the divisor: "))
        logger.info(divisor)
    except SyntaxError:
        logger.log(logging.CRITICAL, "No dividend or divisor value entered!")
        return
    if divisor == 0:
        logger.error("Attempting to divide by 0!")
        return
    else:
        return dividend/divisor
#result = division()
#if result == None:
#    logger.warning("The result value is None!")

```

FINAL - COMPLETED:

```

import logging
import sys

logger = logging.getLogger(__name__)
stream_handler = logging.StreamHandler(sys.stdout)
logger.addHandler(stream_handler)
logger.setLevel(logging.DEBUG)

def division():
    logger.debug("Starting Division!")
    try:
        dividend = float(input("Enter the dividend: " ))
        logger.info(dividend)
        divisor = float(input("Enter the divisor: "))
        logger.info(divisor)
    except SyntaxError:
        logger.log(logging.CRITICAL, "No dividend or divisor value entered!")
        return None
    if divisor == 0:
        logger.error("Attempting to divide by 0!")
        return None
    else:
        return dividend/divisor
result = division()
if result == None:
    logger.warning("The result value is None!")

```

Logging to a File

Up until now, we have used the console for our logged messages. The downside to this is that the logged messages are lost once the execution of the application finishes. There is another helpful stream that we can use to direct our logged messages: files.

Writing logs to a saved file makes our logs persistable after program execution, and it allows the log file to be searched or ingested into other applications like [Splunk](#) or an [ELK Stack](#).

To write logs to a saved file, we can use the `logging` module class `FileHandler(filename)`. The `filename` initialization

[parameter](#)

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis()`: `js function tripleThis(x) { return x * 3; } tripleThis(6);`

is a

[string](#)

Preview: Docs Stores a sequence of indexed characters that can be of any length and is contained within a pair of single or double quotes.

value that represents the filename for the written log file.

Similarly to the `StreamHandler`, we can add the `FileHandler` object to the logger by using the `addHandler(hdlr)`

[method](#)

Preview: Docs Loading link description

as shown below:

```
file_handler = logging.FileHandler("my_program.log")
logger.addHandler(file_handler)
```

Copy to Clipboard

We can practice more with adding a file handler to our logger now.

Instructions

Checkpoint 1 Enabled

1.

Create a `FileHandler` object called `file_handler` that sends logs to a file called **output.log**. Add this handler to the logger object.

Click Check Work to check your work.

Your code should be fairly similar to the example below:

```
file_handler = logging.FileHandler("my_program.log")
logger.addHandler(file_handler)
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Uncomment the last three lines in the program. In the terminal, execute `python3 script.py` with the following inputs:

Dividend = 6

Divisor = 3

View the contents of **output.log** by finding it in the file explorer. You can find the file explorer by clicking on the folder icon next to **script.py**. Once you see the contents of **output.log**, click Check Work.

Note that there is no output to the console when running this code. All logs are inside the **output.log** file.

```
import logging
import sys

logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)

def division():
    logger.debug("Starting Division!")
    try:
        dividend = float(input("Enter the dividend: "))
        logger.info(dividend)
        divisor = float(input("Enter the divisor: "))
        logger.info(divisor)
    except SyntaxError:
        logger.log(logging.CRITICAL, "No dividend or divisor value entered!")
        return
    if divisor == 0:
```

```

        logger.error("Attempting to divide by 0!")
        return
    else:
        return dividend/divisor
# result = division()
# if result == None:
#     logger.warning("The result value is None!")
import logging
import sys

logger = logging.getLogger( name )
logger.setLevel(logging.DEBUG)
file_handler = logging.FileHandler("output.log")
logger.addHandler(file_handler)

def division():
    logger.debug("Starting Division!")
    try:
        dividend = float(input("Enter the dividend: "))
        logger.info(dividend)
        divisor = float(input("Enter the divisor: "))
        logger.info(divisor)
    except SyntaxError:
        logger.log(logging.CRITICAL, "No dividend or divisor value entered!")
        return
    if divisor == 0:
        logger.error("Attempting to divide by 0!")
        return
    else:
        return dividend/divisor
result = division()
if result == None:
    logger.warning("The result value is None!")

```

Logging to Console and File

The `logging` module provides some nifty capabilities to add several handlers to a single logger object. This means that we can easily direct our log messages to multiple streams, such as both a console and a file.

Logging to both a console and file can be useful in cases where we want certain information to go to the console, like debugging information and other log information like errors to write to a file so that we can later search within the file.

We can do this by adding multiple `addHandler(hdlr)` calls for each handler object. An example of this is shown below:

```

logger.addHandler(file_handler)
logger.addHandler(stream_handler)

```

Copy to Clipboard

Let's practice with adding multiple handlers and see it in a real program.

Instructions

Checkpoint 1 Enabled

1.

Create a `FileHandler` object called `file_handler` that sends logs to a file called **calculator.log**. Add this handler to the logger object.

Click Check Work to check your work.

Your code should be fairly similar to the example below:

```

file_handler = logging.FileHandler("my_program.log")
logger.addHandler(file_handler)

```

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a `StreamHandler` object called `stream_handler` that sends logs to the output console. Add this handler to the `logger` object.

Click Check Work to check your work.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Uncomment the last three lines in the program. In the terminal, execute `python3 script.py` with the following inputs:

Dividend = 6

Divisor = 0

View the contents of the **calculator.log** by finding it in the file explorer. You can find the file explorer by clicking on the folder icon next to **script.py**. Verify that the output to the console matches the log messages that are inside the **calculator.log** file. Once you verify this, click Check Work.

```
import logging
import sys

logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)
file_handler = logging.FileHandler("calculator.log")
stream_handler = logging.StreamHandler(sys.stdout)

logger.addHandler(file_handler)
logger.addHandler(stream_handler)

def division():
    logger.debug("Starting Division!")
    try:
        dividend = float(input("Enter the dividend: "))
        logger.info(dividend)
        divisor = float(input("Enter the divisor: "))
        logger.info(divisor)
    except SyntaxError:
        logger.log(logging.CRITICAL, "No dividend or divisor value entered!")
        return
    if divisor == 0:
        logger.error("Attempting to divide by 0!")
        return
    else:
        return dividend/divisor

result = division()
if result == None:
    logger.warning("The result value is None!")
```

Formatting the Logs

A crucial benefit of using the `logging` module is the ability to format log messages. This allows us to include helpful information with each logged message, including:

- timestamps
- the module name
- the line number
- and much more.

If no custom formatting is specified, Python uses the default formatting for all log messages:

```
%(levelname)s: %(name)s: %(message)s
```

Copy to Clipboard

`%(levelname)s` is a [String](#) that represents the log level name
`%(name)s` represents the string of the name of the logger
`%(message)s` is the logged message string.

A colon separates each of these strings. Let's examine the default formatting on a logged message.

```
import logging
import sys

logger = logging.getLogger( name )
stream_handler = logging.StreamHandler(sys.stdout)
logger.addHandler(stream_handler)
logger.warning("This is a warning!")
```

produces the following log message output:

```
WARNING:script:This is a warning!
```

Imagine that we want to include a timestamp and line number in our log. We can create a custom formatter object using the `logging` module's `Formatter` class, which accepts the formatted string as the first input value. We can do the following to add in our timestamp and line number formatting:

```
formatter = logging.Formatter("[% (asctime)s] %(levelname)s: %(name)s: %(lineno)d: %(message)s")
```

To set this formatter object to our logger, we use the `setFormatter(fmt)` [method](#) where the `fmt` [Parameter](#) is the `Formatter` object. Our revised code will resemble this:

```
import logging

logger = logging.getLogger(__name__)
stream_handler = logging.StreamHandler(sys.stdout)
formatter = logging.Formatter("[% (asctime)s] %(levelname)s: %(name)s: %(lineno)d: %(message)s")
stream_handler.setFormatter(formatter)
logger.addHandler(stream_handler)
logger.warning("This is a warning!")
```

which will now produce the following log message output:

```
[2021-11-04 02:58:51,847] WARNING:script:This is a warning!
```

The `setFormatter` method will work on several different types of handler classes, including `StreamHandler` as seen above and `FileHandler`. This means that we can set other formatting options for various handlers. For example, if we choose to log to both console and a file, we can have different formatting for the log messages for each one by applying two separate `Formatter` objects to each handler. This can be useful if we need to, for example, log `DEBUG` messages to console with more information in each log message than what we need in our written log file.

You can find more information about formatting options in the Python documentation [here](#).

Instructions

Checkpoint 1 Enabled

1. In `script.py`, we have preloaded a `FileHandler` object called `file_handler`. Format the object to match the following:

```
[% (asctime)s] {% (levelname)s} %(name)s: #%(lineno)d - %(message)s
```

To do this, create a `Formatter` object called `formatter1`. Then set the format of `file_handler` to the `formatter1` object.

Click Check Work to check your work.

Your code should be similar to the following example:

```
formatter = logging.Formatter("[% (asctime)s]
%(levelname)s: %(name)s: %(lineno)d: %(message)s")
file_handler.setFormatter(formatter)
```

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2. In `script.py`, we have preloaded a `StreamHandler` object called `stream_handler`. Format the object to match the following:

```
[%(asctime)s] %(levelname)s] - %(message)s
```

To do this, create a `Formatter` object called `formatter2`. Then set the format of `stream_handler` to the `formatter2` object.

Click Check Work to check your work.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3. Uncomment the last three lines in the program. Run the program by typing `python3 script.py` in the terminal and clicking Enter.

View and verify that the **formatted.log** file has differently formatted log messages than the console log messages. Once you have done this, click Check Work to move on to the next exercise.

```
import logging
import sys

logger = logging.getLogger( name )
logger.setLevel(logging.DEBUG)
file_handler = logging.FileHandler("formatted.log")
stream_handler = logging.StreamHandler(sys.stdout)
logger.addHandler(file_handler)
logger.addHandler(stream_handler)

formatter1 = logging.Formatter("[% (asctime)s] %(levelname)s] %(name)s: #%(lineno)d -
%(message)s")
formatter2 = logging.Formatter("[% (asctime)s] %(levelname)s] - %(message)s")
file_handler.setFormatter(formatter1)
stream_handler.setFormatter(formatter2)

def division():
    logger.debug("Starting Division!")
    try:
        dividend = float(input("Enter the dividend: "))
        logger.info(dividend)
        divisor = float(input("Enter the divisor: "))
        logger.info(divisor)
    except SyntaxError:
        logger.log(logging.CRITICAL, "No dividend or divisor value entered!")
        return
    if divisor == 0:
        logger.error("Attempting to divide by 0!")
        return
    else:
        return dividend/divisor

result = division()
if result == None:
    logger.warning("The result value is None!")
```

Using basicConfig()

Now that we have covered all of the primary features of the `logging` module, let's cover another [Method](#) that allows for the configuration of some components in one easy step.

The `basicConfig()` method allows for the basic configuration of the logger object by configuring the log level, any handlers, log message formatting options, and more in one line of code. Let's say we want to simplify our calculator code by using the `basicConfig()` method. We can use this method in the following way:

```
logging.basicConfig(filename='calculator.log', level=logging.DEBUG, format='[% (asctime)s]
%(levelname)s] - %(message)s')
```

This code automatically adds a `FileHandler` object that will write to the filename `calculator.log`, set the log level to `DEBUG`, and format all logged messages to have a timestamp, log level, and message information.

Suppose we provide no arguments to the `basicConfig()` method. In that case, a `StreamHandler` object will automatically be created for the logger with default formatting, and it will log to the default `sys.stderr`.

After the configuration is set, the logger object is still retrieved using the `getLogger(name)` method.

If you want to dig more into the `basicConfig()` method, feel free to peruse The Python documentation [here](#), which covers all the arguments that `basicConfig()` supports.

Let's practice using `basicConfig()` to simplify our logger configurations.

Instructions

Checkpoint 1 Enabled

1. Create a logger that writes to a file called `basic_config.log`, sets a log level of `INFO`, and formats the logs with the following format:

```
[%(asctime)s] %(levelname)s - %(message)s
```

using `basicConfig()`.

Run the program by typing `python3 script.py` in the terminal and clicking Enter. Click Check Work once the program is complete.

Your code should be similar to the following example:

```
logging.basicConfig(filename='calculator.log', level=logging.DEBUG, format='%(asctime)s\n%(levelname)s - %(message)s')
```

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2. Execute the calculator code with the following inputs:

Dividend = 0

Divisor = 0

View the `basic_config.log` file and verify that the formatting of the logs matches the formatting set previously. Also, verify that there are no `DEBUG` log messages found within the log file.

Click Check Work once you are ready to move onto the final exercise.

```
import logging
import sys

logger = logging.getLogger(__name__)
logging.basicConfig(filename='basic_config.log', level=logging.INFO, format='%(asctime)s\n%(levelname)s - %(message)s')

def division():
    logger.debug("Starting Division!")
    try:
        dividend = float(input("Enter the dividend: "))
        logger.info(dividend)
        divisor = float(input("Enter the divisor: "))
        logger.info(divisor)
    except SyntaxError:
        logger.log(logging.CRITICAL, "No dividend or divisor value entered!")
        return
    if divisor == 0:
        logger.error("Attempting to divide by 0!")
        return
    else:
        return dividend/divisor

result = division()
if result == None:
    logger.warning("The result value is None!")
```

Review

Congratulations! You completed the Logging lesson! Within this lesson, we learned:

- The benefits of using the `logging` module over print statements
- How to create a logger object
- What log levels are
- How to log errors and messages with a given log level
- How to set the default log level
- How to log to the console, to a file, and to both
- How to format log messages
- How to simplify log configuration using `basicConfig()`

Let's practice these concepts once more!

Instructions

1. If you would like some extra practice, take a lookover **script.py**. Try playing around with the program to get a solid understanding of how it works. See if you can do any of the following:
 - Create a logger named `logger` that will log to a file named `cashier.log` and log all levels INFO and above.
2. Set formatting for the logger to match the following formatted log:

```
2021-11-30 13:59:15,083 - main - INFO - This is your log message!
```
5. Copy to Clipboard
 - Determine the appropriate log levels to log each of the following items and add log statements. to the code.
 - Log the menu options. Log all values entered by the cashier. Log any caught exceptions. Log a message if the returned change is negative.
 - View the output of the `cashier.log` file.

```
print("Welcome to Food in a Jiffy!")

food_options = {
    "H": {
        "name": "Hamburger",
        "price": 2.49
    },
    "C": {
        "name": "Cheeseburger",
        "price": 4.99
    },
    "F": {
        "name": "Fries",
        "price": 2.50
    },
    "D": {
        "name": "Drink",
        "price": 1.99
    },
    "E": {
        "name": "End Order",
        "price": 0.00
    }
}

def menu():
    for food in food_options:
        print("[{key}] {food_name}: ${food_price}".format(key = food, food_name =
food_options[food]["name"], food_price = food_options[food]["price"]))

menu()
cost = 0

choice = str(input("\nSelect a letter corresponding to the menu choice to order: ")).upper()
while choice != "E":
    for key in food_options:
        if choice == key:
            food = food_options[key]
            try:
                num = int(input("\nHow many of the " + food['name'] + " would the customer like to order? "))
            except ValueError:
                print("\nInvalid number entered. Using quantity of 1.")
                num = 1
            food_price = round(food['price'] * num, 2)
```

```
        cost += food_price
        print("\n" + food['name'] + "\t ${food_price}".format(food_price = food_price))
        break

    #if match == 0:
    #    print("Invalid Choice, please select again.")
    choice = str(input("\nSelect a letter corresponding to the menu choice to order: ")).upper()

if choice == "E":
    print("\nThe total for the order is ${cost}".format(cost = round(cost, 2)))
    money = float(input("\nEnter the amount paid by the customer:"))
    if money < cost:
        money = float(input("\nThere is not enough money. Please re-enter the amount paid:"))
    print("The customer's change is ${change}".format(change = round(money - cost, 2)))
```

Introduction to Command Line

A brief overview of the [Learn the Command Line](#) course.

What is the Command Line?

The command line is a text interface for the computer's operating system. You can use it to traverse and edit your computer's filesystem. Through the command line, you can create new files, edit the contents of those files, delete files, and more!

On Mac and Linux systems, we access the command line through something called *Bash*. In this course, we will focus on Bash. Although Windows comes with a different built-in command line, Windows users can also download Bash. We'll go into what that means and how to set it up on your personal device later in the course, but feel free to look ahead [now](#).

What will you learn?

This course will take you through the various functionalities of how to use the command line, starting with navigation, and ending with configuring the environment. At any point, you can access the cheatsheet of commands for this course [here](#).

Navigation

In this module, you will learn how to navigate through a computer's filesystem solely through the command line. You'll learn how to view the contents of your filesystem, move through the different directories (folders), and make new files and directories.

Manipulation

In this module, you will learn how to manipulate the contents of the filesystem with the command line. You'll learn how to move files, delete files and directories, and more.

Redirection

In this module, you will learn how to redirect standard input, standard output, and standard error messages. You'll learn how to display the contents of a file, write to that file, search through directories, and more.

Configuration

In this module, you will learn how to configure your local environment. You'll learn how to set and edit your settings and preferences, assign aliases (shortcuts), how to use nano, and more.

In this final module, we will also sum up [how the command line works for Windows users](#).

Let's get started!

Navigating the File System

`pwd` Print Working Directory

The shell command `pwd` displays the file path from the root directory to the current working directory.

```
$ pwd
/Users/sonny/Downloads
```

Copy to clipboard

`mkdir` Make Directory

The shell command `mkdir` is used to make a new directory in the filesystem according to its argument. If a file path is given, the new directory will be placed at the end. Otherwise, it will create a new directory in the current working directory.

```
$ mkdir new-directory
$ ls
old-directory  new-directory
```

Copy to clipboard

`ls` List

The shell command `ls` is used to list the contents of a directory. If no arguments are given, it will list the contents of the current working directory.

```
$ ls Desktop
resume.pdf
photo.png
```

Copy to clipboard

`cd` Change Directory

The shell command `cd` is used to move throughout the filesystem of a computer. It accepts a variety of arguments:

- Full file paths.
- Names of children of the current directory.
- `..` the parent of the current directory.

```
$ cd some-directory
$ cd ..
```

Copy to clipboard

Filesystem Structure

A computer's filesystem organizes the data stored by a computer, so that it can be easily retrieved by the user.

Files are generally represented in a tree-like structure, in which any parent directory can have any number of children. The root directory is then found at the base of the tree.

`touch` Create New File

The shell command `touch` creates a new file in the current working directory with the name provided.

```
$ touch grocery-list.txt
```

Copy to clipboard

The Command Line

The command line allows a user to navigate the filesystem and run built-in programs or custom scripts. In Unix, the command line interface is called Bash, and the shell prompt is the `$`.

```
$
```

Copy to clipboard

Windows Command Line

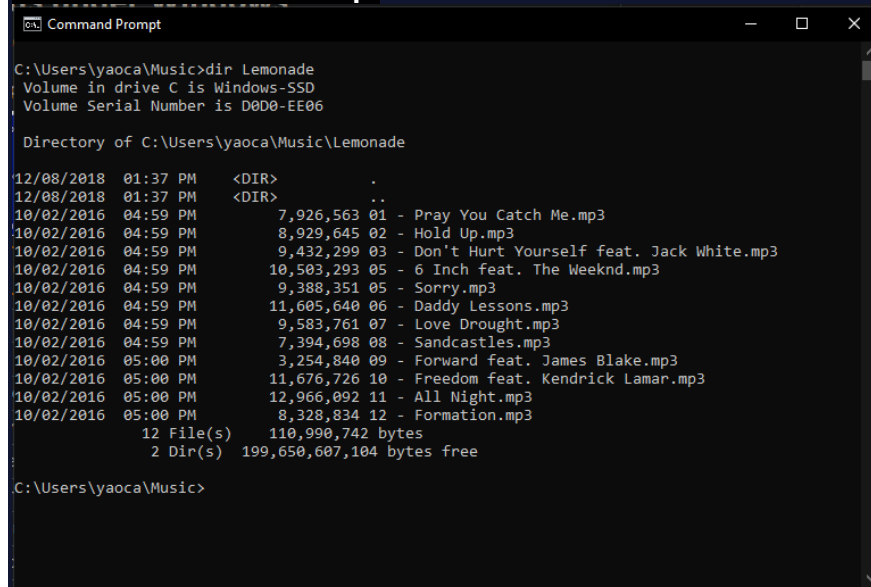
This article is for the Windows user! We will offer more detail about how to apply the knowledge from the Command Line course in the Windows environment.

How do Windows users access the command line?

We will discuss a few options for accessing the command line in Windows. We'll introduce Windows Command Prompt, which is the system default for basic tasks, and Powershell, which is a more advanced tool often used by system administrators. We will also touch on Git Bash, which will allow you to use all the commands we teach in this course on your Windows system.

At the end of the article, we will also provide Windows Command Prompt equivalents of the Bash commands from this course.

Windows Command Prompt



```
Command Prompt

C:\Users\yaoca\Music>dir Lemonade
Volume in drive C is Windows-SSD
Volume Serial Number is D0D0-EE06

Directory of C:\Users\yaoca\Music\Lemonade

12/08/2018  01:37 PM    <DIR>          .
12/08/2018  01:37 PM    <DIR>          ..
10/02/2016  04:59 PM             7,926,563 01 - Pray You Catch Me.mp3
10/02/2016  04:59 PM             8,929,645 02 - Hold Up.mp3
10/02/2016  04:59 PM             9,432,299 03 - Don't Hurt Yourself feat. Jack White.mp3
10/02/2016  04:59 PM             10,503,293 05 - 6 Inch feat. The Weeknd.mp3
10/02/2016  04:59 PM             9,388,351 05 - Sorry.mp3
10/02/2016  04:59 PM             11,605,640 06 - Daddy Lessons.mp3
10/02/2016  04:59 PM             9,583,761 07 - Love Drought.mp3
10/02/2016  04:59 PM             7,394,698 08 - Sandcastles.mp3
10/02/2016  05:00 PM             3,254,840 09 - Forward feat. James Blake.mp3
10/02/2016  05:00 PM             11,676,726 10 - Freedom feat. Kendrick Lamar.mp3
10/02/2016  05:00 PM             12,966,092 11 - All Night.mp3
10/02/2016  05:00 PM             8,328,834 12 - Formation.mp3
               12 File(s)          110,990,742 bytes
               2 Dir(s)    199,650,607,104 bytes free

C:\Users\yaoca\Music>
```

Windows Command Prompt comes preloaded into Windows systems. This tool allows for basic system-related tasks. For example, you can run the following command to shut down the computer in two hours:

```
shutdown -s -t 7200
```

This command prompt is quite limited in its functionality especially when used for software development, but has been kept to maintain backwards compatibility. You can access this tool by bringing up the Start menu and typing `cmd`. We'll share some equivalent commands to Mac/Linux Bash at the end of the article!

Powershell

Powershell is an important tool for system administrators in Windows environments. It was introduced in 2006 to allow users to write much [more powerful scripts](#), and to redirect outputs of commands like one can in Bash.

PowerShell has its own syntax where commands are literal verbs of what's happening. For example, the following command changes the current directory to `C:\Desktop`.

```
Set-Location "C:\Desktop"
```

Windows Terminal

As of summer 2020, Windows has released [Windows Terminal](#), an interface that combines Command Prompt, Powershell, and, if it detects that you have Windows Subsystem for Linux installed, Linux Bash!

Git Bash

Git Bash lets you use Bash commands on your Windows computer. It is our most recommended Windows command line setup for users that need the command line primarily for software development. Many software tools include Bash commands to install, configure, and run them.

Below is a tutorial for setting Git Bash up on Windows:

Bash to Windows Command Prompt Translation Guide

Here's a quick table of Bash commands and their Windows Command Prompt equivalents if you want to try them out on Windows!

Navigation

ls

dir

pwd

chdir

cd

cd

mkdir

md

touch

echo "hello world" > hello.txt

clear

cls

Manipulation

cat

type

type

cp

copy

copy

mv

ren

ren

rm

del

del

>

>

>

Redirection

>

>

>

>

>

>

>

>

>

```
[REDACTED]
```

```
>>
```

```
>>
```

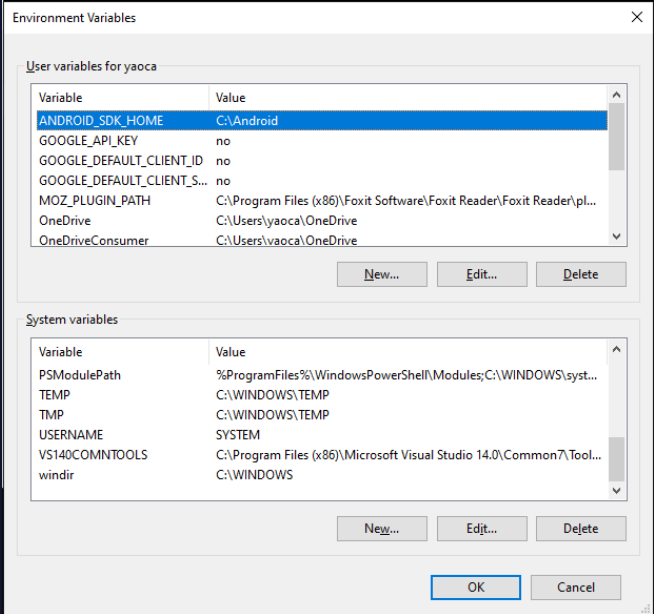
```
[REDACTED]
```

```
grep
```

```
find
```

Environment

In Windows, there are two types of environment variables: System Variables and User Variables. Each variable is in the form of a name and a value. You can view and edit the variables through a GUI by opening the Start menu and search "Edit environment variables" to open up the following window:



To read the environment variables in the command line, these are the Windows commands compared to the Mac/Linux Bash command.

```
[REDACTED]
```

```
[REDACTED]
```

```
[REDACTED]
```

```
[REDACTED]
```

```
env
```

```
set
```

```
[REDACTED]
```

```
export VAR=value
```

```
setx variable value
```

```
[REDACTED]
```

```
echo $VAR
```

```
echo %VAR%
```

Additional Resources

We've covered the basics of Windows Command Prompt, but if you're looking for more, here are some good resources:

[Windows commands cheatsheet](#)

[Microsoft's documentation](#)

What you've learned along the way

<https://www.codecademy.com/learning-outcomes/0e6fd5f0a9e84ddbca19e81fa6357bb>

pwd Print Working Directory

The shell command `pwd` displays the file path from the root directory to the current working directory.

```
$ pwd
/Users/sonny/Downloads
```

mkdir Make Directory

The shell command `mkdir` is used to make a new directory in the filesystem according to its argument. If a file path is given, the new directory will be placed at the end. Otherwise, it will create a new directory in the current working directory.

```
$ mkdir new-directory
$ ls
old-directory  new-directory
```

ls List

The shell command `ls` is used to list the contents of a directory. If no arguments are given, it will list the contents of the current working directory.

```
$ ls Desktop
resume.pdf
photo.png
```

cd Change Directory

The shell command `cd` is used to move throughout the filesystem of a computer. It accepts a variety of arguments:

- Full file paths.
- Names of children of the current directory.
- `..` the parent of the current directory.

```
$ cd some-directory
$ cd ..
```

Filesystem Structure

A computer's filesystem organizes the data stored by a computer, so that it can be easily retrieved by the user.

Files are generally represented in a tree-like structure, in which any parent directory can have any number of children. The root directory is then found at the base of the tree.

touch Create New File

The shell command `touch` creates a new file in the current working directory with the name provided.

```
$ touch grocery-list.txt
```

The Command Line

The command line allows a user to navigate the filesystem and run built-in programs or custom scripts. In Unix, the command line interface is called Bash, and the shell prompt is the `$`.

```
$
```

Helper Commands

Helper commands for the command line include:

- `clear` to clear the terminal
- `tab` to autocomplete the line
- `↑` and `↓` to cycle through previous commands

Command line environment

The *environment* of the command line refers to the settings and preferences of the current user. It enables users to set greetings, `alias` commands, variables, and much more.

Shell Command env

For Unix-based systems like Mac OS and Linux (not Windows), the shell command `env` returns a list of environment variables for the current user.

Alias

The shell command `alias` is used to assign commonly used commands to shortcuts (or aliases). The assigned commonly used command should be wrapped in double quotes.

```
# The following command creates an alias 'pd' for the command 'pwd'
```

```
alias pd="pwd"
```

Environment Variables

Variables that can be used across terminal commands are called *environment variables*. They also hold information about the shell's environment.

Source Bash Profile

All the commands in `~/.bash_profile` are executed with the shell command `source ~/.bash_profile`. So when changes are made to `~/.bash_profile`, run this command to activate the changes in the current session.

history Command

The `history` shell command is used to get a history of commands (also known as "events") that were executed in the current session.

The command also allows us to perform operations on this list of commands that have been executed, such as selecting or manipulating a command in the history.

Export command

The `export` command makes a given variable available to all child sessions initiated from the current session.

```
# This command will make the environment variable USER available
# to all child sessions with the value "Jane Doe".
export USER="Jane Doe"
```

HOME Environment Variable in Unix Systems

`HOME` is an environment variable present in command line environments. It is used to get the path to the current user's home directory. This makes it easy for programs to access the home directory when needed.

```
# To show the path of the home directory use the following command:
```

```
echo $HOME
```

cp Copy

The shell command `cp` is used to copy files or directories.

The basic argument structure is `cp source destination`, where the `source` is the file/directory to copy to the `destination` file/directory.

```
$ cp file1 file1_copy
```

```
$ cp file1 file2 destination folder
```

Command Options

Options can be used to modify the behavior of shell commands. Shell command *options* are commonly represented by a single letter preceded by a `-`. For example, `-l`, `-a`, and `-d` could all be options that follow a shell command.

mv Move

The shell command `mv` is used to move a file into a directory. Use `mv` with the source file as the first argument and the destination directory as the second argument.

```
$ mv index.html website/
```

rm Remove

The shell command `rm` is used to delete files and directories. The `-r` flag deletes a directory and all of its files and directories (`rm -r`).

```
$ rm -r bad selfies
```

ls List Command Options

The shell command `ls` is used to list the contents in a directory. It can be combined with the following command options:

- `-a`: lists all contents, including hidden files and directories.
- `-l`: lists all contents, in long format.
- `-t`: lists all contents, by the time they were last modified.

```
$ ls -a
```

```
$ ls -l
```

```
$ ls -t
```

Append Redirect shell command

The `>>` shell command is used to redirect the standard output of the command on the left and append (add) it to the end of the file on the right.

```
# This command will append "Hello World!" to greetings.txt
```

```
echo "Hello World!" >> greetings.txt
```

Pipe shell command

The `|` command is called a *pipe*. It is used to *pipe*, or transfer, the standard output from the command on its left into the standard input of the command on its right.

```
# First, echo "Hello World" will send Hello World to the standard output.
```

```
# Next, pipe | will transfer the standard output to the next command's standard input.
```

```
# Finally, wc -w will count the number of words from its standard input, which is 2.
```

```
echo "Hello World" | wc -w
```

Redirecting Output

The `>` symbol is used to redirect output by taking the output from the command on the left and passing as input to the file on the right.

```
echo "Hello" > hello.txt
```

cat Display

The shell command `cat` displays the contents of one or more files to the terminal.

```
$ cat poem.txt
```

```
$ cat poem.txt kitties.txt
```

grep Search

The shell command `grep` is used to search files for lines that match a pattern and returns the results. Various options can be specified along with the `grep` command to specify the search.

In the provided example, the lines in the file `names.txt` which contain "sonny" will be returned.

```
grep 'sonny' names.txt
```

Case insensitive search

The shell `grep` command searches files for a particular pattern. The `grep` command with the `-i` option can be used to search files for lines that match a pattern, case insensitive, and returns the results.

grep -R shell command

The shell command `grep` has a `-R` option (`grep -R`) that searches all files in a directory, including its subdirectories, and outputs filenames and lines containing matched results.

Command Line Redirection

On a command line, *redirection* is the process of using the input/output of a file or command to use it as an input for another file. It is similar but different from pipes, as it allows reading/writing from files instead of only commands.

Redirection can be done by using the operators `>` and `>>`.

```
ls > directories_list.txt
```

```
ls >> directories_list.txt
```

Command Line - Introduction

The **command line** is a text interface for your computer. It's a program that takes in commands and passes them on to the computer's operating system to run.

From the command line, you can navigate through files and folders on your computer, just as you would with Finder on Mac OS or Windows Explorer on Windows. The difference is that the command line is fully text-based.

The advantage of using the command line is its power. You can run programs, write scripts to automate common tasks, and combine simple commands to handle difficult tasks. All of these traits come together to make it an important programming tool.

This course is for unix-based systems such as

[Linux](#)

Preview: Docs Loading link description

and Mac OS X. You can also download programs on Windows that will allow you to use the same commands. An appendix of all commands taught in this course is available [here](#).

Instructions

When using the command line, we refer to folders as *directories*. Files and directories on your computer are organized into a *filesystem*.

Move on to the next exercise to find out how the filesystem works.

Filesystem

A filesystem organizes a computer's files and directories into a tree structure:

The first directory in the filesystem is the *root directory*. It is the parent of all other directories and files in the filesystem.

Each parent directory can contain more child directories and files. In the filesystem on the right, **blog/** is the parent of **2014/**, **2015/**, and **hardware.txt**.

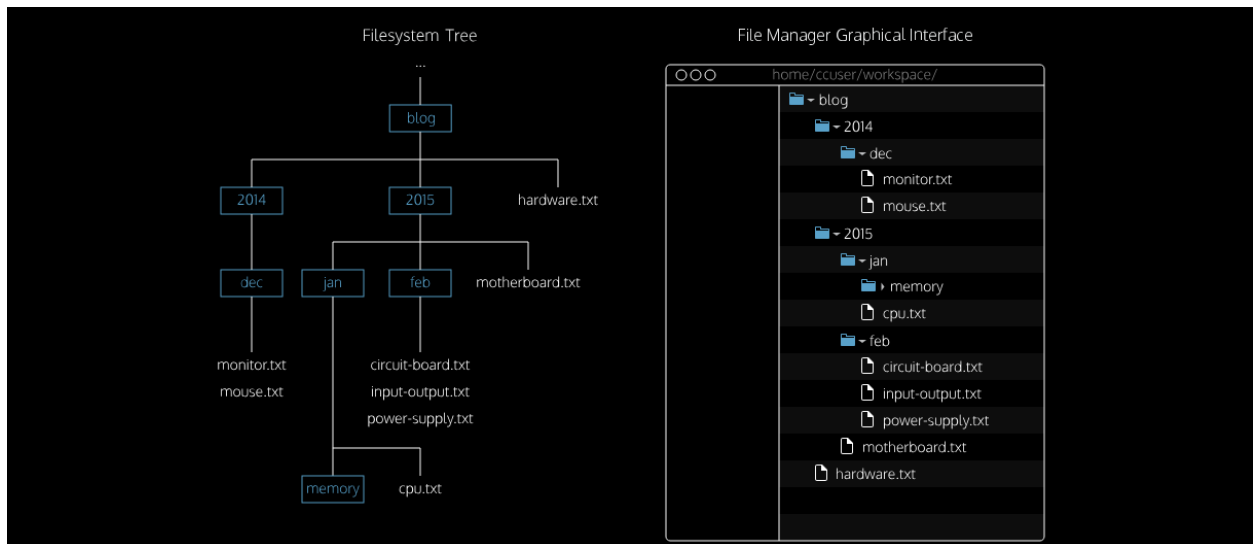
Each directory can contain more files and child directories. The parent-child relationship continues as long as directories and files are nested.

You're probably already familiar with this tree structure - Mac Finder and Windows Explorer represent the filesystem as trees as well.

We are going to use the pictured filesystem for the rest of the lesson. You can reference it at any point [here](#).

Instructions

Move on to the next exercise to start learning about the different commands!



ls

The first command we're going to look at is `ls`. A *command* is a directive to the computer to perform a specific task. When you type `ls`, the command line looks at the directory you are in, and then "**lists**" **all the files and directories inside of it**. Be sure to type the letter `l` as in "List" and not the number 1.

In the [Terminal](#) A terminal is a command-line interface (CLI) that allows users to interact with a computer's operating system by executing text-based commands., the first thing you see is `$`. This is called a *shell prompt*. It appears when the terminal is ready to accept a command.

If we typed:

```
$ ls
```

Copy to Clipboard

the terminal would display our current directory's files and directories:

```
2014 2015 hardware.txt
```

Copy to Clipboard

The directories **2014/**, **2015/**, and the file **hardware.txt** are the contents of the current directory.

Instructions

Checkpoint 1 Enabled

1.

The exercises in this lesson use the **Check Work** button with the terminal. To move through the instructions, perform the tasks and click the **Check Work** button. If you accomplish the task correctly, you can move to the next task or exercise.

To start, after the `$` in the terminal, type:

```
ls
```

Copy to Clipboard

and press enter.

You should see the three items print out below the command.

Click **Check Work** to see how you did.

Be sure to type the letter `l` as in "list" and not the number 1.

After pressing **enter**, the contents of the working directory will appear.

Did you click the **Check Work** button?

pwd

The next command we're going to look at is `pwd`, which stands for "**print working directory**." It outputs the name of the directory you are currently in, called the *working directory*.

Here the working directory is **blog/**. In Codecademy courses, your working directory is usually inside the `/home/ccuser/workspace/` directory.

Together with `ls`, the `pwd` command is useful to show where you are in the filesystem.

Instructions

Checkpoint 1 Enabled

1.

Let's continue with more commands. In the terminal, print the working directory.

Make sure to click the **Check Work** button once you've completed each checkpoint.

Use `pwd` to print the working directory.

Did you click the **Check Work** button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

List all files and directories in the working directory.

Click the "Check

Work" button when you are done.

cd

Our next command is `cd`, which stands for "change directory." Just as you would click on a folder in Windows Explorer or Finder, `cd` switches you into the directory you specify. In other words, `cd` changes the working directory.

Let's say the directory we change into is **2015/**:

```
$ cd 2015
```

Copy to Clipboard

When a file, directory, or program is passed into a command, it is called an

[argument](#)

Preview: Docs An argument is the actual value of a variable passed into a function.

. Here the **2015/** directory is an argument for the `cd` command.

The `cd` command takes a directory name as an argument and switches into that directory.

It is also important to move up one directory. For this, we use:

```
$ cd ..
```

Copy to Clipboard

The argument `..` stands for the directory above the current working directory. Assuming we are in `/home/ccuser/workspace/blog/2015/`:

```
$ pwd
/home/ccuser/workspace/blog/2015
$ cd ..
$ pwd
/home/ccuser/workspace/blog
```

Copy to Clipboard

The above example uses the command `cd ..` to navigate up to the `/home/ccuser/workspace/blog/` directory.

Instructions

Checkpoint 1 Enabled

1.

Our current working directory is `/home/ccuser/workspace/blog/`. Change into the **2015/** directory.

Confirm you are in the **2015/** directory using `pwd`.

Make sure to click the Check Work button once you've completed each checkpoint.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now move back up to the **blog/** directory using the `..` notation.

Confirm your location again with `pwd` and click the Check Work button?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Let's move further into the filesystem. We are currently in the **blog/** directory. From here we want to move into the **2015/** directory, then into a directory named **jan/** and lastly, a directory named **memory/**. (You can reference the filesystem for this lesson [here](#).)

Move down the filesystem into the **memory/** directory and print the working directory again to see the new location.

cd II

In the previous exercise, we used `cd` multiple times to go from the **blog/** directory to the **memory/** directory. You can reference the file system [here](#).

To move across multiple directories with a single command, we can provide `cd` a relative path to the **memory/** directory as an

[argument](#)

Preview: Docs Loading link description

From the **blog/** directory use:

```
$ cd 2015/jan/memory
```

Copy to Clipboard

The relative path is a forward slash (/) separated list of all the directories leading to the goal directory. In the above example, we navigate directly to **memory/** from **blog/** using `cd` with the relative path `2015/jan/memory`.

We can also move up multiple directories using the `..` argument. To go from **memory** up 2 directories to **2015**, we use `../..` such as:

```
$ pwd
/home/ccuser/workspace/blog/2015/jan/memory
$ cd ../..
$ pwd
/home/ccuser/workspace/blog/2015
```

Copy to Clipboard

The relative path `../..` can be read: the directory above and the directory above that.

Lastly, we can move to an adjacent directory using `..` and then a directory name. If we are in the **2015/** directory and we want to go into the **2014/** directory, we go up one directory then into the **2014/** directory:

```
$ ls
2014 2015 hardware.txt
$ cd 2015
$ pwd
/home/ccuser/workspace/blog/2015
$ cd ../2014
$ pwd
/home/ccuser/workspace/blog/2014
```

Copy to Clipboard

The relative path is the directory above **2015/** (which is **blog/**) and then the **2014/**.

Instructions

Checkpoint 1 Enabled

1.

For this exercise, we want to start in the **blog/2015/** directory. Check your location and navigate to this directory. Remember, you can always use `cd` with the absolute path, `/home/ccuser/workspace/blog/2015/`.

Once in the **blog/2015/** directory, change the directory to the **feb/** directory using a single command.

When in the **feb/** directory, check the working directory.

Make sure to click the Check Work button once you've completed each checkpoint.

To get to the **feb/** directory from **blog/2015/** use:

```
cd feb
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now we're in **2015/feb/**, but what if we want to move to the adjacent directory **2015/jan/**? (You can reference the filesystem for this lesson [here](#).)

Using a single `cd` command, navigate from **2015/feb/** to **2015/jan/** and confirm your working directory.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Navigate back to **/home/ccuser/workspace/blog** using a single `cd` command and then use `pwd` to confirm your working directory.

Click the Check Work button when you are done.

mkdir

Now that we can traverse the existing filesystem, let's try editing it by making directories (folders) through the command line. The command for that is `mkdir`:

```
$ mkdir media
```

Copy to Clipboard

The `mkdir` command stands for "make directory". It takes in a directory name as an [argument](#)

Preview: Docs Loading link description

and then creates a new directory in the current working directory.

Here we used `mkdir` to create a new directory named **media/** inside our working directory.

Instructions

Checkpoint 1 Enabled

1.

Navigate to the **/home/ccuser/workspace/blog/2014/dec/** directory.

Then list all files and directories in the working directory to see what's currently in there.

Make sure to click the Check Work button once you've completed each checkpoint.

You can use `cd 2014/dec` to move into the **2014/dec** directory. Remember to use `ls` to list the contents of your current directory.

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a new directory named **media**. If you list the contents of the working directory again, you should see your new directory.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Within our new **media/** directory, create another directory named **tv/**.

You may use either of the two approaches:

use `mkdir` from our current position by using the relative path `media/tv` as an argument

Or

`cd` into **media/** and then use `mkdir` (without a relative path)

Click the Check Work button when you are done.

touch

Now we know how to create directories through the command line, but how do we create new files?

We can do this using the command `touch`:

```
$ touch keyboard.txt
```

Copy to Clipboard

The `touch` command creates a new file inside the working directory. It takes in a filename as an

[argument](#)

Preview: Docs Loading link description
and then creates an empty file with that name in the current working directory.
Here we used `touch` to create a new file named **keyboard.txt**.

Instructions

Checkpoint 1 Enabled

1.

Make sure you are in the **/home/ccuser/workspace/blog/** directory. List your working directory's current contents.

Make sure to click the Check Work button once you've completed each checkpoint.

To navigate into the blog directory and list the contents run these commands:

```
cd /home/ccuser/workspace/blog
ls
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a new file named **keyboard.txt** inside the working directory. If you list its contents again, you should now see your file listed.

Helper Commands

Now that we've covered the basics of navigating your filesystem from the command line, let's look at some helpful commands that will make using it easier!

`tab` can be used to autocomplete your command. When you are typing the name of an existing file or directory, you can use `tab` to finish the rest of the name.

The up and down arrows (`↑` and `↓`) can be used to cycle through your previous commands. `↑` will take you up through your most recent commands, and `↓` will take you back through to the most recent one.

`clear` is used to clear your terminal, which is useful when it's full of previous commands and outputs. It doesn't change or undo your previous commands, it just clears them from view. As mentioned above, you can use the up and down arrow keys to review your command history anytime.

Instructions

1. Experiment with the helper commands! Some things you could try are:

When your working directory is **home/ccuser/workspace/blog**, type `cd 2` and then use `tab` - it should autocomplete upto `cd 201`. This is because both possible directories (**2014** and **2015**) start with **201**.

Use `ls` to see what contents are in your working directory. Then use `cd` with the first letter of one of the files or directories and use `tab` to autocomplete.

Use the up and down arrows to cycle through your previous commands.

Use `clear` to clear your previous commands and output from the terminal.

Review

Congratulations! You've learned five commands commonly used to navigate the filesystem from the command line. What can we generalize so far?

The *command line* is a text interface for the computer's operating system. To access the command line, we use the terminal.

A *filesystem* organizes a computer's files and directories into a tree structure. It starts with the *root directory*. Each parent directory can contain more child directories and files.

From the command line, you can navigate through files and folders on your computer:

`pwd` outputs the name of the current working directory.

`ls` lists all files and directories in the working directory.

`cd` switches you into the directory you specify.

`mkdir` creates a new directory in the working directory.

`touch` creates a new file inside the working directory.

You can use helper commands to make navigation easier:

`clear` clears the terminal

`tab` autocompletes the name of a file or directory

`↑` and `↓` allow you to cycle through previous commands

Command Line Interface Setup

Navigate your operating system like a professional programmer

Setting Up Your Command Line

The command line is a powerful tool used by developers to find, create, and manipulate files and folders. This short tutorial will walk you through the steps for setting up the command line application on your computer.

Command Line Interfaces (CLIs) come in many forms. The CLI we'll use is called Bash

What is Bash?

Bash, or the `__B__ourne-__A__gain __SH__ell`, is a CLI that was created in 1989 by Brian Fox as a free software replacement for the Bourne Shell. A shell is a specific kind of CLI. Bash is "[open source](#)", meaning anyone can read the code and suggest changes. Since its beginning, it has been supported by a large community of engineers who have worked to make it an incredible tool. Bash is the default shell for Linux and Mac up through macOS 10.14 (Mojave). For these reasons, Bash is the most used and widely distributed shell. If you want to learn more about Bash, [this Wikipedia article](#) is a good place to start. Better yet, try it yourself with one of our [Bash courses](#).

Bash Setup for Mac and Windows

Mac users:

As mentioned before, Bash is the default shell on Linux and MacOS up through version 10.14 (Mojave), so good news, you don't have to install anything! MacOS version 10.15 (Catalina) and higher uses a similar, but slightly different default shell called Z shell or Zsh. In most ways Zsh is an exact replacement for Bash, so there is no need to switch over or install Bash instead.

To access Bash in MacOS, you can use an application called **Terminal**.

Continue to the "Try it Out!" section below for some simple first steps with your new tool.

Windows users:

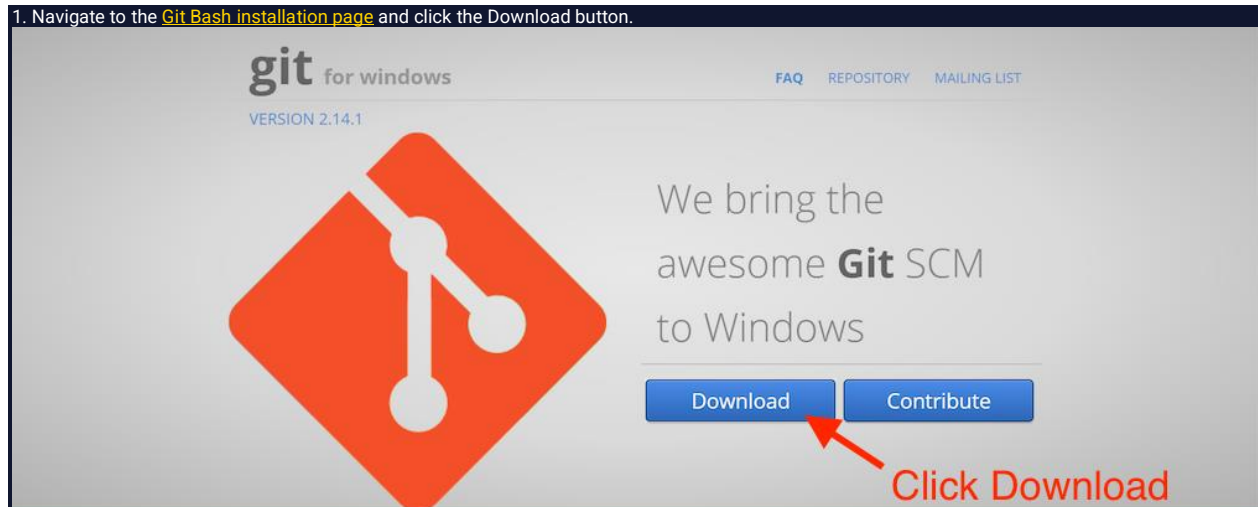
Windows has a different CLI, **Command Prompt**. While this has many of the same features as Bash, Bash is much more popular. Because of the strength of the open source community and the tools it provides, mastering Bash is a better investment than mastering the Command Prompt.

To use Bash on a Windows computer, we will download and install a program called **Git Bash**. Git Bash allows us to easily access Bash as well as another tool we'll be using later called Git, inside the Windows environment. (If you haven't used Git before, try out [Learn Git & GitHub course](#).)

You can either watch the following video, or read the rest of this article.

How to install Git Bash:

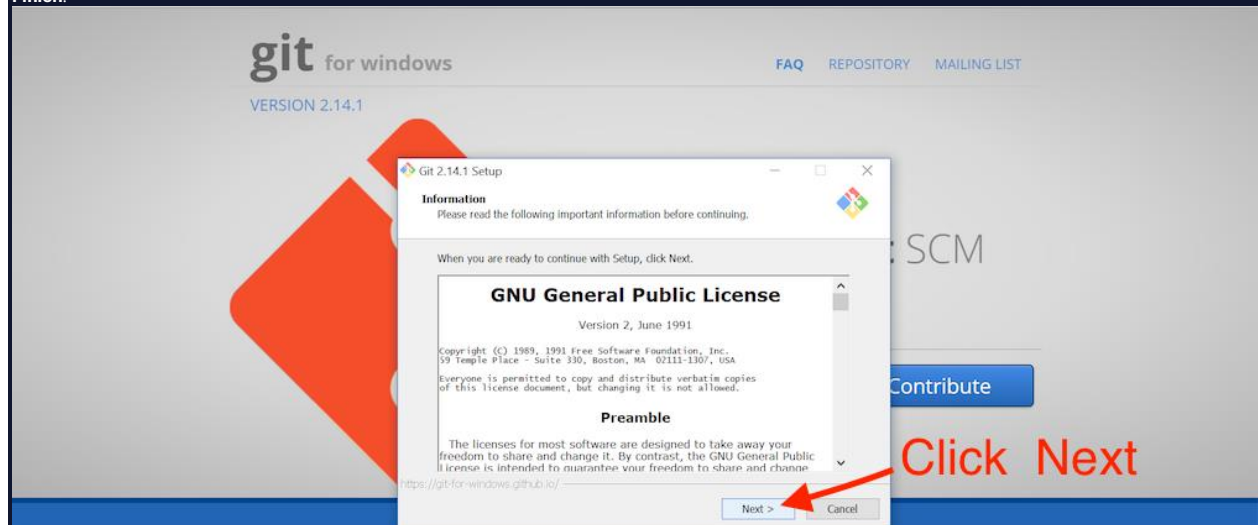
1. Navigate to the [Git Bash installation page](#) and click the Download button.



Tools & Features

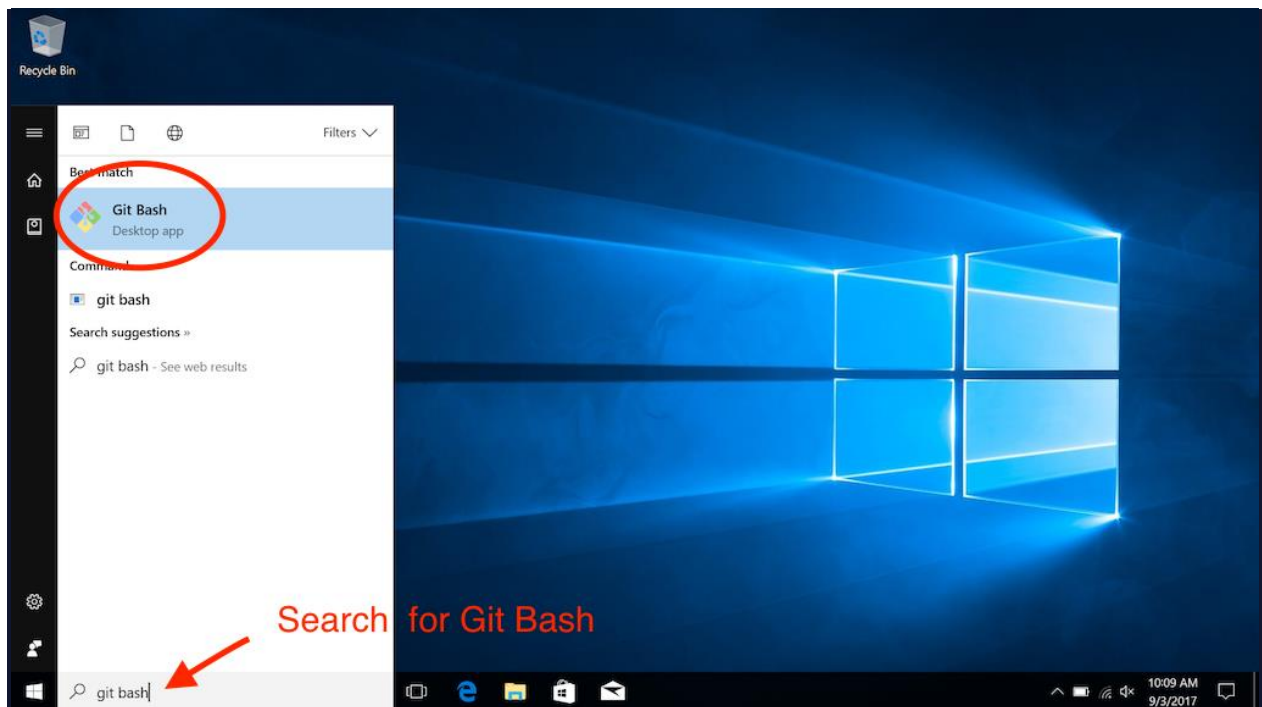
2. Once Git Bash is downloaded, run the downloaded .exe file and allow the application to make changes to your PC. You will get a prompt that says "Do you want to allow this app to make changes to your device?" Click **Yes**.

3. To keep things simple, we will use the default settings for everything in this installation, so all you need to do now is keep clicking **Next**, and finally **Finish**.



Tools & Features

4. Open the Start menu by clicking on the Windows icon and typing "Git Bash" into the search bar. The icon for Git Bash and the words "Git Bash Desktop App" will appear. Click on the icon or the words "Git Bash Desktop App" to open Git Bash.



5. A new window will open. This is the Git Bash CLI where we will run Bash commands. Whenever a new window of the Git Bash app is opened, you will always be placed in the same directory, your **home directory**. The home directory is represented by the tilde sign, `~`, in the CLI after `MINGW64`. The tilde is another way to say `/c/Users/username` in Git Bash or `C:\home\Users\username` in Windows' Command Prompt. The absolute path of your current working directory, how you got from the root directory to the directory you are currently in, will always be noted at the top of the window:



Git Bash works by giving you a CLI that acts like a Bash CLI. That means you can now work with your files and folders using Bash commands instead of Windows commands.

Congratulations, you now have Bash installed on your computer, ready to use! In case you wanted to try out the built-in Windows command line for basic system tasks, we talk about it in [this article](#).

Now that you have your Command Line Interface open on your desktop, you are ready to use it. Go ahead and try some of the commands on your personal computer. Here are some good commands for practice:

```

1. ls to list the contents of the current directory. It may look something like this:
$ ls
Applications          Pictures
Codecademy            Public
Desktop              Downloads
Documents             Library

2. mkdir test to make a new directory named test. Now, when you type ls you should see a folder called test:
$ ls
Applications          Pictures
Codecademy            Public
Desktop              Downloads
Documents             Library
test

3. cd test to navigate into the new directory. You won't see an output when you do this.
4. echo "Hello Command Line" >> hello_cli.txt to create a new file named hello_cli.txt and add Hello Command Line to that file.
When you type ls, you should see the following:
$ ls
hello_cli.txt

5. cat hello_cli.txt to print the contents of the hello_cli.txt file to the terminal. You should see something like:
$ cat hello_cli.txt
Hello Command Line

Good job! You're ready to explore the world of the Command Line Interface on your own computer.

```

Manipulation BASH

Introduction

So far we've used the command line to navigate the filesystem.

We can view directories and files with more detail. We can also use the command line to copy, move, and remove files and directories. This lesson will explore these abilities!

You can reference the filesystem for this lesson [here](#). The file structure will stay mostly the same throughout this lesson.

Instructions

Checkpoint 1 Passed

1.

The exercises in this lesson use the `Check Work` button with the terminal. To move through the instructions, perform the tasks and click the `Check Work` button. If you accomplish the task correctly, you can move to the next task or exercise.

To find out where we are in the file system right now, in the terminal, type

```
pwd
```

Copy to Clipboard

You should be in the `/home/ccuser/workspace/movies` directory. If you are not in that directory, use the `cd` to navigate back in.

Make sure to click the `Check Work` button once you've completed each checkpoint.

To navigate to the correct directory use:

```
cd /home/ccuser/workspace/movies
```

Copy to Clipboard

then run:

```
pwd
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 2 Passed

2.

Type the the following command into the terminal:

```
ls
```

Copy to Clipboard

This will list the files and directories contained in the **movies/** directory.

Click the Check Work button when you are done.

To navigate to the correct directory use:

```
cd /home/ccuser/workspace/movies
```

Copy to Clipboard

then run:

```
ls
```

Copy to Clipboard

Did you click the Check Work button?

ls, revisited

The `ls` command lists all files and directories in the working directory. We can use `ls` as is, or attach an *option*. Options modify the behavior of commands.

For example:

```
ls -a
```

Copy to Clipboard

The command above displays the contents of the working directory in more detail. This command displays all the files and directories, including those starting with a dot (`.`). Files starting with a dot are normally hidden, and don't appear when using `ls` alone.

The `-a` is called an *option*. Options modify the behavior of commands.

In addition to `-a`, the `ls` command has several more options. Here are three common ones:

- `-a` - lists all contents, including hidden files and directories
- `-l` - (a lowercase "L") lists all contents of a directory in long format, as well as the file permissions
- `-t` - orders files and directories by the time they were last modified.

Let's practice using these options below.

Instructions

Checkpoint 1 Enabled

1.

In the terminal, type:

```
ls -a
```

Copy to Clipboard

Do you see the differences between the outputs of both `ls` and `ls -a`?

Make sure to click the Check Work button once you've completed each checkpoint.

Run the command:

```
ls -a
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Then type

```
ls -t
```

Copy to Clipboard

Do you see the differences between the outputs of both `ls -a` and `ls -t`?

Click the **Check Work** button when you are done.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

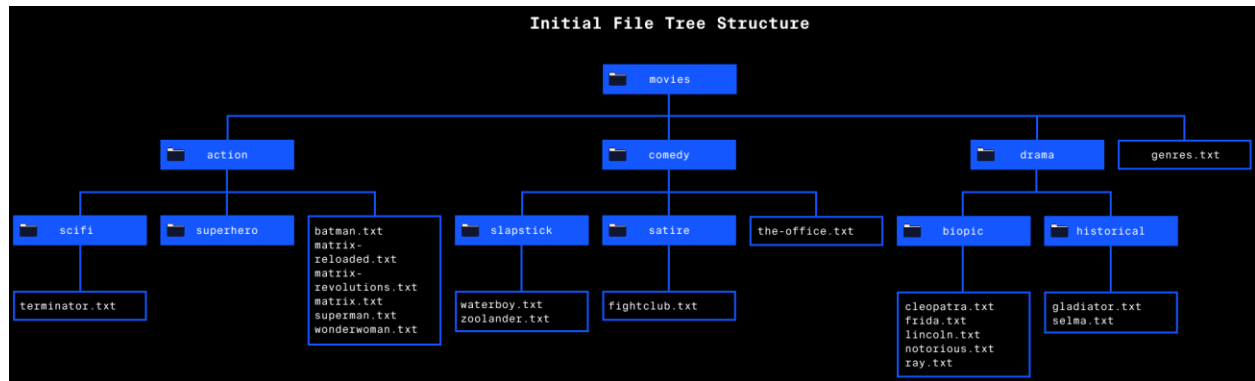
In the terminal, type

```
ls -l
```

Copy to Clipboard

Note that the last character is a lowercase “l”.

Click **Check Work** then “Next” to find out what these columns mean.



Is & Combining Options

```
$ ls -l
drwxr-xr-x 5 cc eng 4096 Jun 24 16:51 action
drwxr-xr-x 4 cc eng 4096 Jun 24 16:51 comedy
drwxr-xr-x 6 cc eng 4096 Jun 24 16:51 drama
-rw-r--r-- 1 cc eng 0 Jun 24 16:51 genres.txt
```

Copy to Clipboard

The `-l` (lowercase “l”) option lists files and directories as a table. Here there are four rows, with seven columns separated by spaces. Here’s what each column means:

Access rights. These indicate the read, write, and execute permissions on the file or directory allowed to the owner, the group, and all users. You can read more about [file permissions](#).

Number of hard links. This represents the number of hard links to the file or directory. This number includes the parent directory link (..) and current directory link (..).

The username of the file’s owner. Here the username is `cc`.

The name of the group that owns the file. Here the group name is `eng`.

The size of the file in bytes.

The date & time that the file was last modified.

The name of the file or directory.

In addition to using each option separately, like `ls -a` or `ls -l`, multiple options can be used together, like `ls -alt`. We can group letter options for other commands in the same way.

`ls -alt` lists all contents, including hidden files and directories, in long format, ordered by the date and time they were last modified.

Instructions

Checkpoint 1 Enabled

1.

Let’s navigate to the **comedy/** directory and confirm you are in the correct working directory using `pwd`.

Make sure to click the **Check Work** button once you’ve completed each checkpoint.

The **comedy/** directory is inside the current directory `/home/ccuser/workspace/movies`, so you can use a single command:

```
cd comedy
```

Copy to Clipboard
to get there.

If you want to specify the entire path, you could use:

```
cd /home/ccuser/workspace/movies/comedy
```

Copy to Clipboard
Then run:

```
pwd
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Then type

```
ls -alt
```

Copy to Clipboard

To view all files and directories in **comedy/**, including the hidden ones, in a long table format, sorted by time last modified.

Manipulation

cat

How do we view the contents of an individual file in our terminal? We can use `cat`.

The `cat` command outputs the contents of a specified file. For example, running the following inside the **movies/** directory:

```
cat genres.txt
```

Copy to Clipboard

will output all the text from **genres.txt**.

This is a useful command for peeking at files without opening them, and confirming the result of other commands that change the contents of files.

We can also get output using paths to a specified file. To output the contents of **batman.txt** within the **action** directory, we can run:

```
cat action/batman.txt
```

Copy to Clipboard

This will output the contents of **batman.txt** within the **action/** directory.

Try it out below!

Instructions

Checkpoint 1 Enabled

1.

Start by moving into the **action/** directory. Once there, output the contents of **superman.txt**.

Make sure to click the Check Work button once you've completed each checkpoint.

To navigate to the correct directory use:

```
cd /home/ccuser/workspace/movies/action
```

Copy to Clipboard
then run:

```
cat superman.txt
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

While inside the **action/** directory, output the contents of **terminator.txt**, which is inside the **scifi/** directory.

Manipulation

cp Part I

Let's move on to copying, moving, and removing files and directories from the command line. The `cp` command copies files or directories. Below, we copy the contents of a source file into a destination file:

```
cp source.txt destination.txt
```

Copy to Clipboard

For example if we are in the **scifi/** directory and want to make a backup of **terminator.txt** file we would run:

```
cp terminator.txt terminator.bak
```

Copy to Clipboard

This will make a copy of **terminator.txt** and save it as **terminator.bak**.

NOTE: The **".bak"** file extension is commonly used to notate a file as a backup of a file with the same name.

We could also copy a file to a destination directory:

```
cp source.txt directory
```

Copy to Clipboard

If we are in the **comedy/** directory and wanted to put a copy of **the-office.txt** in the **slapstick/** directory we would run:

```
cp the-office.txt slapstick
```

Copy to Clipboard

The above example will put an exact copy named **the-office.txt** in the **slapstick/** directory.

If we wanted to rename the file as well, we would include the name in the path of the second argument:

```
cp the-office.txt slapstick/the-office-us.txt
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

For this exercise, we need to start in the **drama/biopic** directory. Use `pwd` to check where you are. If you aren't in the **drama/biopic** directory, you can use the command `cd /home/ccuser/workspace/movies/drama/biopic` to quickly get there.

Then, list all files and directories in the working directory using `ls` (without any options).

Make sure to click the Check Work button once you've completed each checkpoint.

Use `cd` and the absolute path to change into the correct directory:

```
cd /home/ccuser/workspace/movies/drama/biopic
```

Copy to Clipboard

Then list the contents:

```
ls
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

While in the **biopic/** directory, copy the file **frida.txt** to a file named **frida.bak**.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use `cat` on **frida.bak** to confirm that the information about Frida Kahlo has been copied into the **frida.bak** file.

Make sure to click the Check Work button once you've completed the task.

To clear the terminal of the text after its output, use the `clear` command.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Great job! You just copied the contents of a file using the command line!

Now, let's navigate one level up to the **drama/** directory, then list all files and directories in the working directory.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

In a single command, make a copy of **cleopatra.txt** to go into the **historical/** directory. Note that **cleopatra.txt** is in the **biopic/** directory.

Pay attention to your current position, which you can always access using the command `pwd`.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Now move into the **historical/** directory and list all the contents. You should see a new copy of **cleopatra.txt** in this directory.

Make sure to click the Check Work button once you've completed the task.

cp Part II

To copy multiple files into a directory, use `cp` with a list of source files as the first arguments, and the destination directory as the last

[argument](#)

Preview: Docs Loading link description

. Here, we copy the files **file1.txt** and **file2.txt** into the same directory.

```
cp file1.txt file2.txt my_directory/
```

Copy to Clipboard

Note that you can access the layout of the filesystem [here](#).

Instructions

Checkpoint 1 Enabled

1.

For this exercise, we need to start in the **drama/** directory. Use `pwd` to check where you are. If you aren't in the **drama/** directory, you can use the command `cd /home/ccuser/workspace/movies/drama` to quickly get there.

Once in the **drama/** directory, list the contents using `ls` with no options.

Check your progress with Check Work.

Use the `cd /home/ccuser/workspace/movies/drama` command to navigate directly into **drama/**.

Use the `ls` command to list the contents.

You can reference the filesystem for this lesson [here](#).

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

From the **drama/** directory, copy **biopic/ray.txt** and **biopic/notorious.txt** into the **historical/** directory.

Once you copy the files, click the Check Work button.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Navigate to the **historical/** directory and use `ls` without options to look at the contents.

Manipulation

Wildcards

In addition to using exact filenames as arguments, we can use special characters like `*` to select groups of files. These special characters are called *wildcards*.

For example, here we use `cp` to copy all files in the current working directory into another directory.

```
cp * my_directory/
```

Copy to Clipboard

To move just the `.txt` files you can use a wild card with a suffix:

```
cp *.txt my_directory/
```

Copy to Clipboard

In addition to suffixes, prefix text can be used with wildcards. Using `w*.txt` selects all files in the working directory starting with "w" (prefix) and ending with ".txt" (suffix), and copies them to **my_directory/**.

```
cp w*.txt my_directory/
```

Copy to Clipboard

Let's try out the wildcard symbol in this filesystem!

Note that you can access the layout of the filesystem [here](#).

Instructions

Checkpoint 1 Enabled

1.

For this exercise, we need to start in the **comedy/slapstick/** directory. Once there, list the contents of the directory using `ls` (without any options).

You can use the command `cd /home/ccuser/workspace/movies/comedy/slapstick` to quickly get there.

Click the Check Work button when done.

Use `cd /home/ccuser/workspace/movies/comedy/slapstick` to navigate directly into **comedy/slapstick/**.

Use the `ls` command to list the contents.

You can reference the filesystem for this lesson [here](#).

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Using a wildcard, copy all the files in the **slapstick/** directory to the **comedy/** directory.

Check your progress with Check Work.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Move up to the **comedy/** directory and list its contents.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Using `cp`, the letter 'z' and a wildcard, copy just **zoolander.txt** into the **satire/**

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Switch to the **satire/** directory and list its contents (don't use any options with `ls`).

Check your progress with Check Work.

Manipulation

mv

The `mv` command moves files. It's similar to `cp` in its usage, except `mv` moves a file without making a copy.

To move a file into a directory, use `mv` with the source file as the first

[argument](#)

Preview: Docs Loading link description

and the destination directory as the second argument. Here we move **my_file.txt** into **my_directory/**.

```
mv my_file.txt my_directory/
```

Copy to Clipboard

To move multiple files into a directory, use `mv` with a list of source files as the first arguments, and the destination directory as the last argument. Here, we move **my_file_1.txt** and **my_file_2.txt** into **my_directory/**.

```
mv my_file_1.txt my_file_2.txt my_directory/
```

Copy to Clipboard

`mv` can also be used to rename a file.

```
mv file_original.txt file_renamed.txt
```

Copy to Clipboard

By moving **file_original.txt** to **file_renamed.txt**, we rename the file as **file_renamed.txt** within the working directory.

Note that you can access the layout of the filesystem [here](#).

Instructions

Checkpoint 1 Enabled

1.

For this exercise, we need to start in the **action/** directory. Use `pwd` to check where you are. If you aren't in the **action/** directory, you can use the command `cd /home/ccuser/workspace/movies/action` to quickly get there.

After moving into the **action/** directory, list the contents using `ls` with no options.

Check your progress with Check Work.

Use the `cd /home/ccuser/workspace/movies/action` command to navigate directly into **action/**.

Use the `ls` command to list the contents.

You can reference the filesystem for this lesson [here](#).

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

While in the **action/** directory, move the file **superman.txt** into the directory **superhero/**.

Feel free to list the contents of **action/** to ensure that **superman.txt** is gone.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now try and move multiple files at the same time!

Using one command, move both **batman.txt** and **wonderwoman.txt** into the **superhero/** directory.

Feel free to list the contents of **action/** to ensure that **batman.txt** and **wonderwoman.txt** are gone.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Lastly, move into the **superhero/** directory and list the contents using `ls` with no options.

Check your progress with Check Work.

Manipulation

rm

The `rm` command deletes files and directories. Here we remove the file **unwanted_file.txt** from the filesystem.

```
rm unwanted_file.txt
```

Copy to Clipboard

The `-r` (recursive) option is used to delete a directory and all of its child directories.

```
rm -r unwanted_directory
```

Copy to Clipboard

Be careful when you use `rm`! It deletes files and directories permanently. There isn't an undelete command, so once you delete a file or directory with `rm`, it's gone.

Note that you can access the layout of the filesystem [here](#).

Instructions

Checkpoint 1 Enabled

1.

For this exercise, we need to start in the **comedy/slapstick/** directory. Use `pwd` to check where you are. If you aren't in the **comedy/slapstick** directory, you can use the command `cd /home/ccuser/workspace/movies/comedy/slapstick` to quickly get there.

Now that you are in the correct directory, list the contents using `ls` without any options.

Check your progress with Check Work.

Use:

```
cd comedy/slapstick
```

Copy to Clipboard

or

```
cd /home/ccuser/workspace/movies/comedy/slapstick
```

Copy to Clipboard

to navigate to the correct directory.

Then run:

```
ls
```

Copy to Clipboard

to list the contents of the **slapstick/** directory.

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Inside the current directory, remove **waterboy.txt**. List the contents to see that the file has been removed.

Check your progress with Check Work.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Navigate up one directory from **comedy/slapstick/** to **comedy/**. List the contents using `ls` without any options.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

In the **comedy/** directory, remove the **slapstick/** directory and its contents. List the contents of **comedy/** to see that the directory has been removed.

Check your progress with Check Work.

Manipulation

Review

Congratulations! You learned how to use the command line to view and manipulate the filesystem. What can we generalize so far?

Options modify the behavior of commands:

`ls -a` lists all contents of a directory, including hidden files and directories

`ls -l` lists all contents in long format

`ls -t` orders files and directories by the time they were last modified

Multiple options can be used together, like `ls -alt`

From the command line, you can also copy, move, and remove files and directories:

`cp` copies files

`mv` moves and renames files

`rm` removes files

`rm -r` removes directories

Wildcards are useful for selecting groups of files and directories

Instructions

Feel free to play around with the commands you've learned in this lesson in the terminal to the right!

The more you practice with the command line, the more it will start to feel like home.

Redirection

Introduction

Up until now, we have run commands in the command line and received a stream of output in the

[terminal](#)

Preview: Docs Loading link description

. In this lesson, we'll focus on input and output (I/O) redirection.

Through *redirection* you can direct the input and output of a command to and from other files and programs, and chain commands together in a pipeline. Let's try it out.

You can reference the filesystem for this lesson [here](#).

What happens when you type this command?

```
$ echo "Hello"
Hello
```

Copy to Clipboard

The `echo` command accepts the

[string](#)

Preview: Docs Loading link description

"Hello" as *standard input*, and echoes the string "Hello" back to the terminal as *standard output*.

Let's learn more about standard input, standard output, and standard error:

standard input, abbreviated as `stdin`, is information inputted into the terminal through the keyboard or input device.

standard output, abbreviated as `stdout`, is the information outputted after a process is run.

standard error, abbreviated as `stderr`, is an error message outputted by a failed process.

Redirection reroutes standard input, standard output, and standard error to or from a different location.

Instructions

Move on to the next exercise when you're ready to start redirecting!

Your First Redirect

How does redirection work?

```
$ echo "Hello" > hello.txt
```

Copy to Clipboard

The `>` command redirects the standard output to a file. Here, "Hello" is entered as the standard input, and is then redirected to the file **hello.txt** by `>`.

As a reminder, the `cat` command outputs the contents of a file to the

[terminal](#)

Preview: Docs Loading link description

. When you type:

```
$ cat hello.txt
```

Copy to Clipboard

the contents of **hello.txt** are displayed.

Instructions

Checkpoint 1 Enabled

1.

The exercises in this lesson use the **Check Work** button with the terminal. To move through the instructions, perform the tasks and click the **Check Work** button. If you accomplish the task correctly, you can move to the next task or exercise.

Let's try our first redirect!

In the terminal, type:

```
echo "Hello World" > hello.txt
```

Copy to Clipboard

Did you run `echo "Hello World" > hello.txt` and click the **Check Work** button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Then type:

```
cat hello.txt
```

Copy to Clipboard

You should see "Hello World" printed to the console.



```
$ cat deserts.txt > hello.txt
```

Copy to Clipboard

`>` takes the standard output of the command on the left, and redirects it to the file on the right. Here the standard output of `cat deserts.txt` is redirected to **hello.txt**.

Note that `>` overwrites all original content in **hello.txt**. When you view the output data by using `cat` on **hello.txt**, you will see only the contents of **deserts.txt**.

Instructions

Checkpoint 1 Enabled

1.

Let's practice redirection some more. The working directory is **geography/**. If you need to get back to this directory, you can change into it using the absolute path:

```
cd /home/ccuser/workspace/geography
```

Copy to Clipboard

This is the filesystem we'll work with. You can reference it anytime [here](#).

To start, output the working directory's contents using `ls` without any options. When finished, click the Check Work button.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use `cat` to view the contents of **continents.txt**.

Click Check Work when finished.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now use `>` to redirect the contents of **oceans.txt** to **continents.txt**.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Use `cat` to view the contents of **continents.txt** again.

You should now see oceans as output. If this is not the case, double-check the redirect command. Feel free to reset the exercise and try again.

Check Work to continue.

Redirection



Now we know how to overwrite a file's contents, but what if we want to be able to add to a file without losing the original text? We can use the `>>` command!

```
$ cat forests.txt >> hello.txt
```

Copy to Clipboard

`>>` takes the standard output of the command on the left and *appends* (adds) it to the file on the right.

Here, the output data of **hello.txt** will contain the original contents of **hello.txt** with the content of **forests.txt** appended to it.

Instructions

Checkpoint 1 Passed

1.

Use `>>` to append the contents of **glaciers.txt** to **rivers.txt**.

Click Check Work when done.

Use:

```
cat glaciers.txt >> rivers.txt
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 2 Passed

2.

Use `cat` to view the contents of **rivers.txt**.

Notice that we see rivers *and* glaciers as output.

Use:

```
cat rivers.txt
```

Copy to Clipboard

Did you click the Check Work button?



```
$ cat < deserts.txt
```

Copy to Clipboard

< takes the standard input from the file on the right and inputs it into the command on the left. Here, **deserts.txt** is the standard input for the `cat` command. The standard output appears in the terminal.

While this above example accomplishes the same thing as `cat deserts.txt` (without the redirect), it is important to know that input can be redirected.

Instructions

Checkpoint 1 Enabled

1.

In the terminal type:

```
cat < lakes.txt
```

Copy to Clipboard

Click Check Work when done.

Did you click the Check Work button?



`|` is a “pipe.” The `|` takes the standard output of the command on the left, and *pipes* it as standard input to the command on the right. You can think of this as “command to command” redirection.

To count the words in **volcanoes.txt** using the word count command `wc`:

```
$ cat volcanoes.txt | wc
```

Copy to Clipboard

Above, the output of `cat volcanoes.txt` becomes the standard input of `wc`. In turn, the `wc` command outputs the number of lines, words, and characters in **volcanoes.txt**, respectively.

```
$ cat volcanoes.txt | wc | cat > count.txt
```

Copy to Clipboard

Multiple `|`s can be chained together. Above the standard output of `cat volcanoes.txt` is “piped” to the `wc` command. The standard output of `wc` is then “piped” to `cat`. Finally, the standard output of `cat` is redirected to `count.txt`.

You can view the output data of this chain by typing `cat count.txt`.

Instructions

Checkpoint 1 Enabled

1.

Let’s try it out. In the terminal, type:

```
cat volcanoes.txt | wc
```

Copy to Clipboard

Click Check Work when done.

(Tip: If you are unable to type `|`, you can find help [here](#).)

Use:

```
cat volcanoes.txt | wc
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Type:

```
cat volcanoes.txt | wc | cat > volcanoes count.txt
```

Copy to Clipboard

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use `cat` to output the contents of `volcanoes_count.txt`. Notice that three numbers appear as the output. Click [Check Work](#) when done.

Redirection

sort

```
$ sort continents.txt
```

Copy to Clipboard

`sort` takes the standard input and orders it alphabetically for the standard output (it doesn't change the file itself). Here, the continents in `continents.txt` will be listed in alphabetical order:

```
Africa
Antarctica
Asia
Australia
Europe
North America
South America
```

Copy to Clipboard

```
$ cat glaciers.txt | sort > sorted-glaciers.txt
```

Copy to Clipboard

Here, the command takes the standard output from `cat glaciers.txt` and "pipes" it to `sort`. The standard output of `sort` is redirected to a new file named `sorted-glaciers.txt`.

Instructions

Checkpoint 1 Passed

1.

A few commands are particularly powerful when combined with redirection. Let's try them out.

First, use `cat` to view the contents of `lakes.txt`.

Click [Check Work](#) when done.

Did you click the [Check Work](#) button?

Checkpoint 2 Passed

2.

Sort the contents of `lakes.txt`.

You could use either of these commands to sort the `lakes.txt` file:

```
sort lakes.txt
```

Copy to Clipboard

or

```
cat lakes.txt | sort
```

Copy to Clipboard

Did you click the [Check Work](#) button?

Checkpoint 3 Passed

3.

Now, use the `>` symbol to save the sorted `lakes.txt` file as `sorted-lakes.txt`.

There are two ways to do this. You can use `cat` and `sort`:

```
cat lakes.txt | sort > sorted-lakes.txt
```

Copy to Clipboard

Or you can just use `sort`:

```
sort lakes.txt > sorted-lakes.txt
```

Copy to Clipboard

Also, don't forget to click the [Check Work](#) button!

Checkpoint 4 Enabled

4.

Use `cat` to output the contents of `sorted-lakes.txt`.

Click [Check Work](#) when done.

Redirection

uniq

`uniq` stands for “unique.” It filters out adjacent, duplicate lines in a file.

```
$ cat deserts.txt
Arctic Desert
Sahara Desert
Sahara Desert
Arabian Desert
Gobi Desert
Kalahari Desert
Great Basic Desert
Syrian Desert
Kalahari Desert

$ uniq deserts.txt
Arctic Desert
Sahara Desert
Arabian Desert
Gobi Desert
Kalahari Desert
Great Basic Desert
Syrian Desert
Kalahari Desert
```

Copy to Clipboard

Above `uniq deserts.txt` filters out duplicates of "Sahara Desert", because its duplicate directly follows the previous instance. The "Kalahari Desert" duplicates are not adjacent, and thus remain.

```
$ sort deserts.txt | uniq
```

Copy to Clipboard

A more effective way to use `uniq` is to call `sort` to alphabetize a file, and “pipe” the standard output to `uniq`. By piping `sort deserts.txt` to `uniq`, all duplicate lines are alphabetized (and thereby made adjacent) and filtered out.

```
sort deserts.txt | uniq > uniq-deserts.txt
```

Copy to Clipboard

Here we simply send filtered contents to `uniq-deserts.txt`, which you can view with the `cat` command.

Instructions

Checkpoint 1 Passed

1.

Use `cat` to output the contents of **deserts.txt**. Note the duplicate entries.

Click Check Work when done.

Did you click the Check Work button?

Checkpoint 2 Passed

2.

Use `uniq` to remove adjacent duplicates in **deserts.txt**.

Type:

```
uniq deserts.txt
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 3 Passed

3.

```
sort deserts.txt | uniq
```

Copy to Clipboard

Use the command above to sort **deserts.txt** before piping to `uniq`. You'll see the output contains no duplicates.

Click Check Work when done.

Redirection

grep I

```
$ grep America continents.txt
```

Copy to Clipboard

`grep` stands for “global regular expression print.” It searches files for lines that match a pattern and then returns the results. It is also case sensitive. Above, `grep` searched for anything that matched “America” in `continents.txt`.

```
$ grep -i America continents.txt
```

Copy to Clipboard

`grep -i` enables the command to be case insensitive. Here, `grep` searched for capital or lowercase strings that match “America” in `continents.txt`. Note that we don’t use quotes in our command.

The above commands are a great way to get started with `grep`. If you are familiar with [regular expressions](#)

Preview: Docs Loading link description

, you can also use regular expressions to search for patterns in files.

Instructions

Checkpoint 1 Passed

1.

Use `cat` to output the contents of `mountains.txt`.

Click Check Work when done.

Did you click the Check Work button?

Checkpoint 2 Passed

2.

Use `grep` to search for anything that matches `Mount` in `mountains.txt`. This search should be case sensitive.

You can use:

```
grep Mount mountains.txt
```

Copy to Clipboard

We don’t want to use `grep -i` since this search should be case sensitive.

Did you click the Check Work button?

Checkpoint 3 Passed

3.

Search for matches again, but without case sensitivity.

Click Check Work when done.

Now we can use `-i` to make the search case insensitive:

```
grep -i Mount mountains.txt
```

Copy to Clipboard

Did you click the Check Work button?

Redirection

grep II

```
$ grep -R Arctic /home/ccuser/workspace/geography

/home/ccuser/workspace/geography/deserts.txt:Arctic Desert
/home/ccuser/workspace/geography/oceans.txt:Arctic Ocean
```

Copy to Clipboard

`grep -R` searches all files in a directory and outputs filenames and lines containing matched results. `-R` stands for “recursive”. Above, `grep -R` searched the `/home/ccuser/workspace/geography` directory for the

[string](#)

Preview: Docs Loading link description

“Arctic” and outputted filenames and lines with matching results.

```
$ grep -Rl Arctic /home/ccuser/workspace/geography
```

```
/home/ccuser/workspace/geography/deserts.txt
/home/ccuser/workspace/geography/oceans.txt
```

Copy to Clipboard

`grep -Rl` searches all files in a directory and outputs only filenames with matched results (so no lines).

`l`, which is a lowercase `l` (not a capital `i`, not the number `1`), stands for “files with matches.” Here `grep -Rl` searched the `/home/ccuser/workspace/geography` directory for the string “Arctic” and outputs filenames with matched results.

Instructions

Using `grep`, search through the working directory (`/home/ccuser/workspace/geography`) for anything that matches `Island`. We want the matching filenames *and* lines.

Click Check Work when done.

Since we want the matching lines along with the filenames, we need to use `grep -R` and *not* `grep -Rl`:

```
grep -R Island /home/ccuser/workspace/geography
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 2 Passed

2.

Conduct the same search using `grep`, but without the matching lines.

Now that we only want the matching files we can use `grep -Rl`:

```
grep -Rl Island /home/ccuser/workspace/geography
```

Copy to Clipboard

Did you click the Check Work button?

Redirection

sed

```
sed 's/snow/rain/' forests.txt
```

Copy to Clipboard

`sed` stands for “stream editor.” It accepts standard input and modifies it based on an *expression*, before displaying it as output data. It is similar to “find and replace.”

Let’s look at the expression `'s/snow/rain/':`

`s`: stands for “substitution.” It is *always* used when using `sed` for substitution.

`snow`: the search

[string](#)

Preview: Docs Loading link description

, or the text to find.

`rain`: the replacement string, or the text to add in place.

In this case, `sed` searches `forests.txt` for the word “snow” and replaces it with “rain.” Importantly, the above command will only replace the first instance of “snow” on a line.

```
sed 's/snow/rain/g' forests.txt
```

Copy to Clipboard

The above command uses the `g` expression, meaning “global.” Here `sed` searches `forests.txt` for the word “snow” and replaces it with “rain” *globally*. This means *all* instances of “snow” on a line will be turned to “rain.”

`sed` as we’ve used it will only rewrite the command line output and the actual file won’t be changed. In order to rewrite the actual file, we need to use `-i` at the beginning of the command:

```
sed -i 's/snow/rain/g' forests.txt
```

Copy to Clipboard

The above command will rewrite **forests.txt** and replace *all* instances (since we're also using **g**) of "snow" with "rain".

Instructions

Checkpoint 1 Enabled

1.

Use **cat** to display the contents of **forests.txt**

Click Check Work when done.

Did you click the Check Work button?

Redirection

sed

```
sed 's/snow/rain/' forests.txt
```

Copy to Clipboard

sed stands for "stream editor." It accepts standard input and modifies it based on an *expression*, before displaying it as output data. It is similar to "find and replace."

Let's look at the expression **'s/snow/rain/'**:

s: stands for "substitution." It is *always* used when using **sed** for substitution.

snow: the search

string

Preview: Docs Loading link description

, or the text to find.

rain: the replacement string, or the text to add in place.

In this case, **sed** searches **forests.txt** for the word "snow" and replaces it with "rain." Importantly, the above command will only replace the first instance of "snow" on a line.

```
sed 's/snow/rain/g' forests.txt
```

Copy to Clipboard

The above command uses the **g** expression, meaning "global." Here **sed** searches **forests.txt** for the word "snow" and replaces it with "rain" *globally*. This means *all* instances of "snow" on a line will be turned to "rain."

sed as we've used it will only rewrite the command line output and the actual file won't be changed. In order to rewrite the actual file, we need to use **-i** at the beginning of the command:

```
sed -i 's/snow/rain/g' forests.txt
```

Copy to Clipboard

The above command will rewrite **forests.txt** and replace *all* instances (since we're also using **g**) of "snow" with "rain".

Instructions

Checkpoint 1 Passed

1.

Use **cat** to display the contents of **forests.txt**

Click Check Work when done.

Did you click the Check Work button?

Checkpoint 2 Passed

2.

Use **sed** on **forests.txt** to replace *just* the first appearance of "snow" on each line with "rain".

Since we only want to replace the first occurrence on each line we don't need to add **g**:

```
sed 's/snow/rain/' forests.txt
```

Copy to Clipboard

Remember to add the **/** at the end of **'s/snow/rain/'**!

Did you click the Check Work button?

Checkpoint 3 Passed

3.

Use **sed** again to replace "snow" with "rain" but this time replace every occurrence of "snow".

Click Check Work when done.

Now we can use `g` to make the search *global* throughout the file:

```
sed 's/snow/rain/g' forests.txt
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 4 Passed

4.

If you use `cat` to view the contents of **forests.txt**, you'll see that no changes have been made to the file.

Use `sed` one last time to rewrite the file such that *all* occurrences of "snow" are permanently replaced by "rain".

Use `-i` at the beginning of the command in order to rewrite the file:

```
sed -i 's/snow/rain/g' forests.txt
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 5 Enabled

5.

Use `cat` one more time to see the changes you've made to **forests.txt**.

Click Check Work when done.

Redirection

Review

Congratulations! You learned how to use the command line to redirect standard input and standard output. What can we generalize so far?

Redirection reroutes standard input, standard output, and standard error.

The common redirection commands are:

- > redirects standard output of a command to a file, overwriting previous content.
- >> redirects standard output of a command to a file, appending new content to old content.
- < redirects standard input to a command.
- | redirects standard output of a command to another command.

A number of other commands are powerful when combined with redirection commands:

- `sort`: sorts lines of text alphabetically.
- `uniq`: filters duplicate, adjacent lines of text.
- `grep`: searches for a text pattern and outputs it.
- `sed`: searches for a text pattern, modifies it, and outputs it.

Configuring the Command Line Environment

The command line is an indispensable tool for developing projects. You can use the command line to manage all your project files; you can move files, delete files, reorganize them, search and replace terms in your project, and so much more. You can also control the execution of programs you write and run commands from development software. The terminal becomes a busy place! This is why the command line is an environment in and of itself.

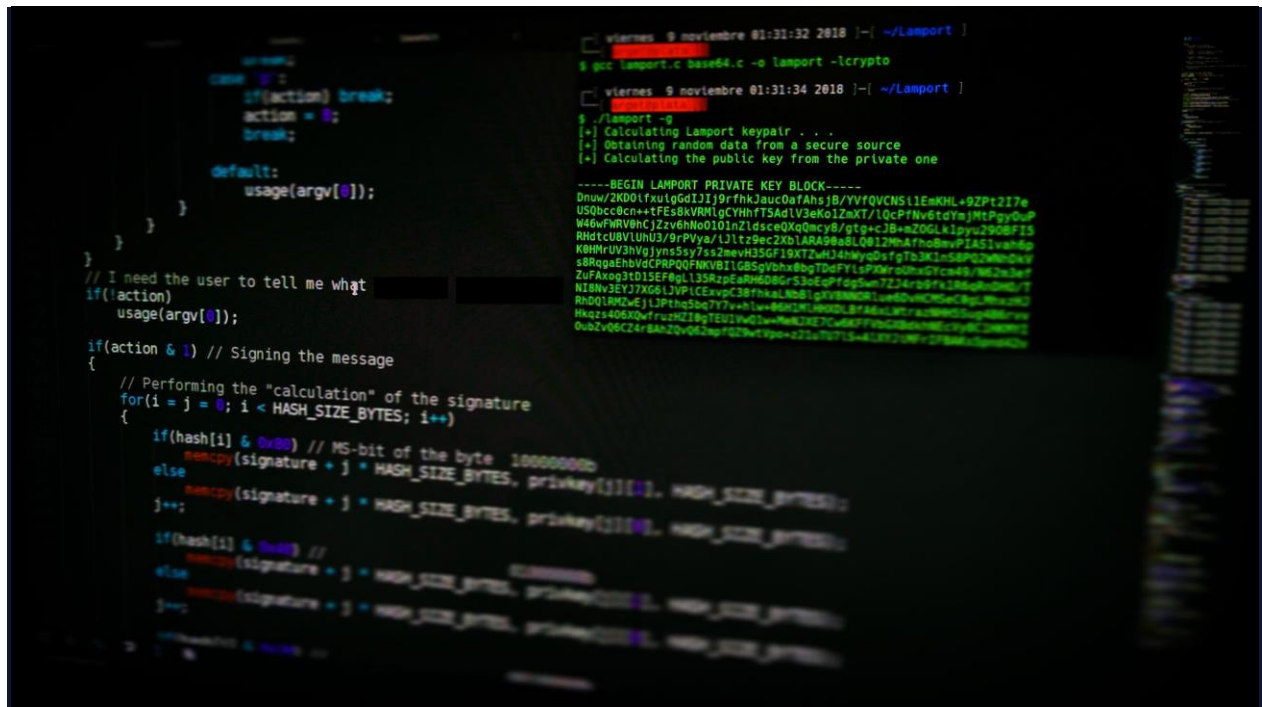


Photo by [Arget](#) on [Unsplash](#)

Within the command line environment, you can have many things running at once! For example, you can execute a Python project and keep track of output as a program runs (and redirect that output to a file), you can edit files directly inside the command line, and you can run commands from development software like [Git](#) to keep things organized. As you become a more seasoned developer, you will find it necessary to configure the command line environment to your liking and convenience.

In this lesson, you will add special settings to customize the command line environment. Specifically, you will learn:

- how to use nano, a keyboard-only text editor that exists in the command line
- what the *bash profile* is
- how to store shorthands (called *alias*) for commonly used commands
- how to change the way your command prompt looks
- how to establish and access global command-line variables
- what the *PATH* variable is and how to check which directories executable commands come from

Ready? Let's get started!

Environment

Intro to Environment

Each time we launch the terminal application, it creates a new session. The session immediately loads settings and preferences that make up the command line *environment*.

We can configure the environment to support the commands and programs we create. This enables us to customize greetings and create nicknames (aliases) for commands, and create variables to share across commands and programs.

You can reference the filesystem for this lesson [with this diagram](#).

In this first exercise, let's pull up a basic command line text editor called *nano*. Follow the steps below to create and save your first file in the terminal.

Note: If you are having trouble in nano and need to exit, use Ctrl + X.

Instructions

Checkpoint 1 Passed

1.

The exercises in this lesson use the Check Work button with the terminal. To move through the instructions, perform the tasks and click the Check Work button. If you accomplish the task correctly, you can move to the next task or exercise.

For your first task, type in the terminal:

```
nano hello.txt
```

Copy to Clipboard

This will open the nano text editor.

Click Check Work when done.

Did you click the Check Work button?

Checkpoint 2 Passed

2.

In nano, at the top of the window, type:

```
Hello, I am nano.
```

Copy to Clipboard

Using the menu at the bottom of the terminal for reference, type Ctrl + O (the letter, not zero) to save the file.

The keys are not case-sensitive, so don't worry about capitalizing.

Press Enter, when prompted about the filename to write.

Then type Ctrl + X to exit nano.

You can run the command `cat hello.txt` to confirm that there are now contents in **hello.txt**.

Click Check Work and move to the next exercise when done.

If you are having trouble in nano and need to force-exit, use Ctrl + Z.

Alternatively, the function key F2 can also be used to exit.

Did you click the Check Work button?

Environment

Nano

Nice job! You just edited a file in the nano text editor. How does it work?

```
$ nano hello.txt
```

Copy to Clipboard

nano is a command line text editor. It works the same way as a desktop text editor like TextEdit or Notepad, except that it is accessible from the command line and only accepts keyboard input.

Let's walk through what we did in the previous exercise:

The command `nano hello.txt` opens a new text file named **hello.txt** in the nano text editor.

`Hello, I am nano` is a text string entered in nano at the line indicated by the cursor.

The menu of keyboard commands at the bottom of the window allow us to save changes to **hello.txt** and exit nano. The `^` stands for the Ctrl key.

```
^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Page  ^K Cut Text   ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^N Next Page  ^U UnCut Text ^T To Spell
```

Ctrl + O saves a file. `o` stands for output. Again, not case-sensitive.
Ctrl + X exits the nano program. `x` stands for exit.
Ctrl + G opens a help menu.

Outside of nano, you can also use `clear` to clear the text in the current terminal window, moving the command prompt to the top of the screen. This is useful for when you want to make the terminal more readable after many commands.

In this lesson, we'll use nano to implement changes to the environment. You can learn more about nano [here](#).

Instructions

Checkpoint 1 Passed

1.

Open `artists/hiphop.txt` with `nano`.

Once the file is opened in `nano`, click the Check Work button.

Did you click the Check Work button?

Checkpoint 2 Passed

2.

In this opened file, add `Kendrick Lamar` to the first line.

Type Ctrl + O to save the file.

Press Enter to write the filename.

Type Ctrl + X to exit.

To verify your edits were saved, type

```
cat artists/hiphop.txt
```

Copy to Clipboard

When complete, click the Check Work button.

If you are having trouble with nano, use Ctrl + Z to force-exit and try again.

If you made a typo, you can open the file again in nano and save the corrections!

Did you click the Check Work button?

Environment

Customization with `.bash_profile`

A bash profile is a file used to store environment settings for your

terminal

Preview: Docs Loading link description

On most computer systems, the file is in the home directory and is accessible by the name `.bash_profile`.

When a session starts, it loads the contents of the bash profile before executing commands.

The `.` indicates a hidden file.

The name `.bash_profile` is important since this is how the command line recognizes the bash profile.

Please Note: Throughout the next several exercises, we will work on `.bash_profile` in the starting directory, not the home directory. This exercise's starting directory is `customization/`. When you work within the local bash shell of your computer, you should look for `.bash_profile` in your home directory, for example, `~/.bash_profile`.

To open and edit the bash profile in the `customization/` directory, you can use the command:

```
nano .bash_profile
```

Copy to Clipboard

When you edit the bash profile, you can add commands to execute every time a new terminal session is started.

For example, if you have an `echo` statement in the bash profile, that will `echo` when a terminal session begins.

To activate the changes made in `.bash_profile` for the current session, use the following command:

```
source .bash_profile
```


Copy to Clipboard

This makes the changes in the bash profile available right away without closing the terminal and needing to start a new session.

Instructions

Checkpoint 1 Passed

1.

Let's edit the environment settings!

In the terminal, type

```
nano .bash_profile
```

Copy to Clipboard

This opens up the existing, currently blank bash profile file in nano. Once complete, click the [Check Work](#) button.

Checkpoint 2 Passed

2.

In **.bash_profile**, at the top of the file, type:

```
echo "Welcome, Jane Doe"
```

Copy to Clipboard

You can use your name in place of "Jane Doe."

Type Ctrl + O to save the file.

Press Enter to write the filename.

Type Ctrl + X to exit nano.

Then, once you've exited nano and are back in the terminal, click the [Check Work](#) button.

Did you click the [Check Work](#) button?

Checkpoint 3 Passed

3.

Finally, to see this greeting immediately, use:

```
source .bash_profile
```

Copy to Clipboard

Once complete, click the [Check Work](#) button.

Did you click the [Check Work](#) button?

Environment

Aliases I

Please Note: In this exercise, we will work on **.bash_profile** in the starting directory, not the home directory. This exercise's starting directory is **aliases-1/**.

As we mentioned in the last exercise, you can add settings and commands that execute every time a new

[terminal](#)

Preview: Docs Loading link description

session is started. One type of setting you can create is called an **alias**:

```
alias pd="pwd"
```

Copy to Clipboard

The **alias** command allows you to create keyboard shortcuts, or aliases, for commonly used commands.

Here, `alias pd="pwd"` creates the alias **pd** for the **pwd** command, which is then saved in the bash profile.

The **pd** alias will be available each time we open a new terminal session, and the output of **pd** will be the same as the **pwd** command.

`source .bash_profile` will make the alias **pd** available in the current session.

Instructions

Checkpoint 1 Passed

1.

Let's continue configuring the environment by adding command aliases.

Open **.bash_profile** in nano. Once complete, click the [Check Work](#) button.

```
nano filename
```

Copy to Clipboard
Opens that file in nano!
Did you click the Check Work button?
Checkpoint 2 Passed

2.

In **.bash_profile**, on a new line, type:

```
alias pd="pwd"
```

Copy to Clipboard
Save the changes and exit out of nano.
Once you've exited nano and are back in the terminal, click the Check Work button.
Note: Make sure to type the command as shown. In bash, spaces are not allowed on either side of the = character.

Type Ctrl + O to save the file. Press Enter to write the filename.

And Ctrl + X to exit nano.

If you are having trouble with nano, force exit using Ctrl + Z.

Did you click the Check Work button?

Checkpoint 3 Passed

3.

To make the alias in the current terminal sessions, type

```
source .bash profile
```

Copy to Clipboard
Once complete, click the Check Work button.

Checkpoint 4 Passed

4.

Let's try out the alias. Type:

```
pd
```

Copy to Clipboard
You should see the same output as you would by typing the pwd command.

Did you click the Check Work button?

Environment

Aliases II

Please Note: In this exercise, we will work on **.bash_profile** in the starting directory, not the home directory. This exercise's starting directory is **aliases-2/**.

We can add as many aliases as we want in a bash profile. Here are two more examples:

```
alias hy="history"
```

Copy to Clipboard
hy is set as an alias for the history command in the bash profile. The alias is then made available in the current session through source. By typing hy, the command line outputs a history of commands that were entered in the current session.

```
alias ll="ls -la"
```

Copy to Clipboard
Likewise, the code above sets ll as an alias for ls -la. Once the alias is made available in the current session through source, the command ll now executes ls -la and outputs all contents and directories in long format, including all hidden files.

Let's add these to our bash profile!

Instructions

Checkpoint 1 Passed

1.

Open **.bash_profile** in nano.

Once complete, click the Check Work button.

Checkpoint 2 Passed

2.

In the bash profile, beneath the previous alias, add the alias `hy` for the command `history`.
On the next line, add the alias `ll` for the command `ls -la`. Note that `ll` and `ls -la` have lowercase "L"s and NOT the number 1.
Save the file, and close out of `nano`.
Once you've exited nano and are back in the terminal, click the Check Work button.

An alias can be set like so:

```
alias alias_name="real_command"
```

Copy to Clipboard

Type Ctrl + O to save the file. Press Enter to write the filename.

And Ctrl + X to exit nano.

If you are having trouble with nano, force exit using Ctrl + Z.

Did you click the Check Work button?

Checkpoint 3 Passed

3.

In the command line, use `source` to activate the changes to the bash profile for the current session.

Use the following command to activate the bash profile changes:

```
source .bash_profile
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 4 Passed

4.

Try out the alias `hy`.

Once complete, click the Check Work button.

In the command line, type:

```
hy
```

Copy to Clipboard

Did you click the Check Work button?

Checkpoint 5 Passed

5.

Try out the alias `ll`.

You should see the same result as `ls -la`!

In the command line, type:

```
ll
```

Copy to Clipboard

Did you click the Check Work button?

Environment

Environment Variables

Please Note: In this exercise, we will work on `.bash_profile` in the starting directory, not the home directory. This exercise's starting directory is **env-variables**.

Environment variables are

[variables](#)

Preview: Docs Loading link description

that can be used across commands and programs and hold information about the environment.

What happens when you store this in `.bash_profile`?

```
export USER="Jane Doe"
```

Copy to Clipboard

The line `USER="Jane Doe"` sets the environment variable `USER` to a name "Jane Doe". Usually the `USER` variable is set to the name of the computer's owner.

The line `export` makes the variable available to all child sessions initiated from the session you are in. This is a way to make the variable persist across programs.

At the command line, the command `echo $USER` returns the value of the variable. Note that `$` is always used when returning a variable's value. Here, the command `echo $USER` returns the name set for the variable.

Instructions

Checkpoint 1 Passed

1.

Now that you are familiar with configuring greetings and aliases, let's move on to setting *environment variables*. Open **.bash_profile** in nano and click the **Check Work** button.

Checkpoint 2 Passed

2.

In the bash profile, beneath the aliases, on a new line, type:

```
export USER="Jane Doe"
```

Copy to Clipboard

Feel free to use your own name. Then, save the file and exit.

Then, once you've exited nano and are back in the terminal, click the **Check Work** button.

Checkpoint 3 Passed

3.

In the command line, use `source` to activate the changes in the bash profile for the current session.

To activate the changes made to the bash profile in the current terminal session, we use this command:

```
source .bash profile
```

Copy to Clipboard

Did you click the **Check Work** button?

Checkpoint 4 Enabled

4.

Type:

```
echo $USER
```

Copy to Clipboard

This should output the value of the variable that you set.

Tip: If you are unable to type `$`, you can find help [here](#).

Did you click the **Check Work** button?

Environment

PS1 Environment Variable

Please Note: In this exercise, we will work on **.bash_profile** in the starting directory, not the home directory. This exercise's starting directory is **ps1-variable**.

`PS1` is an environment variable that defines the makeup and style of the command prompt.

What happens when this is stored in **.bash_profile**?

```
export PS1=">> "
```

Copy to Clipboard

`export PS1=">> "` sets the command prompt variable and exports the variable. Here we change the default command prompt from `$` to `>> .`

After using the `source` command, the command line displays the new command prompt.

Let's try it out!

Instructions

Checkpoint 1 Passed

1.

Open **.bash_profile** in nano.

Did you click the **Check Work** button?

Checkpoint 2 Passed

2.

On a new line, beneath the last entry, set the `PS1` variable equal to `">> "` (make sure there is one space following `>>`). Save the file and exit nano, and click the Check Work button.

In the bash profile, in order to set the `PS1` variable, write the following on a new line:

```
export PS1=">> "
```

Copy to Clipboard

Open the bash profile with `nano .bash_profile`.

Type `Ctrl + O` to save the file.

Press `Enter` to write the filename.

And `Ctrl + X` to exit nano.

If you are having trouble with nano, you can force exit using `Ctrl + Z`

Did you click the Check Work button?

Checkpoint 3 Passed

3.

Use `source` to make the changes in `.bash_profile` available. The command prompt should now look like `>>` .

Hit `Enter` with or without typing in commands to see that the prompt is now changed to `>>`. Click the Check Work button when done.

Did you click the Check Work button?

Environment

HOME Environment Variable

What happens when you type this command?

```
$ echo $HOME
```

Copy to Clipboard

The `HOME` variable is an environment variable that displays the path of the home directory `~`. You can specify and change the `HOME` variable if needed, but in most cases this is not necessary.

Here, by typing `echo $HOME`, the

[terminal](#)

Preview: Docs Loading link description
displays the path `/home/ccuser` as output.

Instructions

Checkpoint 1 Passed

1.

Let's see the value of `$HOME` currently.

In the command line, type:

```
echo $HOME
```

Copy to Clipboard

This returns the value of the `HOME` variable. Click the Check Work button when done.

Did you click the Check Work button?

Environment

PATH Environment Variable

`PATH` is an environment variable that stores a list of directories separated by a colon.

What happens when you type this command?

```
$ echo $PATH

/home/ccuser/.gem/ruby/2.0.0/bin:/usr/local/sbin:
/usr/local/bin:/usr/bin:/usr/sbin:/sbin:/bin
```

Copy to Clipboard

Looking carefully, `echo $PATH` lists the following directories, separated by `:`

```
/home/ccuser/.gem/ruby/2.0.0/bin
/usr/local/sbin
/usr/local/bin
/usr/bin
/usr/sbin
/sbin
/bin
```

Each directory contains scripts for the command line to execute. The `PATH` variable simply lists which directories contain scripts.

For example, many commands we've learned are scripts stored in the `/bin` directory.

```
/bin/pwd
```

Copy to Clipboard

This is the script that is executed when you type the `pwd` command.

```
/bin/ls
```

Copy to Clipboard

This is the script that is executed when you type the `ls` command.

In advanced cases, you can customize the `PATH` variable when adding scripts of your own.

Instructions

Checkpoint 1 Passed

1.

In the command line, type:

```
echo $PATH
```

Copy to Clipboard

This prints the `PATH` variable. Click the `Check Work` button when complete.

Did you click the `Check Work` button?

Checkpoint 2 Passed

2.

Let's confirm that most of the terminal commands we've learned so far are in `/bin/`.

Type the following into the terminal:

```
/bin/pwd
```

Copy to Clipboard

This should have the same output as the `pwd` command.

Did you click the `Check Work` button?

Checkpoint 3 Passed

3.

Type:

```
/bin/ls
```

Copy to Clipboard

This should have the same output as the `ls` command. Click the `Check Work` button when complete.

Did you click the `Check Work` button?

Environment

env

The `env` command stands for “environment,” and returns a list of the environment

[variables](#)

Preview: Docs Stores data that can be manipulated and referenced throughout the code.
for the current user.

What happens when you type the `env` command?

```
env
```

Copy to Clipboard

The `env` command returns a number of variables, including `PATH`, `PWD`, `PS1`, and `HOME`. To select the value of a particular environment variable, let's say `PATH`, you can use the following command:

```
env | grep PATH
```

Copy to Clipboard

the command above displays the value of the `PATH` environment variable. Here the standard output of `env` is “piped” to the `grep` command. `grep` searches for the value of the variable `PATH` and outputs it to the

[terminal](#)

Preview: Docs Loading link description

. Note that this is the same output as `echo $PATH`.

Instructions

Checkpoint 1 Passed

1.

In the command line type:

```
env
```

Copy to Clipboard

You should see a list of variables and their values. Click the Check Work button when complete.

Did you click the Check Work button?

Checkpoint 2 Enabled

2.

Then specifically display the value of the `PS1` environment variable using `env`, `grep`, and redirection.

The output should match what the command prompt looks like right now!

```
env | grep PATH
```

Copy to Clipboard

The command above displays the value of the `PATH` environment variable. Use the same format to display `PS1`.

(Tip: If you are unable to type `|`, you can find help [here](#).)

Did you click the Check Work button?

Environment

Review

Congratulations! You learned to use the bash profile to configure the environment. What can we generalize so far?

The *environment* refers to the preferences and settings of the current user.

The *nano* editor is a command line text editor used to configure the environment.

.bash_profile is where environment settings are stored. You can edit this file with nano.

Environment variables are

[variables](#)

Preview: Docs Loading link description

that can be used across commands and programs and hold information about the environment.

`export VARIABLE="Value"` sets and exports an environment variable.

`USER` is the name of the current user.

`PS1` is the command prompt.

`HOME` is the home directory. It is usually not customized.

`PATH` returns a colon `:` separated list of

[file paths](#)

Preview: Docs Loading link description

. It is customized in advanced cases.

`env` returns a list of environment variables. You can redirect the output, using `grep` to select the variable you want to see.

Feel free to use this

[terminal](#)

Preview: Docs Loading link description

to practice before moving on!

Windows Command Line

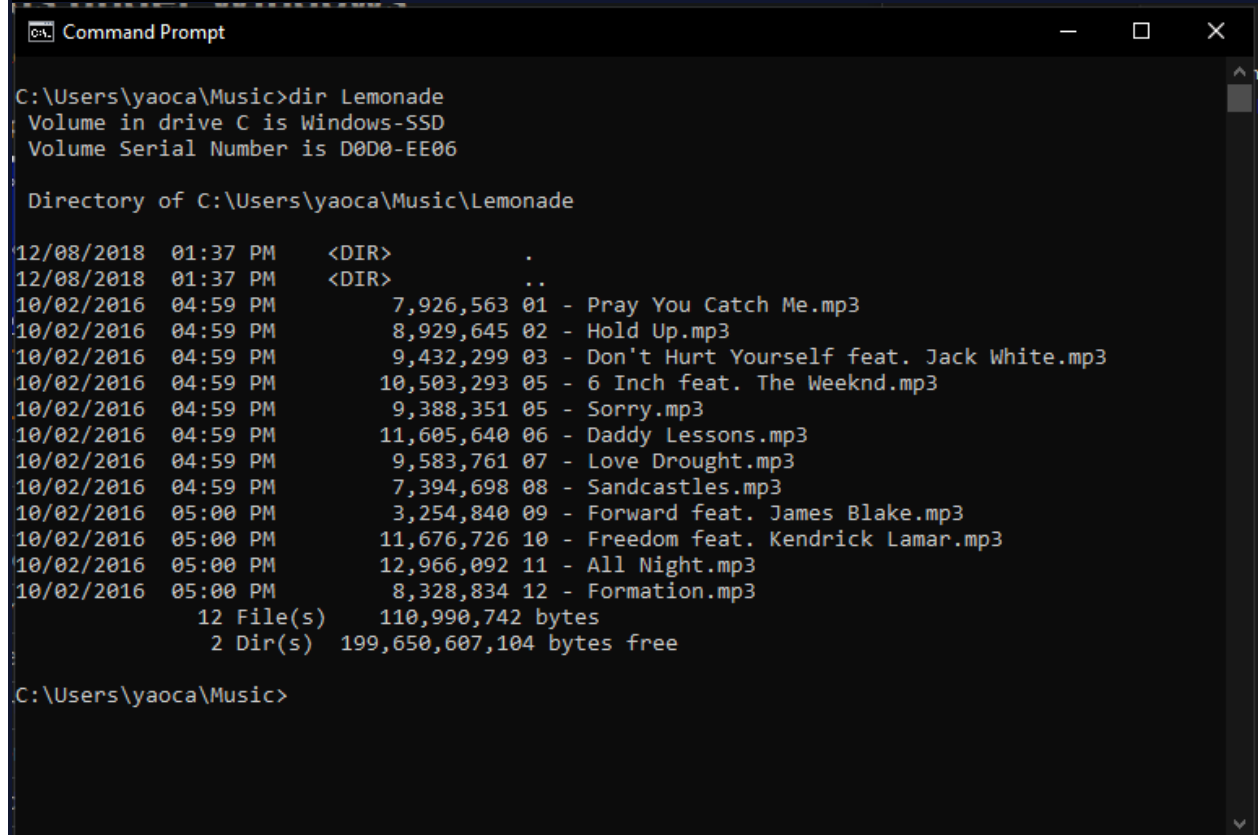
This article is for the Windows user! We will offer more detail about how to apply the knowledge from the Command Line course in the Windows environment.

How do Windows users access the command line?

We will discuss a few options for accessing the command line in Windows. We'll introduce Windows Command Prompt, which is the system default for basic tasks, and Powershell, which is a more advanced tool often used by system administrators. We will also touch on Git Bash, which will allow you to use all the commands we teach in this course on your Windows system.

At the end of the article, we will also provide Windows Command Prompt equivalents of the Bash commands from this course.

Windows Command Prompt

A screenshot of the Windows Command Prompt window. The title bar says "Command Prompt". The command prompt shows the current directory as C:\Users\yaoca\Music. The user has entered the command 'dir Lemonade'. The output shows the directory listing for C:\Users\yaoca\Music\Lemonade. It lists 12 files and 2 directories. The files are: Pray You Catch Me.mp3, Hold Up.mp3, Don't Hurt Yourself feat. Jack White.mp3, 6 Inch feat. The Weeknd.mp3, Sorry.mp3, Daddy Lessons.mp3, Love Drought.mp3, Sandcastles.mp3, Forward feat. James Blake.mp3, Freedom feat. Kendrick Lamar.mp3, All Night.mp3, and Formation.mp3. The total size of the files is 110,990,742 bytes. The total size of the directory is 199,650,607,104 bytes free.

```
C:\Users\yaoca\Music>dir Lemonade
Volume in drive C is Windows-SSD
Volume Serial Number is D0D0-EE06

Directory of C:\Users\yaoca\Music\Lemonade

12/08/2018  01:37 PM    <DIR>          .
12/08/2018  01:37 PM    <DIR>          ..
10/02/2016  04:59 PM         7,926,563 01 - Pray You Catch Me.mp3
10/02/2016  04:59 PM         8,929,645 02 - Hold Up.mp3
10/02/2016  04:59 PM         9,432,299 03 - Don't Hurt Yourself feat. Jack White.mp3
10/02/2016  04:59 PM        10,503,293 05 - 6 Inch feat. The Weeknd.mp3
10/02/2016  04:59 PM         9,388,351 05 - Sorry.mp3
10/02/2016  04:59 PM        11,605,640 06 - Daddy Lessons.mp3
10/02/2016  04:59 PM         9,583,761 07 - Love Drought.mp3
10/02/2016  04:59 PM         7,394,698 08 - Sandcastles.mp3
10/02/2016  05:00 PM         3,254,840 09 - Forward feat. James Blake.mp3
10/02/2016  05:00 PM        11,676,726 10 - Freedom feat. Kendrick Lamar.mp3
10/02/2016  05:00 PM        12,966,092 11 - All Night.mp3
10/02/2016  05:00 PM         8,328,834 12 - Formation.mp3
               12 File(s)      110,990,742 bytes
               2 Dir(s)  199,650,607,104 bytes free

C:\Users\yaoca\Music>
```

Windows Command Prompt comes preloaded into Windows systems. This tool allows for basic system-related tasks. For example, you can run the following command to shut down the computer in two hours:

```
shutdown -s -t 7200
```

This command prompt is quite limited in its functionality especially when used for software development, but has been kept to maintain backwards compatibility. You can access this tool by bringing up the Start menu and typing `cmd`. We'll share some equivalent commands to Mac/Linux Bash at the end of the article!

Powershell

Powershell is an important tool for system administrators in Windows environments. It was introduced in 2006 to allow users to write much [more powerful scripts](#), and to redirect outputs of commands like one can in Bash.

PowerShell has its own syntax where commands are literal verbs of what's happening. For example, the following command changes the current directory to `C:\Desktop`.

```
Set-Location "C:\Desktop"
```

Windows Terminal

As of summer 2020, Windows has released [Windows Terminal](#), an interface that combines Command Prompt, Powershell, and, if it detects that you have Windows Subsystem for Linux installed, Linux Bash!

Git Bash

Git Bash lets you use Bash commands on your Windows computer. It is our most recommended Windows command line setup for users that need the command line primarily for software development. Many software tools include Bash commands to install, configure, and run them.

Below is a tutorial for setting Git Bash up on Windows:

Bash to Windows Command Prompt Translation Guide

Here's a quick table of Bash commands and their Windows Command Prompt equivalents if you want to try them out on Windows!

Navigation

Bash to Windows Command Prompt Translation Guide

Here's a quick table of Bash commands and their Windows Command Prompt equivalents if you want to try them out on Windows!

Navigation

	<code>ls</code>	<code>dir</code>
	<code>pwd</code>	<code>chdir</code>
	<code>cd</code>	<code>cd</code>
	<code>mkdir</code>	<code>md</code>
	<code>touch</code>	<code>echo "hello world" > hello.txt</code>
	<code>clear</code>	<code>cls</code>

Manipulation

File	File	File
cat	cat	type
cp	cp	copy
mv	mv	ren
rm	rm	del

Redirection

[REDACTED]

>

>

[REDACTED]

|

|

[REDACTED]

>>

>>

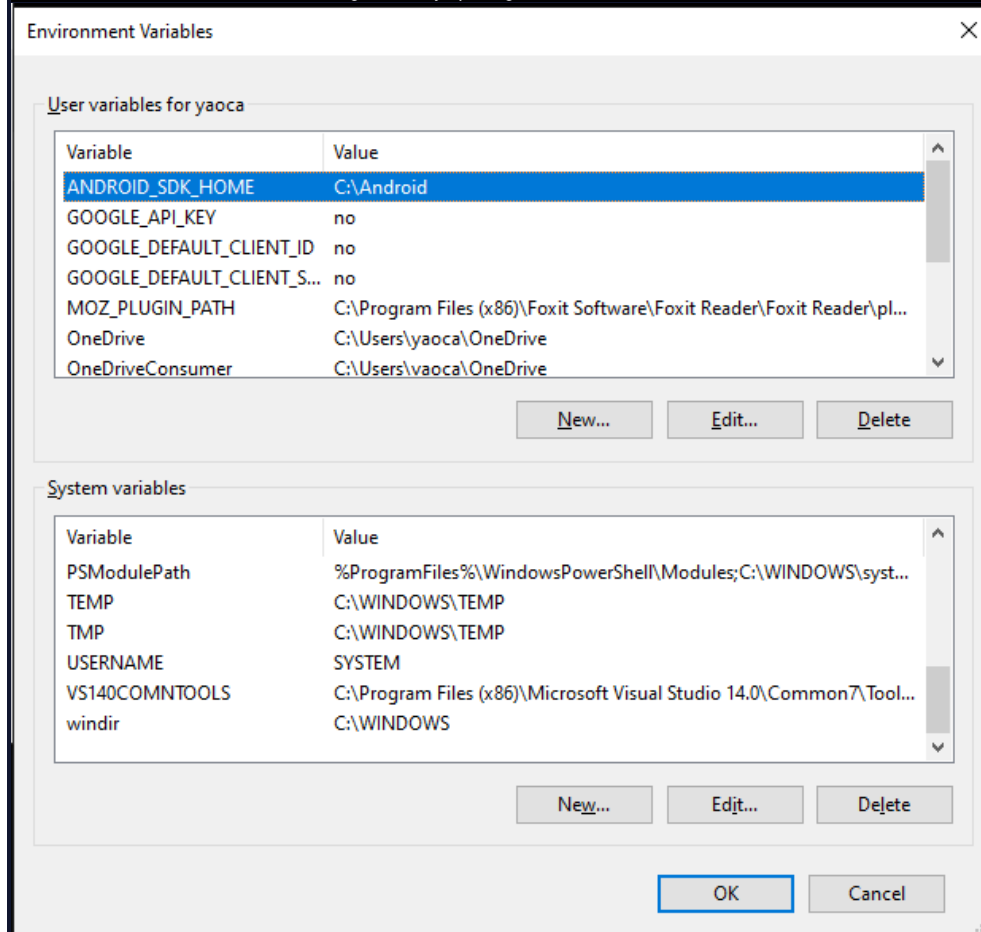
[REDACTED]

grep

find

Environment

In Windows, there are two types of environment variables: System Variables and User Variables. Each variable is in the form of a name and a value. You can view and edit the variables through a GUI by opening the Start menu and search "Edit environment variables" to open up the following window:



To read the environment variables in the command line, these are the Windows commands compared to the Mac/Linux Bash command.

[REDACTED]

[REDACTED]

[REDACTED]

```
set /a VAR=10
```

```
env
```

```
set
```

```
setx VAR value
```

```
export VAR=value
```

```
setx variable value
```

```
set /a VAR=10
```

```
echo $VAR
```

```
echo %VAR%
```

Additional Resources

We've covered the basics of Windows Command Prompt, but if you're looking for more, here are some good resources:

[Windows commands cheatsheet](#)

[Microsoft's documentation](#)

Introduction: Git & GitHub

Introduction to our Learn Git & GitHub course.

In this course, we will cover the basic functions of Git & GitHub.

Through lessons, projects, articles, and tutorials, you'll practice using basic Git commands in the terminal to add version control to your project. This means you can switch between different versions of your progress, and branch off to create new features while preserving a main working copy.

We will go over how to integrate Git with GitHub, one of the most-used web applications by developers across the world to collaborate in teams and share code.

By the end of this course, you'll be able to:

- Create a Git project and setup a remote copy on GitHub.
- Label code changes and move between different versions of your project.
- Write text on the GitHub interface using Markdown to describe your project and code changes.
- Create branches in your Git project so you can collaborate with others.
- Use Git commands to make sure your local copy of code matches the remote copy on GitHub.
- Use GitHub pull requests to discuss code changes with others.
- Be familiar with Git rebase, GitHub repository settings, and how to keep a remote repository organized.
- How to create a GitHub profile and explore code written by others
- Be familiar with intermediate to advanced GitHub features that allow you manage tasks within GitHub itself and automate actions

There's quite a bit in this course, so let's dive in!

Git & GitHub

Introduction: Git & GitHub

Introduction to our [Learn Git & GitHub course](#).

In this course, we will cover the basic functions of Git & GitHub.

Through lessons, projects, articles, and tutorials, you'll practice using basic Git commands in the terminal to add version control to your project. This means you can switch between different versions of your progress, and branch off to create new features while preserving a main working copy.

We will go over how to integrate Git with GitHub, one of the most-used web applications by developers across the world to collaborate in teams and share code.

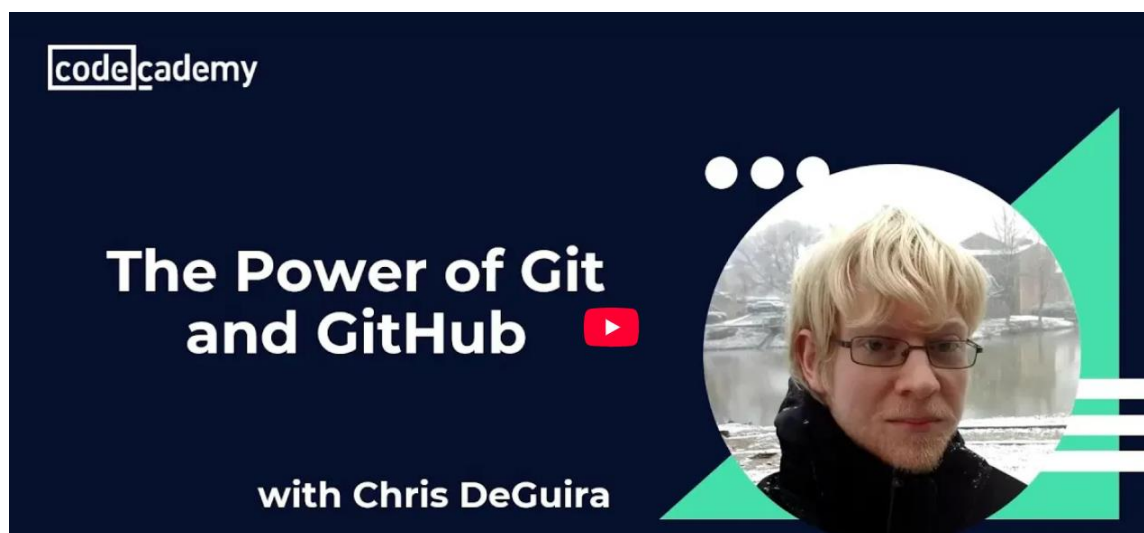
By the end of this course, you'll be able to:

- Create a Git project and setup a remote copy on GitHub.
- Label code changes and move between different versions of your project.
- Write text on the GitHub interface using Markdown to describe your project and code changes.
- Create branches in your Git project so you can collaborate with others.
- Use Git commands to make sure your local copy of code matches the remote copy on GitHub.
- Use GitHub pull requests to discuss code changes with others.
- Be familiar with Git rebase, GitHub repository settings, and how to keep a remote repository organized.
- How to create a GitHub profile and explore code written by others
- Be familiar with intermediate to advanced GitHub features that allow you manage tasks within GitHub itself and automate actions

There's quite a bit in this course, so let's dive in!

The Power of Git and GitHub

<https://youtu.be/eNw8MgguLsQ>



In this video we explore some of the capabilities found with Git and Github, and take a look at how they're used together in a project. The video includes examples of using cloning repositories, Git terminal commands such as `git log`, `git commit`, and `git add`, as well as GitHub features like pull requests, pull request reviews, and adding collaborators to repositories.

Basic Git Workflow

Hello Git

Git is a software that allows you to keep track of changes made to a project over time. Git works by recording the changes you make to a project, storing those changes, then allowing you to reference them as needed.

We'll learn Git by using it to help us write a screenplay called *Harry Programmer and the Sorcerer's Code*.

Instructions

Checkpoint 1 Enabled

1.

We'll get started by taking a look at the screenplay project.

In **scene-1.txt**, add this text:

```
Harry Programmer and the Sorcerer's Code: Scene 1
```

Copy to Clipboard

Then press `enter` to create a new empty line. Once you've created the new line, click Run.

Basic Git Workflow

git init

Now that we have started working on the screenplay, let's turn the **sorcerers-code** directory into a Git project. We do this with:

```
git init
```

Copy to Clipboard

The word `init` means *initialize*. The command sets up all the tools Git needs to begin tracking changes made to the project.

Instructions

Checkpoint 1 Failed, try again

1.

In the terminal, initialize a new Git project.

Notice the output:

```
Initialized empty Git repository in /home/ccuser/workspace/sorcerers-code/.git/
```

Copy to Clipboard

The Git project was created. Click Next to continue.

Basic Git Workflow

Git Workflow

Nice! We have a Git project. A Git project can be thought of as having three parts:

A *Working Directory*: where you'll be doing all the work: creating, editing, deleting and organizing files

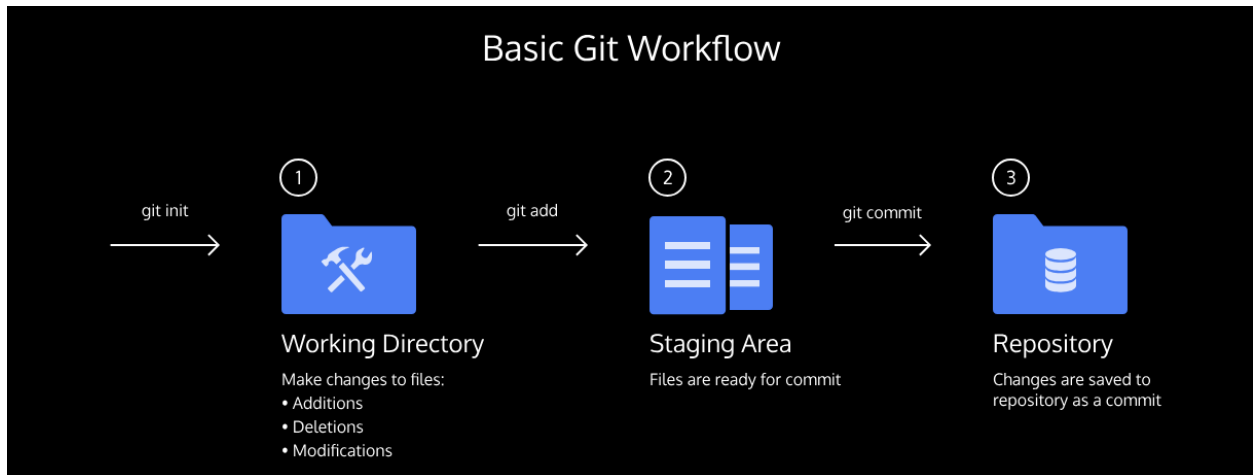
A *Staging Area*: where you'll list changes you make to the working directory

A *Repository*: where Git permanently stores those changes as different *versions* of the project

The Git workflow consists of editing files in the working directory, adding files to the staging area, and saving changes to a Git repository. In Git, we save changes with a *commit*, which we will learn more about in this lesson.

Instructions

Take a look at the diagram. Before we move on, it will help to be familiar with the three parts of the Git workflow. Click Next to continue.



Basic Git Workflow

git status

As you write the screenplay, you will be changing the contents of the working directory. You can check the status of those changes with:

```
git status
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

From the terminal, check the status of the **sorcerers-code** project.

In the output, notice the file in red under `untracked files`. Untracked means that Git sees the file but has not started tracking changes yet.

If you're having trouble accessing the terminal, try refreshing the webpage.

Basic Git Workflow

git add

In order for Git to start tracking **scene-1.txt**, the file needs to be added to the staging area.

We can add a file to the staging area with:

```
git add filename
```

Copy to Clipboard

The word `filename` here refers to the name of the file you are editing, such as **scene-1.txt**.

Instructions

Checkpoint 1 Passed

1. Add **scene-1.txt** to the staging area in Git. Recall that you will need to identify the file by its name.

Checkpoint 2 Passed

2. Check the status of the project in Git.

In the output, notice that Git indicates the changes to be committed with "new file: scene-1.txt" in green text. Here Git tells us the file was added to the staging area.

The output of `git status` should look similar to this:

```
$ git status
On branch master

Initial commit

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

        new file:   scene-1.txt
```

Basic Git Workflow

git diff

Good work! Now you know how to add a file to the staging area.

Imagine that we type another line in **scene-1.txt**. Since the file is tracked, we can check the differences between the working directory and the staging area with:

```
git diff filename
```

Copy to Clipboard

Here, `filename` is the actual name of the file. If the name of my file was **changes.txt** the command would be

```
git diff changes.txt
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

In the code editor, add this text to **scene-1.txt**:

```
Dumblediff: I should've known you would be here, Professor McGonagit.
```

Copy to Clipboard

Click Run.

Checkpoint 2 Passed

2.

From the terminal, use the new command to check the difference between the working directory and the staging area.

Notice the output:

“Harry Programmer and the Sorcerer’s Code: Scene 1” is in the staging area, as indicated in white.

Changes to the file are marked with a `+` and are indicated in green.

If you get stuck in “diff mode”, press `q` on your keyboard to exit.

Checkpoint 3 Passed

3.

Add the changes to the staging area in Git. Recall that you will need to identify the file by its name.

We can add a file to the staging area with:

```
git add filename
```

Basic Git Workflow

git commit

A *commit* is the last step in our Git workflow. A commit permanently stores changes from the staging area inside the repository.

`git commit` is the command we'll do next. However, one more bit of code is needed for a commit: the *option* `-m` followed by a message. Here's an example:

```
git commit -m "Complete first line of dialogue"
```

Copy to Clipboard

Standard Conventions for Commit Messages:

- Must be in quotation marks
- Written in the present tense
- Should be brief (50 characters or less) when using `-m`

Instructions

Checkpoint 1 Enabled

1.

Make your first commit! From the terminal, type the command along with a commit message. The message should describe the point of the commit.

If you're having trouble thinking of a good commit message, reflect on how the project has changed since it began.

git commit

A *commit* is the last step in our Git workflow. A commit permanently stores changes from the staging area inside the repository.

`git commit` is the command we'll do next. However, one more bit of code is needed for a commit: the *option* `-m` followed by a message. Here's an example:

```
git commit -m "Complete first line of dialogue"
```

Copy to Clipboard

Standard Conventions for Commit Messages:

- Must be in quotation marks
- Written in the present tense
- Should be brief (50 characters or less) when using `-m`

Instructions

Checkpoint 1 Passed

1.

Make your first commit! From the terminal, type the command along with a commit message. The message should describe the point of the commit.

If you're having trouble thinking of a good commit message, reflect on how the project has changed since it began.

Basic Git Workflow

git log

Often with Git, you'll need to refer back to an earlier version of a project. Commits are stored chronologically in the repository and can be viewed with:

```
git log
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

From the terminal, log a list of your commits.

In the output, notice:

A 40-character code, called a *SHA*, that uniquely identifies the commit. This appears in orange text.

The commit author (you!)
The date and time of the commit
The commit message

Click "Next" to continue.

Here's what an example log looks like:

```
commit a42ab9bee5f9e90ec60df0dd5dff9c4e5af99b3c
```

```
Author: codecademy <ccuser@codecademy.com>
```

```
Date: Fri Mar 15 17:54:08 2019 +0000
```

Complete first line of dialogue

Basic Git Workflow

Generalizations

You have now been introduced to the fundamental Git workflow. You learned a lot! Let's take a moment to generalize:

Git is the industry-standard

[version control](#)

Preview: Docs Loading link description

system for web developers

Use Git commands to help keep track of changes made to a project:

`git init` creates a new Git repository

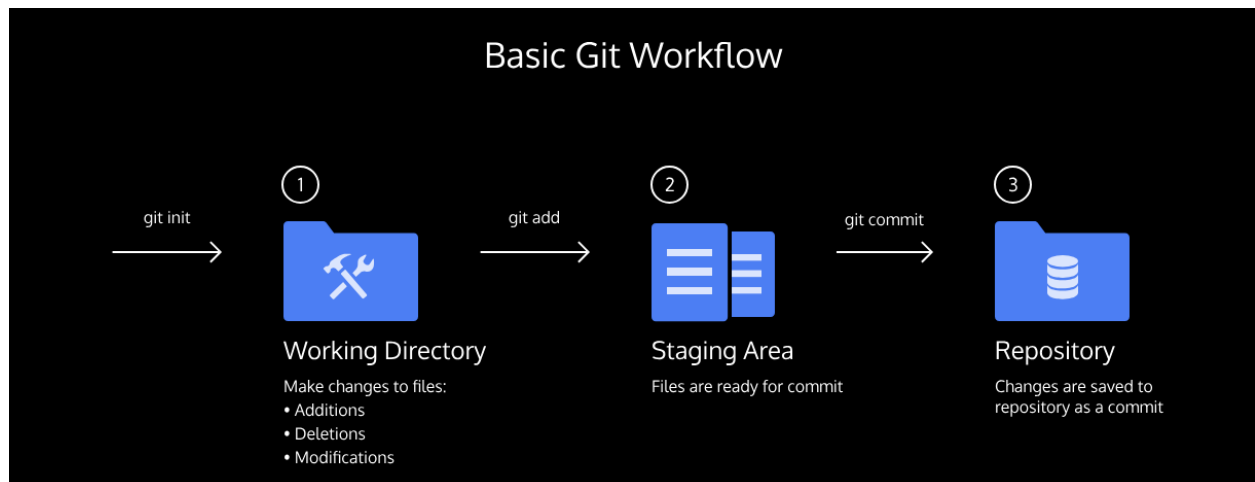
`git status` inspects the contents of the working directory and staging area

`git add` adds files from the working directory to the staging area

`git diff` shows the difference between the working directory and the staging area

`git commit` permanently stores file changes from the staging area in the repository

`git log` shows a list of all previous commits



https://www.codecademy.com/paths/machine-learning-engineer/tracks/mle-software-engineering-for-machine-learning-ai-engineers/modules/mle-learn-git-introduction-to-version-control/external_resources/fecp-external-git-cheat-sheet

How to Backtrack

Backtracking Intro

When working on a Git project, sometimes we make changes that we want to get rid of. Git offers a few eraser-like features that allow us to undo mistakes during project creation. In this lesson, we'll learn some of these features.

To start out, let's review the basic Git workflow.

Instructions

Checkpoint 1 Passed

1.

You are in a Git project titled **hamlet-prince-of-denmark**. In the code editor, you'll be working on **scene-5.txt**. Here, Hamlet encounters the ghost of his father. Add this text to the file:

```
Ghost:
My hour is almost come,
When I to sulphurous and tormenting flames
Must render up myself.
```

Copy to Clipboard

Checkpoint 2 Passed

2.

From the terminal, add **scene-5.txt** to the staging area.

[Here's a hint](#) on how to do it.

Checkpoint 3 Passed

3.

Commit the changes to the repository with a good commit message.

[Here's a hint](#) on how to do it.

How to Backtrack

head commit

In Git, the commit you are currently on is known as the `HEAD` commit. In many cases, the most recently made commit is the `HEAD` commit.

To see the `HEAD` commit, enter:

```
git show HEAD
```

Copy to Clipboard

The output of this command will display everything the [git log command](#) displays for the `HEAD` commit, plus all the file changes that were committed.

Instructions

Checkpoint 1 Passed

1.

Enter the command to show the `HEAD` commit.

Notice the output. The ghost's most recently added line is in green text.

How to Backtrack

git checkout

What if you decide to change the ghost's line in the working directory, but then decide you wanted to discard that change?

You could rewrite the line how it was originally, but what if you forgot the exact wording? The command

```
git checkout HEAD filename
```

Copy to Clipboard

will restore the file in your working directory to look exactly as it did when you last made a commit.

Here, `filename` again is the actual name of the file. If the file is named **changes.txt**, the command would be

```
git checkout HEAD changes.txt
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Change the ghost's words in some way. Here's a fun suggestion:

```
Ghost:
My hour is almost come,
When I to sulphurous and tormenting balloons
Must render up myself.
```

Copy to Clipboard

Checkpoint 2 Passed

2.

From the terminal, use `git diff` to see the difference between **scene-5.txt** as it appears in the working directory vs. how it appears in your last commit.

You may need to press `q` on your keyboard to restore the terminal.

Checkpoint 3 Passed

3.

Use the new Git command to restore the file in your working directory to look as it did when you last made a commit. Close the **scene-5.txt** file and then re-open it to see that the changes you had previously made to the ghost's line have been discarded.

To revert changes that you have made to a file, use the following command:

```
git checkout HEAD filename
```

Copy to Clipboard

where `filename` is the file whose changes you wish to discard.

How to Backtrack

more git add

The **hamlet** repository we are working on contains five files. In Git, it's common to change many files, add those files to the staging area, and commit them to a repository in a single commit.

For example, say you want to change the character "LARRY" to "LAERTES" in the script. The name currently appears in two files. After you change the name in both files, you could add the changed files to the staging area with:

```
git add filename_1 filename_2
```

Copy to Clipboard

Note the word `filename` above refers to the name of the file you are adding to the staging area, such as **scene-3.txt**.

Instructions

Checkpoint 1 Passed

1.

The code editor is open to **scene-3.txt** and **scene-7.txt**. In **scene-3.txt**, everywhere you see the name "LARRY" change it to "LAERTES."

Checkpoint 2 Passed

2.

Now change all instances of "LARRY" to "LAERTES" in **scene-7.txt**.

Checkpoint 3 Passed

3.

Add the files to the staging area together using a single git command.

How to Backtrack

git reset I

Great! The files you've added to the staging area belong in the same commit.

What if, before you commit, you accidentally delete an important line from **scene-2.txt**? Unthinkingly, you add **scene-2.txt** to the staging area. The file change is unrelated to the Larry/Laertes swap and you don't want to include it in the commit.

We can *unstage* that file from the staging area using

```
git reset HEAD filename
```

Copy to Clipboard

This command *resets* the file in the staging area to be the same as the `HEAD` commit. It does not discard file changes from the working directory, it just removes them from the staging area.

Instructions

Checkpoint 1 Passed

1.

To try out the new command, let's make a mistake on purpose!

The code editor is open to **scene-2.txt**. Delete any line from the file and click Run.

Checkpoint 2 Passed

2.

From the terminal, add **scene-2.txt** to the Git staging area.

Checkpoint 3 Passed

3.

Now check the `status` of the Git project.

In the output, notice **scene-2.txt** under "Changes to be committed".

Checkpoint 4 Passed

4.

Use the new Git command to *unstage* **scene-2.txt** from the staging area.

Notice in the output, "Unstaged changes after reset":

```
M    scene-2.txt
```

Copy to Clipboard

`M` is short for "modification"

Checkpoint 5 Passed

5.

Now that changes made to **scene-2.txt** have been booted out of the staging area, you're ready to commit. From the terminal, make a commit to save the Larry/Laertes name swap in **hamlet**.

2 - `git add scene-2.txt`

3 - `git status`

4 `git reset HEAD scene-2.txt`

How to Backtrack

git reset II

Creating a project is like hiking in a forest. Sometimes you take a wrong turn and find yourself lost.

Just like retracing your steps on that hike, Git enables you to rewind to the part before you made the wrong turn. You can do this with:

```
git reset commit_SHA
```

Copy to Clipboard

This command works by using the first 7 characters of the SHA of a previous commit. For example, if the SHA of the previous commit is `5d692065cf51a2f50ea8e7b19b5a7ae512f633ba`, use:

```
git reset 5d69206
```

Copy to Clipboard

HEAD is now set to that previous commit.

Instructions

Checkpoint 1 Passed

1.

From the terminal, print out your Git commit log.

Note: If your cursor gets stuck in “git log” mode in the terminal, press q on your keyboard to escape.

Checkpoint 2 Enabled

2.

From the terminal, enter the command to `reset` to a previous commit, using the first 7 characters of one of the past commit SHAs in your Git log.

Next, print the Git commit log again.

Notice anything interesting? The commits that came after the one you reset to are gone. The `HEAD` commit has been reassigned. You just changed history.

Make sure you don’t use the SHA of the most recent commit, as this will not result in anything resetting! You’ll need to use the SHA of a *previous* commit. You can use the dates in the log to determine the order in which the commits were published.

[How to Backtrack](#)

git reset II

Creating a project is like hiking in a forest. Sometimes you take a wrong turn and find yourself lost.

Just like retracing your steps on that hike, Git enables you to rewind to the part before you made the wrong turn. You can do this with:

```
git reset commit_SHA
```

Copy to Clipboard

This command works by using the first 7 characters of the SHA of a previous commit. For example, if the SHA of the previous commit is `5d692065cf51a2f50ea8e7b19b5a7ae512f633ba`, use:

```
git reset 5d69206
```

Copy to Clipboard

HEAD is now set to that previous commit.

Instructions

Checkpoint 1 Passed

1.

From the terminal, print out your Git commit log.

Note: If your cursor gets stuck in “git log” mode in the terminal, press q on your keyboard to escape.

Checkpoint 2 Passed

2.

From the terminal, enter the command to `reset` to a previous commit, using the first 7 characters of one of the past commit SHAs in your Git log.

Next, print the Git commit log again.

Notice anything interesting? The commits that came after the one you reset to are gone. The `HEAD` commit has been reassigned. You just changed history.

Make sure you don’t use the SHA of the most recent commit, as this will not result in anything resetting! You’ll need to use the SHA of a *previous* commit. You can use the dates in the log to determine the order in which the commits were published.

1 - git log

2 - git reset “write 7 characters from git log (commit ref.)”

2 - git log [again]

[How to Backtrack](#)

git reset review

To better understand `git reset commit_SHA`, notice the diagram on the right. Each circle represents a commit.

Before reset:

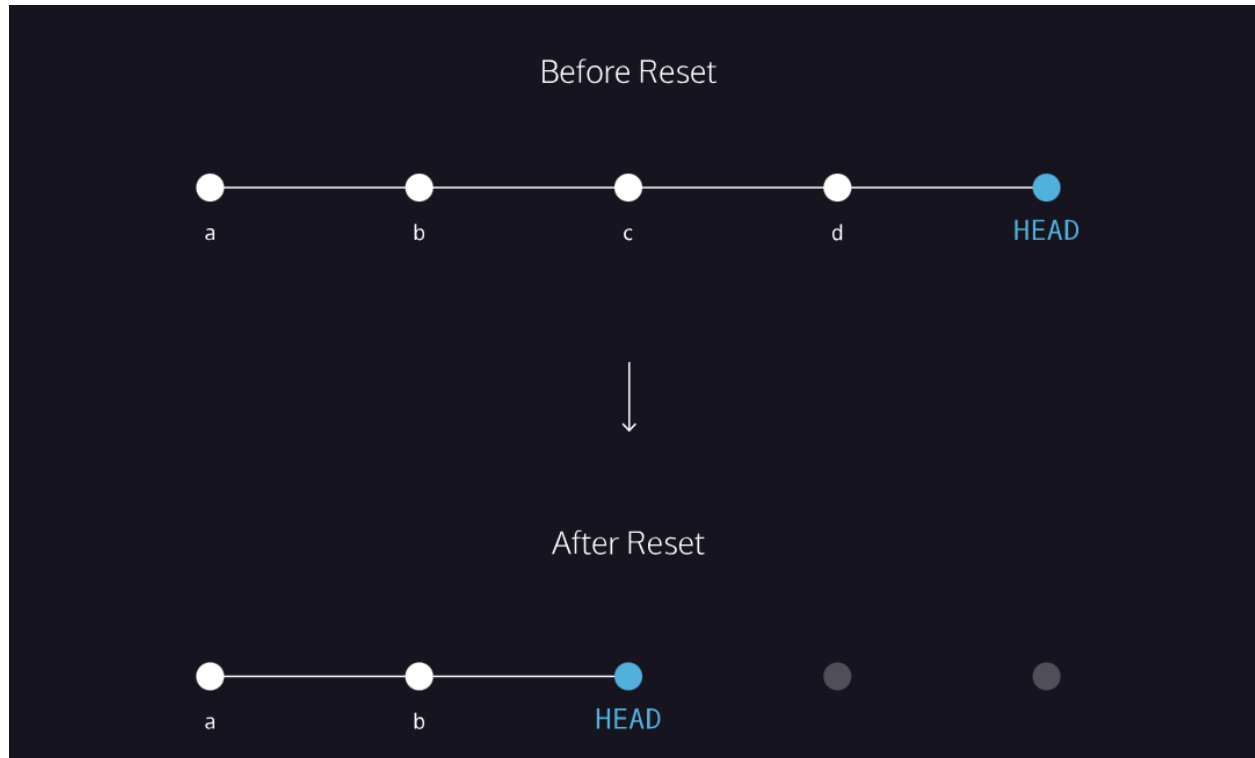
HEAD is at the *most recent commit*

After resetting:

HEAD goes to a *previously made commit* of your choice

The gray commits are no longer part of your project

You have in essence rewound the project's history



How to Backtrack

Generalizations

Congratulations! You've learned three different ways to backtrack in Git. You can use these skills to undo changes made to your Git project.

Let's take a moment to review the new commands:

`git checkout HEAD filename`: Discards changes in the working directory.

`git reset HEAD filename`: Unstages file changes in the staging area.

`git reset commit_SHA`: Resets to a previous commit in your commit history.

Additionally, you learned a way to add multiple files to the staging area with a single command:

```
git add filename_1 filename_2
```

Copy to Clipboard

Instructions

Click Next to finish the lesson!


```
IN: git init
    OUT: Initialized empty Git repository in /home/ccuser/workspace/sorcerers-code/.git/

IN: git status
    OUT:

IN: git add filename

    OUT:

IN: git commit -m "name the change which you did"
    OUT:

IN: git log
    OUT:

IN:
    OUT:

IN:
    OUT:
```

Copy to Clipboard

Handy Git Operations

Git provides us with a vast number of different commands that are listed on the [documentation](#) which can be intimidating at first. We will break down a couple that are powerful for daily tasks.

Introduction

Git provides us with a vast number of different commands that are listed on the [documentation](#) which can be intimidating at first. We will break down a couple that are powerful for daily tasks.

Git stash

Let's say you're working on experimental code on a fresh branch and realize that you forgot to add something to a previous commit in order to continue your work. In order to go to a different branch, one must always be at a clean commit point. In this case you don't want to commit your experimental code since it's not ready but you also don't want to lose all the code you've been working on.

A good way to handle this is by using `git stash`, which allows you to get back to a clean commit point with a synchronized working tree, and avoid losing your local changes in the process. You're "stashing" your local work temporarily in order to update a previous commit and later on retrieve your work.

The flow when using `git stash` might look something like this:

While working on a file, you find a small bug in a separate file from a previous commit that needs to be fixed before you continue.

```
$ git stash
```

Running the command above will store your work temporarily for later use in a hidden directory.

At this point, you can switch branches and do work elsewhere.

Once the bug is fixed, you want to retrieve the code you were working on previously, you can "pop" the work that was stored when you used `git stash`.

```
$ git stash pop
```

From here, you can continue your work and commit it when ready.
Coming up we have a short video demo'ing this in action.

Git log

At this point you might be familiar with the command `git log`, which allows you to view the commit history of the branch you currently have checked out:

```
git log
commit 9a6ea47f5a5cce49274ac46f715ed2ed04972d95 (HEAD -> cg_feature_express_server_skeleton, origin/cg_feature_express_server_skeleton)
Author: Carlos Grijalva <grijalva.carlos@gmail.com>
Date: Wed Oct 20 15:34:03 2021 +0200

    Add .gitignore file

commit a92d0f0c54b4b21e7c4f5d32793a5a49bba0280a
Author: Carlos Grijalva <grijalva.carlos@gmail.com>
Date: Wed Oct 20 15:33:25 2021 +0200

    Add skeleton for an express server

commit 72d16eca350cc94dc3e5b27ed1551801780c1d33 (origin/main, main)
Merge: 6c263fd 4284d67
Author: Carlos Grijalva <grijalva.carlos@gmail.com>
Date: Wed Oct 20 15:11:44 2021 +0200

    Merge pull request #1 from CGrijalva90/cg_feature_new_file

    Added a new file

commit 4284d67a23c048becf5edfc3935ac8ea946891c3 (origin/cg_feature_new_file, cg_feature_new_file)
Author: Carlos Grijalva <grijalva.carlos@gmail.com>
Date: Wed Oct 20 14:30:28 2021 +0200

    Added a new file

commit 6c263fd1eea7cc9f8f1c3acf19cda13b1dc77c0a
Author: Carlos Grijalva <grijalva.carlos@gmail.com>
Date: Wed Oct 20 14:29:43 2021 +0200

    first commit
```

There are other ways you can use `git log` in order to view recorded changes. Here are a few examples:

```
git log --oneline
```

```
git log --oneline
9d8987c (HEAD -> cg_feature_express_server_skeleton, origin/cg_feature_express_server_skeleton) Add get request
9a6ea47 Add .gitignore file
a92d0f0 Add skeleton for an express server
72d16ec (origin/main, main) Merge pull request #1 from CGrijalva90/cg_feature_new_file
4284d67 (origin/cg_feature_new_file, cg_feature_new_file) Added a new file
6c263fd first commit
```

```
git log -S "keyword"
```

```
git log -S "Add"
```

```
git log -S "Add"
commit a92d0f0c54b4b21e7c4f5d32793a5a49bba0280a
Author: Carlos Grijalva <grijalva.carlos@gmail.com>
Date: Wed Oct 20 15:33:25 2021 +0200

    Add skeleton for an express server
```

```
git log --oneline --graph --graph
```

```
--oneline
```

```
~/CodecademyGit/Demo P cg_feature_express_server_skeleton
git log --oneline --graph
* 9d8987c (HEAD -> cg_feature_express_server_skeleton, origin/cg_feature_express_server_skeleton) Add get request
* 9a6ea47 Add .gitignore file
* a92d0f0 Add skeleton for an express server
* 72d16ec (origin/main, main) Merge pull request #1 from CGrijalva90/cg_feature_new_file
| \
| * 4284d67 (origin/cg_feature_new_file, cg_feature_new_file) Added a new file
|/
* 6c263fd first commit
```

Git commit amend

Git's `--amend` flag is extremely useful when updating a commit, it allows you to correct mistakes and edit commits easily instead of creating a completely new one.

Let's say you finish working on a lengthy feature and everything seems to be working fine so you commit your work. Shortly after, you realize you missed a few semicolons in one of your functions. You could technically create a new commit, but ideally, you want to keep all commits specific, clean, and succinct. To avoid creating a new one, you could create your changes, stage them with `git add` and then type the command `git commit --amend` to update your previous commit.

It's important to note that although it seems like `--amend` is simply updating the commit, what Git actually does is replace the whole previous commit. For this reason, when you execute the command `git commit --amend`, your terminal editor asks you to update your commit message:

```
1 Merge branch 'cg_feature_express_server_skeleton' of https://github.com/CGrijalva90/demo into cg_feature_express_server_skeleton
2 # Please enter the commit message for your changes. Lines starting
3 # with '#' will be ignored, and an empty message aborts the commit.
4 #
5 # Date:      Wed Oct 27 12:54:56 2021 -0400
6 #
7 # On branch cg_feature_express_server_skeleton
8 # Your branch is up to date with 'origin/cg_feature_express_server_skeleton'.
9 #
10 # Changes to be committed:
11 #   modified:   helpers.js
12 #
```

However, if you want to keep the same commit message, you can simply add the flag `--no-edit`:

```
$ git commit --amend --no-edit
```

Git alias commands

When grouping commands together, you can end up writing very long lines of Git commands in the terminal such as:

```
$ git log --pretty=format:"%h %s" --graph
```

Fortunately, Git offers a helpful feature that can make your Git experience simpler, easier, and more familiar: aliases.

If you have a set of commands that you use regularly and want to save some time from typing them, you can easily set up an alias for each command using Git config.

Below are a couple of examples:

```
$ git config --global alias.co "checkout"
$ git config --global alias.br "branch"
$ git config --global alias.glop "log --pretty=format:"%h %s" --graph"
```

Once the aliases are configured, next time you want to check out to another branch you could type the command:

```
$ git co example_branch
```

Instead of:

```
$ git checkout example_branch
```

Using Git aliases can create a much more fluid and efficient workflow experience when using Git. By getting creative with your aliases, you're able to wrap a sequence of Git commands into one in order to save time and effort.

What is GitHub?

Learn how to sign up for a GitHub account and share your code

What is GitHub?

One of the best ways to share what you've been learning with other people is to put your code on GitHub. GitHub is both a website and a service that facilitates software development by allowing you to store your code in containers, called *repositories*, and by tracking changes made to your code. In addition, it offers a hosting service and tools to build, test, and deploy code.

GitHub uses [Git, a version-control development tool](#), to manage your projects by tracking changes to files and allowing multiple people to work on the same project. Although both GitHub and Git have similar names, GitHub is a service, while Git is a development tool that can be used by you outside of and without GitHub.

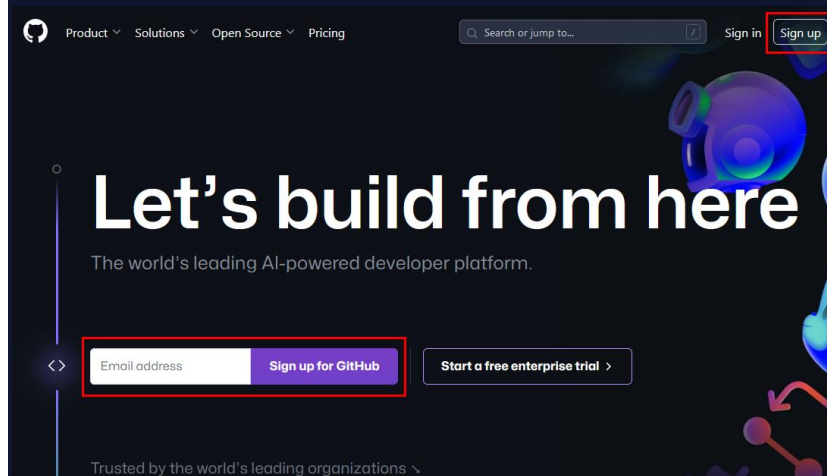
Why is GitHub important?

There are many reasons why knowing about GitHub is important for your personal growth as a developer. A large part of the appeal of Github is the access it grants developers to the massive community of developers around the world who openly share their code, projects, and software development tools with each other. Therefore, if you want to continue working on your Git skills, creating your programming portfolio, or finding work, GitHub can help.

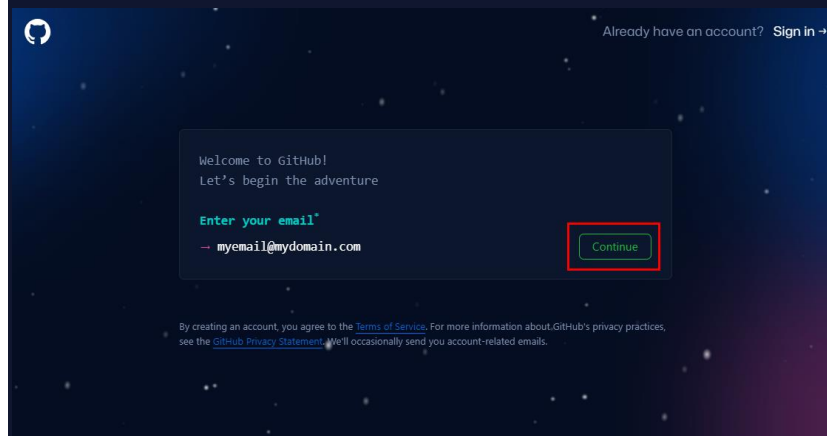
How to Sign Up for an Account

Now that you're aware of GitHub's benefits, you probably want to sign up for an account and try it out yourself. First, navigate to the home page of the GitHub website, <https://github.com>.

You can click on the **Sign Up** button in the upper right corner or fill out your email and click on **Sign up for GitHub**, as highlighted in this screenshot:

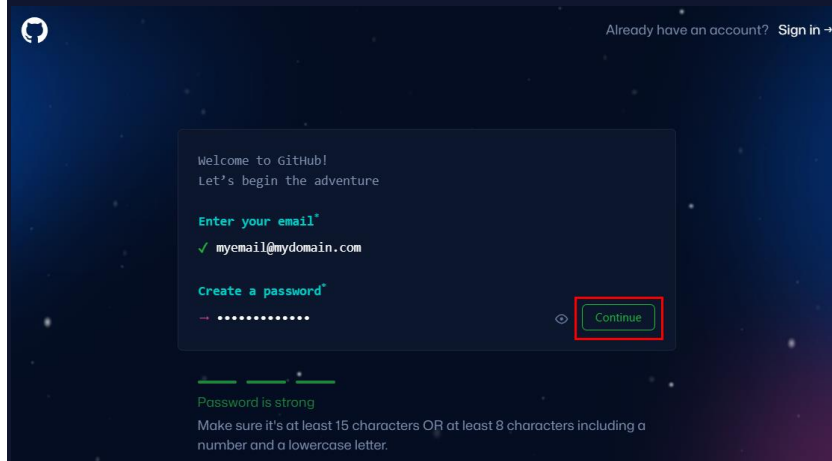


You'll be taken to a page asking you to fill out your email. If you did so on the home page, it will already be filled out for you. If it isn't filled out, enter the email you wish to use.

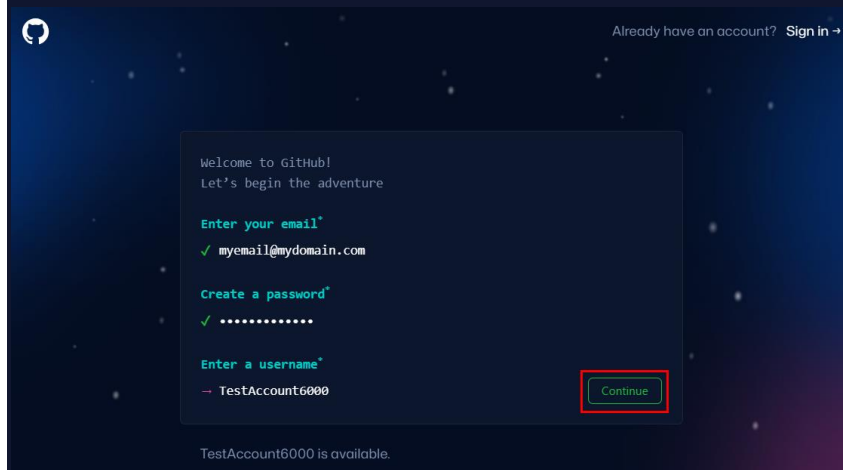


Pick an email that you're comfortable sharing with peers and potential hires. Because of the way Git works, it's important to note that your email can be exposed publicly when you make a pull request or merge in code to a repository, making it visible to anyone looking through your projects. When you sign up for a new GitHub account, your email address is hidden by default.

Once you have an email entered, click on "continue." GitHub will then ask you to create a password. Fill out a password for your new account.

A screenshot of the GitHub sign-up interface. At the top left is the GitHub logo, and at the top right is a link that says "Already have an account? Sign in →". The main content area has a dark background with a starry pattern. It contains a white box with the text "Welcome to GitHub! Let's begin the adventure". Below this, there are two input fields. The first is labeled "Enter your email*" and contains the text "myemail@mydomain.com" with a green checkmark to its left. The second is labeled "Create a password*" and contains a series of dots with a green checkmark to its left. To the right of the password field is a red-outlined box containing the word "Continue". Below the password field, there is a progress bar with three segments, the first two of which are green. Below the progress bar, it says "Password is strong" and "Make sure it's at least 15 characters OR at least 8 characters including a number and a lowercase letter."

When you click on "continue," GitHub will then ask you for a username. Fill out the user name you would like to use (you may need to try a few times to find one that's available).

A screenshot of the GitHub sign-up interface, showing the next step after password creation. The layout is similar to the previous screenshot. The "Enter your email*" field now has a green checkmark and the text "myemail@mydomain.com". The "Create a password*" field now has a green checkmark and a series of dots. The "Enter a username*" field contains the text "TestAccount6000" with a red-outlined box around the "Continue" button to its right. Below the username field, it says "TestAccount6000 is available."

Like with email addresses, when choosing a username, it's wise to choose one you wouldn't mind future employers or colleagues seeing. A combination of your first and last name, like `firstnamelastname`, or using initials, like `i_lastname`, are good because they make it easy to find you on GitHub or identify you when you make [pull requests](#) or [reviews](#). Remember, you're likely using this account to share or access code.

Also be aware that usernames are first come, first serve and may not be available if someone else already claimed the username. Additionally, usernames may only contain alphanumeric characters and hyphens are not allowed at the beginning or end.

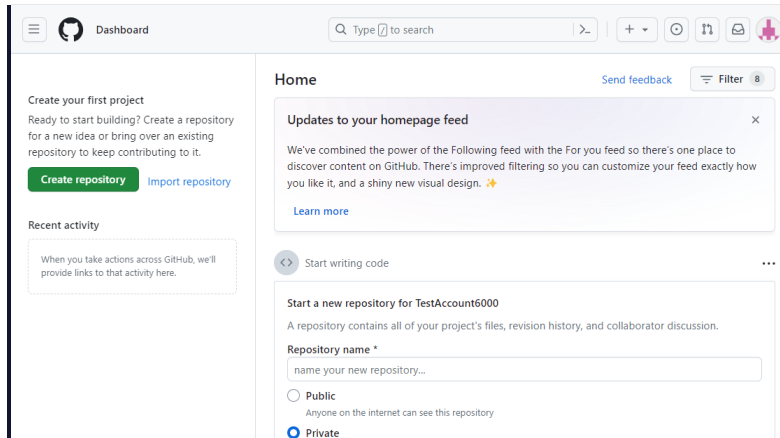
After you click on "continue," GitHub will prompt to ask if you want to receive product updates and announcements. Make your selection and click "continue," again. GitHub will then prompt you to solve a puzzle to prove you're a human being. Click on "Verify" to view the puzzle.

Once you submit the solution to the puzzle, a "Create Account" button will appear. Click on it and GitHub will prompt you for an 8-digit code. GitHub will send you the code to the email you entered earlier. Enter the code from the email and GitHub will create your account.

Settings

Once your account is created, GitHub will ask a couple of questions to personalize your experience. Fill these out or click on the "skip personalization" link at the bottom of the page.

You should now see a personal dashboard with a section for your projects and some messages:



Now you have your own GitHub account! You can continue to customize your account by:

[setting your email to private](#)
[setting up two-factor authentication](#)

That's it; you now have your very own GitHub account. ☺

Recap

With more people working remotely and with teams distributed across different countries and time zones, GitHub and Git can be valuable tools for collaborating on projects. You can also use GitHub to work on any file-based project, such as writing documentation.

Let's review what you did in this article:

Learned that GitHub is both a website and service for storing and sharing code
Learned that GitHub uses Git to facilitate software development by tracking changes
Created your own GitHub account
Enabled security features like keeping your email private and turning on two-factor authentication

Going Further

Once you feel comfortable navigating GitHub, consider doing the following:

[Add information about yourself](#) in your profile, including an avatar, bio, location, etc.
[Set your status](#) in your profile to let people know what you're doing.
If you feel comfortable enough with Git, you can take a look at [this article on GitHub Pages](#), GitHub's hosting service that allows you to create a personal website, "project site", based on a repository to make your portfolio site.
If you're interested in paid accounts, GitHub has a [pricing page](#) with various types of accounts and features you can look at.

Show the world what you can do with your code!

Set Up with Git and GitHub

Learn how to install Git, configure it, and connect to GitHub to manage your code with version control and collaboration.

In this tutorial, we walk through the process for using Git locally on your personal computer and using GitHub to back it up. Specifically, we'll walk through creating your personal GitHub account, setting up Git on your computer, starting your first Git repository, and connecting that repository to a GitHub repository.

This tutorial assumes that you've completed the lessons on [Learn the Command Line](#) and [Learn Git](#). Now, prepare to use those skills on your personal computer! If some steps in this tutorial are confusing, have no fear; it will all come together by the end.

What are Git and GitHub?

This tutorial refers to Git and GitHub repeatedly. *Git* is a widely-used version control system used to manage code. Git allows you to save drafts of your code so that you can look back at previous versions and potentially undo complicated errors. A project managed with Git is called a *Git repository*.

GitHub is a popular hosting service for Git repositories. GitHub allows you to store your local Git repositories in the cloud. With GitHub, you can backup your personal files, share your code, and collaborate with others.

In short, GitHub is a tool for working with Git. There are other services to host Git repositories, but GitHub is a trusted, free service used by organizations across the world, big and small.

Create a GitHub Account

To use GitHub, you will need a GitHub account.

In your own browser:

Open a new browser tab

Navigate to <https://github.com/>

Create an account

If you already have a GitHub account, continue to the next exercise.

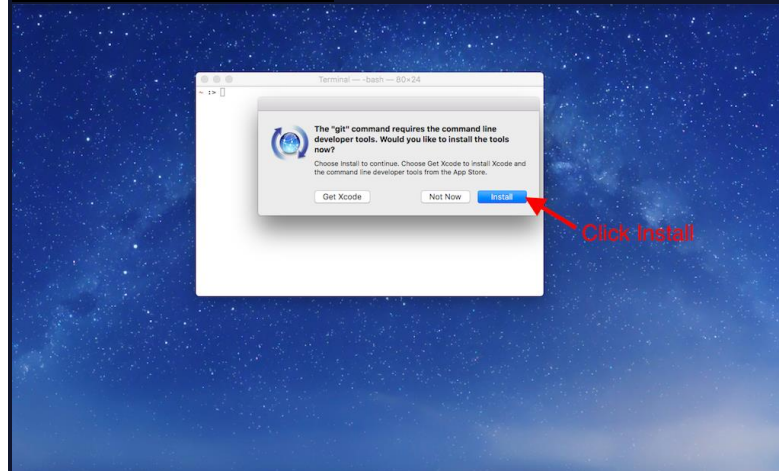
After you sign up, you will receive a verification e-mail. Be sure to verify your e-mail address to GitHub by following the instructions in that e-mail.

Git Setup for Mac and Windows

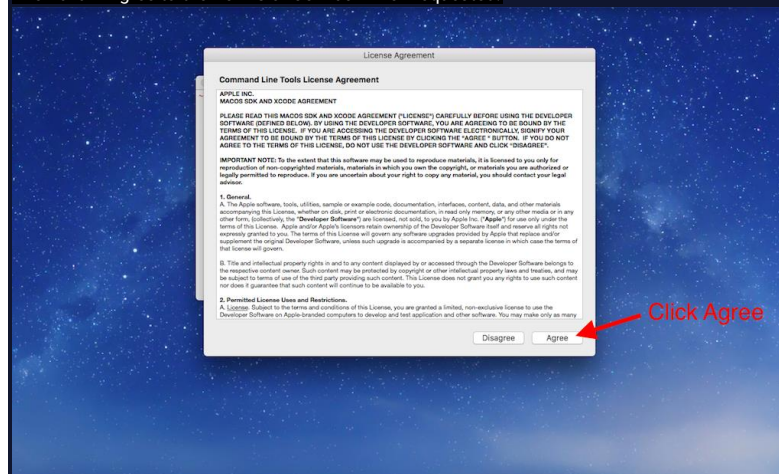
Next, we will set up Git on your personal computer. Follow the instructions for your operating system.

Mac users:

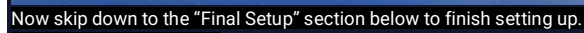
1. Launch the **Terminal** application. You can find it in **/Applications/Utilities/**. You can also use the **Spotlight** search tool (the little magnifying glass in the top right of your screen) to search for **Terminal**. Once **Spotlight** locates it, click on the result that says **Terminal**.
2. When **Terminal** opens, type in `git` and press enter.
3. If you don't already have Git installed, a dialog will appear saying that "The 'git' command requires the command line developer tools. Would you like to install the tools now?" Click "Install".



Then click "Agree to the Terms of Service" when requested.



4. When the download finishes, the installer will go away on its own signifying that Git is now installed! Click "Done" to finish the installation process.

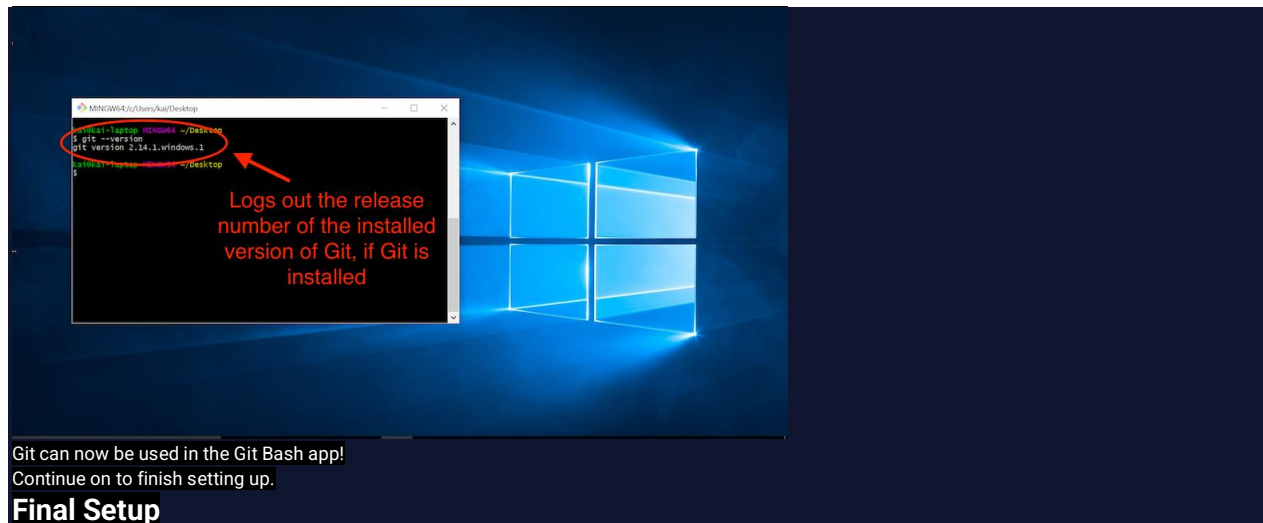


This portion of the guide assumes you have already installed a program called Git Bash which allows us access to Git on Windows. If you have not installed Git Bash, please refer to the previous tutorial on Command Line Interface (CLI) Setup and follow the instructions for installing Git Bash on Windows. Once you complete that you can continue with this guide.

-
- A screenshot of the Windows Start menu search interface. The search bar at the top contains the text "git bash". Below the search bar, a list of search results is displayed. The first result, "Git Bash", is highlighted with a red circle. Below it, the text "Desktop app" is visible. At the bottom of the search results, the text "Search for Git Bash" is written in red, with a red arrow pointing to the search bar.

The screenshot shows a Windows 10 desktop with the Windows logo wallpaper. A terminal window titled "kali@kali-laptop: ~/Desktop" is open. The terminal displays the command `cd /mnt/c/Users/raj/Desktop` and the prompt `raj@kali-laptop: ~/Desktop`. Two red arrows point from text labels to the terminal: one points to the command with the label "Name of the current directory", and the other points to the prompt with the label "Name of the current device".

2. If you want to make sure that Git is installed, run `git --version` in the CLI. You should see a response that gives you the version of Git installed. It will look like this:



Final Setup

Navigate to GitHub's articles on setting up your [Git username](#) and [email](#) and follow the instructions for each using Git Bash.

GitHub offers two authentication options, HTTPS and SSH, to keep your work secure. This is a security measure that prevents anyone who isn't authorized from making changes to your GitHub repository. In this article, we will use HTTPS.

To authenticate yourself in the terminal, you will need to generate a *Personal Access Token* on GitHub. Navigate to GitHub's article on [creating a personal access token](#) and follow the instructions for creating a classic token. Be sure to check the box that grants the token *repo* scope — this will allow you to write to the repository from the terminal on your machine.

Once you get the token, make sure to copy it or keep the page open — you will not be able to view the token again once you navigate away from the page and you'll need it later in the instructions!

Try It Out!

Now you have everything you need to practice your Git skills on your local computer. Take a moment to run the commands below to initialize a Git repository. We will use this Git repository again later in this tutorial so make sure you complete these steps exactly as described.

`mkdir git_practice` to make a new directory to practice.

`cd git_practice` to make the new directory your working directory.

`git init` to turn the current, empty directory into a fresh Git repository.

`echo "Hello Git and GitHub" >> README.txt` to create a new README file (more on this later) with some sample text.

`git add README.txt` to add the new file to the Git staging area.

`git commit -m "First commit"` to make your first commit with the new README file.

Your First Remote Repository on GitHub

Finally, we'll create a repository on GitHub and then link it to a local repository on your computer. This allows you to backup your work constantly and safely, so you never need to worry about losing your work again!

Now, let's connect our local Git repository to GitHub.

Instructions

1. In your Command Line Interface, make sure your current working directory is your new Git repository. Navigate there if not.
2. Check the status of which files and folders are new or have been edited. There should be no files modified.

```
$ git status
```

3. On GitHub, create a new repository by clicking the **New repository** button on the home page.
4. On the new repository page, give your repository a name. It's not necessary, but it would be convenient to name it the same as the directory,

`git_practice`. After naming the repository, click **Create repository**.

5. After creating a repository, GitHub displays the repository page. At the top of the page, make sure "HTTPS" is selected.

6. The repository is empty, so it's time to connect it to your existing work. Copy the Git commands on the GitHub page, under the title "...or push an existing repository from the command line", and paste them into your Command Line Interface. Running these commands will add a remote repository, and then push your local repository to the remote repository.

When asked for a username and password, type in your GitHub username as the username and paste your personal access token as the password, pressing `Enter` (or `return`) after each. Don't be alarmed if you can't see the characters you are typing, they are intentionally hidden as a security

measure.

7. Once your Command Line Interface reports that the push is complete, refresh the page on GitHub. You should now see the text you wrote earlier in the README file, "Hello Git and GitHub."

GitHub automatically displays the contents of a file named **README.txt** if it exists in the repository. The README file is the perfect place to write a description of your project.

There you have it! Your first GitHub repository, linked to your local Git repository. You've taken some huge leaps, so be proud! Now you can use your knowledge of Git to track progress on your local computer and push that progress to GitHub whenever you want. You can rest easy knowing that each step of your progress is safely stored in GitHub.

Getting Started with Git and GitHub (Videos)

This article introduces the basic concepts of Git and GitHub, and how to get started setting up Git on your computer for both Mac and Windows.

The videos below will walk you through setting up Git and GitHub on your computer for both Mac and Windows so that you can start to use version control and collaboration tools like a pro.

Mac OSX

Windows PCs

Mac OSX

<https://youtu.be/tMDbJ0DDuug>

(Set up github account on MAC)

Windows PCs

<https://youtu.be/-sMmrYTtQgE>

(Set up github account on WINDOWS)

Starting a Code Base on GitHub

<https://youtu.be/zF XuHc9AUzq>

How to Push Code to GitHub

<https://youtu.be/wrb7Gge9yoE>

The GitHub Flow

Introduction

In this lesson, we'll discuss the flow of using

[GitHub](#)

Preview: Docs Loading link description

with your code development process. When working on a project, things can scale rather quickly. Following a specific workflow allows the project to move in more orderly way.

Remember that Git is defined as a

[version control](#)

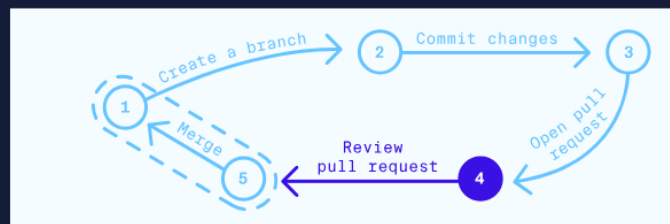
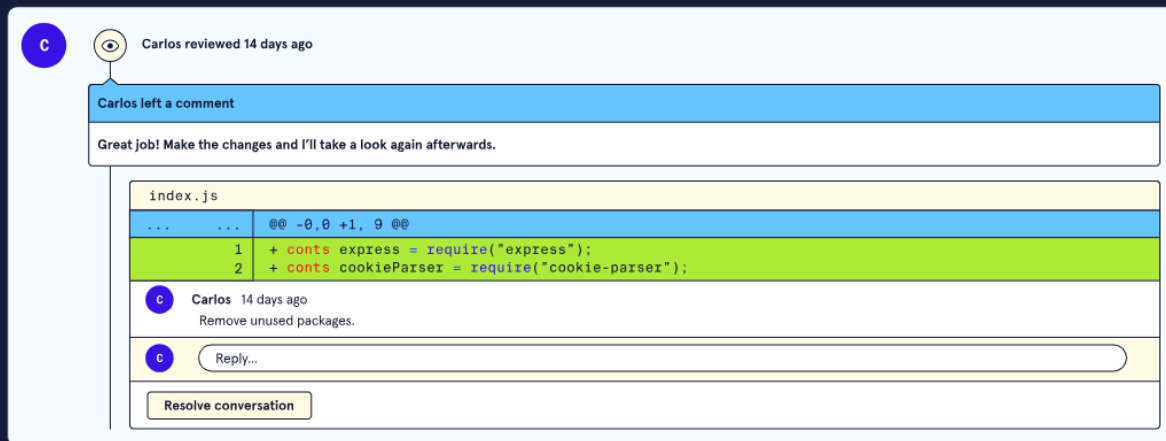
Preview: Docs Loading link description

system that's maintained on your local system. It helps you manage and record your source code history. GitHub, on the other hand, is a hosting service for repositories. Simply put, Git is the tool and GitHub is the service for projects that use Git.

In this lesson we'll take a look at the basic workflow used with GitHub, which goes like this:

- Create a branch
- Commit changes
- Create a pull request
- Review pull request
- Merge and delete branch

By sticking to that workflow, team members are able to isolate their work and avoid any conflicting code from being merged. In the following lesson, we'll take a deeper look at each of these steps.



Managing Branches

Whenever we're working on a team creating multiple versions of a project code, it's important to isolate each teammate's work in order to avoid any conflicts. With Git, each teammate can create their own branch off of the main project in order to work on bug-fixes, new features, experimental code, etc.

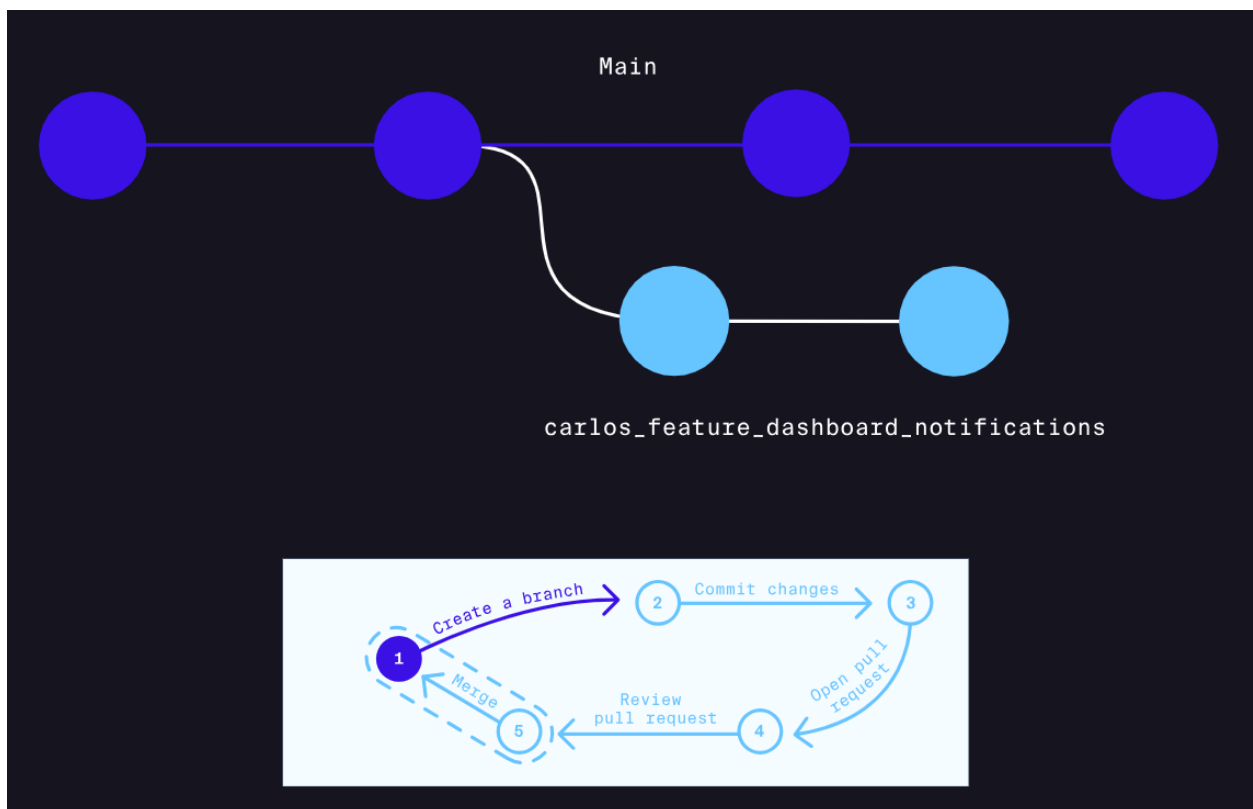
A branch is essentially a divergence from the main project. When you branch out, git is essentially making a new state of your current code, upon which you can work, without affecting the important main state of the code. One can create as many branches as they wish and even create branches off of other branches.

By using separate branches, the main project remains intact and unaffected before the changes are reviewed and merged into the project.

Each repository can have one or more branches. The main branch — the one where all changes eventually get merged back into, is called `main`. The `main` branch is usually the working version of a project and contains the production code, so it's very important to only merge clean and correct code into it!

When someone wants to create a new feature, fix a bug, or just experiment, they should always create their own branch with a descriptive name.

Each team will adopt their own best practices when working together and figuring out naming conventions. For example, the branch name `carlos feature dashboard notifications` includes the author, branch type, and short branch description. Other teams may pick branch names to correspond to ticket numbers from their project management tool.



Adding and Committing Changes

Let's assume you were recently assigned to a team to develop a feature for an app. You clone (download) the entire app repository from

[GitHub](#)

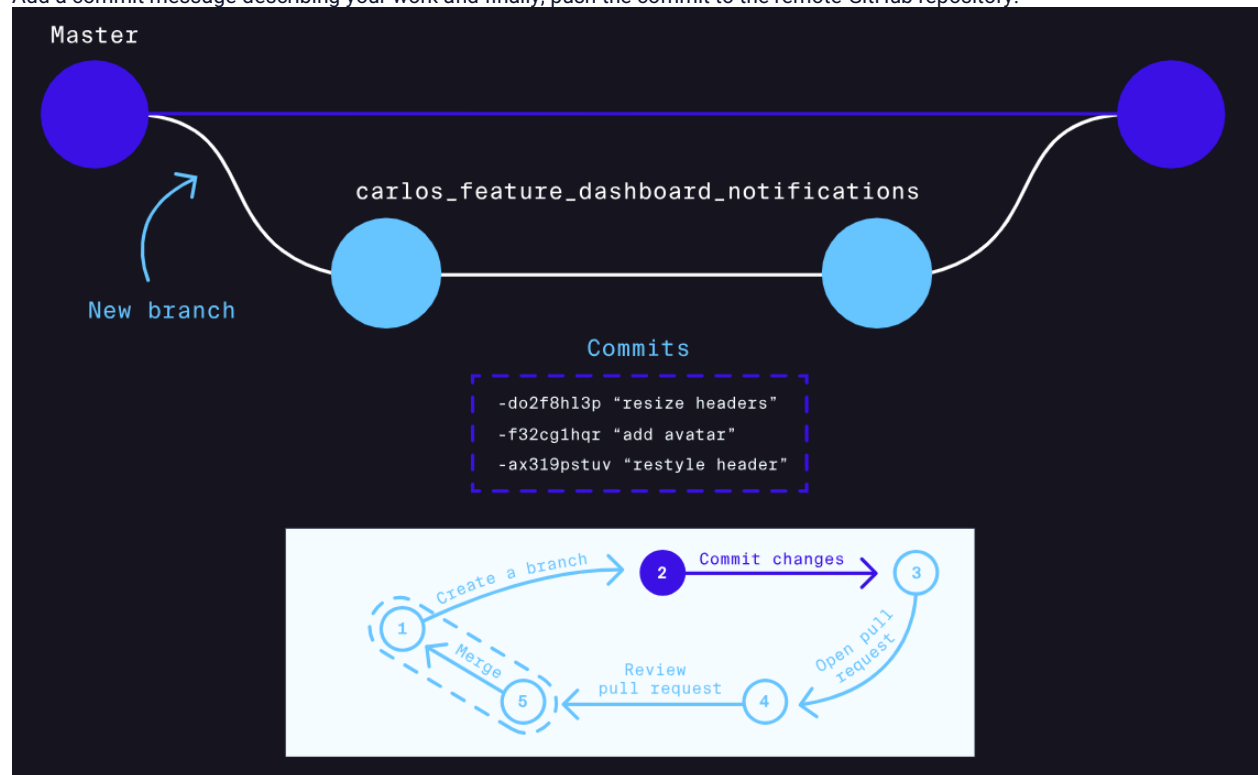
Preview: Docs Loading link description

, create a new branch from the `main` branch for your feature, and begin coding a new file in your local Git environment.

After testing your code and ensuring that everything is running correctly, it's time to commit the changes and push them to the remote repository!

As a refresher, the `git commit` command records changes to one or more files in your branch, assigning the commit a unique ID that identifies who created the changes, what changes were made, and *when* the changes were committed.

Add a commit message describing your work and finally, push the commit to the remote GitHub repository.



The GitHub Flow

Creating a Pull Request

At this point, your work is ready to be reviewed before it's integrated into the official project. You'll start by opening a pull request.

Pull requests on

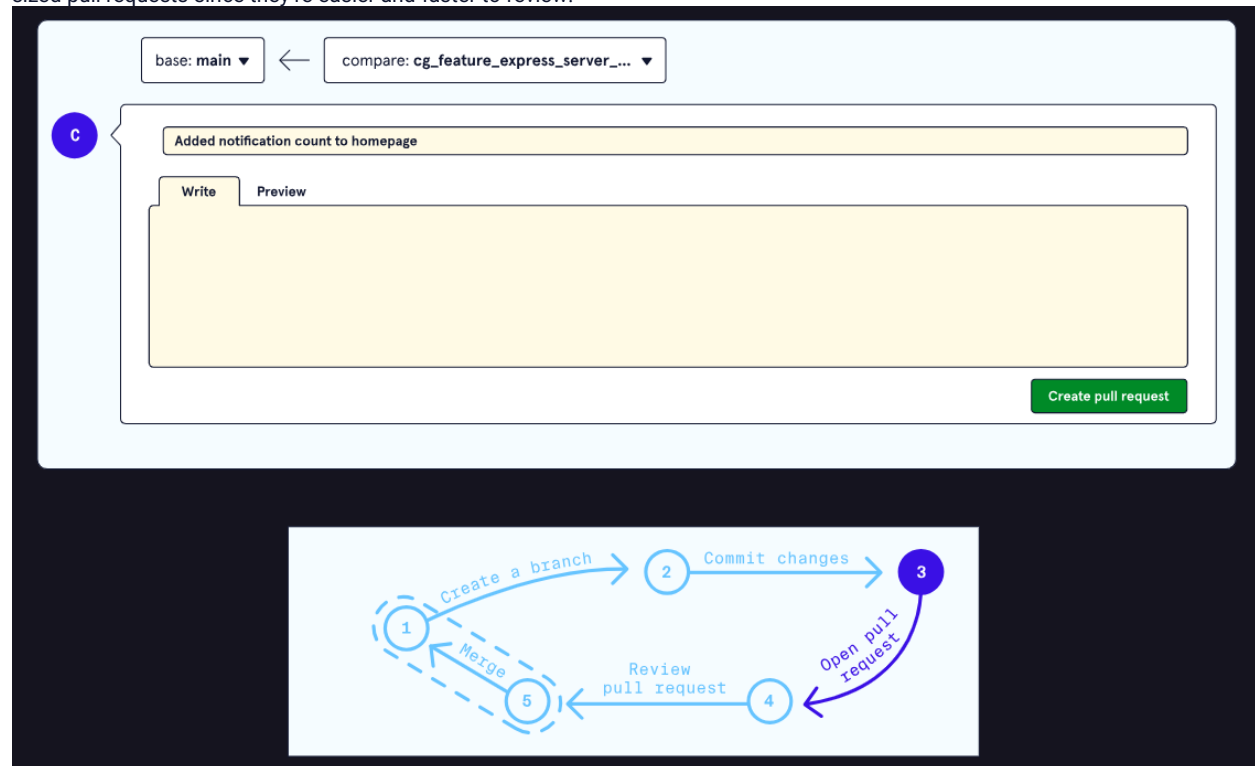
[GitHub](#)

Preview: Docs Loading link description

allow collaborators to review and give feedback on proposed code changes before they are merged into the main branch. Through a process of discussion and potentially some extra code changes, the pull request can be ultimately approved, which means you can merge the changes into the official project on the `main` branch.

When creating pull requests, it's imperative that you include as much relevant detail in the description as possible in order to save review time. Add any comments or images that might be useful for your reviewer.

It's also essential to ensure that your code is running properly with the updated repository in order to prevent anything from crashing. Lastly, you don't want to submit a pull request with 50 files containing a plethora of changes — instead, stick to smaller-sized pull requests since they're easier and faster to review.



The GitHub Flow

Reviewing and Merging a Pull Request

Once you've created a pull request, other members in your team can review it up on

[GitHub](#)

Preview: Docs Loading link description

.

The pull request should include a description and GitHub will display all the files with the changes created. Each line of code will have a clickable "+" button where you can add a

[comment](#)

Preview: Docs A comment is a note or explanation in the source code of a computer program. They are added with the purpose of making the code easier for ourselves or other developers to understand in the future, and they are generally ignored by compilers and interpreters.

in regards to the line.

While reviewing, it's important to be constructive with feedback and be precise about what needs to be changed. Here are few best practices when reviewing code:

Don't only comment on *what* should be changed, but *why* it should be changed. Feel free to provide resources to make your point.

Be as clear as possible with your comments and make sure to be clear as to what to modify.

Look at the bigger picture and try to spot potential errors. Would the submitted code produce any obstacles if the project scales?

Once all the feedback is added, collaborators can click on “Submit Review” and wait for a response. If all goes well, the pull request will eventually be merged into `main`!

The top part of the image is a screenshot of a GitHub pull request review interface. It shows a comment from 'Carlos' 14 days ago: 'Great job! Make the changes and I'll take a look again afterwards.' Below the comment is a code diff for `index.js`. The diff shows two lines being added (highlighted in green):
1 `+ const express = require("express");`
2 `+ const cookieParser = require("cookie-parser");`
Below the code diff, there is another comment from 'Carlos' 14 days ago: 'Remove unused packages.' At the bottom of the review section, there is a 'Reply...' input field and a 'Resolve conversation' button.

The bottom part of the image is a diagram illustrating the GitHub workflow. It consists of five numbered steps connected by arrows:
1. 'Create a branch' (from step 1 to step 2)
2. 'Commit changes' (from step 2 to step 3)
3. 'Open pull request' (from step 3 to step 4)
4. 'Review pull request' (from step 4 to step 5)
5. 'Merge' (from step 5 back to step 1)
Steps 1, 2, and 3 are enclosed in a dashed oval, indicating a sequence of actions to create and commit changes.

The GitHub Flow

Deleting a Branch and Review

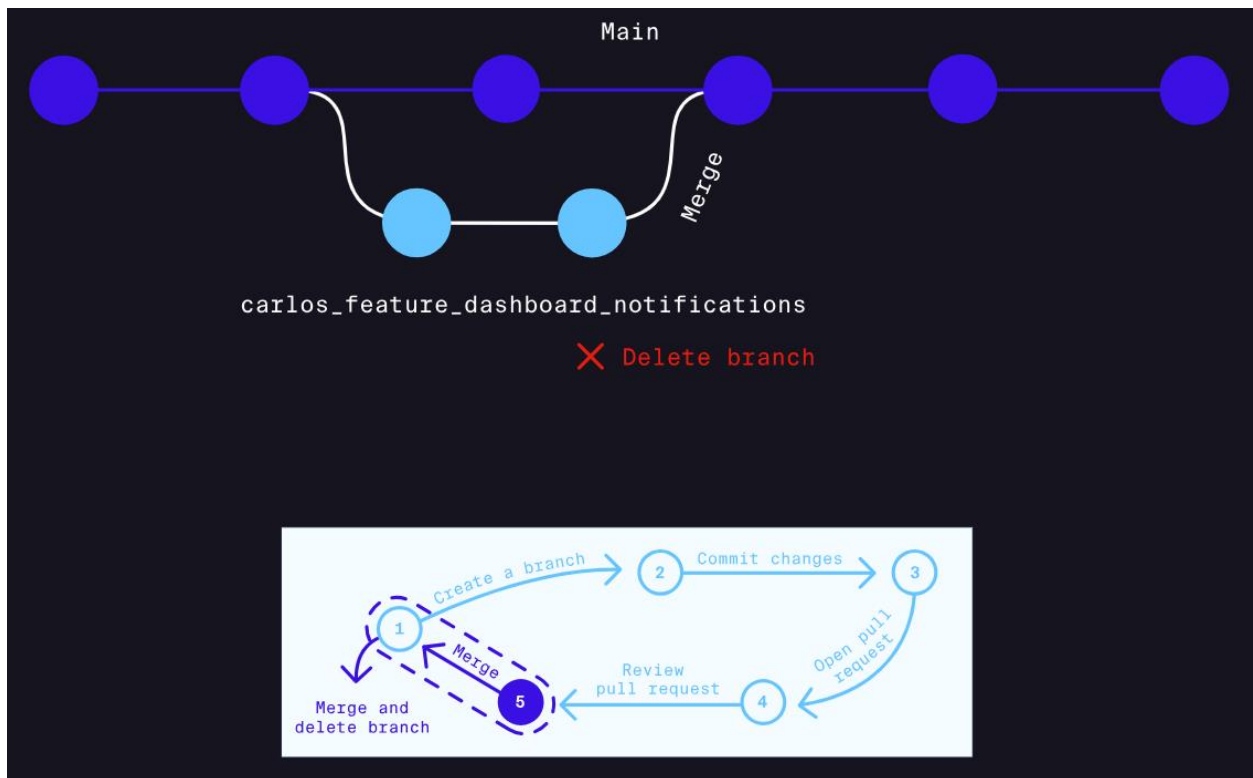
Once changes are merged, in order to keep things organized and managed, it's imperative to only keep active branches and delete the closed ones.

With that in place, this wraps up the flow of working on a project using Github. We explored:

- The importance of creating branches and isolating work from the `main` branch.
- Best practices of naming branches and making commits on branches.
- What a pull request is: a discussion page for a set of code changes between one branch and another.
- Merging a branch and delete it once it's merged.

This covers the main steps of working with a team and managing the workflow using Github.

Github provides us with a number of useful tools that expand on Git functionality, especially if we're collaborating with teammates!



Exceptions

Introduction to Exceptions

It is truly an amazing feeling when our code works exactly the way we want it to. On the other hand, it can be equally frustrating when our code runs into errors. Since errors are such an integral part of working with Python, it's important to know how to control errors and use them to our advantage effectively. In this lesson, we will explore a specific type of error, an *exception*.

At this point, we are probably very familiar with the most common type of error: a *syntax error*. Syntax errors are mistakes in the structure of Python code. They are caught during a special parsing stage before a program is executed. They always prevent the entire program from running. For example, here is the error output of a syntax error:

```
File "script.py", line 1
  def print_five
    ^
SyntaxError: invalid syntax
```

As opposed to a syntax error, an exception is a different kind of error that can occur with syntactically correct code. Exceptions are *runtime errors* because they occur during program execution, only when the offending code (the code causing the error) is reached. An example of an exception, and one we have probably seen before, is a `NameError`:

```
Traceback (most recent call last):
  File "script.py", line 1, in <module>
    print(five)
NameError: name 'five' is not defined
```

Although the `NameError` has a similar output to a `SyntaxError` (both end with

`Error`

Preview: Docs Loading link description

), it falls under the category of exceptions. Exceptions and syntax errors make up the two core categories for any error we will run into.

We'll encounter many different kinds of exceptions, some of which will be unfamiliar. Luckily, as we saw in the example above, Python gives us a tool for gaining insight into exceptions - the *traceback*. A traceback is a summary that includes the exception type, a message, and the series of function calls preceding the exception, along with file names and line numbers. Here is another example of a traceback for a small program:

```
# Imaginary file script.py
print(1/0)
```

Copy to Clipboard

Would output:

```
Traceback (most recent call last):
  File "script.py", line 1, in <module>
    print(1/0)
ZeroDivisionError: division by zero
```

In the traceback above, reading from the bottom line, we see the exception type (`ZeroDivisionError`) followed by a message (division by zero). Going up, we see that the exception originated on line 1 of a file called `script.py` while calling

`print(1/0)`. We'll be using tracebacks throughout the rest of the lesson to track and identify why and where our exceptions are occurring.

Let's get some practice debugging syntax errors and exceptions. For this lesson, let's imagine we are hired by Instrument World, a musical instrument company with retail and online stores.

Instructions

Checkpoint 1 Passed

1.

Take a look at the code in `welcome.py`. There is a syntax error on line 3 (the extra closing parenthesis). Will any of the previous code be executed? Press "Run" to find out.

Checkpoint 2 Passed

2.

Fix the error from the extra closing parenthesis from line 3. Run the code to see what happens.

Checkpoint 3 Passed

3.

We hit an exception on line 3 because of the misspelled variable name! However, the previous lines of code were executed (observe `Welcome to` in the output) because exceptions occur at runtime.

Fix the variable name on line 3 to continue!

The variable `store` is misspelled on line 3!

Exceptions

Built-in Exceptions

In the previous exercise (and probably many times before), we saw one type of exception called the `NameError`. The `NameError` is just one of the many *built-in exceptions* — exceptions that are built into the Python language. Other built-in exceptions cover fields ranging from mathematical errors all the way to operating system errors. We don't need to memorize them all, but it's helpful to be familiar with some common ones and, more importantly, understand where they come from inside Python.

Exceptions are objects just like anything else. Most exceptions inherit directly from a class called `Exception`; however, they all are derived directly or indirectly from the `BaseException` class. We can examine the base classes by using the `__bases__` attribute on any specific exception:

```
print(NameError.__bases__)
```

Copy to Clipboard

Will output:

```
<class 'Exception'>
```

Copy to Clipboard

We can even call `__bases__` on the `Exception` class to see its origins:

```
print(Exception.__bases__)
```

Copy to Clipboard

Will output:

```
<class 'BaseException'>
```

Copy to Clipboard

The full hierarchy of built-in exceptions is the following:

```
BaseException
+-- Exception
    +-- StopIteration
    +-- StopAsyncIteration
    +-- ArithmeticError
    |   +-- FloatingPointError
    |   +-- OverflowError
    |   +-- ZeroDivisionError
    +-- AssertionError
    +-- AttributeError
```

```

+-- BufferError
+-- EOFError
+-- ImportError
|   +-- ModuleNotFoundError
+-- LookupError
|   +-- IndexError
|   +-- KeyError
+-- MemoryError
+-- NameError
|   +-- UnboundLocalError
+-- OSError
|   +-- BlockingIOError
|   +-- ChildProcessError
|   +-- ConnectionError
|       |   +-- BrokenPipeError
|       |   +-- ConnectionAbortedError
|       |   +-- ConnectionRefusedError
|       |   +-- ConnectionResetError
|   +-- FileExistsError
|   +-- FileNotFoundError
|   +-- InterruptedError
|   +-- IsADirectoryError
|   +-- NotADirectoryError
|   +-- PermissionError
|   +-- ProcessLookupError
|   +-- TimeoutError
+-- ReferenceError
+-- RuntimeError
|   +-- NotImplementedError
|   +-- RecursionError
+-- SyntaxError
|   +-- IndentationError
|   +-- TabError
+-- SystemError
+-- TypeError
+-- ValueError
|   +-- UnicodeError
|       +-- UnicodeDecodeError
|       +-- UnicodeEncodeError
|       +-- UnicodeTranslateError

```

Copy to Clipboard

Note that there are **a lot** of exceptions built into the language of Python. Again, we don't need to memorize all of them, but at some point, we may see them pop up in our programs. We can find details on each of the exceptions listed above in the [Python documentation](#).

Later in this lesson, we'll be using the `Exception` base class to create custom exceptions. For now, let's get some practice encountering built-in exceptions and reading their tracebacks.

Instructions

Checkpoint 1 Passed

1.

Instrument World has a program that prints some of the most popular instruments it has on sale.

Take some time to look over the program, and then run the code in `instruments.py`. We'll encounter another exception that we might not have seen before. What might this exception be telling us?

A `TypeError` can sometimes occur when we try to concatenate a string and an integer.

Checkpoint 2 Passed

2.

Take a look over traceback - it ends with the exception type and a brief message. Above that is the exact line that caused the exception.

Fix this line so that the exception no longer occurs and then re-run the code.

The `len()` function returns an integer type, which cannot be concatenated with strings. Use the `str()` built-in function to convert integers into a string.

Checkpoint 3 Passed

3.

Looks like the exception we saw in the previous step was a `TypeError`. Let's confirm which base class it is derived from. Use `print()` to output a `TypeError.__bases__`.

The `__bases__` property can be examined on any error that occurs. To examine it, we simply attach it to the end of an exception:

```
Exception.__bases__
```

Copy to Clipboard

Checkpoint 4 Passed

4.

There is another exception that gets hit - once again, read the traceback, fix the exception, and re-run the code.

An `IndexError` commonly occurs when trying to access an index that does not exist. The index of the last element in `sale_instruments` is 2.

Exceptions

Raising Exceptions

Encountering exceptions isn't always an accident. We can throw an exception at any time by using the `raise` keyword, even when Python would not normally throw it.

We might want to raise an exception anytime we think a mistake has or will occur in our program. This lets us stop program execution immediately and provide a useful error message instead of allowing mistakes to occur that may be difficult to diagnose at a later point.

One way to use the `raise` keyword is by pairing it with a specific exception class name. We can either call the class by itself or call a constructor and provide a specific error message. So for example we could do:

```
raise NameError
# or
raise NameError('Custom Message')
```

Copy to Clipboard

When only the class name is provided (as in the first example), Python calls the constructor

[method](#)

Preview: Docs Loading link description

for us without any arguments (and thus no custom message will come up).

For a more concrete example, let's examine raising a `TypeError` for a function that checks if an employee tries to open the cash register but does not have the correct access:

```
def open_register(employee_status):
    if employee_status == 'Authorized':
        print('Successfully opened cash register')
    else:
        # Alternatives: raise TypeError() or TypeError('Message')
        raise TypeError
```

Copy to Clipboard

When an `employee_status` is not `'Authorized'`, the function `open_register()` will throw a `TypeError` and stop program execution.

Alternatively, when no built-in exception makes sense for the type of error our program might experience, it might be better to use a generic exception with a specific message. This is where we can use the base `Exception` class and provide a single

[argument](#)

Preview: Docs Loading link description

that serves as the error message. Let's modify the previous example, this time raising an `Exception` object with a message:

```
def open_register(employee_status):
    if employee_status == 'Authorized':
        print('Successfully opened cash register')
    else:
        raise Exception('Employee does not have access!')
```

Copy to Clipboard

As a general rule of thumb, use an exception that provides the best explanation for the expected error for both the user and anyone that will read the code.

Later in this lesson, we'll explore how to customize our exceptions further by creating user-defined exceptions. For now, let's practice using the `raise` keyword.

Instructions

Checkpoint 1 Passed

1.

Instrument World has a program that attempts to print the price of several instruments. Take some time to examine the program and then run the code and observe the output!

What do we expect to happen?

What kind of exception would happen if we try to access a key that does not exist in the `instrument_catalog` dictionary?

Checkpoint 2 Passed

2.

We hit a `KeyError` since 'Piano' is not a key in the `instrument_catalog` dictionary.

Let's provide a custom message by raising the exception ourselves. To accomplish this goal we will use a simple conditional.

First, inside of `print_instrument_price`, add an `if` statement that checks whether or not the `instrument` parameter is found in `instrument_catalog`. If it is, use the provided pre-written print statement to print the price.

The `in` keyword can be used to check if a key is found in a dictionary.

Checkpoint 3 Passed

3.

Finally, let's add an `else` block where we will print our custom `KeyError` exception if the key does not exist. Inside, it should raise a `KeyError` with the message `instrument + ' is not found in instrument catalog!'`.

When instantiating an exception, the first argument passed to the constructor method is the message. Here is an example using `TypeError`:

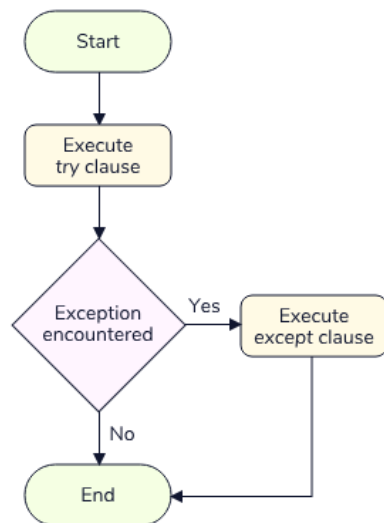
```
raise TypeError('Custom TypeError Message')
```

Exceptions

Try / Except

So far, the exceptions we've encountered have caused our programs to stop executing. However, it is possible for programs to continue executing even after encountering an exception. This process is known as *exception handling* and is accomplished using the Python `try/except` clauses.

The following flow chart demonstrates the mechanics of `try/except`:



Let's break it down:

Python will first attempt to execute code inside the `try` clause code block.

If no exception is encountered in the code, the `except` clause is skipped and the program continues normally. If an exception does occur inside of the `try` code block, Python will immediately stop executing the code and begin executing the code inside the `except` code block (sometimes called a *handler*).

Let's see this in action in a small program that prints colors:

```
colors = {
    'red': '#FF0000',
    'blue': '#0000FF',
    'yellow': '#FFFF00',
}

for color in ('red', 'green', 'yellow'):
    try:
        print('The hex value of ' + color + ' is ' + colors[color])
    except:
        print('An exception occurred! Color does not exist.')
    print('Loop continues...')
```

Copy to Clipboard

We get the following output:

```
The hex value of red is #FF0000
Loop continues...
An exception occurred! Color does not exist.
Loop continues...
The hex value of yellow is #FFFF00
Loop continues...
```

Copy to Clipboard

In the above code, the `try` block runs until it hit an exception. The hex value of the color red was successfully printed before it tried to access the hex value of green, which caused a `KeyError` since green is not in our `colors`

[dictionary](#)

Preview: Docs Loading link description

and ran the code in the `except` block. However, the exception was handled so Python continued executing our code and went onto print the hex value of yellow.

Exception handling is a powerful tool that lets us gain more flexibility in dealing with errors in our applications. We can use it to perform an action multiple times until it succeeds, or perhaps simply print a message when a non-critical part of our program doesn't work properly.

Let's try performing some exception handling!

Instructions

Checkpoint 1 Passed

1.

Instrument World has a program that prints a staff report for all of the Instrument World locations in the `staff` dictionary. Take some time to review the code. Spot any issues?

What error might happen when we try to calculate the staff ratio in the Melbourne location?

Checkpoint 2 Passed

2.

We successfully printed the staff report for Austin, but we hit an exception (`ZeroDivisionError`) when trying to print out the ratio for Melbourne since we attempted to divide 8 by 0.

Let's use exception handling to manage this error and keep our program running. First, wrap the function call

`print_staff_report()` in a `try` clause.

A `try` clause starts with `try:` followed by an indented code block.

Checkpoint 3 Passed

3.

Immediately after the `try` clause, add an `except` clause which prints `'Could not print sales report for ' + location`.

Run the code and observe our exception handling!

An `except` clause starts with `except:` followed by an indented code block. The exception handling will look like this when completed:

```
try:
    # Some code to try
except:
    # Some code to run when an exception occurs
```

Exceptions

Catching Specific Exceptions

The exception handlers from the previous exercise handled any exception hit during the `try` clause. However, in most cases, we will have an idea of the types of exceptions that might occur within our code. It is generally considered best practice to be as specific as possible with the exceptions we want to raise unless there is a specific reason for catching any type of exception.

We can catch a specific exception by listing it after the `except` keyword, as in the example below:

```
try:
    print(undefined_var)
except NameError:
    print('We hit a NameError')
```

Copy to Clipboard

In this case, the `except` block is only executed if a `NameError` is encountered (in the `try` block) rather than any exception. If any other exception occurs, it is unhandled, and the program terminates.

When we specify exception types, Python also allows us to capture the exception object using the `as` keyword. The exception object hosts information about the specific error that occurred. Examine our previous function but now capturing the exception object as `errorObject`:

```
try:
    print(undefined_var)
except NameError as errorObject:
    print('We hit a NameError')
    print(errorObject)
```

Copy to Clipboard

Would output:

```
We hit a NameError
name 'undefined_var' is not defined
```

Copy to Clipboard

Its worth noting `errorObject` is an arbitrary name and can be replaced with any name we see fit. The following code would work exactly the same:

```
try:
    print(undefined_var)
except NameError as e:
    print('We hit a NameError')
    print(e)
```

Copy to Clipboard

Let's get some practice capturing specific exceptions.

Instructions

Checkpoint 1 Passed

1.

Let's improve upon the exception handler we built in the previous exercise.

Change the `except` clause so that it only handles a `ZeroDivisionError`. Store the `ZeroDivisionError` we intend to capture into a variable called `e`.

The `except` keyword can be followed by a specific exception type, the `as` keyword, and then a variable name. Take a look at an example:

```
try:
    # Some code to try
except specificException as e:
    # Some code to run after exception
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Print `e` as the second `print()` statement in the `except` block.

Exceptions

Handling Multiple Exceptions

While handling a single exception is useful, Python also gives us the ability to handle multiple exceptions at once. We can list more than one exception type in a

[tuple](#)

Preview: Docs Loading link description

with a single `except` clause. Here is what the syntax would look like:

```
try:
    # Some code to try!
except (NameError, ZeroDivisionError) as e:
    print('We hit an Exception!')
    print(e)
```

Copy to Clipboard

In the above example, we expect to encounter either a `NameError` or a `ZeroDivisionError`. We can list any number of exceptions in this tuple format as long as it makes sense for the code in our `try` block. This is where we can see the benefit of capturing our exception object (via the `as` clause) since it enables us to print (or operate on) the specific exception that is caught. In addition to catching multiple exceptions, we can also pair multiple `except` clauses with a single `try` clause, enabling specific exceptions to be handled differently. For example:

```
try:
    # Some code to try!
except NameError:
    print('We hit a NameError Exception!')
except KeyError:
    print('We hit a TypeError Exception!')
except Exception:
    print('We hit an exception that is not a NameError or TypeError!')
```

Copy to Clipboard

In the above program, a `NameError` or `KeyError` will trigger one of the first two exception handlers. Any other exception will trigger the third handler. Note that the order of handlers is important here - if an exception is encountered, Python will execute the first one that matches its type. In this case, and a valid strategy for exception handling, we use the last `except` clause as a generic `Exception` as a backup if no other specific exception gets caught.

Let's now practice handling multiple exceptions!

Instructions

Checkpoint 1 Passed

1.

Instrument World has a program that allows the user to apply a discount to an instrument price.

Take some time to look over the program. Spot any issues? Run the code to find out!

What exception gets thrown when we try to access a key that does not exist in a dictionary?

Checkpoint 2 Passed

2.

Looks like we hit a `KeyError`! Let's apply some exception handling to handle this exception!

Wrap the `display_discounted_price()` function call in a `try` clause. In addition, add an `except` clause which handles a `KeyError` exception. Inside the `except` clause, print 'An invalid instrument was entered!'.

Checkpoint 3 Passed

3.

Awesome! Now our program can account for any `KeyError` we encounter. Let's see what happens when we use a key that does exist in our `instrument_prices` dictionary.

Change `instrument = 'Clarinet'` so that `instrument` is equal to `'Banjo'`. Before you run the code, take some time to ponder if our program will run into any error.

Take a look at the data type of the `discount` variable. What would happen when our `display_discounted_price()` reaches the line where we calculate discount percentage?

Checkpoint 4 Passed

4.

We hit a `TypeError`!

This happened because the `discount` variable was set to a string, not a number. Let's adjust our exception handling to also account for a `TypeError`.

After the exception handler for `KeyError`, add another `except` clause which catches a `TypeError`. Inside the `except` clause, print 'Discount percentage must be a number!'.

A single `try` clause can be followed by multiple `except` clauses. Here is an example:

```
try:
    # Some code to try!
except SpecificException:
    # Some code to run if the exception is hit!
except SomeOtherSpecificException:
    # Some code to run if the exception is hit!
```

Copy to Clipboard

Checkpoint 5 Passed

5.

We now have exception handlers for when we hit a `KeyError` or `TypeError`, but what if some other unexpected exception occurs?

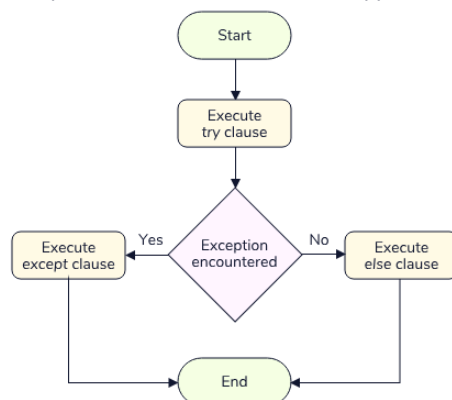
Add a final exception handler which will catch any `Exception` object. Inside, print `'Hit an exception other than KeyError or TypeError!'`

Exceptions

The else Clause

We've seen how exception handlers get executed when we encounter exceptions during a `try` clause - but what if we want to run some code only if we do not encounter an exception? Python provides us a way to do this as well - the `else` clause.

Our updated flow chart shows what happens when an `else` clause is added to the mix:



Python will only execute the `else` clause if no exception was encountered in the `try` clause.

Let's examine a hypothetical program that authenticates a user. For now, we will use two imaginary functions `check_password()` and `login_user()`. Here is what the program looks like:

```
try:
    check_password()
except ValueError:
    print('Wrong Password! Try again!')
else:
    login_user()
    # 20 other lines of imaginary code
```

Copy to Clipboard

In this program, we can assume if our function `check_password()` fails, it will return a `ValueError`. Thankfully, our exception handler takes care of this scenario. However, if our function doesn't fail, the `else` clause allows us to log the user in!

Now, one could argue, we could have written our program a different way to achieve a similar outcome:

```
try:
    check_password()
    login_user()
    # 20 other lines of imaginary code
except ValueError:
    print('Wrong Password! Try again!')
```

Copy to Clipboard

Here, if our `check_password()` ever fails, we will be able to catch the exception just like before. Python does offer a bit of insight on this scenario in the official documentation:

The use of the `else` clause is better than adding additional code to the `try` clause because it avoids accidentally catching an exception that wasn't raised by the code being protected by the `try ... except` statement.

This suggestion is valid in this case since in the alternative style, the `ValueError` could occur in any of the other lines of code other than `check_password()`, and it would be challenging to tell where it came from.

Let's give the `else` clause a try!

Instructions

Checkpoint 1 Passed

1.

Our Instrument World stores have a customer rewards program. Examine the code which displays a customer's account number. Spot any issues? Run the code to find out!

What happens when we try to access a key that does not exist in a dictionary?

Checkpoint 2 Passed

2.

Looks like our pesky `KeyError` is back! Let's try to account for this scenario by using exception handling.

Wrap `rewards_number = customer_rewards[customer]` inside of a `try` clause. Add an `except` clause which catches a `KeyError` and prints `'Customer was not found in rewards program!'`.

This is the standard syntax for a `try/except` for a specific exception:

```
try:
    # Some code to try!
except SpecificException:
    # Some code to run if we hit an exception!
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Lastly, add an `else` clause and move `print('Rewards account number is: ' + str(rewards_number))` so that it is inside of the `else` clause.

This is the standard syntax for a `try/except/else` for a specific exception:

```
try:
    # Some code to try!
except SpecificException:
    # Some code to run if we hit an exception!
else:
    # Some code to run if we don't hit an exception
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Change the value of the `customer` variable so that it is equal to `'Mario'`.

What we should expect from our output given our new exception handling structure? Ponder the question and then run the code to find out!

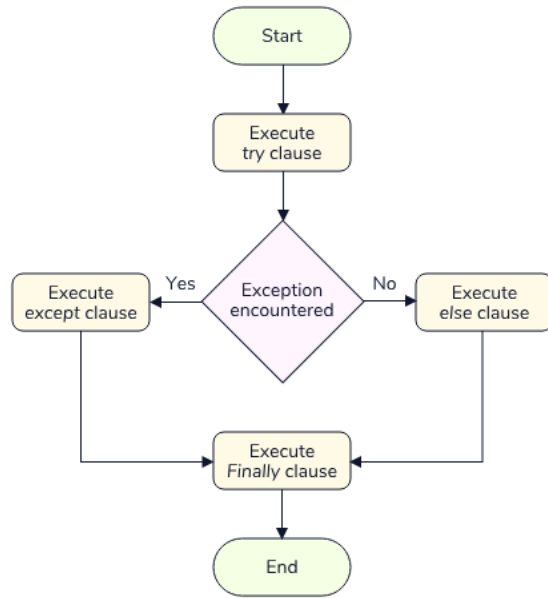
Which clause would be hit in our `try/except/else` structure?

Exceptions

The finally Clause

With `try/except/else`, we've seen how to run certain code when an exception occurs and other code when it does not. There is also a way to execute code regardless of whether an exception occurs - the `finally` clause.

Here is our final flow chart demonstrating `try/except/else/finally`:



Let's return to our fictional login program from earlier and examine a use case for the `finally` clause:

```

try:
    check_password()
except ValueError:
    print('Wrong Password! Try again!')
else:
    login_user()
    # 20 other lines of imaginary code
finally:
    load_footer()
  
```

Copy to Clipboard

In the above program, most of our code stayed the same. The one change we made was we added the `finally` clause to execute no matter if the user fails to login or not. In either case, we use an imaginary function called `load_footer()` to load the page's footer. Since the footer area of our imaginary application stays the same for both states, we always want to load it, and thus call it inside of the `finally` clause.

Note that the `finally` clause can be used independently (without an `except` or `else` clause). This is a convenient way to guarantee that a behavior will occur, regardless of whether an exception occurs:

```

try:
    check_password()
finally:
    load_footer()
    # Other code we always want to run
  
```

Copy to Clipboard

Let's put the `finally` clause into practice for our Instrument World application!

Instructions

Checkpoint 1 Passed

1.

Instrument World maintains a database (in this case a large dictionary) with instrument information that any store can access.

The current program displays information from the database for a particular instrument. Take some time to look over `database.py` and `instrument.py` to get better acquainted with the program.

Run the code to examine the output!

Checkpoint 2 Passed

2.

Since the database server Instrument World uses can only have a limited number of users connected to it, we want to make sure that we disconnect from it after attempting to retrieve information, even if an exception occurs.

Add a `finally` clause that calls `database.disconnect_from_database()`. Observe the output of the exception handling when we hit an exception by running the code!

The `finally` clause comes last in our exception handling structure.

Checkpoint 3 Passed

3.

Change `instrument` to have a value of `'Kora'`. Run the code to observe the `finally` clause executing even when we don't hit an exception.

```
import database

instrument = 'Kora'
database.connect_to_database()

try:
    database.display_instrument_info(instrument)
except KeyError:
    print('Oh no! This instrument does not exist.')
else:
    print(instrument)
finally:
    database.disconnect_from_database()
```

Exceptions

User-defined Exceptions

So far we have seen how to raise and manage built-in exceptions. In most programs, using built-in exceptions won't always be the most detailed way to describe an error occurring. What if we could create custom exceptions that are more specific to a program or module? Well, Python gives us the ability to create *user-defined exceptions*.

User-defined exceptions are exceptions that we create to allow for better readability in our program's errors. The core syntax looks like this:

```
class CustomError(Exception):
    pass
```

Copy to Clipboard

All we have to do to create a custom exception is to derive a subclass from the built-in `Exception` class. Although not required, most custom exceptions end in "Error" similar to the naming of the built-in exceptions. We'll learn how to customize these exceptions in the next exercise, but for now, let's see how a simple custom exception helps us better document our errors.

Let's imagine that Instrument World has an optional delivery service for instruments. If someone tries to schedule a delivery but their address is too far, we want to raise a custom `LocationTooFarError` exception. This isn't a type of exception that is built into Python, but rather one that is specific to our program and use case. Here is what our program might look like utilizing this custom exception:

```
class LocationTooFarError(Exception):
    pass

def schedule_delivery(distance_from_store):
    if distance_from_store > 10:
        raise LocationTooFarError
    else:
        print('Scheduling the delivery...')
```

Copy to Clipboard

Here, we have a class called `LocationTooFarError` that inherits from the `Exception` class. By doing so, we are telling Python that we would like to be able to use the class as our own custom exception.

Now, if we call `schedule_delivery(20)`, we get the following output:

Traceback (most recent call last):

```
File "inventory.py", line 10, in <module>
    schedule_delivery(20)
File "inventory.py", line 6, in schedule_delivery
    raise LocationTooFarError
__main__.LocationTooFarError
```

Since our class name populates into the traceback, even this simple class proves to be more useful than a generic `Exception` object or any built-in types! Users and developers alike will appreciate having specific exception details to work with.

Let's practice creating our own simple custom exceptions!

Instructions

Checkpoint 1 Passed

1.

Instrument World has a program for submitting an online order for an instrument and then updating the inventory. Take some time to look over the current state of the program.

Can we imagine any specific errors occurring? Run the code to see what happens!

What would happen if a customer requested to buy a quantity greater than the current quantity in `inventory`?

Checkpoint 2 Passed

2.

Sometimes we will receive orders that can't be fulfilled because there is not enough inventory for a specific instrument. Let's add some custom exception handling to the program to handle this specific situation.

Create a class called `InventoryError`, which inherits from `Exception`. The body of the class should be a single `pass` statement.

A simple custom expression looks like this:

```
class CustomExceptionName(Exception):  
    pass
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Now, let's deal with the logic of capturing if an exception occurs. Inside of `submit_order()`, add an `if` statement after the `supply` variable is assigned.

The `if` statement should check if `quantity` is greater than `supply`. If it is, then we want to raise our custom `InventoryError` exception.

Here is the pseudocode for using `if` block with an exception:

```
if <A conditional statement> :  
    raise <An exception>
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Lastly, add an `else` clause after the `if` clause, and move the remainder of the function inside of it. This will make sure we execute the rest of the function if the exception is not hit.

Here is the pseudocode for using `if/else` block with an exception:

```
if <A conditional statement> :  
    raise <An exception>  
else  
    <Some other code to run>
```

```
# Checkpoint 1  
  
inventory = {  
    'Piano': 3,  
    'Lute': 1,  
    'Sitar': 2  
}  
  
# Checkpoint 2  
class InventoryError(Exception):  
    pass  
  
def submit_order(instrument, quantity):  
    supply = inventory[instrument]  
  
    # Checkpoint 3  
    if quantity > supply:  
        raise InventoryError  
    # Checkpoint 4  
    else:  
        inventory[instrument] -= quantity  
        print('Successfully placed order! Remaining supply: ' + str(inventory[instrument]))  
  
instrument = 'Piano'
```

```
quantity = 5
submit_order(instrument, quantity)
```

Exceptions

Customizing User-defined Exceptions

We've just seen how defining a simple exception class can provide a more specific and useful error to users. Defining a simple class is just the first step to creating better exceptions in our programs. Python does not stop us from customizing our custom exception classes even further.

Let's say we wanted to expand our `LocationTooFarError` exception from earlier to also provide a custom error message. Here is what the custom class might look like:

```
class LocationTooFarError(Exception):
    def __init__(self, distance):
        self.distance = distance

    def __str__(self):
        return 'Location is not within 10 km: ' + str(self.distance)
```

Copy to Clipboard

Let's break this down:

Our class definition doesn't look much different from before. We have a class named `LocationTooFarError` that still inherits from the built-in `Exception` class.

We have added a constructor that is going to take in a distance [argument](#)

Preview: Docs Loading link description

when we instantiate our exception class. Here, we have overridden the constructor of the `Exception` class to accept our own custom argument of `distance`. The reason for taking in a distance is to use it in our

`__str__` [method](#)

Preview: Docs Loading link description

that will return a custom error message when the exception is hit!

The `__str__` method provides our exception a custom message by returning a [string](#)

Preview: Docs Loading link description

with the distance property from the constructor.

If we now ran it using our script from earlier:

```
def schedule_delivery(distance_from_store):
    if distance_from_store > 10:
        raise LocationTooFarError(distance_from_store)
    else:
        print('Scheduling the delivery...')
```

Copy to Clipboard

We would see our expanded custom exception in action:

Traceback (most recent call last):

```
File "inventory.py", line 14, in <module>
    schedule_delivery(20)
```

```
File "inventory.py", line 11, in schedule_delivery
    raise LocationTooFarError(distance_from_store)
```

```
__main__.LocationTooFarError: Location is not within 10 km: 20
```

Let's practice customizing our exceptions even further!

Instructions

Checkpoint 1 Passed

1.

Let's customize the `InventoryError` from our previous exercise to return a custom error message. Inside the class, replace the `pass` statement with an `__init__` method which takes two arguments: `self`, and `supply`.

Inside the method, store `supply` into the variable `self.supply`.

Checkpoint 2 Passed

2.

Define a `__str__` method which returns, `'Available supply is only ' + str(self.supply)`.

Checkpoint 3 Passed

3.

Modify `raise InventoryError` by passing in `supply` to the exception's constructor method.

```
class InventoryError(Exception):
    def __init__(self, supply):
        self.supply = supply

    def __str__(self):
        return 'Available supply is only ' + str(self.supply)

inventory = {
    'Piano': 3,
    'Lute': 1,
    'Sitar': 2
}

def submit_order(instrument, quantity):
    supply = inventory[instrument]
    if quantity > supply:
        raise InventoryError(supply)
    else:
        inventory[instrument] -= quantity
        print('Successfully placed order! Remaining supply: ' + str(inventory[instrument]))

instrument = 'Piano'
quantity = 5
submit_order(instrument, quantity)
```

Exceptions

Review

Congratulations, you have now mastered many techniques for interacting with exceptions in Python! We learned:

- How exceptions differ from syntax errors

- How to read tracebacks

- How `try/except/else/finally` provides us with a powerful [control flow](#)

- Preview: Docs Loading link description for handling exceptions

- How to create and raise custom exceptions to provide more helpful errors to users of our code

These tools will get you very far as a Python developer!

Instructions

Checkpoint 1 Enabled

1.

The code in `families.py` prints some instruments and the instrument families they belong to. It has some bugs in it. Can you find and fix the bugs? Consider adding some exception handlers so that you can print custom error messages the next time someone runs into these bugs!

Unit Testing

Introduction to Testing

When working with Python, or any programming language, there is a lot that can go wrong with our code. There are syntax errors and exceptions, but there are also mistakes in the program logic which cause it to behave in unexpected ways.

For these reasons, testing is crucial to creating quality software. The goal of testing isn't just to find bugs but to find them quickly. Leaving bugs unfound and unresolved can lead to massive consequences in the real world. Take a look at some of the [most infamous bugs that have ever occurred in software](#).

Don't worry though - by following some common practices and using the tools built into Python, we can start creating quality tests in no time. To dive in, first, let's talk about the different types of testing styles that exist.

The world of testing can generally be divided into two categories:

Manual Testing:

With manual testing, a physical person interacts with software much as a user would. In fact, we have been manually testing our code any time we run it and observe the results!

Automated Testing:

With automated testing, tests are performed with code. Generally, automated testing is faster and less prone to human error.

In this lesson, we'll be diving into the world of testing. For most of the exercises, let's imagine we've been hired to create tests for a new airline company called Small World Air. Before we start writing automated tests, let's do some manual testing and see what kind of shape their software is in.

Instructions

Checkpoint 1 Passed

1.

Small World Air provided us access to a program that displays the flight status for all current flights. Manually test the code by running it and observing the output.

Checkpoint 2 Passed

2.

Uh oh, we hit a `TypeError`! Can you spot and fix the error? Manually test your bug fix by running the program again. Integers cannot be concatenated with strings. Check out line 11!

Unit Testing

The assert Statement

While the previous example was simple to test, most code will be much more complex. It would be very tedious to have to perform these tests manually. Our time would be better spent writing automated tests.

Luckily, Python provides an easy way to perform simple tests in our code - the `assert` statement. An `assert` statement can be used to test that a condition is met. If the condition evaluates to `False`, an `AssertionError` is raised with an optional error message.

The general syntax looks like this:

```
assert <condition>, 'Message if condition is not met'
```

Copy to Clipboard

Consider the following example that demonstrates the `assert` statement paired with a function called `times_ten`. Note there is a bug in the function for demonstration purposes.

```
def times_ten(number):  
    return number * 100  
  
result = times_ten(20)  
assert result == 200, 'Expected times_ten(20) to return 200, instead got ' + str(result)
```

Copy to Clipboard

Here, we want to test if our `times_ten()` function works as intended. We use the `assert` statement to evaluate the expression `result == 200` since we expect that our function would return 200 given an input of 20. Since this is not the case, this expression evaluates to `False` (there is a bug in `times_ten` - it actually multiplies by 100!), we get the following exception:

```
AssertionError: Expected times_ten(20) to return 200, instead got 2000
```

An `assert` statement is a quick and powerful way to verify that a program is in the correct state. They can be used to catch mistakes early and make sure we avoid any catastrophes. Let's practice using `assert` to get a feel for automated testing!

Instructions

Checkpoint 1 Passed

1.

Small World Air has a program that runs at the check-in kiosk and asks passengers for their destination airport code. Currently, Small World only flies to three destinations: Budapest (BUD), Casablanca (CMN), and Istanbul (IST). Take some time to examine the program.

What should the program return for the current set destination of 'HND' (an airport in Tokyo, Japan)?

What kind of exception gets thrown for a key not existing in a dictionary?

Checkpoint 2 Passed

2.

This error wasn't a very user-friendly experience! We also want to make sure that users are not entering destinations that Small Air World does not travel to. Let's add an automated test using `assert`!

Add an `assert` statement that checks if `destination` is in the `destinations` keys. If it isn't, the `AssertionError` message should read, 'Sorry, Small World currently does not fly to this destination!'

The `assert` keyword is followed by a condition and a message, separated by a comma.

The condition should use th

Unit Testing

Unit Testing

Assertion statements are a good start to ensuring our programs are being tested, but they don't necessarily tell us *what* we should test. Generally, we can start by testing the smallest unit of a program.

For example, in the real world, if we were testing the functionality of a door, we could test a multitude of units. The handle could be an example of a single unit that we must check to make sure a door functions, followed by the hinges and maybe even the lock.

In programming, these types of individual tests are called *unit tests*. Like our door handle, we can test a single unit of a program, such as a function, loop, or variable. A unit test validates a single behavior and will make sure all of the units of a program are functioning properly.

Let's say we wanted to test a single function (a single unit). To test a single function, we might create several test cases. A test case validates that a specific set of inputs produces an expected output for the unit we are trying to test. Let's examine a test case for our `times_ten()` function from the previous exercise:

```
# The unit we want to test  
def times_ten(number):  
    return number * 10  
  
# A unit test function with a single test case  
def test_multiply_ten_by_zero():  
    assert times_ten(0) == 0, 'Expected times_ten(0) to return 0'
```

Copy to Clipboard

Great, now we have a simple test case that validates that `times_ten()` is behaving as expected for a valid input of 0! We can improve our testing coverage of this function by adding some more test cases with different inputs. A common approach is to create test cases for specific edge case inputs as well as reasonable ones. Here is an example of testing two extreme inputs:

```
def test_multiply_ten_by_one_million():
```

```

assert times_ten(1000000) == 10000000, 'Expected times_ten(1000000) to return 10000000'

def test_multiply_ten_by_negative_number():
    assert times_ten(-10) == -100, 'Expected times_ten(-10) to return -100'

```

Copy to Clipboard

Now we have several test cases for a wide variety of inputs: a large number, a negative number, and zero. We can create as many test cases as we see fit for a single unit, and we should try to test all the unique types of inputs our unit will work with.

Now, let's create a variety of unit tests for another feature of Small World Air.

Instructions

Checkpoint 1 Passed

1.

At Small Air World, every plane seat has a monitor which displays the nearest emergency exit.

This monitor relies on a function called `get_nearest_exit()`, which takes a row number and then returns an exit location depending on where the row is. Let's make sure our function is working properly by creating a unit test.

Create a function called `test_row_1()` that will host a test case. Inside the function, assert that a call of `get_nearest_exit(1)` should equal to `'front'`, along with a message, `'The nearest exit to row 1 is in the front!'`.

We can call functions within the condition of an `assert` statement.

Checkpoint 2 Passed

2.

Create another test case function called `test_row_20()`.

Inside the function, call `get_nearest_exit(20)` and assert that the return value is equal to `'middle'`, along with the message, `'The nearest exit to row 20 is in the middle!'`

We can call functions within the condition of an `assert` statement.

Checkpoint 3 Passed

3.

Finally, create another test case function called `test_row_40()`.

Inside the function, call `get_nearest_exit(40)` and assert that the return value is equal to `'back'`, along with the message, `'The nearest exit to row 40 is in the back!'`

We can call functions within the condition of an `assert` statement.

Checkpoint 4 Passed

4.

At the bottom of the file, call each of the three test functions we created. What would be the expected output?

What happens in the `get_nearest_exit()` function when we test it with the value of `40`?

Checkpoint 5 Passed

5.

Looks like our tests caught a logic error in our function `get_nearest_exit()`! If the row number is larger than `30`, we actually want to return `'back'`.

Adjust the function and fix the error so all of our tests pass (we should see no output).

What should happen in our `else` statement in the `get_nearest_exit()` function for a value of `40`?

Unit Testing

Python's unittest Framework

There are some problems with the approach to our previous unit tests that would make them difficult to maintain. First, we had to call each function specifically when a new test was created. We also didn't have any way of grouping tests, which is necessary when the number of tests increases. Perhaps most importantly, if one test failed, the `AssertionError` would prevent any remaining tests from running!

Luckily, Python provides a

[framework](#)

Preview: Docs Loading link description

that solves these problems and provides many other tools for writing unit tests. This framework lives in the `unittest` module which is included in the standard library. It can be imported like so:

```

import unittest

#The rest of our program...

```

Copy to Clipboard

The `unittest` module provides us with a *test runner*. A test runner is a component that collects and executes tests and then provides results to the user. The framework also provides many other tools for test grouping, setup, teardown, skipping, and other features that we'll soon learn about.

First, let's refactor our tests for the `times_ten` function to use the `unittest` framework. There are several things we need to do: First, we must create a class which inherits from `unittest.TestCase`, as follows:

```
import unittest

class TestTimesTen(unittest.TestCase):
    pass
```

[Copy to Clipboard](#)

This class will serve as the main storage of all our

[unit testing](#)

[Preview: Docs Loading link description](#)

functions. Once we have the class, we need to change our test functions so that they are methods of the class. The `unittest` module requires that test functions begin with the word `'test'`, so our existing names work well:

```
import unittest

class TestTimesTen(unittest.TestCase):
    def test_multiply_ten_by_zero(self):
        pass

    def test_multiply_ten_by_one_million(self):
        pass

    def test_multiply_ten_by_negative_number(self):
        pass
```

[Copy to Clipboard](#)

Lastly, we need to change our `assert` statements to use the `assertEqual`

[method](#)

[Preview: Docs Loading link description](#)

of `unittest.TestCase`. The framework requires that we use special methods instead of standard `assert` statements. Don't worry we'll cover these methods in the remainder of this lesson, for now, simply get used to the syntax. Here is what our class looks after the change:

```
import unittest

class TestTimesTen(unittest.TestCase):
    def test_multiply_ten_by_zero(self):
        self.assertEqual(times_ten(0), 0, 'Expected times_ten(0) to return 0')

    def test_multiply_ten_by_one_million(self):
        self.assertEqual(times_ten(1000000), 10000000, 'Expected times_ten(1000000) to return 10000000')

    def test_multiply_ten_by_negative_number(self):
        self.assertEqual(times_ten(-10), -100, 'Expected add_times_ten(-10) to return -100')
```

[Copy to Clipboard](#)

That's it! Now we can run our tests by calling `unittest.main()`. The `unittest` framework will work its magic to detect any tests in the existing module, run them, and provide us results. Our final code would look like this:

```
# Importing unittest framework
import unittest

# Function that gets tested
def times_ten(number):
    return number * 100

# Test class
class TestTimesTen(unittest.TestCase):
    def test_multiply_ten_by_zero(self):
        self.assertEqual(times_ten(0), 0, 'Expected times_ten(0) to return 0')
```

```

def test_multiply_ten_by_one_million(self):
    self.assertEqual(times_ten(1000000), 10000000, 'Expected times_ten(1000000) to return 10000000')

def test_multiply_ten_by_negative_number(self):
    self.assertEqual(times_ten(-10), -100, 'Expected add_times_ten(-10) to return -100')

# Run the tests
unittest.main()

```

Copy to Clipboard

When we run this code, we would see the following output:

```

FF.
=====
FAIL: test_multiply_ten_by_negative_number (__main__.TestTimesTen)
-----
Traceback (most recent call last):
  File "scratch.py", line 16, in test_multiply_ten_by_negative_number
    self.assertEqual(times_ten(-10), -100, 'Expected add_times_ten(-10) to return -100')
AssertionError: -1000 != -100 : Expected add_times_ten(-10) to return -100

=====
FAIL: test_multiply_ten_by_one_million (__main__.TestTimesTen)
-----
Traceback (most recent call last):
  File "scratch.py", line 13, in test_multiply_ten_by_one_million
    self.assertEqual(times_ten(1000000), 10000000, 'Expected times_ten(1000000) to return 10000000')
AssertionError: 100000000 != 10000000 : Expected times_ten(1000000) to return 10000000

-----
Ran 3 tests in 0.001s

FAILED (failures=2)

```

In the test output, we can see that two of the tests failed (`test_multiply_ten_by_one_million` and `test_multiply_ten_by_negative_number`).

Let's get some practice with `unittest` by refactoring our previous test cases for Small World Air and then we will dive into learning all the useful testing methods we can work with!

Instructions

Checkpoint 1 Passed

1.

First, let's `import unittest` at the top of the file.

To import a built-in Python module we can use the following syntax:

```
import <module name>
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Next, after the `get_nearest_exit()` function, create a class called `NearestExitTests`. It should inherit from `unittest.TestCase`.

To setup a test class, we use the following syntax:

```
class testClass(unittest.TestCase):
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Refactor `test_row_1()`, `test_row_20()`, and `test_row_40()` so that they are methods of this class. Don't forget to add the `self` argument.

Indent the functions to make them part of the class body and thus methods.

Checkpoint 4 Passed

4.

Change the `assert` statements in these functions so that they instead call the `self.assertEqual()` method with the correct arguments.

`self.assertEqual()` should have three arguments: the value returned by `get_nearest_exit()`, the expected return value, and the message. For example:

```
self.assertEqual(<function call>, <value to compare function call to>, <error message>)
```

Copy to Clipboard

Checkpoint 5 Passed

5.

Remove the three function calls from the bottom of the file. Replace them with a single call to `unittest.main()`.

Observe the output of our tests!

Checkpoint 6 Passed

6.

Looks like it's the same bug from before! Fix the error in the `get_nearest_exit()` function. We should no longer get any failed tests when we run our script.

```
# Checkpoint 1
import unittest

def get_nearest_exit(row_number):
    if row_number < 15:
        location = 'front'
    elif row_number < 30:
        location = 'middle'
    else:
        # Checkpoint 6
        location = 'back'
    return location

# Checkpoint 2
class NearestExitTests(unittest.TestCase):
    # Checkpoint 3
    def test_row_1(self):
        self.assertEqual(get_nearest_exit(1), 'front', 'The nearest exit to row 1 is in the front!')

    # Checkpoint 3
    def test_row_20(self):
        self.assertEqual(get_nearest_exit(20), 'middle', 'The nearest exit to row 20 is in the middle!')

    # Checkpoint 3
    def test_row_40(self):
        self.assertEqual(get_nearest_exit(40), 'back', 'The nearest exit to row 40 is in the back!')

# Checkpoint 4
unittest.main()
```

Unit Testing

Assert Methods I: Equality and Membership

In the last exercise, we saw how to check for equality between two values in the `unittest` framework using the `.assertEqual` method of the `TestCase` class. The framework relies on built-in assert methods instead of `assert` statements to track results without actually raising any exceptions. Specific assert methods take arguments instead of a condition, and like assert statements, they can take an optional message argument.

Let's go over three commonly used assert methods for testing equality and membership, their general syntax, and their `assert` statement equivalents.

assertEqual: The `assertEqual()` method takes two values as arguments and checks that they are equal. If they are not, the test fails.

```
self.assertEqual(value1, value2)
```

Copy to Clipboard

assertIn: The `assertIn()` method takes two arguments. It checks that the first argument is found in the second argument, which should be a container. If it is not found in the container, the test fails.

```
self.assertIn(value, container)
```

Copy to Clipboard

assertTrue: The `assertTrue()` method takes a single argument and checks that the argument evaluates to `True`. If it does not evaluate to `True`, the test fails.

```
self.assertTrue(value)
```

Copy to Clipboard

The equivalent `assert` statements would be the following:

Method	Equivalent
<pre>self.assertEqual(2, 5)</pre>	<pre>assert 2 == 5</pre>
<pre>self.assertIn(5, [1, 2, 3])</pre>	<pre>assert 5 in [1, 2, 3]</pre>
<pre>self.assertTrue(0)</pre>	<pre>assert bool(0) is True</pre>

The full list for equality and membership can be seen in the [Python documentation](#). Let's put these methods into practice!

Instructions

Checkpoint 1 Passed

1.

Small World Air planes are equipped with an on-board entertainment system which we need to create some tests for. Take some time to review `entertainment.py` and `tests.py`.

Note the three functions in `entertainment.py`:

`get_daily_movie()`: returns the movie of the day.

`get_licensed_movies()`: returns a list of licensed movies the plane can play.

`get_wifi_status()`: returns the current wifi status of the plane.

Note the two test cases in `tests.py`:

`test_movie_license()`: is intended to test if a daily movie is licensed

`test_wifi_status()`: is intended to test if the wifi is currently active

Run the code to proceed.

Checkpoint 2 Passed

2.

Every flight has a free movie. We want to create a test that checks our database to make sure that the uploaded movie has a valid license, or else we could pay some hefty fines.

Inside of our `test_movie_license()` test method, we have two variables defined:

`daily_movie`: which stores the value of the current free daily movie.

`licensed_movies`: which stores the value of all the current licensed movies.

To test if we have a license for the current daily movie, we need to compare if `daily_movie` exists inside of `licensed_movies`.

Use the `self.assertIn()` assert method inside of `test_movie_license()` to check if the `daily_movie` is licensed.

`self.assertIn()` has two required arguments: an element, followed by a container that we expect to contain that element.

Checkpoint 3 Passed

3.

Our entertainment system also provides WiFi as a purchase option for passengers. We want to make a test that ensures the WiFi is enabled.

Inside the test method called `test_wifi_status()` we have a variable called `wifi_enabled` which stores the boolean value of whether the wifi is turned on or not.

Use the `self.assertTrue()` assert method inside of `test_wifi_status()` to test that `wifi_enabled` is `True`. The test should fail because currently wifi is disabled.

`self.assertTrue()` has one required argument - an element we want to check the boolean value of!

```
# Checkpoint 1
import unittest
import entertainment

class EntertainmentSystemTests(unittest.TestCase):

    def test_movie_license(self):
        daily_movie = entertainment.get_daily_movie()
        licensed_movies = entertainment.get_licensed_movies()
        # Checkpoint 2
        self.assertIn(daily_movie, licensed_movies)

    def test_wifi_status(self):
        wifi_enabled = entertainment.get_wifi_status()
        # Checkpoint 3
        self.assertTrue(wifi_enabled)

unittest.main()
```

Unit Testing

Assert Methods II: Quantitative Methods

Often we need to test conditions related to numbers. The `unittest` module provides a handful of assert methods to achieve this. Let's take a look at two common assert methods related to quantitative comparisons, their general syntax, as well as their `assert` statement equivalents.

`assertLess`: The `assertLess()` method takes two arguments and checks that the first argument is less than the second one. If it is not, the test will fail.

```
self.assertLess(value1, value2)
```

Copy to Clipboard

`assertAlmostEqual`: The `assertAlmostEqual()` method takes two arguments and checks that their difference, when rounded to 7 decimal places, is 0. In other words, if they are almost equal. If the values are not close enough to equality, the test will fail.

```
self.assertAlmostEqual(value1, value2)
```

Copy to Clipboard

The equivalent `assert` statements would be the following:

Method	Equivalent
<code>self.assertLess(2, 5)</code>	<code>assert 2 < 5</code>
<code>self.assertAlmostEqual(.22, .225)</code>	<code>assert round(.22 - .225, 7) == 0</code>

The full list of quantitative methods can be seen in the [Python documentation](#). Let's put these methods into practice!

Instructions

Checkpoint 1 Passed

1.

Our `entertainment.py` file has a function called `get_maximum_display_brightness()` that returns the max screen brightness value.

Create a test method called `test_maximum_display_brightness()`. Inside the method, do the following:

Call `entertainment.get_maximum_display_brightness()` and store the return value in a variable called `brightness`.

Next, call `self.assertAlmostEqual()` to make sure that `brightness` is almost equal to 400.

`self.assertAlmostEqual()` requires two numbers as arguments.

Checkpoint 2 Passed

2.

Our `entertainment.py` file has a method called `entertainment.get_device_temp()` that returns the current device temperature.

Create a test method called `test_device_temperature()`. Inside the method do the following:

Call `entertainment.get_device_temp()` and store the return value in a variable called `device_temp`.

Then call `self.assertLess()` to make sure that `device_temp` is less than 35.

The test should fail because the current temperature is 40.

`self.assertLess()` requires two numbers as arguments. The first one is expected to be less than the second.

```
import unittest
import entertainment

class EntertainmentSystemTests(unittest.TestCase):

    def test_movie_license(self):
        daily_movie = entertainment.get_daily_movie()
        licensed_movies = entertainment.get_licensed_movies()
        self.assertIn(daily_movie, licensed_movies)

    def test_wifi_status(self):
        wifi_enabled = entertainment.get_wifi_status()
        self.assertTrue(wifi_enabled)

    # Checkpoint 1
    def test_maximum_display_brightness(self):
        brightness = entertainment.get_maximum_display_brightness()
        self.assertAlmostEqual(brightness, 400)

    # Checkpoint 2
    def test_device_temperature(self):
        device_temp = entertainment.get_device_temp()
        self.assertLess(device_temp, 35)

unittest.main()
```

Unit Testing

Assert Methods III: Exception and Warning Methods

There is another group of assert methods related to exceptions and warnings. Note that while we haven't covered [warnings](#) in detail yet, they are a type of exception. Let's go over two of these methods and their general syntax.

[assertRaises](#): The `assertRaises()`

[method](#)

Preview: Docs Loading link description

takes an exception type as its first [argument](#)

Preview: Docs An argument is the actual value of a variable passed into a function.

, a function reference as its second, and an arbitrary number of arguments as the rest.

It calls the function and checks if an exception is raised as a result. The test passes if an exception is raised, is an error if another exception is raised, or fails if no exception is raised. This method can be used with custom exceptions as well!

```
self.assertRaises(specificException, function, functionArguments...)
```

Copy to Clipboard

[assertWarns](#): The `assertWarns()` method takes a warning type as its first argument, a function reference as its second, and an arbitrary number of arguments for the rest.

It calls the function and checks that the warning occurs. The test passes if a warning is triggered and fails if it isn't.

```
self.assertWarns(specificWarningException, function, functionArguments...)
```

Copy to Clipboard

There are no particular concise ways to replicate these tests using the `assert` keyword so it is recommended to use these methods instead when possible!

The full list of exception and warning methods can be seen in the [Python documentation](#). Let's put these methods into practice!

Instructions

Checkpoint 1 Passed

1.

We need to create some tests for the airplane alert system so that the flight crew is properly notified of critical events.

Let's start by creating a class called `SystemAlertTests` which inherits from `unittest.TestCase`.

A class can inherit from another using the following syntax:

```
class CustomClass(classToInherit):
```

Copy to Clipboard

Checkpoint 2 Passed

2.

We are going to create a test for any power outages that might occur on the airplane. Check out the custom exception

`PowerError` and our function that raises the error `power_outage_detected()` in the `alerts.py` file. The file is already imported into `tests.py` for us.

In our `SystemAlertTests` class, create a test method called `test_power_outage_alert()`.

Inside the new method, use `self.assertRaises()` to check that an `alerts.PowerError` is raised whenever `alerts.power_outage_detected` is called with an argument of `True`.

This test should pass since we are passing a value of `True` and the exception is raised.

The general syntax for `assertRaises()` is as follows:

```
self.assertRaises(specificException, function, functionArguments...)
```

Copy to Clipboard

For our scenario here is each component:

SpecificException: This would be our `PowerError` custom exception from `alerts.py`. Remember, to use an imported function, we must preface it with `alerts.` since that is the file it originated from.

Function: This would be our `power_outage_detected()` function. For this syntax, we omit the parentheses `()` since we are not calling the function. Remember, to use an imported function, we must preface it with `alerts.` since that is the file it originated from.

Function Arguments: We want to pass the function above the argument of `True`

Alternatively, we can pass the function arguments directly into the function, in which case we do not omit the parentheses `()` and instead place `True` inside the parentheses, with no other arguments.

Checkpoint 3 Passed

3.

We are going to create a test for any water level warnings that occur on the airplane. Check out the custom exception

`WaterLevelWarning` and our function that raises the warning `water_levels_check()` in the `alerts.py` file. The file is already imported into `tests.py` for us.

In our `SystemAlertTests` class, create a test method called `test_water_levels_warning()`.

Inside the new method, use `self.assertWarns()` to check that an `alerts.WaterLevelWarning` is raised whenever `alerts.water_levels_check` is called with an argument of 150 liters.

This test should pass since we are passing a value less than 200 and a warning occurs.

The general syntax for `assertWarns()` is as follows:

```
self.assertWarns(specificWarning, function, functionArguments...)
```

Copy to Clipboard

For our scenario here is each component:

SpecificWarning: This would be our `WaterLevelWarning` custom warning from `alerts.py`. Remember, to use an imported function, we must preface it with `alerts.` since that is the file it originated from.

Function: This would be our `water_levels_check()` function. For this syntax, we omit the parentheses `()` since we are not calling the function. Remember, to use an imported function, we must preface it with `alerts.` since that is the file it originated from.

Function Arguments: We want to pass the function above the argument of `150`

Alternatively, we can pass the function arguments directly into the function, in which case we do not omit the parentheses `()` and instead place the `150` inside the parentheses, with no other arguments.

```
import unittest
import alerts

# Checkpoint 1
class SystemAlertTests(unittest.TestCase):

    # Checkpoint 2
    def test_power_outage_alert(self):
        self.assertRaises(alerts.PowerError, alerts.power_outage_detected, True)

    # Checkpoint 3
    def test_water_levels_warning(self):
        self.assertWarns(alerts.WaterLevelWarning, alerts.water_levels_check, 150)

unittest.main()
```

Unit Testing

Parameterizing Tests

In previous examples, we created test cases for the `times_ten()` function with various inputs. However, the actual logic of our tests really didn't change. To decrease repetition, Python provides us a specific toolset for tests with only minor differences. This is known as *test parameterization*. By parameterizing tests, we can leverage the functionality of a single test to get a large amount of coverage of different inputs.

To accomplish test parameterization, the `unittest` [framework](#)

Preview: Docs Loading link description

provides us with the [subTest context manager](#). Let's refactor our previous test class to utilize it and see it in action:

```
import unittest

# The function we want to test
def times_ten(number):
    return number * 100

# Our test class
class TestTimesTen(unittest.TestCase):

    # A test method
    def test_times_ten(self):
        for num in [0, 1000000, -10]:
            with self.subTest():
                expected_result = num * 10
                message = 'Expected times_ten(' + str(num) + ') to return ' +
str(expected_result)
                self.assertEqual(times_ten(num), expected_result, message)
```

Copy to Clipboard

Here, in our test

[method](#)

Preview: Docs Loading link description

`test_times_ten()`, instead of writing individual test cases for each input of `0`, `10`, and `1000000`, we can test a collection of inputs by using a loop followed by a `with` statement and our `subTest` context manager.

By using `subTest`, each iteration of our loop is treated as an individual test. Python will run the code inside of the context manager on each iteration, and if one fails, it will return the failure as a separate test case failure.

Just like before, we are using the `assertEqual()` method to check the expected result, and we are expecting (due to an error in `times_ten()`) that the cases of using an input of `-10` and `1000000` will fail.

Here is the new output:

```
=====
FAIL: test_times_ten (__main__.TestTimesTen) (<subtest>)
-----
Traceback (most recent call last):
  File "scratch.py", line 12, in test_times_ten
    self.assertEqual(times_ten(num), expected_result, message)
AssertionError: 100000000 != 10000000 : Expected times_ten(1000000) to return 10000000

=====
FAIL: test_times_ten (__main__.TestTimesTen) (<subtest>)
-----
Traceback (most recent call last):
  File "scratch.py", line 12, in test_times_ten
    self.assertEqual(times_ten(num), expected_result, message)
AssertionError: -1000 != -100 : Expected times_ten(-10) to return -100

-----
Ran 1 test in 0.000s

FAILED (failures=2)
```

If we want to expand our test coverage, we can simply modify the list that our loop iterates over. We can test a range of thousands of inputs simply by using the context manager setup to achieve test parameterization.

Optionally, we can give our subtests better readability by making a small change in our code for the first

[argument](#)

Preview: Docs Loading link description

of `self.subTest()`. The below code has most of our script omitted for brevity but uses the same script we executed above:

```
# ... more code above..

for num in [0, 1000000, -10]:
    with self.subTest(num):

# ... more code below ....
```

Copy to Clipboard

This makes our test clearer, because our test error message goes from:

```
FAIL: test_times_ten (__main__.TestTimesTen) (<subtest>)
```

to:

```
FAIL: test_times_ten (__main__.TestTimesTen) [1000000]
```

When working with large amounts of test inputs, it is much easier to distinguish which case failed. We can actually use any message we want as the first argument, but using the tested case is usually the best way to increase readability for ourselves and other developers.

By using test parameterization, we made our codebase much cleaner and more maintainable. Let's get some practice by refactoring some of our previous tests!

Instructions

Checkpoint 1 Passed

1.

Small World Air is growing and has added many more movie options to the entertainment system (we can see them inside of `entertainment.py`).

Let's adjust our `EntertainmentSystemTests` class to make sure they all get tested. Replace the call to `entertainment.get_daily_movie()` with `entertainment.get_daily_movies()` (our new method).

Lastly, for better readability, update the variable name `daily_movie` to `daily_movies`. After updating this variable name, update the first argument in the call to `self.assertIn()` to use this new variable name.

Checkpoint 2 Passed

2.

Under our two variables, write a `for` loop that iterates over `daily_movies` and stores each iteration value into a variable called `movie`. For now, let's simply print `movie` on each iteration.

The basic structure of a `for` loop looks like this:

```
for element in collection:
    #some code to execute
```

Copy to Clipboard

In this case, our `element` is named `movie` and our collection is `daily_movies`.

Checkpoint 3 Passed

3.

Indent our `self.assertIn()` call to be inside the `for` loop and change the first argument in `self.assertIn()` from `daily_movie` to `movie` to represent the individual movies on each iteration of the loop.

Note: Creating this structure might be okay at first glance (and may even make you wonder why we need the context manager), but if we run our test, we will see that the test will fail in the middle of our `movies` collection and won't check the rest (it stops at *Black Widow* and not *Spiral*!) This is because like many testing frameworks, `unittest` will fail and stop on the first failure it encounters.

Checkpoint 4 Passed

4.

Lastly, under our print statement of `movie` but before our `assertIn()` call, insert a `self.subTest()` to wrap our test method. To make sure we can distinguish test cases between each movie, pass a single argument of `movie` into `self.subTest()`.

Don't forget to preface the context manager with a `with` statement and indent our `self.assertIn()` statement. Now, we can observe testing multiple movies and if they are licensed or not.

Our general structure should look like this (with spacing representing indentation levels):

```
for loop
    print statement
    with statement using subTest()
        assertIn() test case
```

Copy to Clipboard

```
import unittest
import entertainment

class EntertainmentSystemTests(unittest.TestCase):

    def test_movie_license(self):
        # Checkpoint 1
        daily_movies = entertainment.get_daily_movies()
        licensed_movies = entertainment.get_licensed_movies()

        # Checkpoint 2
        for movie in daily_movies:
            print(movie)
            # Checkpoint 3 & 4
            with self.subTest(movie):
                self.assertIn(movie, licensed_movies)

unittest.main()
```

Unit Testing

Test Fixtures

One of the most important principles of testing is that tests need to occur in a known state. If the conditions in which a test runs are not controlled, then our results could contain false negatives (invalid failed results) or false positives (invalid passed results).

This is where *test fixtures* come in. A test fixture is a mechanism for ensuring proper test setup (putting tests into a known state) and test teardown (restoring the state prior to the test running). Test fixtures guarantee that our tests are running in predictable conditions, and thus the results are reliable.

Let's say we are testing a Bluetooth device. The device's Bluetooth module can sometimes fail. When this happens, the device needs to be power cycled (shut off and then on) to restore Bluetooth functionality. We would not want tests to run if the device was already in a failed state because these results would not be valid. Furthermore, if our tests cause the Bluetooth module to fail, we want to restore it to a working state after the tests run. So, we add a test fixture to power cycle the device before and after each test. Here is how we might do it:

```
def power_cycle_device():
    print('Power cycling bluetooth device...')

class BluetoothDeviceTests(unittest.TestCase):
    def setUp(self):
        power_cycle_device()

    def test_feature_a(self):
        print('Testing Feature A')

    def test_feature_b(self):
        print('Testing Feature B')

    def tearDown(self):
        power_cycle_device()
```

[Copy to Clipboard](#)

The `unittest`
[framework](#)

Preview: Docs Loading link description

automatically identifies setup and teardown methods based on their names. A

[method](#)

Preview: Docs Loading link description

named `setUp` runs before each test case in the class. Similarly, a method named `tearDown` gets called after each test case. Now, we can guarantee that our Bluetooth module is in a working state before and after every test. Here is the output when these tests are run:

```
Power cycling bluetooth device...
Testing Feature A
Power cycling bluetooth device...
.Power cycling bluetooth device...
Testing Feature B
Power cycling bluetooth device...
.
-----
Ran 2 tests in 0.000s

OK
```

[Copy to Clipboard](#)

Let's consider another scenario. Perhaps our tests rely on working Bluetooth, but there is nothing in the tests that would cause the bluetooth to stop working. In this case, it would be inefficient to power cycle the device before and after every test. Let's refactor the previous example so that setup and teardown only happen once - before and after all tests in the class are run:

```
def power_cycle_device():
    print('Power cycling bluetooth device...')

class BluetoothDeviceTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        power_cycle_device()

    def test_feature_a(self):
        print('Testing Feature A')

    def test_feature_b(self):
        print('Testing Feature B')

    @classmethod
    def tearDownClass(cls):
        power_cycle_device()
```

Copy to Clipboard

We replaced our `setUp` method with the `setUpClass` method and added the `@classmethod` [decorator](#). We changed the [argument](#)

Preview: Docs Loading link description

from `self` to `cls` because this is a class method. Similarly, we replaced the `tearDown` method with the `tearDownClass` class method. Now, we get the following output:

```
Power cycling bluetooth device...
Testing Feature A
Testing Feature B
Power cycling bluetooth device...
```

```
-----
Ran 2 tests in 0.000s
```

```
OK
```

Copy to Clipboard

In addition to calling functions, we can also use setup methods to instantiate objects and or gather any other data needed. Anything stored in our class will be available throughout our test functions.

It's generally good practice to create fixtures that run for every test. However, when a fixture has a large cost (i.e. it takes a long time), then it might make more sense to have it run once per test class rather than once per test. Let's practice setting up test fixtures!

Instructions

Checkpoint 1 Passed

1.

In our `tests.py` file we have some simple tests written for the passenger check-in experience at the kiosk for Small World Air. We also have some functions we are testing written in `kiosk.py`.

Take some time to review the provided code in both files. Run the code to continue!

Checkpoint 2 Passed

2.

We want to make sure the kiosk is powered on before we run any tests. This is a great time to setup some test fixtures! Create a `setUpClass()` method which takes a single argument (`cls`) and calls `kiosk.power_on_kiosk()`. Add the `@classmethod` decorator on top of it!

Python detects setup test fixtures that run once by looking for methods with the name `setUpClass`.

Checkpoint 3 Passed

3.

We don't want to leave the kiosk powered on after all tests are run.

Create a `tearDownClass()` method which takes a single argument (`cls`) and calls `kiosk.power_off_kiosk()`. Add the `@classmethod` decorator on top of it!

Python detects teardown test fixtures that run once by looking for methods with the name `tearDownClass`.

Checkpoint 4 Passed

4.

We also want to make sure that customers are on the welcome page before each test runs. Create a method called `setUp()`. Inside of the method, call `kiosk.return_to_welcome_page()`.

Python detects setup test fixtures that run on every test by looking for methods with the name `setUp`.

```
# Checkpoint 1
import unittest
import kiosk

class CheckInKioskTests(unittest.TestCase):

    def test_check_in_with_flight_number(self):
        print('Testing the check-in process based on flight number')

    def test_check_in_with_passport(self):
        print('Testing the check-in process based on passport')

# Checkpoint 2
@classmethod
```

```
def setUpClass(cls):
    kiosk.power_on_kiosk()

# Checkpoint 3
@classmethod
def tearDownClass(cls):
    kiosk.power_off_kiosk()

# Checkpoint 4
def setUp(self):
    kiosk.return_to_welcome_page()

unittest.main()
```

Unit Testing

Skipping tests

Sometimes we have tests that should only run in a particular context. For example, we might have a group of tests that only runs on the Windows operating system but not

[Linux](#)

Preview: Docs Loading link description

or macOS. For these situations, it's helpful to be able to skip tests.

The `unittest`

[framework](#)

Preview: Docs Loading link description

provides two different ways to skip tests:

The `@unittest` skip decorator

The `skipTest()` method

First, let's examine the skip decorator option. There are two decorator options to accomplish the goal of skipping a test. Let's observe both of them in the example below:

```
import sys

class LinuxTests(unittest.TestCase):

    @unittest.skipUnless(sys.platform.startswith("linux"), "This test only runs on Linux")
    def test_linux_feature(self):
        print("This test should only run on Linux")

    @unittest.skipIf(not sys.platform.startswith("linux"), "This test only runs on Linux")
    def test_other_linux_feature(self):
        print("This test should only run on Linux")
```

Copy to Clipboard

Let's break down both skip decorator options:

The `skipUnless` option skips the test if the condition evaluates to `False`.

The `skipIf` option skips the test if the condition evaluates to `True`.

Both share common requirements. Firstly, both of these skip decorators are prefaced with `@unittest` to denote the decorator pattern. They both take a condition as a first

[argument](#)

Preview: Docs Loading link description

, followed by a

[string](#)

Preview: Docs Loading link description

message as the second. In this example, both decorators achieve the same goal: skipping the test if the operating system is not Linux.

If we ran the tests on a macOS system, we would get the following output:


```

ss
-----
Ran 2 tests in 0.000s

OK (skipped=2)

```

Copy to Clipboard

The second way to skip tests is to call the `skipTest` [method](#)

Preview: Docs Loading link description of the `TestCase` class, as in this example:

```

import sys

class LinuxTests(unittest.TestCase):

    def test_linux_feature(self):
        if not sys.platform.startswith("linux"):
            self.skipTest("Test only runs on Linux")

```

Copy to Clipboard

Here we call `self.skipTest()` from within the test function itself. It takes a single string message as its argument and always causes the test to be skipped when called. When run on macOS we get the following output:

```

s
-----
Ran 1 test in 0.000s

OK (skipped=1)

```

Copy to Clipboard

Skip decorators are slightly more convenient and make it easy to see under what conditions the test is skipped. When the conditions for skipping a test are too complicated to pass into a skip decorator, the `skipTest` method is the recommended alternative.

Let's practice skipping tests in our Small World Air application!

Instructions

Checkpoint 1 Passed

1.

Small World Air continues to grow! They recently added some regional jets for shorter flights. Unfortunately, these planes don't have an onboard entertainment system.

Let's modify our `EntertainmentSystemTests` to make sure these tests get skipped on regional jets. Use the `@unittest.skipIf()` decorator to `test_movie_license()`.

It should skip the test if `entertainment.regional_jet()` returns `True`. The message should output `'Not available on regional jets'`.

The `unittest.skipIf()` takes two arguments. The first is a condition (it can be the result of a functional call). If the condition is `True`, the test is skipped. The second argument is a string which is the skip message.

Checkpoint 2 Passed

2.

Add the `@unittest.skipUnless` decorator to `test_wifi_status()`.

It should skip the test unless `entertainment.regional_jet()` returns `False`. The message should be `'Not available on regional jets'`.

`unittest.skipUnless()` takes two arguments. The first is a condition (it can be the result of a functional call). If the condition is `False`, the test is skipped. The second argument is a string which is the skip message.

Checkpoint 3 Passed

3.

Inside of `test_device_temperature()`, add an `if` statement which calls `entertainment.regional_jet()`. If the return value is `True`, then call `self.skipTest()` with an argument of `'Not available on regional jets'`.

Add the same `if` block to `test_maximum_display_brightness()`.

`skipTest()` does not take a condition argument - it always skips the test when called. It takes a message as its only argument.

```

import unittest
import entertainment

class EntertainmentSystemTests(unittest.TestCase):
    # Checkpoint 1

```

```

@unittest.skipIf(entertainment.regional_jet(), 'Not available on regional jets')
def test_movie_license(self):
    daily_movie = entertainment.get_daily_movie()
    licensed_movies = entertainment.get_licensed_movies()
    self.assertIn(daily_movie, licensed_movies)

# Checkpoint 2
@unittest.skipUnless(entertainment.regional_jet() is False, 'Not available on regional jets')
def test_wifi_status(self):
    wifi_enabled = entertainment.get_wifi_status()
    self.assertTrue(wifi_enabled)

# Checkpoint 3
def test_device_temperature(self):
    if entertainment.regional_jet():
        self.skipTest('Not available on regional jets')
    device_temp = entertainment.get_device_temp()
    self.assertLess(device_temp, 35)

# Checkpoint 3
def test_maximum_display_brightness(self):
    if entertainment.regional_jet():
        self.skipTest('Not available on regional jets')
    brightness = entertainment.get_maximum_display_brightness()
    self.assertAlmostEqual(brightness, 400)

unittest.main()

```

Unit Testing

Expected Failures

Sometimes we have a test that we know will fail. This could happen when a feature has a known bug or is designed to fail on purpose. In this case, we wouldn't want an expected failure to cloud our test results. Rather than simply skipping the test, `unittest` provides a way to mark tests as expected failures. Expected failures are counted as passed in our test results. If the test passes when we expected it to fail, then it is marked as failed in test results.

To setup a test to have an expected failure, we can use the `expectedFailure` decorator. Let's consider the following example:

```

class FeatureTests(unittest.TestCase):

    @unittest.expectedFailure
    def test_broken_feature(self):
        raise Exception("This test is going to fail")

```

Copy to Clipboard

The `expectedFailure` decorator takes no arguments. The test in the example will always fail because an exception was raised during test execution. When run, we get the following output:

```

x
-----
Ran 1 test in 0.000s

OK (expected failures=1)

```

Copy to Clipboard

The test failure did not cause any failures in our test results because it was marked as expected. Let's do the same for some of our Small World Air tests.

Instructions

Checkpoint 1 Passed

1.

The monitors in front of passenger seats on Small World planes contain a customer feedback portal. The class `CustomerFeedbackTests` has tests for both the survey and the complaint form. We can find the tests in `tests.py` and the functions that perform feedback tasks in `feedback.py`. Take some time to get acquainted with the program. Run the code to move on!

Checkpoint 2 Passed

2.

Looks like `test_survey_form()` caught a bug in the customer survey form! It's best not to have this test fail every day while we wait for the bug to get fixed.

Use the `expectedFailure` decorator to mark this test as an expected failure!

The `@unittest.expectedFailure` decorator marks a test function as an expected failure. It expects no arguments and is not executed using parenthesis `()`.

```
import unittest
import feedback

class CustomerFeedbackTests(unittest.TestCase):

    # your code below:
    @unittest.expectedFailure
    def test_survey_form(self):
        self.assertEqual(feedback.issue_survey(), 'Success')

    def test_complaint_form(self):
        self.assertEqual(feedback.log_customer_complaint(), 'Success')

unittest.main()
```

Unit Testing

Review

Awesome job! We've covered a lot of material related to

[unit testing](#)

Preview: Docs Loading link description

in Python. We learned:

- The difference between manual and automated testing.

- What unit tests are.

- How to write simple tests with the `assert` keyword.

- How to create and run test cases with the `unittest`

- [framework](#)

Preview: Docs Loading link description

.

- Best practices for test fixtures, test parameterization, skipped tests and expected failures.

The world of

[software testing](#)

Preview: Docs Software testing refers to the process of verifying and evaluating the function of a software application or product.

is vast and can take time to master, but the basic principles of unit testing will almost always be applicable to any language we work with. Incorporating testing into our software is the best way to prevent unexpected bugs from occurring. The sooner we write tests, the faster we can catch and fix bugs and make our software better!

Instructions

Checkpoint 1 Enabled

1.

The code in `monitor.py` adds some new functionality to the monitor in the Small World plane seats. Read through it and see what it does. Can you create some unit tests for these functions? Add your tests to `tests.py`, and have fun!

```
import warnings

class CustomerRequestWarning(Warning):
    pass

def calculate_remaining_time(km):
    """
    This function estimates the remaining flight time when given the kilometers that remain for
    this flight. The function assumes an average speed of 15 km per minute and rounds down when
    calculating minutes. It returns a tuple of hours and then minutes remaining.
    """
    total_minutes = int(km / 15)
    minutes = total_minutes % 60
    hours = int(total_minutes / 60)
    return (hours, minutes)

def request_flight_attendant():
    """
    This function raises a CustomerRequestWarning to let the flight attendants know that they are
    requested.
    """
    warnings.warn('A customer has requested attention!', CustomerRequestWarning)
```

What is Concurrent Programming

[Learn about concurrent programming!](#)

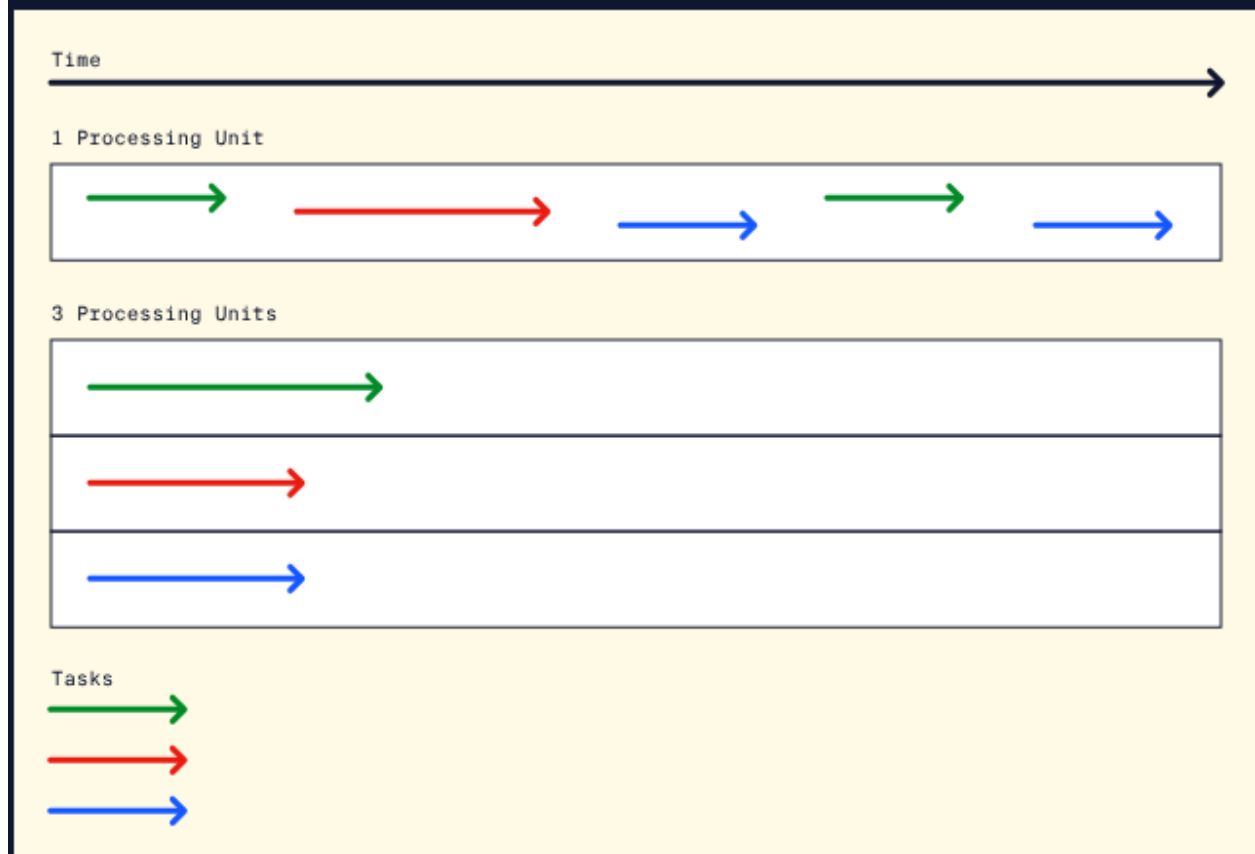
Introduction

In this article, we are going to talk about the following topics:

- Sequential Programming
- Concurrency and Parallelism
- Asynchronous Programming
- threading, multiprocessing, and asyncio modules

Sequential Programming

As you have gone along your Python journey, you have most likely worked with *sequential programs*. These are programs that follow a set order of instructions. We can view this on the following diagram:



For example, if we have the following program:

```
for i in range(4):  
    print(i)  
print("end program")
```

we will see the numbers 0-3 printed in order as the program follows the sequence of events:

```
print(0)  
print(1)  
print(2)
```

```
print(3)
print("end program")
```

Great! We have a deterministic algorithm that suits our needs. However, in this program, we have four steps that do not take a long time. Let's imagine we are a data scientist working on a complex learning model or a backend developer working with an extensive database. Suddenly, we are dealing with many more steps that take a lot more processing power.

In situations like this, the concepts of *concurrent programming* and *parallel programming* come into play. To give us a basic intuition behind each of these paradigms, let's picture three separate scenarios of ordering food at a deli.

Analogies

We can view sequential programming as a line of customers with access to one register. In this scenario, each customer has to wait in a single-file line until they can make their order. This is fine if there are only a few customers. However, as the number of customers piles up, this can be incredibly inefficient for the deli.

We can view concurrent programming as multiple lines of customers with access to one register. In this model, we might have three separate lines. The cashier can pick one of the customers who is ready to order from the front of a line. This can speed up ordering for the deli as if a customer decides they are not quite ready to order yet, they can let one of the other people in the front of the line go ahead.

We can view parallel programming as multiple lines of customers with access to multiple registers. In this model, there are multiple registers that can divvy up the customers. This way, the deli is more likely to avoid long lines and run a more efficient operation.

Finally, we can also have asynchronous programs. If we add this paradigm to our analogy, we can view it as having a wait queue where each customer has a ticket. Once a customer's food is ready, their ticket gets called. However, the tickets don't necessarily get called in order; instead, any ticket can be called once the customer's food is ready. Sequential (synchronous) would mean that customers would have to wait for each customer before them to get their food.

Defining Technical Terms

Now that we have some high-level understanding of these concepts, let's get back to programming mode and develop technical definitions for these terms.

Concurrency is the process in which we have multiple tasks running and completing during overlapping periods of time.

Parallelism is the process in which we simultaneously have multiple tasks or separate parts of the same task running using multiple CPUs (core processing units). These definitions seem quite similar; however, we should identify some key difference between the two processes:

Parallelism needs hardware with multiple processing units, whereas concurrency only utilizes one.

Concurrency requires at least two tasks to exist whereas parallelism only requires one.

Parallelism assigns each task for a core to execute, whereas concurrency executes all tasks by switching tasks simultaneously.

Parallelism means we can do multiple things at once, while concurrency means we can juggle between tasks. These diagrams should remind us of our deli line analogy from before. Think of each processing unit as a register and each task as a line. With one core, we can split up the tasks cleverly to speed up the runtime. With multiple cores, we can run those tasks at the same time.

Methods in Python

In Python, we are going to go over the following libraries:

```
threading
multiprocessing
asyncio
```

Using these, we can create concurrent, parallel, and asynchronous programs that are much faster than synchronous ones.

We can map these libraries one-to-one as:

```
threading -> concurrency
multiprocessing -> parallelism
asyncio -> asynchronous
```

In the next few lessons, we will learn about the concepts of threads and processes. Then we will see how to use these concepts to create powerful and efficient programs and see our conceptual understanding of concurrency, parallelism, and asynchronous programming come to life!

- Sequential Programming
- **Concurrency and Parallelism**
- **Asynchronous** Programming
threading, multiprocessing, and asyncio modules

Processes and Threads

Introduction to Processes

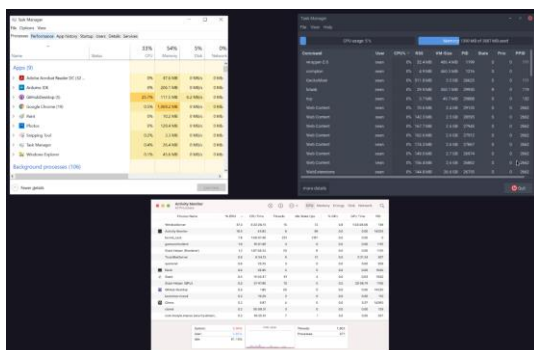
When using a computer, multiple programs can often be found running at the same time. Perhaps one for playing music, another for creating documents, and one for browsing the web. All of these programs have certain functionalities, but on their own they do nothing. To actually make use of them, they must be executed.

While a computer program is a static collection of coded instructions stored on a disk, a *process* is an abstraction representing the program when it is running. A process is created when a program is executed. These processes are not only central for the usability of a computer, but they are the building blocks of an operating system. Managing these processes is central to operating system development.

Processes can sometimes also be called “tasks” or “jobs”, although these definitions are ambiguous. The key defining factor is that processes generally operate independently and do not share data; for example, a music player program will launch a music player process that would be independent of the process managing an office suite.

Instructions

1. All of the processes running on your system can be found in the operating system’s process manager:
 - Task Manager in Windows
 - Activity Monitor in MacOS
 - System Monitor or top in most Linux Distributions



Processes and Threads

Lifecycle of a Process

To best optimize the performance of processes as their priority changes or as they wait for access to a limited resource, processes are put into one of five states:

New: The program has been started and waits to be added into memory in order to become a full process.

Ready: Process fully initialized, loaded into memory, and waiting to be picked up by the processor.

Running: Currently being executed by the processor.

Blocked: The process requires a contested resource that it must wait for.

Finished: The process has been completed.

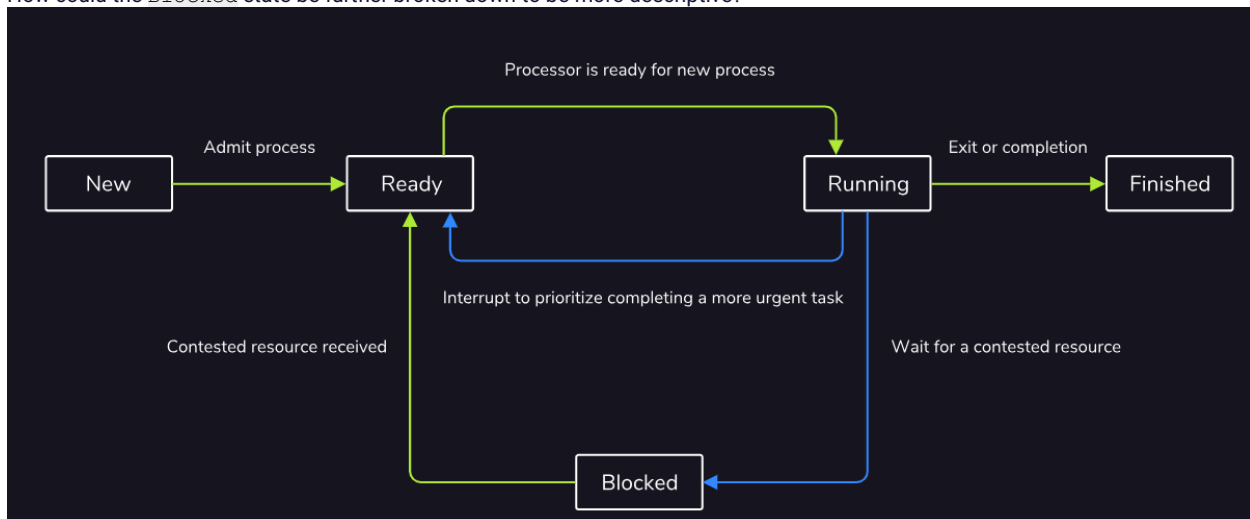
The life cycle of a process is its journey between these five states, beginning with **New** and ending with **Finished**. As **CPU** cores traditionally only executed one task at a time, managing the state of processes allows the processor to interleave these tasks and allows multiple processes to best share these cores and other limited computer resources. For example, instead of a process occupying the processor while waiting for user input, it can be marked as blocked to have the processor focus on another process in the ready state until that input arrives.

Blocking isn't inherently negative as some tasks require more time. Marking these processes as blocked allows the processor to prioritize other tasks, creating a more responsive and efficient system. Similarly, some processes may also be reverted to the **Ready** state through preemption, where tasks are temporarily interrupted by an external scheduler for urgent reasons, such as a hardware interrupt signal asking the system to shutdown.

All of these switching processes do come with overhead that is best to be avoided. This is called *context switching* and is typically an expensive operation as the current state of the process needs to be stored and then be reloaded later to resume execution.

Instructions

How could the **Blocked** state be further broken down to be more descriptive?



Processes and Threads

Process Layout and Process Control Block

When a process is initialized, its layout within memory has four distinct sections:

- A text section for the compiled code

- A data section for initialized variables

- A stack for local

- [variables](#)

- Preview: Docs Loading link description

- defined within functions

- A heap for dynamic memory allocation

Processes are also initialized with a *Process Control Block* that is required by the operating system for managing the process. This contains:

- A unique process ID and the ID of any parent processes that launched the current one

- The current process state

How long the process has been running and any time limits the process may have
Allowed system resources and other permissions
The priority of the process
The program counter for the address of the instruction currently being executed
The address of other registers within the
[CPU](#)

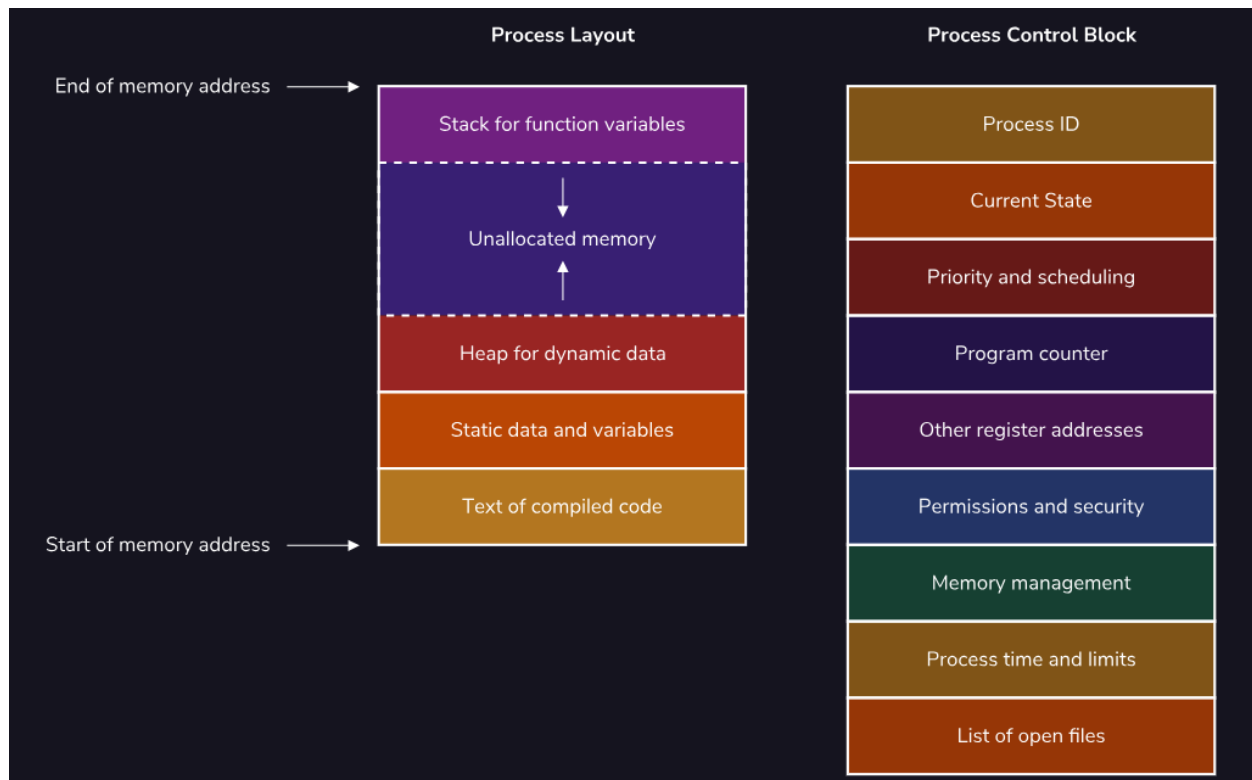
Preview: Docs Loading link description
holding intermediate values

Information required for memory management such as page and segment tables

Additionally, when one process launches another, the original enters a parent-child relationship with the newly-launched process that shares much of the above data. For example, when an existing music player process starts a new process for scanning the user's music library, both of these processes generally share the same system resources and permissions. Parent processes usually also wait for their children to complete before terminating themselves, unless the child was created specifically to run independently in the background.

Instructions

What other information may be useful to store on the Process Control Block?



Processes and Threads

Introduction to Threads

While a process is an abstract data structure that represents all of the necessary information to run a program, a *thread* represents the actual sequence of processor instructions that are actively being executed.

Each process contains at least one thread to be able to execute, although more can be created to allow for concurrent processing if it is supported by the

[CPU](#)

Preview: Docs Loading link description

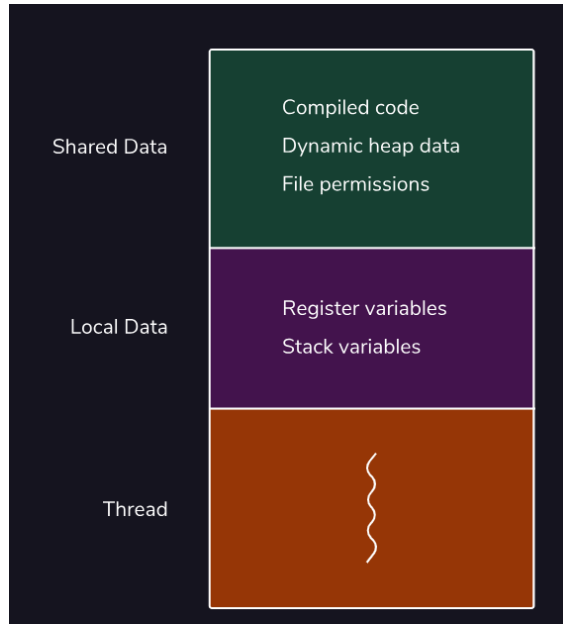
. These threads live within the process and share all of the common resources available to it, such as memory pages and active files, as shown in the image to the right.

These shared resources are critical for the definition of a thread. While each process is typically independent, multiple threads usually work together within the context of a process. By sharing data directly, there is faster communication and context switching between threads than what is possible for processes, all while taking fewer system resources.

For example, within a video game process, multiple threads may exist to manage separate services relating to the operation of the game, such as one thread for collecting user inputs and another for producing sounds. As these threads live within the same process, they can easily share information about the game, such as the type of ground the player is walking on. This can be used to affect both the speed the character moves from the input thread as well as the noises created by the sound thread.

Instructions

Why might a browser application prefer to create each tab as a separate process instead of a thread?



Processes and Threads

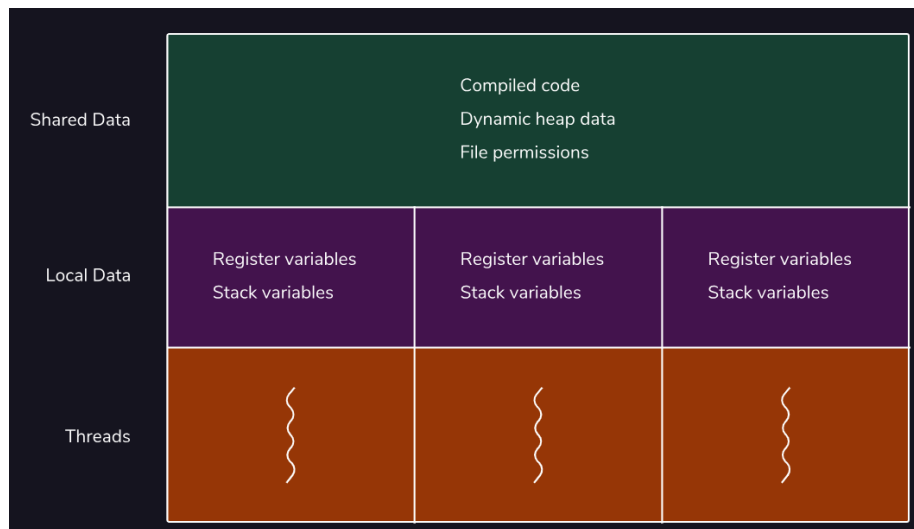
Multithreading

Typically, a single [CPU](#) core can only execute one thread, and therefore one process, at a time. With a clever use of blocking and context switching, this limitation can be obscured to users through nanosecond-long pauses that allow processes to be completed near-simultaneously. With some hardware advances, single CPU cores can now execute multiple threads at once, which is a capability called *multithreading*.

Parallelizing computations have a variety of benefits, such as improved system utilization and system responsiveness. This is because tasks can be more evenly split between multiple threads, exhausting all available computing resources and allowing longer tasks to run in the background, separate from user input. The image to the right shows how threads share data to achieve this.

However, these optimizations come with disadvantages due to the additional complexity required for the implementation. Not only are these programs more difficult to write because of their non-sequential nature, but they also create whole new classes of bugs.

The two of the most common examples are data races, where multiple threads attempt to modify the same piece of data, and deadlocks, where multiple threads all attempt to wait for each other and freeze the system. Also, since these bugs are usually related to the tight timing of CPU interactions, the programs can be considered non-deterministic and therefore untestable, compounding the problem.



Processes and Threads

Kernel Threads vs User Threads

Threads can behave differently depending on the environment they are created in.

A thread built into the existing process is considered a *kernel thread*. This means that the kernel within the operating system is fully aware of these threads and directly manages their execution.

There are also *user threads* that exist solely in userspace and, while functionally identical, are not known or controlled by the kernel. This allows for more fine-grained control by developers. These threads are even more efficient than their kernel counterparts as they save on the costly indirection of making a system call to constantly interact with the kernel.

While these user threads typically operate independently of the kernel, they do need to be mapped to existing kernel threads in order to have the operating system execute them. There are three common models for mapping user threads to kernel threads, as shown in the image to the right:

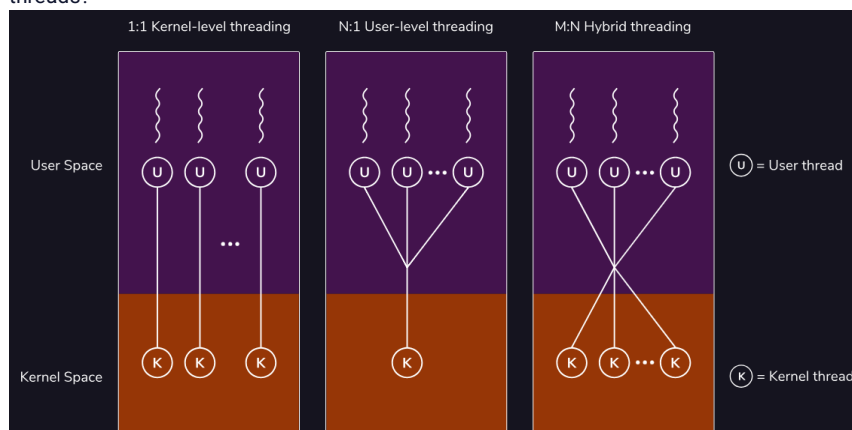
1:1 Kernel-level threading for a simple implementation that best allows for hardware acceleration provided by the kernel threads.

N:1 User-level threading for ultra-light threads that can quickly communicate and context switch, but do not benefit from hardware acceleration due to sharing the same kernel thread.

M:N Hybrid threading to get the best of both of the above solutions: very light and fast threads that can be hardware accelerated as necessary. However, this complex implementation can lead to bugs such as priority inversion where less important tasks are mistakenly prioritized and run first.

Instructions

Which mapping of user threads to kernels threads would be preferred if the computer hardware does not support multiple kernel threads?



Processes and Threads

Review and Wrap-up

Congratulations! You have finished learning about some of the key foundations of an operating system: the processes and threads that all of the other code on the system rests upon.

A process is an abstraction within the operating system that represents the program while it is in execution. These processes exist in five states that are leveraged to allow the [CPU](#) cores to alternate between ready and blocked processes to best take advantage of limited computing resources.

A thread represents the actual sequence of processor instructions that are actively being executed. Each process contains at least one thread and can contain many such structures that all share resources among each other to allow for faster communication and context switching between them. This all allows them to be “lighter” and require fewer system resources. With the hardware advancement of multithreading, individual cores can also execute multiple threads at once, further improving system utilization and responsiveness by more efficiently splitting up tasks.

Threads behave differently depending on the environment they were created in. Kernel threads are constructed through system calls to the kernel while user threads are constructed using local function calls. User threads, therefore, allow for more fine-grained control by developers that can be more efficient than their kernel counterparts. However, these user threads have to be mapped to their kernel counterparts in order to be actually executed.

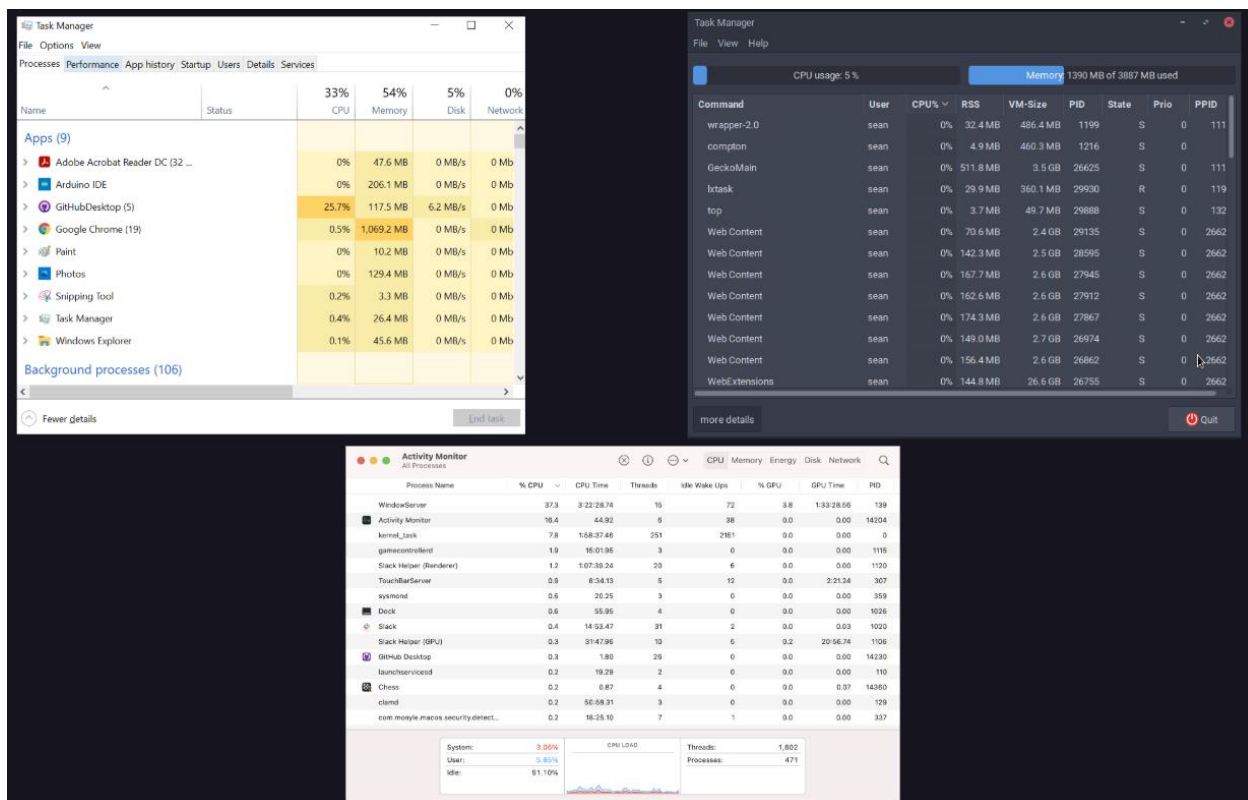
Instructions

1. All of the processes running on your system can be found in the operating system’s process manager:

Task Manager in Windows

Activity Monitor in MacOS

System Monitor or top in most Linux Distributions



Concurrent Programming in Python

Introduction

So far, we have seen what concurrent programming is and learned about processes and threads. Now we are going to apply our knowledge of these concepts using Python! In this lesson, we are going to learn about three modules:

```
threading
asyncio
multiprocessing
```

These modules can help speed up programs and provide design clarity. We will walk through what they are and practice writing short programs with them. Along the way, think back to the previous article and lesson you read. Make sure you understand the connection between these modules and the concepts we have covered already.

Let's get started!

Instructions

Checkpoint 1 Enabled

1.

Much of this lesson will involve comparing the efficiency and behavior of sequential programming with concurrent programming.

In **script.py**, you should see a sequential program with a timer. We will use the timer throughout this lesson to track how long each program takes to run.

After you have looked over the code a bit, click **Run** to move onto the next exercise.

```
import time
def sequential():
    s = time.perf_counter()
    print("Codecademy")
    time.sleep(2)
    print("says hello!")
    elapsed = time.perf_counter() - s
    print("Sequential Programming Elapsed Time: " + str(elapsed) + " seconds")

sequential()
```

Concurrent Programming in Python

The Threading Module

In the previous lesson, we saw what a thread is. Now let's see how we can speed up our Python programs with the `threading` module!

To review, a *thread* is a unique flow of execution. Theoretically, multiple threads mean the ability to run multiple things at the same time. However, in Python, threads do not actually run simultaneously; they merely appear to do so. Still, we will find that threading can increase the speed of our program. Generally, threading will perform much better than sequential programming if tasks within your program spend a lot of time waiting for external events to occur.

To create a thread instance in Python, we use the following code:

```
import threading
example_thread = threading.Thread(target=some_function, args=(some_arg,))
```

Copy to Clipboard

The two main parameters we will focus on are:

`target`: this is the function you want to execute with thread(s). It defaults to `None`.

`args`: this is the argument or set of arguments applied to the target function. It is a tuple and defaults to `None`.

For example, let's say we wanted to apply a thread to the function called `analyze_list()` with arguments `11, 12, 13`. We would use the following code:

```
t = threading.Thread(target=analyze_list, args=(11, 12, 13))
```

Copy to Clipboard

After creating our thread instance, we also have to "start" our thread using `.start()`. Therefore, to make sure `t` executes on Run, we write:

```
t.start()
```

Copy to Clipboard

Let's practice with an example!

Instructions

Checkpoint 1 Passed

1.

Let's apply a thread to the sequential example from our previous exercise. In `script.py`, we have a function called `greeting_with_sleep()`.

To start, call `greeting_with_sleep()` with `'Codecademy'` as the input. Inside the terminal, type `python3 script.py` and press Enter. To check your answer, click [Check Work](#) once your program finishes running. How long does the code take to execute?

You should see this in your output terminal:

```
Codecademy
says hello!
Sequential Programming Elapsed Time: 2.001893345001008 seconds
```

Copy to Clipboard

Note that the elapsed time may be very slightly different.

Checkpoint 2 Passed

2.

Let's try running `greeting_with_sleep()` with a single thread.

Create a thread object called `t`. This thread should execute `greeting_with_sleep()` with `'Codecademy'` as the argument.

Inside the terminal, type `python3 script.py` and press Enter. To check your answer, click [Check Work](#) once your program finishes running. Does the program run faster now? What do you think is happening?

Your threading code should look like the following:

```
t = threading.Thread(target=greeting_with_sleep, args=('Codecademy',))
t.start()
```

Copy to Clipboard

The code runs at essentially the same speed with or without a thread. This is because right now, we have one task and one thread. There isn't any way to speed up the task. In the next exercise, we will see how we can use multiple threads to speed things up!

```
import time
import threading
def greeting_with_sleep(string):
    s = time.perf_counter()
    print(string)
    time.sleep(2)
    print("says hello!")
    elapsed = time.perf_counter() - s
    print("Sequential Programming Elapsed Time: " + str(elapsed) + " seconds")

greeting_with_sleep('Codecademy')

t = threading.Thread(target=greeting_with_sleep, args=('Codecademy',))
t.start()
```

Concurrent Programming in Python

Using Multiple Threads

We have seen how to create a thread in Python; however, we didn't see any speed or efficiency benefits in our previous example. To see the benefits of threading, let's learn how to create multiple threads and test them out in our example.

When creating multiple threads throughout this course, we will use one of the following approaches:

```
# create each thread
t1 = threading.Thread(target=target_function, args=(arg1,))
t2 = threading.Thread(target=target_function, args=(arg2,))
t3 = threading.Thread(target=target_function, args=(arg3,))
# start each thread
t1.start()
t2.start()
t3.start()
```

Copy to Clipboard

or

```
threads = []
# list of arguments to use
args = [arg1, arg2, arg3]
# iterate through the length of arguments
for i in range(len(args)):
    # create thread
    t = threading.Thread(target=target_function, args=(args[i],))
    # add thread to threads list
    threads.append(t)
    # start thread
    t.start()
```

Copy to Clipboard

The benefit of the second approach is that it keeps track of our threads in a list called `threads`. In the next exercise, we will see why this is helpful. Therefore, we will follow this approach going forward.

Instructions

Checkpoint 1 Passed

1.

In **script.py**, we have some code set up for you. `greeting_with_sleep()` is loaded in; however, this time, it does not have the timer code. Instead, a function called `main_threading()` contains the timer code. In this function, you will write code to create and initiate four threads.

Inside `main_threading()`, write a for loop that iterates through `greetings`. Inside this for loop:

- Create a thread called `t` that has `greeting_with_sleep()` as its target and takes one arg from `greetings` at each iteration.

- Start the created thread.

For now, do not worry about creating a threads list.

Finally, call `main_threading()`.

Inside the terminal output, type `python3 script.py` and press **Enter**. To check your answer, click **Check Work** once your program finishes running.

Does your output make sense?

To implement the for loop, you should use the same approach we used above:

```
threads = []
# list of arguments to use
args = [arg1, arg2, arg3]
# iterate through the length of arguments
for i in range(len(args)):
    # create thread
    t = threading.Thread(target=target_function, args=(args[i],))
    # add thread to threads list
    threads.append(t)
    # start thread
    t.start()
```

Copy to Clipboard

This time `target_function()` should be `greeting_with_sleep` and `args` should be `greetings`.

Checkpoint 2 Passed

2.

In the terminal, you should see something similar to the following:

```
Codecademy
Chelsea
Hisham
Ashley
Threading Elapsed Time: 0.0005821380036650226 seconds
says hello!
says hello!
says hello!
says hello!
```

Copy to Clipboard

You also may have noticed that the following showed up instantaneously:

```
Codecademy
Chelsea
Hisham
Ashley
Threading Elapsed Time: 0.0005821380036650226 seconds
```

Copy to Clipboard

But after about two seconds, the next messages showed up:

```
says hello!
says hello!
says hello!
says hello!
```

Copy to Clipboard

We will explore this some more in the next exercise. However, to give you a brief explanation. Each thread has started, *but* Python is waiting for the last thread to finish before terminating. Therefore, while `main_threading()` has completed, `greeting_with_sleep()` is still executing the final thread.

To move onto the next exercise, click [Check Work](#).

Concurrent Programming in Python

Joining a Thread

In the previous exercise, we started working with multiple threads and noticed some interesting behavior. We will investigate this behavior more with the `.join()` threading method.

We can use `.join()` to tell one thread to wait for this thread to stop before moving on. Let's see how we implement this and then try it out in our previous example.

We use `.join()` after each thread has already been initiated.

Click here if you need a reminder of what the first approach looked like.

```
# create each thread
t1 = threading.Thread(target=target_function(), args=(arg1,))
t2 = threading.Thread(target=target_function(), args=(arg2,))
t3 = threading.Thread(target=target_function(), args=(arg3,))
# start each thread
t1.start()
t2.start()
t3.start()
```

Copy to Clipboard

With the first approach from the previous exercise, we do:

```
t1.join()
t2.join()
t3.join()
```

Copy to Clipboard

Click here if you need a reminder of what the second approach looked like.

```
threads = []
# list of arguments to use
args = [arg1, arg2, arg3]
# iterate through the length of arguments
for i in range(len(args)):
    # create thread
    t = threading.Thread(target=target_function, args=(arg[i],))
    # add thread to threads list
    threads.append(t)
    # start thread
    t.start()
```

Copy to Clipboard

With the second approach from the previous exercise, we do:

```
for t in threads:
    t.join()
```

Copy to Clipboard

The second approach is much easier for keeping track of our threads and is much more efficient if we have numerous.

Instructions

Checkpoint 1 Passed

1.

In **script.py**, the code loaded in is almost identical to the final code you had in the previous exercise, but an empty list called `threads` has been added to `main_threading()`.

Within the for loop, beneath `# add append code here`, write a line of code so that `t` is added to list `threads`.

Inside the terminal, type `python3 script.py` and press `Enter`. To check your answer, click `Check Work` once your program finishes running.

You should add the following line of code in the for loop:

```
threads.append(t)
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Underneath `# add join code here`, write a for loop that traverses `threads` and call `.join()` for each thread in `threads`.

Inside the terminal, type `python3 script.py` and press `Enter`. To check your answer, click `Check Work` once your program finishes running.

Is your output different than the output of the previous exercise?

Checkpoint 3 Passed

3.

You should see something similar to the following output:

```
Codecademy
Chelsea
Hisham
Ashley
says hello!
says hello!
says hello!
says hello!
Threading Elapsed Time: 2.002612964999571 seconds
```

Copy to Clipboard

Let's walk through this. Now that we are using `.join()`, `main_threading()` does not complete until each thread has been executed. Therefore, we get a more accurate measurement of two seconds. If we were to run this with sequential programming, it would take eight seconds since calling:

```
greeting_with_sleep('Codecademy')
greeting_with_sleep('Chelsea')
greeting_with_sleep('Hisham')
greeting_with_sleep('Ashley')
```

Copy to Clipboard

would run into `time.sleep(2)` four times.

You may also wonder why the output order is the names, and then the greeting "says hello!". This is because when each thread is blocked by `time.sleep(2)` they begin to work concurrently to complete each task. In the next checkpoint, we'll see some interesting behavior resulting from this concurrency.

Click [Check Work](#) to move onto the next task.

Checkpoint 4 Passed

4.

Change the following print statement:

```
print("says hello!")
```

Copy to Clipboard

to:

```
print(string + " says hello!")
```

Copy to Clipboard

Before running **script.py**, make a prediction about what the order of the outputs will be?

Inside the terminal, type `python3 script.py` and press Enter. To check your answer, click [Check Work](#) once your program finishes running.

Run **script.py** a few times. The order is not deterministic! Essentially, the order the threads finish in is random, but the speed at which they finish, ~2 seconds, is consistent.

The randomness has to do with the operating system and is beyond the scope of this course, but it is an interesting thing to note!

```
import time
import threading
def greeting_with_sleep(string):
    print(string)
    time.sleep(2)
    print("says hello!")

def main_threading():
    s = time.perf_counter()
    # your code goes here
    greetings = ['Codecademy', 'Chelsea', 'Hisham', 'Ashley']
    for i in range(len(greetings)):
        t = threading.Thread(target=greeting_with_sleep, args=(greetings[i],))
        t.start()
    elapsed = time.perf_counter() - s
    print("Threading Elapsed Time: " + str(elapsed) + " seconds")

main_threading()
```

```
import time
import threading
def greeting_with_sleep(string):
    print(string)
    time.sleep(2)
    print(string + " says hello!")

def main_threading():
    s = time.perf_counter()
    threads = []
    greetings = ['Codecademy', 'Chelsea', 'Hisham', 'Ashley']
    for i in range(len(greetings)):
        t = threading.Thread(target=greeting_with_sleep, args=(greetings[i],))
        t.start()
        # add append code here
```

```

    threads.append(t)
    # add join code here
    for t in threads:
        t.join()

    elapsed = time.perf_counter() - s
    print("Threading Elapsed Time: " + str(elapsed) + " seconds")

main_threading()

```

Concurrent Programming in Python

The Asyncio Module

Now we will cover another concurrent programming model: the `asyncio` module.

The `asyncio` module uses `async/await` syntax. `async` and `await` are two keywords that allow you to build and execute asynchronous code in your programs.

The `async` keyword declares a function as a *coroutine*. Coroutines are functions that may return normally with a value or may suspend themselves internally and return a continuation. This is a fancy way of saying they allow tasks to be paused and resumed to mimic multitasking. This is conceptually very similar to what we saw with threads! Coroutines are at the heart of asynchronous programs in Python.

The `await` keyword suspends execution of the current task until whatever is being “await”ed on is completed. For example, if we have an “await function task2” within a coroutine “task1” this tells Python “Suspend task1 until task2 is completed.”

Let’s apply this to some real examples and see how we declare asynchronous functions in Python. The following code block defines `hello_async()`, prints “hello”, waits three seconds, and prints “how are you?”.

```

import asyncio
async def hello_async():
    print("hello")
    await asyncio.sleep(3)
    print("how are you?")

```

Copy to Clipboard

To run the coroutine, we have to use the following syntax:

```

loop = asyncio.get_event_loop()
loop.run_until_complete(hello_async())

```

Copy to Clipboard

Note: this syntax has been updated as of Python 3.7. It is now:

```

asyncio.run(hello_async)

```

Copy to Clipboard

We will have to use the longer syntax in this lesson and in future projects due to the Python version we run in our workspace. We apologize for this inconvenience.

Instructions

Checkpoint 1 Passed

1.

Let’s change `greeting_with_sleep_async()` into an asynchronous function. To do this:

Add in the `async` keyword.

Replace `time.sleep(2)` with the `await` keyword.

Inside the terminal output, type `python3 script.py` and press `Enter`. To check your answer, click `Check Work` once your program finishes running.

Remember to define a coroutine function:

```

async def some_function():

```

Copy to Clipboard

To replace `time.sleep(2)`:

```

await asyncio.sleep(2)

```

Copy to Clipboard

Checkpoint 2 Passed

2.

Use the two lines of `loop` syntax noted in this lesson to run your asynchronous function. Call your function with `string` set equal to `'Codecademy'`. What do you think the output will be?

Inside the terminal output, type `python3 script.py` and press Enter. To check your answer, click Check Work once your program finishes running.

To call `greeting_with_sleep_async()`:

```
loop = asyncio.get_event_loop()
loop.run_until_complete(greeting_with_sleep_async('Codecademy'))
```

```
import time
import asyncio

async def greeting_with_sleep_async(string):
    s = time.perf_counter()
    print(string)
    await asyncio.sleep(2)
    print("says hello!")
    elapsed = time.perf_counter() - s
    print("Asyncio Elapsed Time: " + str(elapsed) + " seconds")

loop = asyncio.get_event_loop()
loop.run_until_complete(greeting_with_sleep_async('Codecademy'))
```

Concurrent Programming in Python

Multiple Asynchronous Tasks

Let's use our knowledge of `async/await` syntax to do some complicated tasks.

If we wanted to run multiple tasks, we can do a setup that is similar to how we created multiple threads. Let's walk through the following code:

```
async def main():
    tasks = [task1(arg1), task2(arg2), task3(arg3)]
    await asyncio.gather(*tasks)
```

Copy to Clipboard

Breaking this down, we define `main()` as a coroutine function. `tasks` is a list of separate function calls. Note that each of `task1()`, `task2()`, and `task3()` are each coroutine functions.

The next line is where the magic happens. `asyncio.gather()` groups all of our tasks together and allows them to be run concurrently. You can read more about it [here](#). It is given the `await` syntax. Finally `*tasks` unpacks the `tasks` lists.

Let's practice this in our own example!

Instructions

Checkpoint 1 Passed

1.

Inside of `main_async()` below `# your code goes here`, use the `.gather()` `asyncio` method to group together the tasks in `greetings`.

Inside the terminal output, type `python3 script.py` and press Enter. To check your answer, click Check Work once your program finishes running.

Remember that you should call:

```
await asyncio.gather()
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Call your `main_async()` function.

Save your current event loop to the variable `loop`. Then run your loop with `.run_until_complete()`.

What do you think the output will be? How long do you think the execution will take?

Inside the terminal output, type `python3 script.py` and press Enter. To check your answer, click Check Work once your program finishes running.

Use `.get_event_loop()` to save your current event loop. Then when you run the loop make sure `main_async()` is the argument inside `.run_until_complete()`.

The loop takes about two seconds to run. If we were to run it sequentially, it would take about eight seconds. For asynchronous methods, it increases speed because when each task reaches `await asyncio.sleep(2)`, the function tells the event loop nothing is happening and pushes for the next task to occur. This allows all three calculations to run in about two seconds rather than eight seconds.

Checkpoint 3 Passed

3.

Change the following print statement:

```
print("says hello!")
```

Copy to Clipboard

to:

```
print(string + " says hello!")
```

Copy to Clipboard

Before running **script.py**, make a prediction about what the order of the outputs will be?

Inside the terminal, type `python3 script.py` and press Enter. To check your answer, click [Check Work](#) once your program finishes running.

This time the outputs are deterministic and in order!

```
import time
import asyncio

async def greeting_with_sleep_async(string):
    print(string)
    await asyncio.sleep(2)
    print(string + " says hello!")

async def main_async():
    s = time.perf_counter()
    greetings = [greeting_with_sleep_async('Codecademy'), greeting_with_sleep_async('Chelsea'),
greeting_with_sleep_async('Hisham'), greeting_with_sleep_async('Ashley')]
    # your code goes here
    await asyncio.gather(*greetings)

    elapsed = time.perf_counter() - s
    print("Asyncio Elapsed Time: " + str(elapsed) + " seconds")

loop = asyncio.get_event_loop()
loop.run_until_complete(main_async())
```

Concurrent Programming in Python

The Multiprocessing Module

We are onto our final Python module, `multiprocessing`. This module is unique from `threading` and `asyncio` in that allows the user to leverage multiple processors on a given machine simultaneously. This is because instead of threads or asynchronous tasks, `multiprocessing` is powered by subprocesses.

Implementation will look very similar to how we approached the `threading` module.

We went over processes in the [previous lesson](#); now we are going to implement them in Python.

To create a process in Python, we do the following:

```
import multiprocessing
p = multiprocessing.Process(target=target_function, args=(arg,))
```

Copy to Clipboard

This might look familiar as the arguments are identical. `target` is the function we execute with the process and `args` is the [argument](#)

Preview: Docs Loading link description
or set of arguments used in the function.

Also identical to the threading module, to start the process, we use:

```
p.start()
```

Copy to Clipboard

Let's practice with an example!

Instructions

Checkpoint 1 Passed

1.

Let's try running `greeting_with_sleep()` with a single process.

Create a process object called `p`. This process should execute `greeting_with_sleep()` with `'Codecademy'` as the argument.

Inside the terminal output, type `python3 script.py` and press Enter. To check your answer, click [Check Work](#) once your program finishes running.

```
import time
import multiprocessing

def greeting_with_sleep(string):
    s = time.perf_counter()
    print(string)
    time.sleep(2)
    print("says hello!")
    elapsed = time.perf_counter() - s
    print("Multiprocessing Elapsed Time: " + str(elapsed) + " seconds")

p = multiprocessing.Process(target=greeting_with_sleep, args=('Codecademy',))
p.start()
```

Concurrent Programming in Python

Using Multiple Processes

We have seen how to create a process in Python; however, we didn't see any speed or efficiency benefits in our previous example.

To see the benefits of processes, let's learn how to create multiple processes and test them out in our example.

We will create multiple processes using one of the following approaches:

```
import multiprocessing

processes = []
# list of arguments to use
args = [arg1, arg2, arg3]
# iterate through the length of arguments
for i in range(len(args)):
    # create process
    p = multiprocessing.Process(target=target_function, args=(args[i],))
    # add process to processes list
    processes.append(p)
    # start process
    p.start()
# join each process
for p in processes:
    p.join()
```

Copy to Clipboard

or

```
# list of arguments to use
args = [arg1, arg2, arg3]
# list comprehension to create processes list
processes = [multiprocessing.Process(target=target_function, args=(args[i],)) for i in range(len(args))]
# start each process
for p in processes:
    p.start()
# join each process AFTER starting each process
for p in processes:
    p.join()
```

Copy to Clipboard

Notice that we also use `.join()` for processes. The reason we use the

[method](#)

Preview: Docs Loading link description

is exactly the same!

Both approaches are effective. For the rest of the lesson, we will use the first approach. We may ask you to try the second approach later on.

Note: you can also use the second approach for the `threading` module!

Instructions

Checkpoint 1 Passed

1.

In `script.py`, we have some code set up for you. `greeting_with_sleep()` is loaded in; however, this time, it does not have the timer code. Instead, a function called `main_multiprocessing()` contains the timer code. In this function, you will write code to create and initiate four processes.

Inside `main_multiprocessing()`, write a for loop that iterates through `greetings`. Inside this for loop:

Create a process called `p` that has `greeting_with_sleep()` as its target and takes one greeting from `greetings` at each iteration.

Add the process to the `processes` list.

Start the created process.

Create a second for loop after this one that iterates through each process in `processes` and `.join()` each one.

Finally, call `main_multiprocessing()`.

Inside the terminal output, type `python3 script.py` and press `Enter`. To check your answer, click `Check Work` once your program finishes running.

The full answer for this code is in the hint. Note that we test that you use the first approach shown in the narrative to implement this code.

You should have the following code:

```
def main_multiprocessing():
    s = time.perf_counter()
    processes = []
    greetings = ['Codecademy', 'Chelsea', 'Hisham', 'Ashley']
    # add your code here
    for i in range(len(greetings)):
        p = multiprocessing.Process(target=greeting_with_sleep, args=(greetings[i],))
        processes.append(p)
        p.start()

    for p in processes:
        p.join()

    elapsed = time.perf_counter() - s
    print("Processing Elapsed Time: " + str(elapsed) + " seconds")

main_multiprocessing()
```

Copy to Clipboard

Checkpoint 2 Passed

2.

You should see something similar to the following in your terminal:

```
Codecademy
```

```
Chelsea
Hisham
Ashley
says hello!
says hello!
says hello!
says hello!
Multiprocessing Elapsed Time: 2.0072066410066327 seconds
```

Copy to Clipboard

Change the following print statement:

```
print("says hello!")
```

Copy to Clipboard

to

```
print(string + " says hello!")
```

Copy to Clipboard

What do you think the order of the print statements will be?

Inside the terminal output, type `python3 script.py` and press `Enter`. To check your answer, click `Check Work` once your program finishes running.

```
import time
import multiprocessing

def greeting_with_sleep(string):
    print(string)
    time.sleep(2)
    print(string + " says hello!")

def main_multiprocessing():
    s = time.perf_counter()
    processes = []
    greetings = ['Codecademy', 'Chelsea', 'Hisham', 'Ashley']
    # add your code here
    for i in range(len(greetings)):
        p = multiprocessing.Process(target=greeting_with_sleep, args=(greetings[i],))
        processes.append(p)
        p.start()

    for p in processes:
        p.join()

    elapsed = time.perf_counter() - s
    print("Multiprocessing Elapsed Time: " + str(elapsed) + " seconds")

main_multiprocessing()
```

Concurrent Programming in Python

Review

Congrats on making it through the Concurrent Programming in Python lesson! We have covered:

- Creating a thread in Python
- Creating and joining multiple threads in Python
- Using `async/await` in Python
- Creating and joining processes in Python

Next up, we have a fun project for you to practice these skills and compare their performances. Happy coding!

Deploying a Simple Python Script With Flask

Learn how to deploy a program in Python using Flask!

Introduction

In this article, we will cover how to locally deploy a [Python](#) application using Flask, including:

The importance of deployment.

How to install Flask using [pip](#).

What is deployment?

After we have designed, built, and thoroughly tested our application, it is time to package it up and ready it for a production state. This process is called *deployment*. This allows our application to successfully execute in an environment other than our development and test environments.

There are several methods for deploying applications, but in this article, we will dive into deploying a simple web application to a local [server](#) using the Python web framework, Flask.

Flask is a lightweight framework for developing web applications. It provides an efficient and easy way to deploy web-based projects. Flask offers a built-in development server that we can use to test and deploy web application code locally (which we will cover in this article). It also includes useful features like templating that allow us to render [HTML](#) templates and provide the functional [back-end](#) code for template components.

Installing Flask

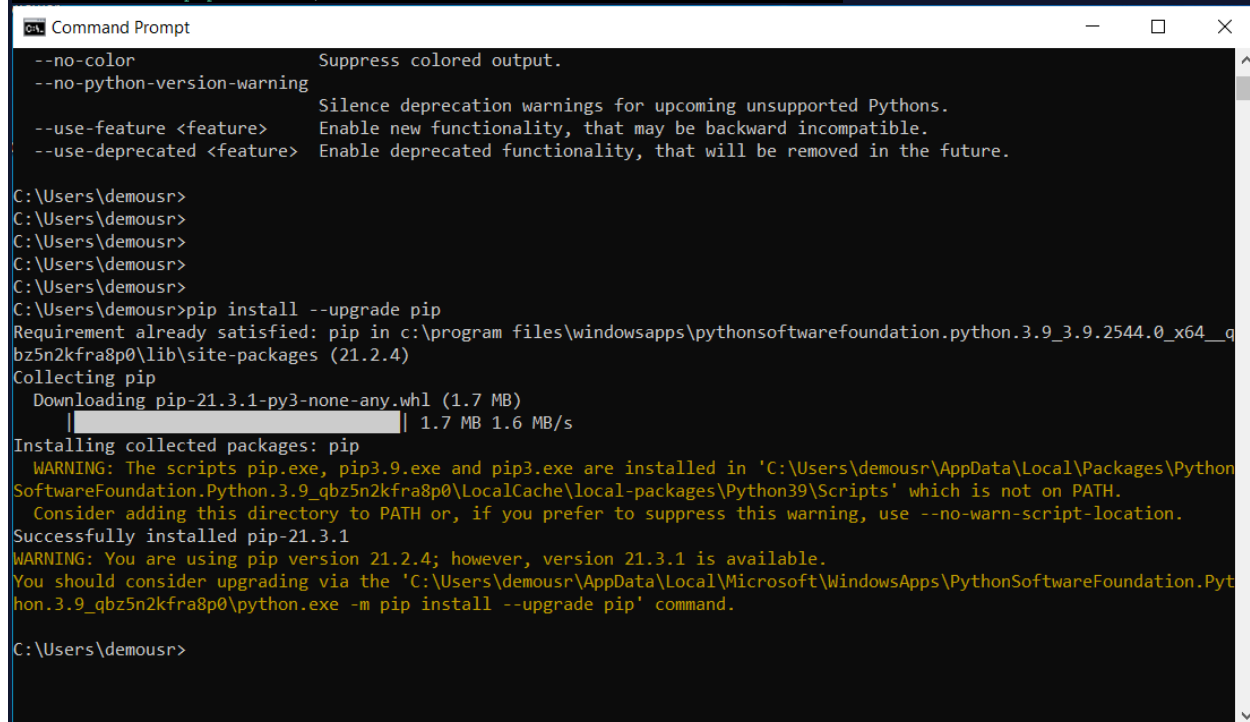
To use the Flask library for deployment, we must first install it in our environment. The following steps will show how to install Flask on a Windows or macOS environment.

To install Flask, we will use [pip](#), the package manager for Python that allows us to install and manage Python packages that are not a part of the Python standard library. If you have Python installed on your computer, you most likely have [pip](#) automatically installed. However, if you need to install it, follow the steps in [pip's documentation](#).

Open a command line [terminal](#) in your environment where Python3 is already installed and set up. First, let's make sure we are running the latest, upgraded version of pip by running the following command:

```
pip install --upgrade pip
```

If a newer version of [pip](#) is available, it will download and install as shown in the screenshots below:



```
Command Prompt

--no-color                Suppress colored output.
--no-python-version-warning
                           Silence deprecation warnings for upcoming unsupported Pythons.
--use-feature <feature>   Enable new functionality, that may be backward incompatible.
--use-deprecated <feature> Enable deprecated functionality, that will be removed in the future.

C:\Users\demoustr>
C:\Users\demoustr>
C:\Users\demoustr>
C:\Users\demoustr>
C:\Users\demoustr>
C:\Users\demoustr>
C:\Users\demoustr>pip install --upgrade pip
Requirement already satisfied: pip in c:\program files\windowsapps\pythonsoftwarefoundation.python.3.9_3.9.2544.0_x64__q
bz5n2kfra8p0\lib\site-packages (21.2.4)
Collecting pip
  Downloading pip-21.3.1-py3-none-any.whl (1.7 MB)
    | 1.7 MB 1.6 MB/s
Installing collected packages: pip
  WARNING: The scripts pip.exe, pip3.9.exe and pip3.exe are installed in 'C:\Users\demoustr\AppData\Local\Packages\Python
SoftwareFoundation.Python.3.9_qbz5n2kfra8p0\LocalCache\local-packages\Python39\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed pip-21.3.1
WARNING: You are using pip version 21.2.4; however, version 21.3.1 is available.
You should consider upgrading via the 'C:\Users\demoustr\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Pyt
hon.3.9_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip' command.

C:\Users\demoustr>
```

Now that we have the latest version of [pip](#) installed, let's install Flask. Run the following command:

```
pip install flask
```

Flask will download and install as shown in the below screenshot:

```
Command Prompt
You should consider upgrading via the 'C:\Users\demoustr\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.9_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip' command.

C:\Users\demoustr>pip install flask
Collecting flask
  Downloading Flask-2.0.2-py3-none-any.whl (95 kB)
    | 95 kB 1.2 MB/s
Collecting itsdangerous>=2.0
  Downloading itsdangerous-2.0.1-py3-none-any.whl (18 kB)
Collecting Jinja2>=3.0
  Downloading Jinja2-3.0.3-py3-none-any.whl (133 kB)
    | 133 kB ...
Collecting click>=7.1.2
  Downloading click-8.0.3-py3-none-any.whl (97 kB)
    | 97 kB 6.8 MB/s
Collecting Werkzeug>=2.0
  Downloading Werkzeug-2.0.2-py3-none-any.whl (288 kB)
    | 288 kB ...
Collecting colorama
  Downloading colorama-0.4.4-py2.py3-none-any.whl (16 kB)
Collecting MarkupSafe>=2.0
  Downloading MarkupSafe-2.0.1-cp39-cp39-win_amd64.whl (14 kB)
Installing collected packages: MarkupSafe, colorama, Werkzeug, Jinja2, itsdangerous, click, flask
WARNING: The script flask.exe is installed in 'C:\Users\demoustr\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.9_qbz5n2kfra8p0\LocalCache\local-packages\Python39\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed Jinja2-3.0.3 MarkupSafe-2.0.1 Werkzeug-2.0.2 click-8.0.3 colorama-0.4.4 flask-2.0.2 itsdangerous-2.0.1

C:\Users\demoustr>
```

Understanding the code

Take a look at the following calculator application code. We must first import the Flask library into our application in order to use it later on.

```
app = Flask( name )

@app.route('/')
def welcome():
    return "Welcome to the Codecademy Calculator!"
```

Next, we create a Flask instance and assign it to the variable named `app`. We can create several routes using the `@app.route decorator`. These routes correspond to the URL web request made to access different pages of the web application. The function that follows the decorator executes upon the web request. The main URL of our application is denoted by `/`. If we want to add additional URL routes for `/division` or `/multiplication`, we can define routes in our Flask application for each.

```
# allows user to input two numbers
num1 = int(input('Enter your first number: '))
num2 = int(input('Enter your second number: '))

@app.route('/division')
def division():
    return "Now dividing " + str(num1) + " and " + str(num2) + "! The result is: {result}".format(result=str(num1/num2))

@app.route('/multiplication')
def multiplication():
    return "Now multiplying " + str(num1) + " and " + str(num2) + "! The result is: {result}".format(result=str(num1*num2))
```

Let's review the following code:

- `num1` and `num2` allow users to input two numbers to multiply or divide.
- `@app.route('/division')` specifies the web request with `/division`.
- `division()` is a function that executes in the `/division` route. This function divides the values `num1` and `num2`.
- `@app.route('/multiplication')` specifies the web request with `/multiplication`.
- `multiplication()` is a function that executes in the `/multiplication` route. This function multiplies the values `num1` and `num2`.

Finally, we need to add the last line to our program:

```
app.run()
```

Calling `app.run()` will start the application server allowing the application to load in the web browser.

Running the code

Now, it's time to run our program and see Flask come to life! You can either copy and paste the code above into a Python file or download the code by clicking [here](#).

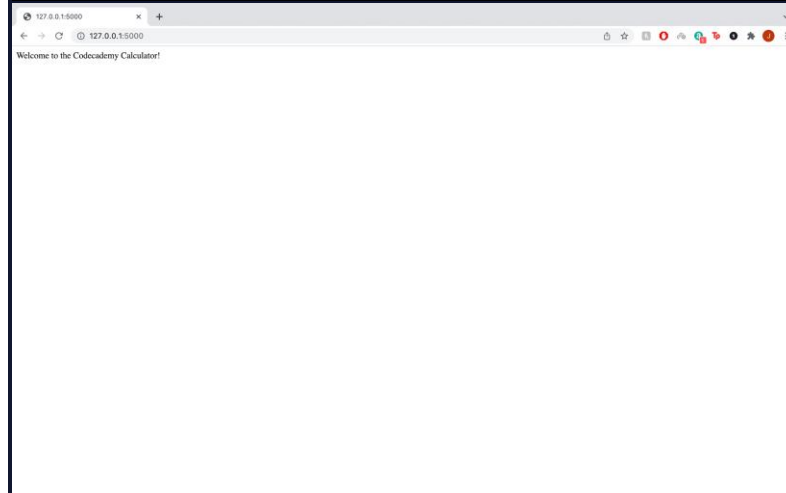
Once you have the code ready on your computer, run it (e.g., `flask --app app run`). You should see the following:

Enter your first number:

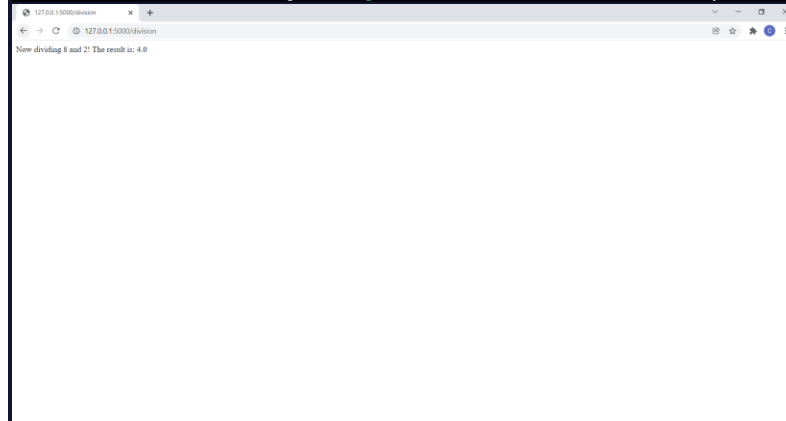
After you enter two numbers, you should see this:

```
* Serving Flask app 'app' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

Visit the localhost `http://127.0.0.1:5000/` on your browser, and the following page should appear:



To go to the `/division` or `/multiplication` route, you can add `division` or `multiplication` after the final `/`. If you wanted to see the `division()` function executed, go to `http://127.0.0.1:5000/division`, and you should see something like the following:



Conclusion

Deploying allows us to quickly package up our application to be easily configured and installed into environments other than where we developed the application. This was a brief introduction to deployment with Flask. If you want to learn more, you can dive deeper into creating web applications in our [Learn Flask](#) course.

Happy coding!

Learn Bash Scripting

Introduction to Bash Scripting

Bash (or shell) scripting is a great way to automate repetitive tasks and can save you a ton of time as a developer. Bash scripts execute within a Bash shell interpreter [Terminal](#). Any command you can run in your terminal can be run in a Bash script. When you have a command or set of commands that you will be using frequently, consider writing a Bash script to perform it. There are some conventions to follow to ensure that your computer is able to find and execute your Bash scripts. The beginning of your script file should start with `#!/bin/bash` on its own line. This tells the computer which type of interpreter to use for the script. When saving the script file, it is good practice to place commonly used scripts in the `~/bin/` directory. The script files also need to have the “execute” permission to allow them to be run. To add this permission to a file with filename: `script.sh` use:

```
chmod +x script.sh
```

Copy to Clipboard

Your terminal runs a file every time it is opened to load its configuration. On [Linux](#) style shells, this is `~/.bashrc` and on OSX, this is `~/.bash_profile`. To ensure that scripts in `~/bin/` are available, you must add this directory to your `PATH` within your configuration file:

```
PATH=~/bin:$PATH
```

Now any scripts in the `~/bin` directory can be run from anywhere by typing the filename.

Instructions

Checkpoint 1 Passed

1.

In the code editor, at the top of the **script.sh** file, add a line to specify that the script should be run using bash.

To make it known that the **script.sh** file is a Bash script, include the following line at the very top of the file.

```
#!/bin/bash
```

Copy to Clipboard

The file editor is the section that has the **script.sh** file name at the very top. You can also distinguish it from the terminal due to the line numbers included on its left side.

The terminal, which you'll interact with in a later step, has a blinking cursor and has a tab titled: **bash**.

Checkpoint 2 Passed

2.

Add an `echo` bash command to **script.sh** to make the script print “Hello Codecademy!” when the script is run.

Include the following line in the terminal:

```
echo "Hello Codecademy!"
```

Copy to Clipboard

Also, “Codecademy” only has 1 “a”. It’s “Codecademy” not “CodeAcademy”!

Checkpoint 3 Passed

3.

In the provided terminal, use the command `./script.sh` to run the script.

In the terminal, type out the command `./script.sh`, it should look like:

```
$ ./script.sh
```

Copy to Clipboard

You do not need to type out the `$`. The `$` is automatically included in the terminal and is used to show that the provided code is a terminal command.

After the command has been properly typed out, press `enter` to run the script.

Learn Bash Scripting

Variables

Within bash scripts (or the

[terminal](#)

Preview: Docs Loading link description
for that matter),

[variables](#)

Preview: Docs Loading link description

are declared by setting the variable name equal to another value. For example, to set the variable `greeting` to "Hello", you would use the following syntax:

```
greeting="Hello"
```

Copy to Clipboard

Note that there is no space between the variable name, the equals sign, or "Hello".

To access the value of a variable, we use the variable name prepended with a dollar sign (`$`). In the previous example, if we want to print the variable value to the screen, we use the following syntax:

```
echo $greeting
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Create a variable `phrase` on the line before the current `echo` statement. Set the `phrase` equal to the string "Hello to you!".

```
phrase="Hello to you!"
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Update the `echo` statement to make use of the variable instead of the fixed string.

```
echo $phrase
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Run the script in the terminal.

Use `./script.sh` to run the script.

Learn Bash Scripting

Conditionals

When bash scripting, you can use

[conditionals](#)

Preview: Docs Loading link description

to control which set of commands within the script run. Use `if` to start the conditional, followed by the condition in square brackets (`[ ]`). Make sure you leave a space between a bracket and the conditional statement! `then` begins the code that will run if the condition is met. `else` begins the code that will run if the condition is not met. Lastly, the conditional is closed with a backwards `fi`.

A complete conditional in a bash script uses the following syntax:

```
if [ $index -lt 5 ]
then
  echo $index
else
  echo 5
fi
```

Copy to Clipboard

Bash scripts use a specific list of operators for comparison. Here we used `-lt` which is "less than". The result of this conditional is that if `$index` is less than 5, it will print to the screen. If it is 5 or greater, "5" will be printed to the screen.

Here is the list of comparison operators for numbers you can use within bash scripts:

Equal: `-eq`

Not equal: `-ne`

Less than or equal: `-le`
Less than: `-lt`
Greater than or equal: `-ge`
Greater than: `-gt`
Is null: `-z`

When comparing strings, it is best practice to put the variable into quotes (`"`). This prevents errors if the variable is null or contains spaces. The common operators for comparing strings are:

Equal: `==`
Not equal: `!=`

For example, to compare if the

[variables](#)

Preview: Docs Loading link description

`foo` and `bar` contain the same string:

```
if [ "$foo" == "$bar" ]
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

We've added two different greetings and a variable to store how many times the user has been greeted before. If this variable is less than 1, we want to use `first_greeting`. Otherwise, we want to use `later_greeting`.

Add a line setting up the `if` conditional. Use the `-lt` operator.

```
if [ $greeting_occasion -lt 1 ]
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Use `echo` to print `$first_greeting` if the conditional is met and `$later_greeting` otherwise. Be sure to close the `if` statement.

Use `then`, `else` and `fi`.

Checkpoint 3 Passed

3.

Run the script in the terminal. Try adjusting the value of `$greeting_occasion` to test your `if` statement.

Use `./script.sh` to run the script.

```
#!/bin/bash
first_greeting="Nice to meet you!"
later_greeting="How are you?"
greeting_occasion=1
if [ $greeting_occasion -lt 1 ]
then
    echo $first_greeting
else
    echo $later_greeting
fi
```

Learn Bash Scripting

Loops

There are 3 different ways to loop within a bash script: `for`, `while` and `until`.

A `for` loop is used to iterate through a list and execute an action at each step. For example, if we had a list of words stored in a variable `paragraph`, we could use the following syntax to print each one:

```
for word in $paragraph
do
    echo $word
done
```

Copy to Clipboard

Note that `word` is being “defined” at the top of the `for` loop so there is no `$` prepended. Remember that we prepend the `$` when accessing the value of the variable. So, when accessing the variable within the `do` block, we use `$word` as usual.

Within bash scripting `until` and `while` are very similar. `while` loops keep looping while the provided condition is true whereas `until` loops loop until the condition is true. Conditions are established the same way as they are within an `if` block, between square brackets. If we want to print the

`index`

Preview: Docs Loading link description

variable as long as it is less than 5, we would use the following `while` loop:

```
while [ $index -lt 5 ]
do
    echo $index
    index=$((index + 1))
done
```

Copy to Clipboard

Note that arithmetic in bash scripting uses the `$((...))` syntax and within the brackets the variable name is not prepended with a `$`.

The same loop could also be written as an `until` loop as follows:

```
until [ $index -eq 5 ]
do
    echo $index
    index=$((index + 1))
done
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Start by opening a `while` loop directly after the `greeting_occasion` variable. We want to loop while `$greeting_occasion` is less than 3.

Remember, the `while` syntax is:

```
while [ $index -lt 5 ]
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Move the existing `if...fi` block inside of the `while` loop. Remember to use the `do` and `done` keywords. Increment `greeting_occasion` on each loop.

Use this syntax to increment a variable:

```
index=$((index + 1))
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Run the script. You should see the first greeting as “Nice to meet you!” followed by two “How are you?” greetings.

Use `./script.sh` to run the script.

```
#!/bin/bash
first_greeting="Nice to meet you!"
later_greeting="How are you?"
greeting_occasion=0
while [ $greeting_occasion -lt 3 ]
do
    if [ $greeting_occasion -lt 1 ]
    then
        echo $first_greeting
    else
        echo $later_greeting
    fi
    greeting_occasion=$((greeting_occasion + 1))
done
```

[Learn Bash Scripting](#)

Inputs

To make bash scripts more useful, we need to be able to access data external to the bash script file itself. The first way to do this is by prompting the user for input. For this, we use the `read` syntax. To ask the user for input and save it to the `number` variable, we would use the following code:

```
echo "Guess a number"
read number
echo "You guessed $number"
```

Copy to Clipboard

Another way to access external data is to have the user add input arguments when they run your script. These arguments are entered after the script name and are separated by spaces. For example:

```
saycolors red green blue
```

Copy to Clipboard

Within the script, these are accessed using `$1`, `$2`, etc, where `$1` is the first [argument](#)

Preview: Docs Loading link description
(here, "red") and so on. Note that these are 1 indexed.

If your script needs to accept an indefinite number of input arguments, you can iterate over them using the `"$@"` syntax. For our `saycolors` example, we could print each color using:

```
for color in "$@"
do
    echo $color
done
```

Copy to Clipboard

Lastly, we can access external files to our script. You can assign a set of files to a variable name using standard bash pattern matching using [regular expressions](#)

Preview: Docs Loading link description

. For example, to get all files in a directory, you can use the `*` character:

```
files=/some/directory/*
```

Copy to Clipboard

You can then iterate through each file and do something. Here, lets just print the full path and filename:

```
for file in $files
do
    echo $file
done
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Let's ask the user how many times the program should greet them.

Just before the `while` loop, print the following to the terminal: "How many times should I greet?"

Use the `echo` command.

Checkpoint 2 Passed

2.

Get user input and assign it to the variable `greeting_limit`. Replace the limit of 3 in the `while` loop with `$greeting_limit`.

You can assign a user's input to a variable `foo` as follows:

```
read foo
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Run the script and try out different `greeting_limits`.

Use `./script.sh` to run the script.


```
#!/bin/bash
first_greeting="Nice to meet you!"
later_greeting="How are you?"
greeting_occasion=0
echo "How many times should I greet?"
read greeting_limit
while [ $greeting_occasion -lt $greeting_limit ]
do
    if [ $greeting_occasion -lt 1 ]
    then
        echo $first_greeting
    else
        echo $later_greeting
    fi
    greeting_occasion=$((greeting_occasion + 1))
done
```

Learn Bash Scripting

Aliases

You can set up aliases for your bash scripts within your `.bashrc` or `.bash_profile` file to allow calling your scripts without the full filename. For example, if we have our `saycolors.sh` script, we can alias it to the word `saycolors` using the following syntax:

```
alias saycolors='./saycolors.sh'
```

Copy to Clipboard

You can even add standard input arguments to your alias. For example, if we always want “green” to be included as the first input to `saycolors`, we could modify our alias to:

```
alias saycolors='./saycolors.sh "green"'
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Our script is updated to take an argument for the number of times the user wants to be greeted:

```
./script.sh 5 #greet 5 times
```

Copy to Clipboard

Let's create an alias so that when you type `greet3` in the terminal, our script greets you three times.

In your own environment, you could add this alias to your `~/.bashrc` to make the alias active every time the terminal is started.

Here, just make the alias in the command line.

Your command should look like the following:

```
alias greet3='./script.sh 3'
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Test out your new alias!

Use the alias by typing `greet3` in the terminal.

```
#!/bin/bash
first_greeting="Nice to meet you!"
later_greeting="How are you?"
greeting_occasion=0
greeting_limit=$1
while [ $greeting_occasion -lt $greeting_limit ]
do
    if [ $greeting_occasion -lt 1 ]
    then
        echo $first_greeting
```

```
else
    echo $later_greeting
fi
greeting_occasion=$((greeting_occasion + 1))
done
```

Learn Bash Scripting

Review

Take a minute to review what you've learned about bash scripting.

Any command that can be run in the

[terminal](#)

Preview: Docs Loading link description
can be run in a bash script.

[Variables](#)

Preview: Docs Loading link description

are assigned using an equals sign with no space (`greeting="hello"`).

Variables are accessed using a dollar sign (`echo $greeting`).

[Conditionals](#)

Preview: Docs Loading link description

use `if`, `then`, `else`, `fi` syntax.

Three types of loops can be used: `for`, `while`, and `until`.

Bash scripts use a unique set of comparison operators:

Equal: `-eq`

Not equal: `-ne`

Less than or equal: `-le`

Less than: `-lt`

Greater than or equal: `-ge`

Greater than: `-gt`

Is null: `-z`

Input arguments can be passed to a bash script after the script name, separated by spaces (`myScript.sh "hello" "how are you"`).

Input can be requested from the script user with the `read` keyword.

Aliases can be created in the `.bashrc` or `.bash_profile` using the `alias` keyword.

Instructions

1. Our completed script is in the code editor. Feel free to edit it to make it your own.
Some ideas:
 - ask the user for different greetings
 - add more than two greetings
 - add more conditions to adjust the greetings over time

GitHub and Markdown

Why learn Markdown?

Once we add our code to GitHub, how do we actually format the text in our repository?

Technically, we could use plaintext on everything. But if we wanted to highlight our project with codeblocks, insert images into our project descriptions, or visually pace our documentation with headings, using Markdown on GitHub is the answer!

With a little bit of Markdown magic, you can achieve text that includes headings, hyperlinks, emojis, checkmarks, and codeblocks like the following file in our [Learn C++ repository!](#)

Learn C++

Curriculum Developer: Sonny Li
(sonny@codecademy.com)  



Curriculum Developer: Mariel Frank (mariel@codecademy.com)  

Artwork: Jared Langel

Course Link

<https://www.codecademy.com/learn/learn-c-plus-plus>

Textbook

[Programming Principles and Practice Using C++](#) (Bjarne Stroustrup)

1. Hello World

```
$ g++ hello.cpp -o hello
$ ./hello
```

- ☒ [hello.cpp](#)
- ☒ [pattern.cpp](#)
- ☒ [letter.cpp](#)
- ☒ [spell.cpp](#)

Block Letters:

- ☒ [initials.cpp](#)
- ☒ [snowman.cpp](#)
- ☒ [codecademy.cpp](#)
- ☒ [daftpunk.cpp](#)

Markdown is an indispensable skill and can be opened in many other applications, not just GitHub. In the next few interactive articles, you'll learn the basics of Markdown and how to use it to write a good README for your project!

Let's get started!

YOUTUBE

<https://youtu.be/f49LJV1i-w>

<https://youtu.be/f49LJV1i-w>

What is Markdown?

An alternative way to style a text document for readability and portability.

Introduction

If you are looking for a simple way to create visually appealing text documents without using any fancy editors, check out [Markdown](#). Invented by John Gruber and Aaron Swartz in 2004, Markdown provides a lightweight syntax to style any text document so that it can be converted to [HTML](#) for viewing and publishing.

Differences between HTML and Markdown

Markdown translates to HTML, but Markdown is not a replacement for HTML. Markdown's syntax can be converted to a small subset of HTML tags to do things like format text, add links, display images, and more. You can even incorporate HTML elements inside a Markdown document. To render Markdown in HTML, though, you would need a tool called a Markdown parser (more about this parser later on).

Benefits of Using Markdown

Why would you use Markdown? Compared to HTML, writing text in Markdown is much faster because the syntax is simpler. The [authors of Markdown](#) intended Markdown to be a language for writing styled text with syntax that is just as easy to read as it is to write.

Imagine that we are writing down some important text on a sheet of paper. If we wanted to emphasize the text we might underline it, twice even!

Similarly, in Markdown, we could write:

```
Title of My Document
=====
```

Notice that we have plain text that has a row of equal signs (=) to produce a first-level header, also known as `<h1>`. On a webpage, the heading will display as such:

Title of My Document

The amount of =s don't matter so long as there's at least one and it goes under the text.

For text that's important, but not as important as a first-level heading, we might just underline it once. In code, we could use a row of hyphens (-), like so:

```
Sub-Title of My Document
-----
```

to produce a second-level header, also known as `<h2>`:

Sub-Title of My Document

Both examples are valid Markdown syntax. It is intuitive and normal for headings to be in either format. Alternatively, you can format headings using the # character. We'll show more examples of different headings, but for now, here's an example of valid syntax for a `<h1>` heading:

```
# Title of My Document
```

Furthermore, a Markdown document without any HTML tags can be published as is because its syntax is already made for easy viewing. For example, the following Markdown document has:

- styled a heading underlined with equal signs (=),
- emphasized `week` in bold with double asterisks (**),
- bulleted a list by prefacing each item with a number and a period,
- and marked several lines as separate quotes with an angle bracket (>) per quote.

```
My Todo List
=====
```

At the end of this **week**, I plan to:

1. Learn Markdown
2. Write Markdown
3. Share a Markdown note

```
My favorite quote is:
> If you didn't get it the first time
> Do not despair
> Try and try again
> ~ Anonymous
```

The HTML equivalent to the above would be:

```
<h1>My Todo List</h1>
<p>At the end of this <strong>week</strong>, I plan to:</p>
<ol>
  <li>Learn Markdown</li>
  <li>Write Markdown</li>
  <li>Share a Markdown note</li>
</ol>
<p>My favorite quote is:</p>
<blockquote>
  <p>
    If you didn't get it the first time<br>
    Do not despair<br>
    Try and try again<br>
    ~ Anonymous
  </p>
</blockquote>
```

When viewed on a Markdown parser such as [Stackedit.io](https://stackedit.io), you will see this:

My Todo List

At the end of this **week**, I plan to:

1. Learn Markdown
2. Write Markdown
3. Share a Markdown note

My favorite quote is:

If you didn't get it the first time
Do not despair
Try and try again
~ Anonymous

or this, based on Codecademy's parser:

My Todo List

At the end of this **week**, I plan to:

- Learn Markdown
- Write Markdown
- Share a Markdown note

My favorite quote is:

If you didn't get it the first time
Do not despair
Try and try again
~ Anonymous

While both parsers take the same Markdown input, they render slightly different HTML based on their internal configuration and CSS.

When to Use Markdown

After knowing the many benefits of Markdown, you might want to consider using Markdown if you ever find yourself in any of these scenarios:

- The only editor available to you supports just plain text.
- Time is of the essence and you can't afford to learn how to use an unfamiliar rich-text editor.
- You need to quickly outline your ideas in a structured but presentable manner.
- You want your document to be portable so that it can convert to HTML, PDF, EPUB, and/or MOBI.

Markdown Document Extension

Contrary to popular belief, Markdown is not a document format. Therefore, it doesn't require a strict file extension naming convention, such as `.md`. As the [official Markdown mailing list](#) explains, Markdown isn't meant to take over the format of a file. Any file extension that you would normally use to name your text document such as `.txt` or `.text` is appropriate. However, organizations such as [GitHub](#) have a preference to expect Markdown documents to appear with an `.md` or `.markdown` extension.

Markdown Applications

Since Markdown has gained a lot of popularity, you will find Markdown content being accepted in many applications.

- Website publishers that accept Markdown content include [Wordpress.com](#), [Ghost](#) and [BeakerBrowser](#). Tools such as [Jekyll](#), [Docusaurus](#) and [MkDocs](#) can convert Markdown documents into a static website geared for technical documentation. Take a look at this [page](#) rendered in HTML, and its text [source](#) styled in Markdown.
- Book publishers such as [LeanHub](#) accepts Markdown manuscripts and convert them to books for publication.
- Slide-show presentations such as [Remark](#) and [Clever](#) can convert Markdown slides into HTML for web viewing.

The Markdown Parser

A Markdown parser is software written to parse the Markdown syntax in a text document and convert it to HTML syntax. The original Markdown parser was written in Perl, but you can find parser [implementations](#) today in almost any programming language. Regardless, a basic Markdown parser should support the [core Markdown syntax](#): paragraphs, headers, blockquotes, phrase emphasis, lists, code, images, and links.

There are Markdown parsers that are freely available on the Web: [StackEdit.io](#), [Dillinger](#), [Parse](#) and [Markdown to HTML Converter](#). In addition to parsing and rendering, both Parse and Markdown to HTML Converter also convert the Markdown document to raw HTML so that you can copy and paste the HTML to be used elsewhere.

Markdown Tutorial

Let's learn a little more about the Markdown syntax. As mentioned above, you can simulate a `<h1>` HTML tag with a `#` Markdown symbol. Since there are six heading tags for HTML, from `<h1>` through `<h6>`, you can simulate this with `#` through `#####` in Markdown. For example this Markdown syntax:

```
# This is a H1 heading
## This is a H2 heading
### This is a H3 heading
#### This is a H4 heading
##### This is a H5 heading
##### This is a H6 heading
```

will render this:

This is a H1 heading

This is a H2 heading

This is a H3 heading

This is a H4 heading

This is a H5 heading

This is a H6 heading

In addition to numbered lists, you can style a bulleted list too. There are three different symbols you can use to mark a line item as a bullet: asterisk (`*`), plus sign (`+`), or hyphen (`-`).

For example:

```
* Markdown
+ HTML
- XML
```

produces this:

```
Markdown
HTML
XML
```

For best practice, it is recommended to use the same marker throughout your list instead of mixing them like above.

The core Markdown syntax does not include nested lists (list within another list), but it allows adding paragraphs between list items. To do so, you need to add a blank line after a list item and indent 4 spaces or 1 tab before starting a paragraph. Some parsers are lenient and do not enforce 4 spaces but there should be some spacing.

```
* Markdown
```

Markdown is a lightweight markup language for styling text.

* HTML

HTML stands for HyperText Markup Language.

For more examples of basic syntax, check out [CommonMark's reference guide](#).

Fill in the blank

Questions

Let's practice styling a page that contains a truncated description of one of Codecademy's courses, [Learn to Program Alexa](#). Fill in the code so that your output should look like this:

Learn to Program Alexa

Why Learn Alexa?

With a comprehensive understanding of the process for developing a custom Alexa skill, you'll be prepared to:

- publish your own program and
- work with other Voice User Interface (VUI) applications.

What You'll Learn

1. Build Your First Alexa Skill

Learn to build skills for Alexa using the Alexa Skills Kit and Amazon Web Services (AWS).

2. Create Your Lambda Function

In this course, you will learn how to create your own Lambda function using the AWS console.

Code

blank 1

Learn to Program Alexa

blank 2

Why Learn Alexa?

With a comprehensive understanding of the process for developing a custom Alexa skill, you'll be prepared to:

blank 3

publish your own program and

blank 4

work with other Voice User Interface (VUI) applications.

blank 5

What You'll Learn

blank 6

Build Your First Alexa Skill

Learn to build skills for Alexa using the Alexa Skills Kit and Amazon Web Services (AWS).

blank 7

Create Your Lambda Function

In this course, you will learn how to create your own Lambda function using the AWS console.

Answer Choices

- #
- #
- ###
-
- 1
- 2
-

Click or drag and drop to fill in the blank

The Markdown Flavors

Because the core Markdown language supports only a subset of HTML features, many independent developers have extended the Markdown syntax to incorporate more HTML compatibilities and customize it for their own organizations. Here are a few popular flavors of Markdown:

CommonMark is a body of special-interest developers who work side-by-side on a proposal to standardize the Markdown syntax and offer extensive test suites to validate Markdown implementations against this specification. This standard has been used by other developers to base their code upon.

GitHub Flavored Markup, or **GFM** is GitHub's expanded dialect of Markdown based on CommonMark and is used throughout the GitHub platform by its active community.

Trello, a popular collaborative tool that organizes and tracks information through virtual boards and cards, implements a custom version of Markdown as well.

Conclusion

You've learned a lot about Markdown, specifically:

- What Markdown is and how it differs from HTML
- How you can benefit from Markdown
- Where and when to apply Markdown in various scenarios
- How to write Markdown to style your text and make it presentable
- What a Markdown parser is and where to locate one
- Where to find other flavors of Markdown that are used in industry

Amazing job getting this far! Don't be shy in applying Markdown in your next document. In fact, this article and many others like this on Codecademy were prepared using Markdown! See? We practice what we preach.

Mastering Markdown

External article for learning Markdown.

- **Mastering Markdown**

In this article, you will learn how to use Markdown for styling text on the web, such as creating links and adding code blocks. This is helpful if you want to be able to control the display of your writing on GitHub and other platforms that support Markdown. After you finish reading the article, return to this page and complete the following assessments.

Multiple choice

Questions

With regard to GitHub, why is it important to learn Markdown?

Answer Choices

Markdown is used to squash coding issues and merge branches into `main`.

Markdown produces SHA-1 hashes.

Markdown can directly edit code within individual commits.

Markdown styles text to make it easier to present lists, images, code, and other content in GitHub gists and comments within issues and pull requests.

Multiple choice

Questions

What is Markdown not used for?

Answer Choices

Creating headers, emphasis, lists, links, blockquotes, and inline code.

Styling code of popular programming languages with syntax highlighting.
Creating code edits to fix issues raised in pull requests in GitHub.
Files with `.md` or `.markdown` extension.

Markdown Cheatsheet

> <https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet>

Git Branching

git branch

Up to this point, you've practiced in a single Git branch. (Note:

[GitHub](#)

Preview: Docs Loading link description

has changed the naming convention of the main branch from `master` to `main`. We will be updating the instructions and code workspaces soon to reflect that. In the meantime, `master` refers to `main`).

Git allows us to create *branches* to experiment with versions of a project. Imagine you want to create a version of a story with a happy ending. You can create a new branch and make the happy ending changes to that branch only. It will have no effect on the `master` branch until you're ready to merge the happy ending to the master branch.

In this lesson, we'll be using Git branching to develop multiple versions of a resumé.

You can use the command below to answer the question: "which branch am I on?"

```
git branch
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Check what branch you are currently on.

In the output, the * (asterisk) is showing you what branch you're on. The project only has one branch at this time.

Click Next to learn how to make a new branch.

Git Branching

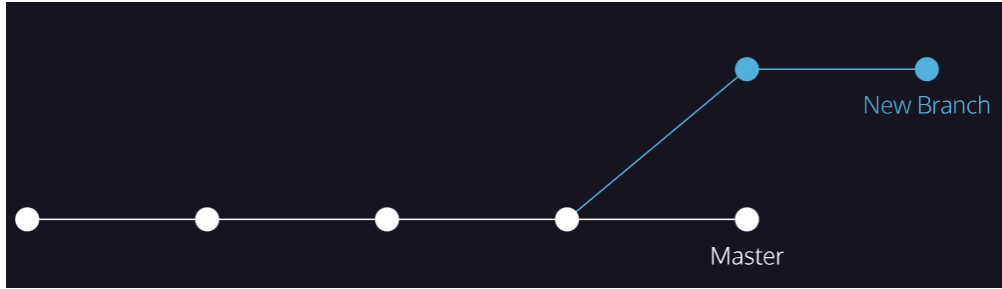
branching overview

The diagram to the right illustrates branching.

The circles are commits, and together form the Git project's commit history.

New Branch is a different *version* of the Git project. It contains commits from the main branch but also has commits that it does not have.

Click Next to make your first new branch.



Git Branching

git branch 2

Right now, the Git project has only one branch: `master`.

To create a new branch, use:

```
git branch new_branch
```

Copy to Clipboard

Here `new_branch` would be the name of the new branch you create, like `photos` or `blurb`. Be sure to name your branch something that describes the purpose of the branch. Also, branch names can't contain whitespaces: `new-branch` and `new_branch` are valid branch names, but `new branch` is not.

Instructions

Checkpoint 1 Enabled

1.

Let's create a new version of a resumé to apply for a fencing instructor role.

Create a new branch called `fencing`.

Remember to spell the word "fencing" correctly.

Next, view your branches as you did in the previous exercise.

Notice in the output there now appear two branches: `master` and `fencing`.

Git Branching

git checkout

Great! You just created a new branch.

The `master` and `fencing` branches are identical: they share the same exact commit history. You can switch to the new branch with

```
git checkout branch_name
```

Copy to Clipboard

Here, `branch_name` is the name of the branch. If the branch's name is `skill`

```
git checkout skill
```

Copy to Clipboard

Once you switch branches, you will now be able to make commits on the branch that have no impact on `master`.

You can continue your workflow, while `master` stays intact!

Instructions

Checkpoint 1 Passed

1.

Switch to the `fencing` branch from the `master` branch.

Checkpoint 2 Passed

2.

Use `git branch` to verify that you have switched branches.

In the output, notice the `*` is now over the `fencing` branch.

Git Branching

commit on a new branch

Congratulations! You have switched to a new branch. All the commands you do on `master`, you can also do on this branch.

For example, to add files to the staging area, use:

```
git add filename
```

Copy to Clipboard

And to commit, use:

```
git commit -m "Commit message"
```

Copy to Clipboard

In a moment, you will make a commit on the `fencing` branch. On the far right, the diagram shows what will happen to the Git project.

Instructions

Checkpoint 1 Passed

1. Print the Git commit log.

Notice the output:

The commits you see were all made in the `master` branch. `fencing` inherited them.
This means that every commit `master` has, `fencing` also has.

Note: if you find that your cursor is stuck in Git log, press `q` to escape.

Checkpoint 2 Passed

2. In `resume.txt`, replace your skill at scheming against Hook with your experience in sword-fights.

Delete this line:

```
-Scheme against Captain Hook
```

Copy to Clipboard

and type this line in its place:

```
-Engage in swordfights with pirates
```

Copy to Clipboard

Checkpoint 3 Passed

3. Make sure you are no longer in `git log`, then add `resume.txt` into the staging area.

If you find that your cursor is stuck in `git log`, press `q` to escape.

Checkpoint 4 Passed

4. Commit the changes to the repository with a commit message.

Git Branching

merge conflict I

The merge was successful because `master` had not changed since we made a commit on `fencing`. Git knew to simply update `master` with changes on `fencing`.

What would happen if you made a commit on `master` *before* you merged the two branches? Furthermore, what if the commit you made on `master` altered the same exact text you worked on in `fencing`? When you switch back to `master` and ask Git to merge the two branches, Git doesn't know which changes you want to keep. This is called a *merge conflict*.

Instructions

Checkpoint 1 Passed

1.

You are on the `master` branch. In the code editor, where you have written:

```
-Engage in swordfights with pirates
```

Copy to Clipboard

Add the word "professional", so the text reads:

```
-Engage in swordfights with professional pirates
```

Copy to Clipboard

Click Run.

Checkpoint 2 Passed

2.

Add **resume.txt** to the staging area.

Checkpoint 3 Passed

3.

Commit the changes to the repository with a commit message.

Checkpoint 4 Passed

4.

Imagine a few weeks have passed, and you'd like to develop your fencing resumé some more.

Switch back to the `fencing` branch.

Checkpoint 5 Passed

5.

From `fencing`, change the line so it reads:

```
-Engage in swordfights with professional pirates such as Smee.
```

Copy to Clipboard

Click Run.

Checkpoint 6 Passed

6.

Once again, add **resume.txt** to the staging area.

Checkpoint 7 Passed

7.

Commit the changes to the repository with a commit message.

Git Branching

delete branch

In Git, branches are usually a means to an end. You create them to work on a new project feature, but the end goal is to merge that feature into the `master` branch. After the branch has been integrated into `master`, it has served its purpose and can be deleted.

The command

```
git branch -d branch name
```

Copy to Clipboard

will delete the specified branch from your Git project.

Now that `master` contains all the file changes that were in `fencing`, let's delete `fencing`.

Instructions

Checkpoint 1 Enabled

1.

Delete the `fencing` branch.

Now, verify that you have indeed deleted `fencing` by listing all your project's branches on the terminal.

Notice in the output that only one branch, `master`, is shown.

Click Next to continue!

Git Branching

Generalizations

Let's take a moment to review the main concepts and commands from the lesson before moving on.

Git *branching* allows users to experiment with different versions of a project by checking out separate *branches* to work on.

The following commands are useful in the Git branch workflow.

`git branch`: Lists all a Git project's branches.

`git branch branch_name`: Creates a new branch.

`git checkout branch_name`: Used to switch from one branch to another.

`git merge branch_name`: Used to join file changes from one branch to another.

`git branch -d branch_name`: Deletes the branch specified.

Git Teamwork

Overview

So far, we've learned how to work on Git as a single user. Git also offers a suite of collaboration tools to make working with others on a project easier.

Imagine that you're a science teacher, developing some quizzes with Sally, another teacher in the school. You are using Git to manage the project.

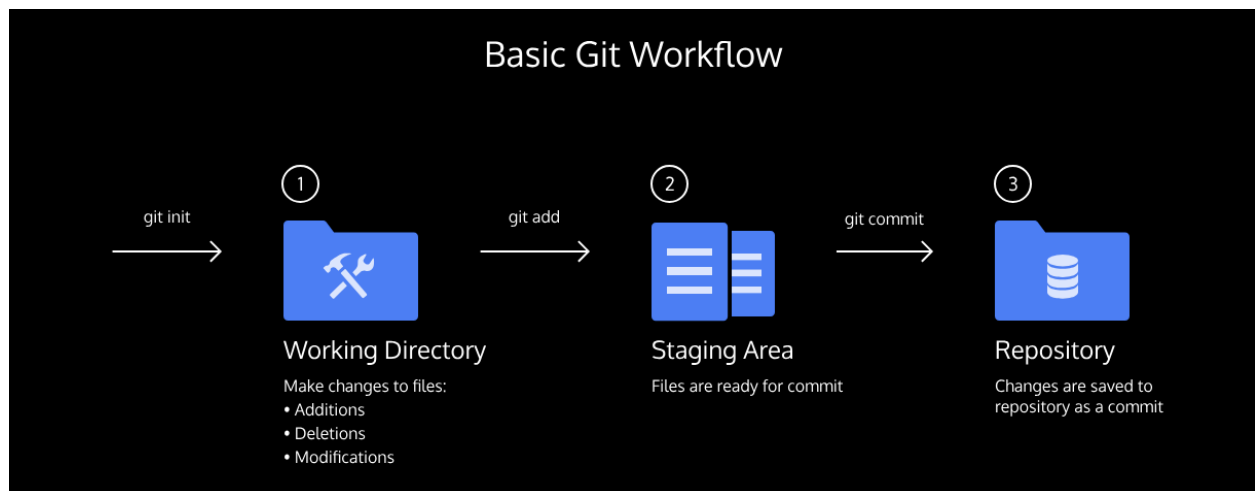
In order to collaborate, you and Sally need:

- A complete replica of the project on your own computers
- A way to keep track of and review each other's work
- Access to a definitive project version

You can accomplish all of this by using *remotes*. A remote is a shared Git repository that allows multiple collaborators to work on the same Git project from different locations. Collaborators work on the project independently, and merge changes together when they are ready to do so.

Instructions

Click Next to start working with your first remote.



Git Teamwork

git clone

Sally has created the remote repository, **science-quizzes** in the directory **curriculum**, which teachers on the school's shared network have access to. In order to get your own replica of **science-quizzes**, you'll need to *clone* it with:

```
git clone remote_location clone_name
```

Copy to Clipboard

In this command:

`remote_location` tells Git where to go to find the remote. This could be a web address, or a filepath, such as:
`/Users/teachers/Documents/some-remote`

Copy to Clipboard

`clone_name` is the name you give to the directory in which Git will clone the repository.

Instructions

Checkpoint 1 Enabled

1.

The Git remote Sally started is called:

```
science-quizzes
```

Copy to Clipboard

Enter the command to clone this remote. Name your clone:

```
my-quizzes
```

Copy to Clipboard

Notice the output:

```
cloning into 'my-quizzes'...
```

Copy to Clipboard

Git informs us that it's copying everything from **science-quizzes** into the **my-quizzes** directory.

my-quizzes is your *local* copy of the **science-quizzes** Git project. If you commit changes to the project here, Sally will not know about them.

If you did this correctly, you will find a new directory named `my-quizzes`.

You can check by running `ls`. You should see this output:

```
$ ls
my-quizzes  science-quizzes
```

```
> git clone science-quizzes my-quizzes
```

```
> ls
```

Git Teamwork

git remote -v

Nice work! We have a clone of Sally's remote on our computer. One thing that Git does behind the scenes when you clone **science-quizzes** is give the remote address the name *origin*, so that you can refer to it more conveniently. In this case, Sally's remote is *origin*.

You can see a list of a Git project's remotes with the command:

```
git remote -v
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Using the file navigator, examine the contents of the cloned Git project. There are a few quiz files here, which we will be working with during this lesson.

Open a file of your choice in the code editor.

Checkpoint 2 Passed

2.

Change directories into the **my-quizzes** directory, enter this command on the terminal:

```
cd my-quizzes
```

Copy to Clipboard

To learn more about `cd`, take a look at our [command line course](#).

Checkpoint 3 Passed

3.

Enter `git remote -v` to list the remotes.

Notice the output:

```
origin  /home/ccuser/workspace/curriculum/science-quizzes (fetch)
origin  /home/ccuser/workspace/curriculum/science-quizzes (push)
```

Copy to Clipboard

Git lists the name of the remote, `origin`, as well as its location.

Git automatically names this remote `origin`, because it refers to the remote repository of origin. However, it is possible to safely change its name.

The remote is listed twice: once for `(fetch)` and once for `(push)`. We'll learn about these later in the lesson.

Git Teamwork

git fetch

After you cloned **science-quizzes**, you had to run off to teach a class. Now that you're back at your computer, there's a problem: what if, while you were teaching, Sally changed the **science-quizzes** Git project in some way. If so, your clone will no longer be up-to-date.

An easy way to see if changes have been made to the remote and bring the changes down to your local copy is with:

```
git fetch
```

Copy to Clipboard

This command will not *merge* changes from the remote into your local repository. It brings those changes onto what's called a *remote branch*. Learn more about how this works below.

Instructions

Checkpoint 1 Passed

1.

Enter this command:

```
cd my-quizzes
```

Copy to Clipboard

to go into the **my-quizzes** directory.

Checkpoint 2 Passed

2.

Fetch any new changes Sally may have made to the remote.

```
cd my-quizzes
```

```
git fetch
```

```
git merge origin/master
```

```
Updating a2ba090..bc87a1a
```

```
Fast-forward
```

```
biology.txt | 2 +-  
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
git log --oneline
```

Add heading and comment to biology quiz

Git Teamwork

Git workflow

Now that you've merged `origin/master` into your local `master` branch, you're ready to contribute some work of your own. The workflow for Git collaborations typically follows this order:

- Fetch and merge changes from the remote
- Create a branch to work on a new project feature
- Develop the feature on your branch and commit your work
- Fetch and merge from the remote again (in case new commits were made while you were working)
- Push your branch up to the remote for review

Steps 1 and 4 are a safeguard against *merge conflicts*, which occur when two branches contain file changes that cannot be merged with the `git merge` command. Step 5 involves `git push`, a command you will learn in the next exercise.

Instructions

Checkpoint 1 Passed

1.

Enter this command:

```
cd my-quizzes
```

Copy to Clipboard

to change directories into the **my-quizzes** directory.

Checkpoint 2 Passed

2.

Enter the Git command:

```
git branch <branch_name>
```

Copy to Clipboard

to create a branch to develop questions for the biology quiz. Name the branch `bio-questions`.

Note: be careful to spell the name "bio-questions" exactly as it appears.

Checkpoint 3 Passed

3.

Switch to your new branch with the command:

```
git checkout <branch_name>
```

Copy to Clipboard

replacing `<branch_name>` with the name of the new branch.

Checkpoint 4 Passed

4.

On your branch, open the file **biology.txt** under the `my-quizzes` folder in the code editor (make sure you are editing the correct **biology.txt** file).

Add a biology question to the file and some sample answers. For example:

```
What is an animal that hunts and eats other animals called?  
a) herbivore  
b) prey  
c) ecosystem  
d) predator
```

Copy to Clipboard

Checkpoint 5 Passed

5.

Add **biology.txt** to the staging area.

Use `git add` to add a file to the staging area:

```
git add <filename>
```

Copy to Clipboard

Checkpoint 6 Passed

6.

Commit the work to the repository with a commit message.

Use `git commit -m` to commit with a commit message.

```
git commit -m "Detailed commit message here."
```



```
cd my-quizzes
git branch bio-questions
git branch bio-questions
git checkout bio-questions
What is an animal that hunts and eats other animals called?
a) herbivore
b) prey
c) ecosystem
d) predator
git add biology.txt
git commit -m "Add biology question about predators"
```

Git Teamwork

git push

Now it's time to share our work with Sally.

The command:

```
git push origin <your_branch_name>
```

Copy to Clipboard

will push your branch up to the remote, `origin`. From there, Sally can review your branch and merge your work into the `master` branch, making it part of the definitive project version.

Instructions

Checkpoint 1 Passed

1.

Enter this command

```
cd my-quizzes
```

Copy to Clipboard

to change directories into the **my-quizzes** directory.

Checkpoint 2 Passed

2.

Push your branch up to the remote.

In the output, notice the line:

```
To /home/ccuser/workspace/curriculum-a/science-quizzes
* [new branch]      bio-questions -> bio-questions
```

Copy to Clipboard

Git informs us that the branch `bio-questions` was pushed up to the remote. Sally can now review your new work and can merge it into the remote's `master` branch.

Use `git push` with the remote name and branch name.

More help

The remote name is `origin` and your branch name is `bio-questions`.

Community Support

Still have questions? Get help from the [Codecademy community](#).

Terminal

Add new terminal

Close bash session

Enter Fullscreen Terminal

7/8

[Back](#)

[Next](#)

Git Teamwork: Git workflow

```
cd my-quizzes
```

```
git push origin bio-questions
```

Expected output will include something like:

```
To /home/ccuser/workspace/curriculum-a/science-quizzes
* [new branch]      bio-questions -> bio-questions
> git push
```

Git Teamwork

Generalizations

Congratulations, you now know enough to start collaborating on Git projects! Let's review.

A *remote* is a Git repository that lives *outside* your Git project folder. Remotes can live on the web, on a shared network or even in a separate folder on your local computer.

The *Git Collaborative Workflow* are steps that enable smooth project development when multiple collaborators are working on the same Git project.

We also learned the following commands

`git clone`: Creates a local copy of a remote.

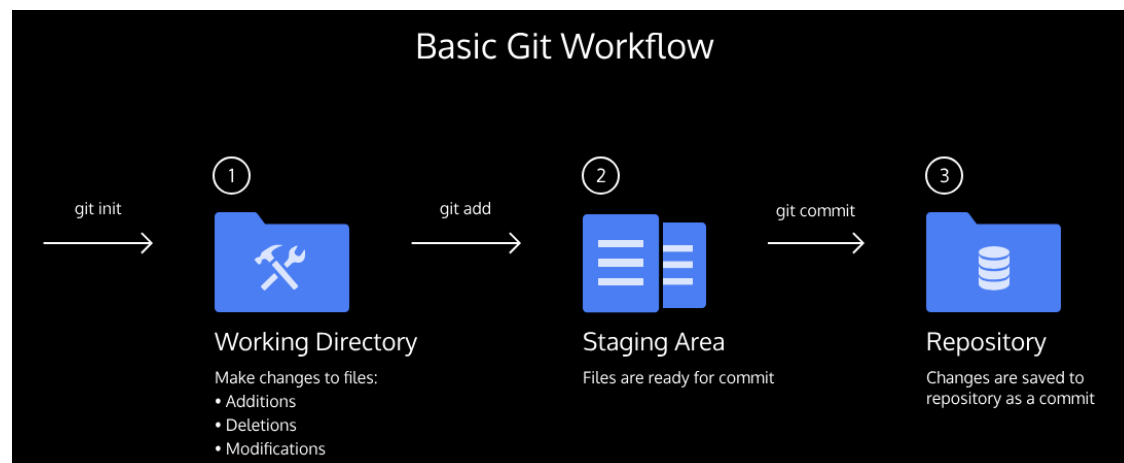
`git remote -v`: Lists a Git project's remotes.

`git fetch`: Fetches work from the remote into the local copy.

`git merge origin/master`: Merges `origin/master` into your local branch.

`git push origin <branch_name>`: Pushes a local branch to the `origin` remote.

Git projects are usually managed on Github, a website that hosts Git projects for millions of users. With Github you can access your projects from anywhere in the world by using the basic workflow you learned here.



Deploying to GitHub Pages

Share your website with the world, for free!

GitHub is a great tool to store projects and to collaborate with others, but its usefulness does not stop there. We'll use a service called GitHub Pages to share our web page creations on the World Wide Web.

What is GitHub Pages?

There are many different ways to deploy a website to the public Internet. We'll be using GitHub's free service called GitHub Pages.

Why GitHub Pages? GitHub Pages offers a lot of features and flexibility, all for free. Some of the benefits include:

Easy setup

Seamless collaboration using Git and GitHub

Free hosting with >95% uptime

Live updating with normal GitHub workflow

What is Deploying?

Deploying is like publishing. When authors are ready for their work to be seen by the world, they publish it. When web developers are ready to share their projects, they deploy to the World Wide Web. Deployment is when a project is packaged and shared on the Internet. Unlike publishing, however, deployment may occur many, many times over the course of a software project.

Deployment on GitHub Pages

Deploying to GitHub Pages is automatic. Once it's set up, deploying happens whenever you push your local changes to your remote, GitHub-hosted repository. Head to GitHub Pages' [setup instructions](#) and follow the steps exactly to get your main GitHub Pages page setup.

When you first navigate to your newly deployed site it is possible that you will receive a 404 error. If this happens, and you are confident that you have followed all the steps as written, check back in 30 minutes to see if the deploy has successfully gone through.

Viewing Your Live Web Page

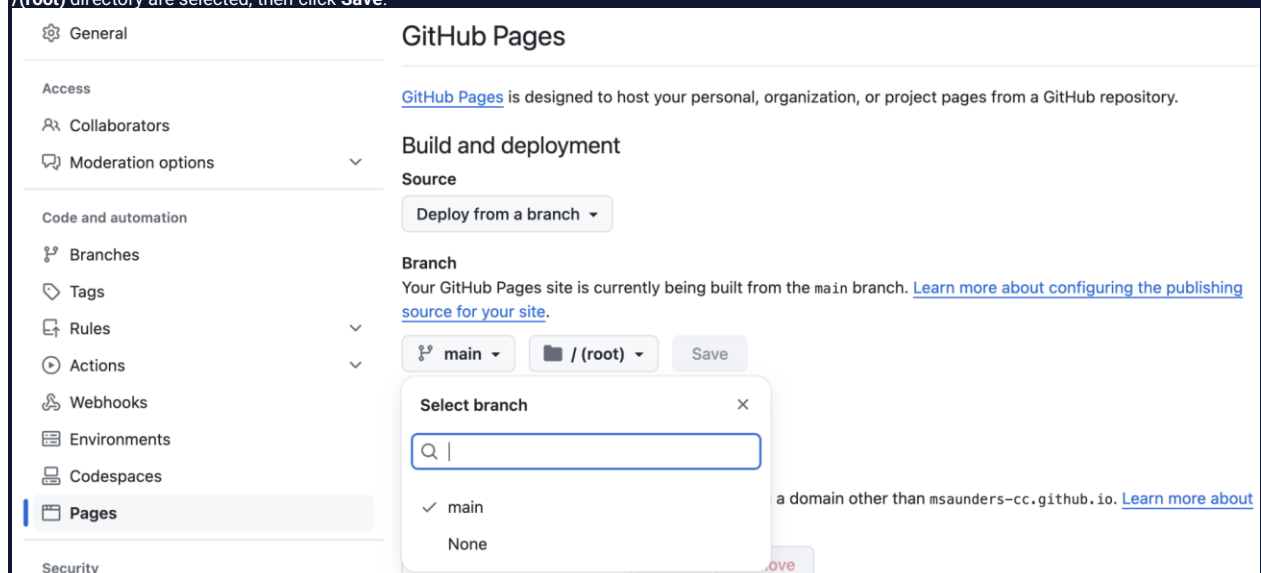
That's it! Your website is deployed to the Internet! You and anyone with whom you share this link can view your project by navigating in your browser to the URL <http://<your-github-username>.github.io>.

Adding GitHub Pages Projects

You can set up your GitHub Pages to deploy every one of your repositories in addition to [.<username>.github.io](http://<username>.github.io). This will allow you to ensure all of your sites are deployed automatically whenever you push to GitHub.

In GitHub, navigate to your [.<username>.github.io](http://<username>.github.io) repository and click **Settings**.

Within **Settings**, navigate to the **Source** section within the **GitHub Pages** section. From the dropdown menus, ensure that the **main** branch and the **/(root)** directory are selected, then click **Save**.

The screenshot shows the GitHub 'Settings' page for a repository, specifically the 'GitHub Pages' section. On the left is a sidebar with navigation links: General, Access, Collaborators, Moderation options, Code and automation, Branches, Tags, Rules, Actions, Webhooks, Environments, Codespaces, Pages (highlighted), and Security. The main content area is titled 'GitHub Pages' and contains the following information: 'GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.' Under 'Build and deployment', the 'Source' is set to 'Deploy from a branch'. The 'Branch' section states 'Your GitHub Pages site is currently being built from the main branch.' and includes a link 'Learn more about configuring the publishing source for your site.' Below this, there are two dropdown menus: the first is set to 'main' and the second is set to '/(root)'. A 'Save' button is to the right of these dropdowns. A 'Select branch' modal is open, showing a search bar and a list of options: 'main' (checked) and 'None'. At the bottom of the main content area, there is a link 'a domain other than msaunders-cc.github.io. Learn more about'.

Now, all of your repositories can be found at <http://<username>.github.io/<repository-name>>. Try creating a new repo with an HTML project inside it (perhaps push an old project to GitHub) and then navigate to the deployed page.

Deploying New Changes

Now that your GitHub Pages site is set up, deploying new changes is easy. Every time you make a change to your site, use the normal GitHub flow. That is, use `git commit` and `git push` to send your changes to GitHub. After this, the GitHub site should update within a few seconds. Just refresh the page in your browser, and you're good to go! Congratulations on your first live web page!

Excursion Project Solution

Compare your work to the official solution!

You can download the solution code to the Excursion project [here](#).

Try not to peek at the solution code until you've completed this part of the project, or unless you're really stuck and your struggle has become totally unproductive. Looking at the code before that moment will deprive you of an important learning opportunity and will make it harder for you to achieve your ultimate outcome.

Welcome to the Intermediate Machine Learning!

Learn about the Intermediate Machine Learning Skill Path!

Welcome to Intermediate Machine Learning! This track will introduce you to intermediate-level Machine Learning (ML) content including algorithms like Random Forests and Support Vector Machines and techniques for improving and tuning ML models like regularization, hyperparameter tuning and ensembling.

Why did we build this?

Our world is increasingly reliant on algorithms powered by machine learning for decision-making. Knowing how machine learning algorithms work and how to implement them is empowering to anyone interested in Tech! The prerequisite for this track is some familiarity with fundamental machine learning algorithms. Check out the [Machine Learning Fundamentals Skill Path](#) if you'd like to learn or brush up on these fundamentals.

What will you learn?

How to apply and evaluate intermediate Supervised Learning Algorithms like SVMs & Naive Bayes

The best ways to improve and tune ML models with Regularization and Hyperparameter Tuning

How to leverage the power of ensembling methods like bagging, boosting, and stacking

Machine Learning Methods

Bayes' Theorem

Review

In this course, we learned several new definitions:

Two events are *independent* if the occurrence of one event does not affect the probability of the second event

If two events are independent then:

$$P(A \cap B) = P(A) \times P(B)$$

Bayes' Theorem is the following:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Naive Bayes Classifier

Review

In this lesson, you've learned how to leverage Bayes' Theorem to create a supervised [machine learning](#)

Preview: Docs Loading link description

[algorithm](#)

Preview: Docs Loading link description

. Here are some of the major takeaways from the lesson:

- A tagged dataset is necessary to calculate the probabilities used in Bayes' Theorem.

- In this example, the features of our dataset are the words used in a product review. In order to apply Bayes' Theorem, we assume that these features are independent.

- Using Bayes' Theorem, we can find $P(\text{class}|\text{data point})$ for every possible class. In this example, there were two classes – positive and negative. The class with the highest probability will be the algorithm's prediction.

Even though our algorithm is running smoothly, there's always more that we can add to try to improve performance. The following techniques are focused on ways in which we process data before feeding it into the Naive Bayes classifier:

- Remove punctuation from the training set. Right now in our dataset, there are 702 instances of "great!" and 2322 instances of "great.". We should probably combine those into 3024 instances of "great".

- Lowercase every word in the training set. We do this for the same reason why we remove punctuation. We want "Great" and "great" to be the same.

- Use a bigram or trigram model. Right now, the features of a review are individual words. For example, the features of the point "This crib is great" are "This", "crib", "is", and "great". If we used a bigram model, the features would be "This crib", "crib is", and "is great". Using a bigram model makes the assumption of independence more reasonable.

These three improvements would all be considered part of the field *Natural Language Processing*.

You can find the baby product review dataset, along with many others, on [Dr. Julian McAuley's website](#).

Instructions

Checkpoint 1 Passed

1.

In the code editor, we've included three Naive Bayes classifiers that have been trained on different datasets. The training sets used are the baby product reviews, reviews for Amazon Instant Videos, and reviews about video games.

Try changing `review` again and see how the different classifiers react!

```
from reviews import baby_counter, baby_training, instant_video_counter, instant_video_training, video_game_counter,
video_game_training
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

review = "this game was violent"

baby_review_counts = baby_counter.transform([review])
instant_video_review_counts = instant_video_counter.transform([review])
video_game_review_counts = video_game_counter.transform([review])

baby_classifier = MultinomialNB()
instant_video_classifier = MultinomialNB()
video_game_classifier = MultinomialNB()

baby_labels = [0] * 1000 + [1] * 1000
instant_video_labels = [0] * 1000 + [1] * 1000
video_game_labels = [0] * 1000 + [1] * 1000

baby_classifier.fit(baby_training, baby_labels)
instant_video_classifier.fit(instant_video_training, instant_video_labels)
video_game_classifier.fit(video_game_training, video_game_labels)

print("Baby training set: " + str(baby_classifier.predict_proba(baby_review_counts)))
print("Amazon Instant Video training set: " + str(instant_video_classifier.predict_proba(instant_video_review_counts)))
print("Video Games training set: " + str(video_game_classifier.predict_proba(video_game_review_counts)))
```

Support Vector Machines

Review

Great work! Here are some of the major takeaways from this lesson on SVMs:

SVMs are supervised

[machine learning](#)

Preview: Docs Loading link description
models used for classification.

An SVM uses support vectors to define a decision boundary. Classifications are made by comparing unlabeled points to that decision boundary.

Support vectors are the points of each class closest to the decision boundary. The distance between the support vectors and the decision boundary is called the margin.

SVMs attempt to create the largest margin possible while staying within an acceptable amount of error.

The C

[parameter](#)

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called tripleThis(): `js function tripleThis(x) { return x * 3; } tripleThis(6);`

controls how much error is allowed. A large C allows for little error and creates a hard margin. A small C allows for more error and creates a soft margin.

SVMs use kernels to classify points that aren't linearly separable.

Kernels transform points into higher dimensional space. A polynomial kernel transforms points into three dimensions while an rbf kernel transforms points into infinite dimensions.

An rbf kernel has a γ parameter. If γ is large, the training data is more relevant, and as a result overfitting can occur.

Linear Discriminant Analysis

<https://www.codecademy.com/paths/machine-learning-engineer/tracks/mle-int-ml/modules/mle-supervised-li/articles/fe-linear-discriminant-analysis>

Learn about the Linear Discriminant Analysis, a dimensionality reduction technique for classification problems, and how to implement it in `scikit-learn`.

Introduction

Linear discriminant analysis (LDA) is a dimensionality reduction technique used for classification problems with continuous independent variables. This technique can be used in a wide variety of applications including image recognition, marketing, biological classification, and more. It works by using linear algebra techniques to find a subspace within the space of independent variables that simplifies classification problems.

How LDA Works

Let's start with a simple example. Here's a classification problem with two continuous independent variables and two classes: a red class and a blue class.

In this example you could easily draw a line that would separate most of the red data points from the blue ones. But could you simplify the problem by reducing the data to a single dimension? If we project the data onto a 1-dimensional subspace, we might be able to separate red from blue with only one dimension instead of two.

Here are two different ways of projecting the data onto 1-dimensional subspaces.

In the first example, there's no good way to separate red from blue. But in the second example you could easily choose a good decision boundary. This second subspace isn't just any subspace, it's actually the subspace that does the best job of separating red and blue. It was obtained by using LDA.

This is what LDA does in general. It finds the best possible subspace for a given classification problem. When there are only two classes, LDA finds a subspace that maximizes the ratio of variance between the classes to variance within the classes. This means that it finds a subspace where classes are far apart from each other, but the observations within each individual class are close to each other. LDA can also be applied when there are more than two classes, but this is slightly more complicated.

Once LDA finds the best subspace, data points can be projected onto that subspace. This yields a data set with fewer dimensions but nearly as much predictive power. LDA can also be used as a classifier by itself with the additional step of computing a decision boundary.

Notice that LDA is similar to Principal Component Analysis (PCA). Both LDA and PCA are dimensionality reduction tools. They both reduce dimensions in a similar way: by using linear algebra to find optimal subspaces for a statistical or machine learning problem.

A key difference between LDA and PCA is that they have different applications. PCA uses linear algebra techniques to find a subspace that maximizes the variance of a data set. This means that PCA basically finds the subspace that is best for linear regression problems. LDA, on the other hand, uses linear algebra techniques to find a subspace that is best for classification problems.

Multiple choice

Questions

Which of the following statements about Linear Discriminant Analysis (LDA) is FALSE?

Answer Choices

LDA can be used as an alternative to PCA when you have a continuous dependent variable.

LDA, like PCA, uses linear algebra to find a subspace of a given data set.

LDA can be used for both dimensionality reduction and classification.

When there are two classes, LDA maximizes the ratio of variance between classes to variance within classes.

Implementing LDA with scikit-learn

Let's take a look at how LDA performs in a real-life classification problem. The `seeds` data set comes from the [UCI Machine Learning Repository](#).

The original data set contains data about three varieties of wheat, but for the sake of simplicity we'll only look at the data for Kama and Rosa wheat. The problem is to classify wheat kernels based on their size and shape. We have seven independent variables to work with:

Scikit-learn's implementation of LDA allows us to do a few different things. First, we can transform the independent variables into a subspace with fewer dimensions. To create an LDA model that will find a 1-dimensional subspace you can use the code `LinearDiscriminantAnalysis(n_components=1)`. Note that scikit-learn requires that `n_components` is less than the number of classes minus one and less than the number of features. Since this dataset has two classes, we can only set `n_components` to 1. After finding the 1-dimensional subspace we can use a classification algorithm.

Here's an example that uses LDA to find a 1-dimensional subspace of the seed data set.

```
# Import libraries
import pandas as pd
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# Load data
seeds = pd.read_csv('seeds.csv')
X = seeds.drop('variety', axis=1)
y = seeds['variety']

# Create LDA model
lda = LinearDiscriminantAnalysis(n_components=1)

# Fit the data and create a subspace X_new
X_new = lda.fit_transform(X, y)
```

`X_new` is a 1-dimensional subspace of `X`. We can use it to train a classifier like logistic regression.

```
# Import library
from sklearn.linear_model import LogisticRegression

# Create logistic regression model
lr = LogisticRegression()

# Fit the model
lr.fit(X_new, y)

# Model accuracy
print(lr.score(X_new, y))
```

Output:

0.9928571428571429

As you can see, the model achieved near-perfect accuracy even though we reduced the data to a single dimension. For the sake of comparison, let's see how logistic regression performs on the original 7-dimensional data set.

```
# Create logistic regression model
lr_no_lda = LogisticRegression()

# Fit the model
lr_no_lda.fit(X, y)

# Model accuracy
print(lr_no_lda.score(X, y))
```

Output:

0.9428571428571428

In this case the model accuracy is still very good, but it happens to be a bit less accurate than the model that was trained on `X_new`. This illustrates the power of LDA. We were able to get similar results while greatly reducing the dimension of the data set.

Coding question

Questions

The beans data set comes from the [UCI Machine Learning Repository](#). The original data set contains data for seven different kinds of beans, but for the sake of simplicity you'll only look at two classes: barbunya beans and cali beans. The task is to classify beans based on their size and shape. There are 16 independent variables.

Use scikit-learn to create an LDA model called `lda`. Use the `fit_transform()` method to fit `lda` to `X` and `y` and create a 1-dimensional subspace called `X_new`.

Fit the logistic regression model to the new subspace `X_new` and `y`. Then create a variable called `lr_acc` and set it to the accuracy of the logistic regression model. You can use the `score()` method to find the accuracy. Then print `lr_acc`.

Code

1
2
3
4
5
6
7
8
9
10
11
12
13

14
15
16
17

Run

Run your code to check your answer

LDA as a classifier

We can also use LDA itself as a classifier. In this case, scikit-learn will project the data onto a subspace and then find a decision boundary that is perpendicular to that subspace. Put simply, it will do LDA and use the result to find a linear decision boundary.

Since we've already fit `lda` to `X` and `y`, we can simply look at the accuracy of the model by using the `score()` method.

```
print(lda.score(X, y))
```

Output:

```
0.9857142857142858
```

Coding question

Questions

Use LDA as a classifier on the beans data set. When using LDA as a classifier, it's not necessary to transform `X` into `X_new`. You can simply use the `fit()` method to fit `lda` to `X` and `y`. After fitting the model, create a variable called `lda_acc` and set it to the model's accuracy (`lda.score(X, y)`). Then print `lda_acc`.

Code

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18

Run

Run your code to check your answer

On this data set, LDA performs comparably to logistic regression.

Assumptions and caveats

As useful as LDA is, it relies on several assumptions. Some of these assumptions arise because LDA relies on linear algebra. In order for all the necessary linear algebra tricks to work out, a data set has to have some specific properties.

Class size versus the number of variables

LDA only works if the size of the smallest class is greater than the number of variables. If you have 10 variables and a class with only 8 observations, then LDA simply won't work. (The technical reason for this is that LDA works by inverting a class's covariance matrix, and that matrix won't be invertible if you don't have more observations than variables.)

Normal variables

LDA relies on the assumption that independent variables in each class are multivariate normal. The technical definition of a multivariate normal distribution is that any linear combination of variables must be univariate normal (the univariate normal distribution is the familiar bell-shaped normal curve). Don't worry too much about the technical definition: in some applications it's fine to look at histograms of variables within each class and see if they look close enough to a bell curve.

Homoscedasticity

Homoscedasticity is the assumption that each class has the same covariance matrix. Intuitively this means that a scatter plot of one class has roughly the same size and shape as the scatter plot of every other class. If the values in one class are spread very far apart and the values in another class are clumped close together, or if the scatter plots of two classes are shaped like long, narrow ellipses that are perpendicular to one another, then LDA won't perform well. If, however, all scatter plots look similar, LDA can be applied. (Side note: if you compute the sample covariance matrices for each class in a data set, the matrices will almost never be exactly the same. It's okay if they're close enough.)

For cases when other assumptions hold true but the data is not homoscedastic, there is a technique called quadratic discriminant analysis (QDA). QDA is very similar to LDA, but it can be used when classes have different covariance matrices.

Other considerations

Another assumption of LDA is that independent variables are not multicollinear. This means that no variable is closely correlated with any other variable or linear combination of variables. This particular assumption is not as strict as the other assumptions, but it's good to be aware that LDA will generally perform worse on a data set with multicollinear variables.

Another consideration for LDA is that the observations in your data set should be statistically independent. If you have dependent variables (for example, multiple observations of the same subjects over time), LDA shouldn't be applied.

It's also important to note that LDA can be sensitive to outliers. If a data set has extreme outliers it may be best to remove them before doing LDA.

Finally, it's good to be aware that LDA is prone to overfitting. Scikit-learn's implementation of LDA addresses this by using a `shrinkage` parameter. This is a regularization technique for LDA that helps to prevent overfitting. We won't address this any further here, but it's something to be aware of if you use LDA in the future.

Multiple choice

Questions

Which of the following is NOT a required assumption of LDA?

Answer Choices

Data within each class should be multivariate normally distributed.

All class sizes should be equal.

The size of the smallest class must be greater than the number of variables.

Each class's covariance matrix should be (at least approximately) equal.

Conclusion

In summary, LDA is a powerful dimensionality reduction technique for classification problems with continuous independent variables. It uses linear algebra to find a subspace of a data set that does the best job of distinguishing between classes. It can also be used as a classifier. The scikit-learn library in Python allows you to easily implement LDA for both dimensionality reduction and classification. LDA does require certain assumptions about a data set, including multivariate normality, homoscedasticity, and the requirement that the size of the smallest class is greater than the number of variables.

An Introduction to Regularization in Machine Learning

Review

In this lesson we've learned the following topics:

- The importance of regularization in minimizing overfitting
- Lasso and Ridge regularization methods and their associated loss functions
- How to implement Lasso and Ridge with Multiple Linear Regression
- How regularization can be used as a feature selection method
- Hyperparameter tuning and the bias-variance tradeoff

The tables here summarize the key concepts we've learned. These form the basis for understanding regularization in more complex non-linear algorithms such as support vector machines and neural networks. We are now ready to learn how to tune hyperparameters and explore different methods of implementing regularization in Python!

1. Differences between Lasso and Ridge

	Lasso Regularization	Ridge Regularization
Regularization / Penalty term	L1 norm (sum of <u>absolute</u> values) of coefficients	L2 norm (sum of <u>squared</u> values) of coefficients
Constraint surface	<u>Diamond</u> shaped in 2D	<u>Circle</u> shaped in 2D
Coefficients after Regularization	All the coefficients shrink in their magnitude and <u>some can be zero</u> .	All the coefficients shrink in their magnitude but <u>never approach zero</u> .
Feature Selection Method	Yes! Features corresponding to the zero value coefficients are eliminated.	No! But it helps in emphasizing the relative feature importance.

2. Hyperparameter Tuning and the Bias-Variance Tradeoff

Low alpha	High s	Low Bias	High Variance	Low Regularization	Overfitting	Low training error
High alpha	Low s	High Bias	Low Variance	High Regularization	Underfitting	High training error

Implementing Regularization Methods in Python

Learn how to implement L1 and L2 regularization using `scikit-learn` and tune the regularization hyperparameter using `GridSearchCV` for linear and logistic regression.

Implementing Regularization with Linear Regression

In the previous lesson, we saw how implementing Lasso and Ridge regularization methods with linear regression in `scikit-learn` was fairly straightforward. Both `Lasso()` and `Ridge()` functions have an input parameter `alpha` that sets the strength of the regularization. The not-so-straightforward part was the question of how to choose the right `alpha` for the data and question at hand. We're now ready to dive into this!

For starters, we're going to run a quick Lasso implementation on the student performance `dataset` that we were working with. The dataset has 41 features relating to various factors that might influence a student's final grade. Let's take a sneak peek first:

```
import pandas as pd
df = pd.read_csv('student_math.csv')
print(df.columns, df.shape)
```

Output:

```
Index(['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'famrel',
      'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2',
      'Final_Grade', 'school_MS', 'sex_M', 'address_U', 'famsize_LE3',
      'Pstatus_T', 'Mjob_health', 'Mjob_other', 'Mjob_services',
      'Mjob_teacher', 'Fjob_health', 'Fjob_other', 'Fjob_services',
      'Fjob_teacher', 'reason_home', 'reason_other', 'reason_reputation',
      'guardian_mother', 'guardian_other', 'schoolsup_yes', 'famsup_yes',
      'paid_yes', 'activities_yes', 'nursery_yes', 'higher_yes',
      'internet_yes', 'romantic_yes'],
      dtype='object') (395, 42)
```

We set our predictor and outcome variables and perform a train-test-split:

```
y = df['Final_Grade']
X = df.drop(columns = ['Final_Grade'])
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
random_state=42)
```

We're now ready to fit a Lasso regularized regression model by using the `Lasso()` function from the `linear_model` module:

```
from sklearn.linear_model import Lasso
lasso = Lasso(alpha = 0.05)
lasso.fit(X_train, y_train)
```

We're going to look at the Mean Squared Error (MSE) using a function from the `metrics` module:

```
from sklearn.metrics import mean_squared_error
pred_train = lasso.predict(X_train)
pred_test = lasso.predict(X_test)
training_mse = mean_squared_error(y_train, pred_train)
test_mse = mean_squared_error(y_test, pred_test)
print('Training Error:', training_mse)
print('Test Error:', test_mse)
```

Output:

```
Training Error: 2.8132075838851405
Test Error: 4.474769444129441
```

We see that the test error is more than 1.5 times that of the training error. This is an indicator that the model is still overfitting the data and that there is room to regularize more.

Tuning the Regularization Hyperparameter

We chose `alpha` to be 0.05 (the `scikit-learn` default is 1.0). It is worth recalling here that an `alpha` of zero is equivalent to no regularization. And when `alpha` is allowed to get too big we have the opposite problem of biasing our model and consequently underfitting the data.

One way to figure out the “Goldilocks zone” in this bias-variance tradeoff is to try multiple values of `alpha`. Iterating over an array of `alpha` values and plotting the resulting training and test errors against the corresponding `alpha`'s gets us the following figure:



We see that the test error goes down as we increase `alpha` until a certain value after which it begins to go up and then seems to slowly but steadily increase. Our optimal `alpha` thus lies somewhere in the blue band shown, likely a number between 0.12 and 0.16.

There is, however, another question to consider: would this be the same for a different train-test split? It is quite possible that our tuned hyperparameter might be different if we did a different train-test split. Luckily, `scikit-learn` has just the feature for us. `GridSearchCV`, short for “Grid Search Cross-validation” helps us to satisfactorily resolve this while automating and speeding up our hyperparameter search - all within a couple of lines of code!

Automate the Hyperparameter Search with `GridsearchCV`

`GridSearchCV` uses a “k-fold” cross-validation method to search for the optimal hyperparameter in a machine learning algorithm. The k-fold method iteratively splits the dataset into train-test splits “k” times such that every point in the sample gets to be within the test data at least once. For instance,

if we wanted to do 5-fold cross-validation, our data would be split into a 80:20 train-test split five times. To visualize this, imagine that our data is contained within this green box as shown below:



Our five “folds” can be represented by the yellow box. Each location of the box shows a specific train-test split. If we iterated our model fit over these five folds, every part of the data gets to be “test data” once. Doing a k-fold cross-validation makes sure that the conclusions we’re drawing about our data are not the result of a sampling effect but are truly representative of all of the data.

`GridSearchCV` takes in the following arguments:

estimator: the machine learning algorithm whose hyperparameters we’re tuning (in our case, `Lasso()` or `Ridge()`)
param_grid: a grid of potential values for the hyperparameters that are being tuned. This has to be in the form of a dictionary with the keys representing the parameter inputs to the model and the values, lists of potential parameter values. For instance in the case of our Lasso implementation, we have only one hyperparameter we’re tuning, i.e., `alpha`.

Since we don’t have an idea of what order of magnitude `alpha` needs to be (i.e., is it between 0.01-0.1 or 0.1-1.0 or 1-10 and so on), using NumPy’s `logspace` function can be very useful. It is similar to `linspace` except that it gets us numbers that are evenly spaced in the logarithmic scale. (It takes in the powers of ten that we’re searching between, so if we wanted to search between 0.01 to 100, we’d set its inputs to -2 and 2 respectively.)

```
import numpy as np
## an array of alpha values between 0.000001 and 1.0
alpha_array = np.logspace(-6, 0, 100)

#dict with key (alpha) and values being alpha_array
tuned_parameters = [{'alpha': alpha_array}]
```

scoring: the metric used to evaluate the performance of the model on the test set. `GridSearchCV` searches for the set of parameters within `param_grid` that maximizes this value.

scikit-learn’s [documentation](#) has an exhaustive list of scoring strategies. The argument for mean squared error, our chosen metric, is `neg_mean_squared_error`. It’s a negative value because `GridSearchCV` *maximizes* the score it’s given.

cv: specifies the way the cross-validation is performed. The default here is the 5-fold cross-validation method that’s described above.

Otherwise one can set a different number of folds (i.e., an alternate positive integer value for `k`)

return_train_score: a Boolean argument specifying whether we would like the output of our fit to return the scores on training data every folds. The default here is `False` but we’re going to set it to `True` as we’re using `GridSearchCV` to correct for overfitting and would like to be kept in the loop about the training score.

Consider our implementation of Lasso on the student performance dataset. Suppose we wanted to check the result of implementing different values of `alpha` and wanted to do multiple train-test splits to make sure we’re covering the entire sample. We could do this in one line as follows:

```
from sklearn.model_selection import GridSearchCV
model = GridSearchCV(estimator = Lasso(), param_grid = tuned_parameters, scoring =
'neg_mean_squared_error', cv = 5, return_train_score = True)
model.fit(X, y)
```

The result of the `fit` object has many attributes, of which we’re going to examine the ones that are most important to us:

The `cv_results_` object gives us the details of **every** model fit corresponding to a particular `alpha` value and the train-test split of each fold. (Our `alpha` array had 100 values and we did a 5-fold cross-validated search - so this is essentially equivalent to performing 500 model fits!) We're specifically going to look at the mean train and test scores across the 5 train-test splits:

```
test_scores = model.cv_results_['mean test score']
train_scores = model.cv_results_['mean train score']
```

Each of the above objects is an array the size of our `param_grid`, i.e., the number of `alpha` values we've specified.

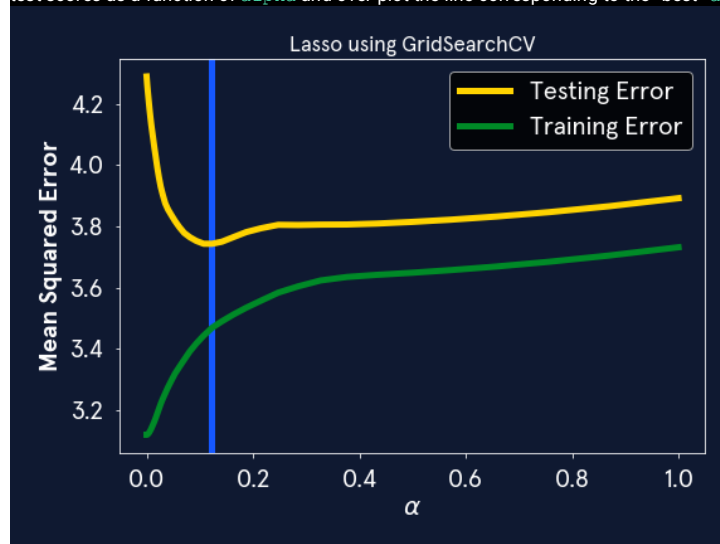
Additionally, the model fit object also gives us: `best_estimator_`, `best_score_` and `best_params_`. To get the `alpha` that is optimal to our scoring strategy, we can do the following:

```
print(model.best_params_ , model.best_score_)
```

Output:

```
{'alpha': 0.12328467394420659} -3.7432949015591155
```

(Note that the score is the *negative* of least test mean squared error!) If we were to replicate the previous plot, i.e., plot our new training and test scores as a function of `alpha` and over plot the line corresponding to the "best" `alpha` value, we get the following:



The blue line here is our tuned hyperparameter and this value of `alpha` (~0.1233) corresponds to the optimal cross-validated test and training error values.

Coding question

Questions

We've loaded the student performance dataset here. Following the same logic as described above, implement hyperparameter tuning using `GridSearchCV` for Linear Regression with Ridge Regularization. Find the optimal `alpha` and the mean squared error corresponding to the best `alpha`. `GridSearchCV` might take about a minute to run - it is, after all, performing 5 (folds) times 100 (hyperparameter values) = 500 model fits! :)

Note: We've already specified a range of `alpha` values between 0.01 and 10000. These are a few orders of magnitude more than Lasso's `alpha` values. Can you think of why this might be the case?

Run

Run your code to check your answer

Note: The `alpha` values are orders of magnitude bigger for Ridge for the following reason: `alpha` is inversely proportional to the size of the regularization constraint. Recall that for Lasso this means that it's proportional to $1/s$ while for Ridge, this would be $1/s^2$ as the constraint surfaces are different. So reducing `s` by a factor of 10 means increasing `alpha` 10x for Lasso but 100x for Ridge!

Implementing Regularization with a Logistic Regression Classifier

So far we've seen different implementations of regularization with linear regression. What if we were working with a classification problem instead? We're going to discuss a couple of ways of implementing L1 and L2 regularization with a logistic regression classifier.

On `alpha`'s, `C`'s and `solver`'s: some `scikit-learn` pointers!

This might come as a surprise to you (it certainly did to me): the default `scikit-learn` logistic regression **implementation** is *already regularized*! It is L2-regularized with an `alpha` of 100. To elaborate, let's take a look at a couple of attributes that go into the logistic regression model:

penalty: The options here are 'l1', 'l2', 'none' and 'elasticnet' and the default is 'l2'.
C: The inverse of regularization strength - the default value is 1.0.
solver: The options are {'lbfgs', 'liblinear', 'newton_cg', 'sag', 'saga'}.

The `LogisticRegression()` function differs from `Lasso()` and `Ridge()` functions for Linear Regression in that its input is the parameter `C`, which is the inverse of `alpha`. This is important to keep in mind, especially while setting up the parameter grid prior to implementing `GridSearchCV`.

Let's take a look at our different options for implementation then:

No regularization:

```
from sklearn.linear_model import LogisticRegression
logistic_no_regularization = LogisticRegression(penalty = 'none')
```

Lasso (L1): The L1 implementation requires setting the *penalty* attribute to 'l1' and also setting the *solver* attribute to 'liblinear'

```
logistic_lasso = LogisticRegression(penalty = 'l1', solver = 'liblinear', C =  
    )
```

It is worth noting here that there is a *solver* attribute in `scikit-learn` for most machine learning algorithms. The default solver for `LogisticRegression()` is `lbfgs`, however the only solver that can be used with Lasso Regularization is the 'liblinear' solver and hence it needs to be explicitly specified. (We're not going to go into further details at the moment but [the docs](#) are always a great place to get some more pointers!)

Ridge (L2): Remember the default is 'l2'! So all we need to do is specify *C* here:

```
logistic_ridge = LogisticRegression(C = )
```

Elasticnet: The *penalty* options 'l1', 'l2' and 'none' are self-evident but we haven't covered 'elasticnet' in this course. Elasticnet Regularization is a combination of L1 and L2 regularization and has two penalty terms, one for L1 and one for L2!

```
logistic_elasticnet = LogisticRegression(penalty = 'elasticnet', solver =  
'saga', C = , l1_ratio = )
```

To implement Elasticnet one needs two hyperparameters: *C*, which specifies the regularization strength and an additional

mixing hyperparameter, 'l1_ratio' which specifies how much L1 regularization used relative to L2.

'l1_ratio' takes values between 0 and 1, with 1 being the same as applying just L1 and 0, being equivalent to applying just L2 penalties respectively.

The solver 'saga' needs to be explicitly specified here.

Tuning hyperparameter *C* using `GridSearchCV` and `LogisticRegressionCV`

Implementing Lasso or Ridge with logistic regression in `scikit-learn` is as simple as picking the right arguments for *penalty* and *C*. We've written some sample code here to find the optimal *C* using `GridSearchCV`: (Remember that *C* is the inverse of `alpha` so greater the *C*, the lesser the amount of regularization!)

```
#Making an array of C's; here we're choosing 100 values between 0.001 and 100
C_array = np.logspace(-3,2, 100)
```

```
#Making a dict to enter as an input to param grid
tuning_C = {'C':C_array}
clf = LogisticRegression(penalty = 'l1', solver = 'liblinear')
gs = GridSearchCV(clf, param_grid = tuning_C, scoring = 'accuracy', cv = 5)
```

Coding question

Questions

A dataset related to different types of candies and their rankings has been loaded for you here. The [data](#) comes from FiveThirtyEight and the article it pertains to is Halloween Candy Rankings from 2017.

But we're going to answer a different question: can we predict whether a candy contains chocolate or not? Build a logistic regression classifier with L2 regularization using `GridSearchCV` to obtain the best *C* as well as the best possible accuracy. Use the parameters `model`, `tuning_C`, `scoring='accuracy'`, and `cv=5` in your `GridSearchCV` model and save your `GridSearchCV` model as `gs`.

Be sure to take a look at the `model` and `tuning_C` variables provided for you, but do not change them.

Code

Since Logistic Regression is such a commonly used algorithm, `scikit-learn` has a function that makes the above faster and neater, the `LogisticRegressionCV()` function. If we were to implement the code challenge using this function, it would look as follows:

```
model = LogisticRegressionCV( Cs=np.logspace(-3,2, 100),
                             penalty='l2',
                             scoring='accuracy', cv=5,
                             random_state=42,max_iter=10000)

model.fit(X, y)
print(model.C_, model.scores_[1].mean(axis=0).max())
```

Output:

```
[2.42012826] 0.8823529411764705
```

Which is the same output as we get from the code challenge! Note that there are slight syntactical differences between the outputs of `GridsearchCV` and `LogisticRegressionCV`. In general, many algorithms in `scikit-learn` have their own special grid search cross-validation function such as `LassoCV()`, `RidgeCV()`, `ElasticnetCV()`, etc.

Conclusion

We have covered multiple ways of implementing regularization with linear and logistic regression in `scikit-learn` in this article. There are a few essential (albeit annoying) differences between these methods. Together they offer a powerful toolkit to find the optimal regularization for regression and classification methods. You can try out all these techniques in the upcoming project!

Hyperparameters in Machine Learning Models

[Learn about hyperparameters in machine learning models.](#)

Introduction

Hyperparameters are an important part of machine learning. We're going to explain what exactly they are along with examples of what they look like for different commonly used ML models. You'll learn why they're important and how they affect model performance. You'll also learn about a few hyperparameter tuning methods that you can implement in your own projects.

What are Hyperparameters?

A *hyperparameter* of a machine learning model is a value that determines part of the learning process and is not affected by training unlike a *parameter*, which is learned during the model training process. You're probably already familiar with some hyperparameters. Let's look at some examples.

In the k-nearest neighbors algorithm, k is a hyperparameter. k determines how many neighbors will be used, and training data does not change k.

Decision trees also use hyperparameters. Before training a decision tree classifier, you might want to specify how deep the tree can go (i.e., how many splits are allowed before arriving at a leaf). You might also want to specify the minimum number of samples that are present in a node in order to split that node. Both of those values are hyperparameters. They determine the structure of the model, but they are not learned during training.

Regularization factors are another example. In linear or logistic regression, regularization is a term that is added to the loss function in order to penalize models with large coefficients. (The coefficients are the *parameters* of the model here! The regularization factor is a hyperparameter. The value of the regularization factor affects how large the coefficients of a regression model will be, but the regularization factor is independent of training data.

Hyperparameter Tuning

Hyperparameters can make or break a machine learning model. Take the k-nearest neighbors algorithm as an example. Recall that in k-nn classification, the class of a sample is determined by the classes of its k nearest neighbors from the training data. If k is too large, classes with a small number of samples might be missed, and the model could underfit the data. If k is too small, it's possible that outliers in the training data could misclassify their neighbors. The model could be overfit to those outliers.

This balance between overfitting and underfitting is called the *bias-variance tradeoff*. It's an important consideration for supervised machine learning.

The bias variance tradeoff is called a tradeoff because decreasing the bias usually increases the variance. The converse is also true: decreasing variance usually increases the bias. Good machine learning models strike a balance. We can reduce bias by making a model more sensitive to training data, but we do so at the risk of overfitting. On the other hand, we can reduce variance by restricting a model so that it is less affected by training data. This, however, runs the risk of underfitting the model.

Ultimately, we want models that perform well on testing data. One way to do that is to choose the right hyperparameters. In the case of supervised machine learning, we want hyperparameters that make a good compromise between bias and variance. This can be done by training a model multiple times, each time with different hyperparameter values, and then using the best ones. This process is called *hyperparameter tuning*. There are several different ways of doing this.

Common Hyperparameter Tuning Methods

The *grid search* algorithm for hyperparameter tuning works by training a model on predetermined lists of hyperparameter values. This method tries every hyperparameter value on the list, and then uses the one that makes the model perform best.

The *random search* algorithm works similarly, but instead of using a predetermined list of hyperparameter values, the values are randomly chosen. As with grid search, it selects the hyperparameter that performed the best.

Bayesian optimization is another approach to hyperparameter tuning. It uses ideas from the field of Bayesian statistics to iterate through different hyperparameter values. Each time the Bayesian optimization algorithm evaluates a new hyperparameter value, it gains more information about where it should look for the best hyperparameter value.

Genetic algorithms are another possible hyperparameter tuning method. These work by going through several generations of hyperparameter values. Within each generation, the fittest (i.e., best-performing) hyperparameter values are slightly mutated (i.e., changed) in order to produce the next generation.

Each of these hyperparameter tuning methods will be covered in more detail in a subsequent lesson. You'll also learn how to implement each of them in Python.

When tuning hyperparameters, it's important to split the data into training, testing, and validation data. Training data is used to train the model. Validation data is used to evaluate hyperparameters. After a hyperparameter is tuned, the model can be tested on testing data. This data allows for an estimate of model performance that isn't affected by the hyperparameter tuning process.

Here's an example. Say there is a classification problem and we want to tune a hyperparameter `alpha`. Say we want to determine whether the model performs best with `alpha=1`, `alpha=2`, or `alpha=3`. We'll train the model three times: once with each possible value for `alpha`. (Since we are looking at a predetermined list of hyperparameter values, this is a grid search.) Let's say the accuracy was as follows.

	Accuracy on training set	Accuracy on validation set	Accuracy on testing set
<code>alpha=1</code>	85%	80%	
<code>alpha=2</code>	85%	84%	82%
<code>alpha=3</code>	88%	83%	

Since the model was most accurate on the validation set with `alpha=2`, we will use that value. However, we can't estimate the model's accuracy based on data that was used to determine the model's hyperparameter. This would likely overestimate the actual accuracy. Therefore, we must measure the accuracy of the final model (with `alpha=2`) on the testing set.

Here's a table that lists several machine learning algorithms, some of their hyperparameters, and possible tuning methods. This table is not exhaustive: some machine learning algorithms have many hyperparameters that aren't listed here. If you want to learn more about the hyperparameters for a specific model in scikit-learn, the documentation is a good place to start.

For example, you can read about the details of scikit-learn's support vector machine classifier [here](#). You'll find descriptions of `c` (the inverse of regularization strength), `kernel`, `degree`, and more. Each of these values is a hyperparameter that can be chosen or tuned.

Hyperparameter Tuning with `scikit-learn`

Hyperparameter Tuning Review

Nice work! You've learned how to tune the hyperparameters in a machine-learning model! Let's go over some of the key points to take away.

In

[machine learning](#)

Preview: Docs [Machine learning](#) is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

, hyperparameter tuning is altering the hyperparameter values of the model to get the model that's most suited to answering a question about the data.

Examples of hyperparameters; the learning rate, regularization penalty, number of neighbors (in a k-means model), tree depth (decision tree model), and minimum number of samples to split (decision tree model).

Grid Search is a

[method](#)

Preview: Docs [Loading link description](#)

of hyperparameter tuning. It works by testing a model on a list of hyperparameter values decided upon beforehand.

With a grid search, if we want to test 2 different types of regularization penalties and 3 different regularization strengths, this would give us 6 different combinations of hyperparameters on our grid to test ($2 * 3 = 6$).

The Random Search method is an opposing method to grid search. In a random search model, a

[dictionary](#)

Preview: Docs [Loading link description](#)

is given that specifies the distributions from which the model will randomly draw values for each hyperparameter. Additionally, the number of times the

[algorithm](#)

Preview: Docs [Loading link description](#)

should randomly draw from these distributions is also specified.

Both `GridSearchCV()` and `RandomSearchCV()` use a technique known as **k-fold cross-validation**. This is when new combinations of the training and testing sets are created (using the same data) to conclude the most accurate model. This ensures a less biased model.

We used the following attributes to evaluate both the `GridSearchCV()` and `RandomSearchCV()` models; `.best_estimator_`, `.best_score_`, `.best_params_`, and the `.cv_results_`.

ML Algorithm	scikit-learn package	Hyperparameters	Parameters in sklearn	Notes	How to tune
Linear regression with gradient descent	<code>sklearn.linear_model.SGDRegressor</code>	Learning rate, regularization penalty	<code>learning_rate</code> , <code>penalty</code> , <code>alpha</code>	Set <code>penalty</code> to <code>l1</code> or <code>l2</code> (default). <code>alpha</code> is the regularization strength. Larger <code>alpha</code> will result in stronger regularization.	<code>learning_rate</code> and <code>C</code> can be tuned with a grid search or random search.
Logistic regression	<code>sklearn.linear_model.LogisticRegression</code>	Regularization penalty	<code>penalty</code> , <code>C</code>	Set <code>penalty</code> to <code>l1</code> , <code>l2</code> (default), or <code>None</code> . <code>C</code> is the inverse of regularization strength. Smaller <code>C</code> will result in stronger regularization.	<code>C</code> can be tuned with a grid search or random search.
k-nearest neighbors	<code>sklearn.neighbors.KNeighborsClassifier</code>	k (the number of neighbors)	<code>n_neighbors</code>		<code>n_neighbors</code> can be tuned with a grid search or random search.
Decision trees	<code>sklearn.tree.DecisionTreeClassifier</code>	Maximum depth of the tree, minimum number of samples to split an internal node	<code>max_depth</code> , <code>min_samples_split</code>		<code>max_depth</code> and <code>min_samples_split</code> can be tuned with a grid search or random search.
k-means	<code>sklearn.cluster.KMeans</code>	k (the number of clusters)	<code>n_clusters</code>		Use the elbow method.
Support vector machines	<code>sklearn.svm.SVC</code>	Regularization penalty	<code>C</code>	Smaller <code>C</code> will result in stronger regularization.	<code>C</code> can be tuned with a grid search or random search.

Introduction to Ensembling Methods

An introduction to Ensembling Methods in Machine Learning.

Introduction

Why use one machine learning model to solve a problem when you can use many at the same time? **Ensemble learning** is the process of building a complex machine learning model by combining multiple base estimators as building blocks. The main goals for employing ensemble methods are to end up with a resultant machine learning model that is more performant and more robust than its components.

Base estimators

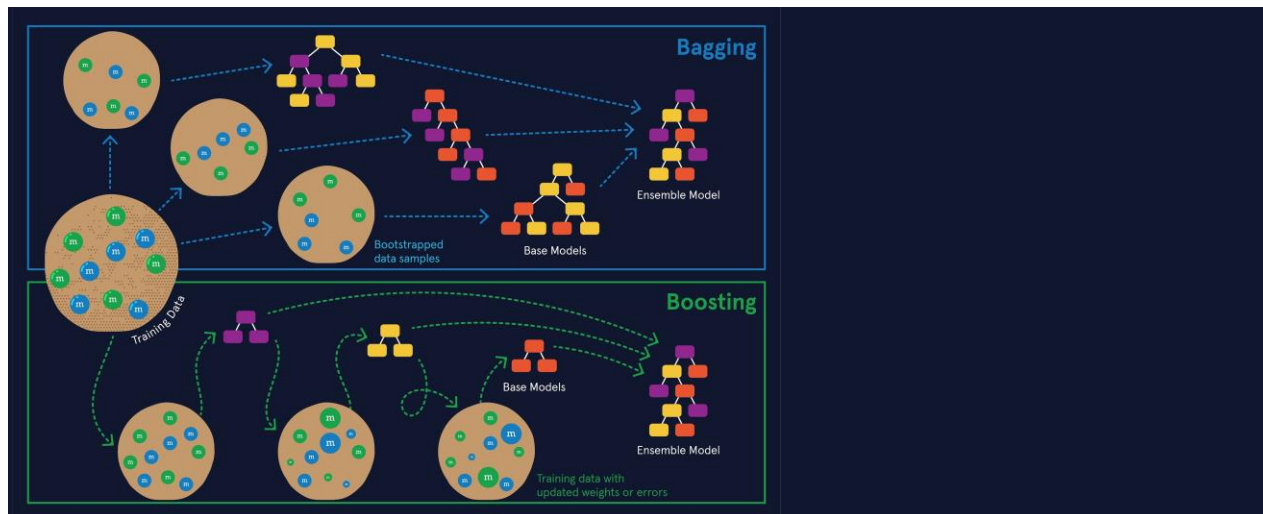
In ensembling, often the base models that are chosen tend to underachieve on their own and are typically referred to as **weak learners**. The reason this is preferable is to have the model implementation be computationally efficient. Combining strong learners doesn't necessarily make the resultant ensemble model more performant so one might as well choose learners that cost less computationally.

What makes a model a "weak learner"? There are many ways in which a base model can be considered weak. For example, it can have high bias or high variance. The nature of the weakness of the base model is typically taken into consideration and is a design choice when determining the best ensembling method to use.

The basic components of ensemble learning are:

It is possible to have an ensemble model that performs worse than any one of its contributing base estimators. To circumvent this outcome it is important that the base estimators are uncorrelated and independent. Having a higher diversity among the trained base estimators leads to a stronger ensemble model.

Three common ensembling methods in machine learning that will be covered briefly in this article and later on in separate modules are Bagging, Boosting, and Stacking.



Bagging

As the name suggests, **Bootstrap AGGREGATING** (also known as **Bagging**) is an ensemble method that combines the concepts of bootstrapping and aggregation. Bagging can be used for both classification and regression problems. Bagging methods use weak learners as base models that are complex and tend to suffer from high variance. Their weakness as models is due to the fact that they are built with only a subset of the available features and on a subset of the training data due to bootstrapping.

If the full dataset is represented by the larger cookie in the figure above, each of the candies atop the cookie represents an individual training data instance. The smaller cookies in the Bagging panel represent a **bootstrapped** sample of the full training dataset. Bootstrapping refers to the method of sampling data with replacement.

Bagging is a learning technique that is done in parallel. Each of the base models is trained independently of the others. Additionally, each base model is trained using only a subset of the original features. Even though the base models tend to be complex, they overfit to both a subset of the available training data and a subset of the available features. This allows them to be diverse from one another, often leading to a very strong ensemble model when aggregated. In the Bagging panel of the figure, the base models are decision trees that are relatively large and overfit to the bootstrapped subset of data provided to each of them.

Once each of the base models is trained, the method for ensembling tends to be a simple aggregation technique over each of the models; a majority vote for classification problems and averaging for regression problems.

A common implementation of a bagging algorithm that uses decision trees as their base model is the Random Forest. You can learn more about Bagging and Random Forests in the [Random Forests module](#).

Boosting

Boosting is an ensemble learning technique where the weak learners are too simple and tend to suffer from high bias. In the Boosting panel of the figure above, the base models are decision trees with only one level, a decision stump. Decision stumps can only make a decision based off of one feature at a time, causing them to underfit the data substantially.

Boosting is a sequential learning technique where each of the base models builds off the previous model. Each subsequent model aims to improve the performance of the final ensembled model by attempting to fix the errors in the previous stage.

In the Boosting panel of the figure, you may notice that some of the candies atop the cookies are larger than the others. These particular training instances were misclassified by the previous decision stump and are therefore given more weight by the next decision stump. This is one method in which boosting methods may learn from their mistakes.

Two common implementations of the boosting algorithm are Adaptive Boosting and Gradient Boosting. You can learn more about Boosting, including these two implementations, in the [Boosting module](#).

Stacking

Stacking is an extremely flexible ensembling technique where a final model is trained to learn how to best combine a set of base models to make strong predictions. In contrast to bagging and boosting, the base models in stacking do not need to be the same type of learning algorithm.

While bagging and boosting are built with base models that are weak learners, that does not necessarily have to be the case for stacking. A stacking algorithm can be used to combine decently performing learners as well.

You can learn more about Stacking in the [Stacking article](#).

Random Forests

Review

Nice work! Here are some of the major takeaways about random forests:

A random forest is an ensemble

[machine learning](#)

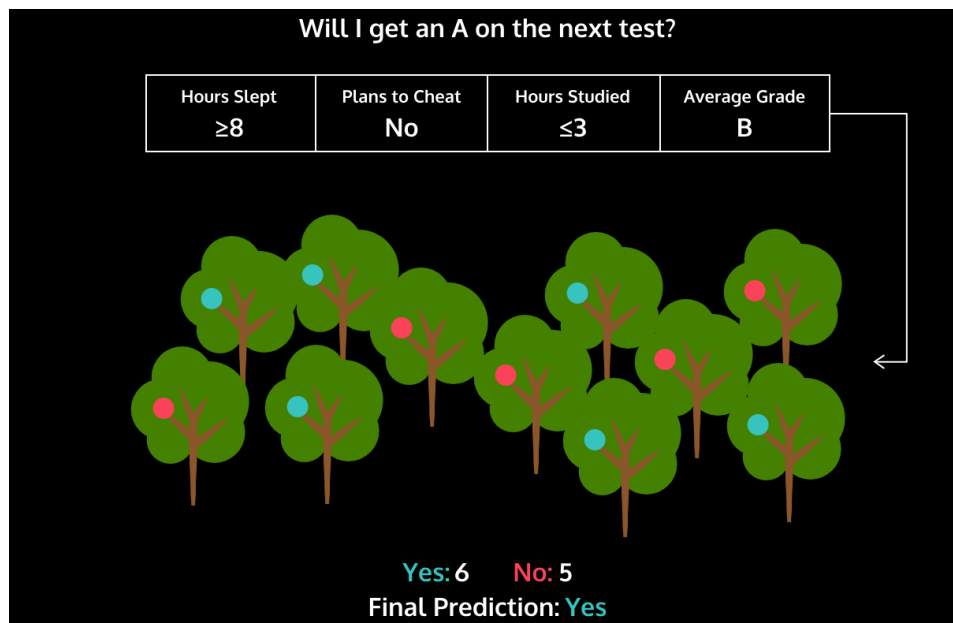
Preview: Docs Loading link description

model. It makes a classification by aggregating the classifications of many decision trees.

Random forests are used to avoid overfitting. By aggregating the classification of multiple trees, having overfitted trees in a random forest is less impactful.

Every decision tree in a random forest is created by using a different subset of data points from the training set. Those data points are chosen at random with replacement, which means a single data point can be chosen more than once. This process is known as bagging.

When creating a tree in a random forest, a randomly selected subset of features are considered as candidates for the best splitting feature. If your dataset has n features, it is common practice to randomly select the square root of n features.



Stacking

Learn about the stacking ensemble learning method.

Introduction

We've seen how ensembling methods such as bagging or boosting can be used to manage bias and variance of a machine learning model and improve performance. In this article, we're going to learn about another ensemble method: Stacking! Stacking fundamentally operates differently from bagging and boosting in two ways:

Stacking is a proven technique for improving prediction accuracy of standalone models. How does this work? Let's take a look!

How does stacking work?

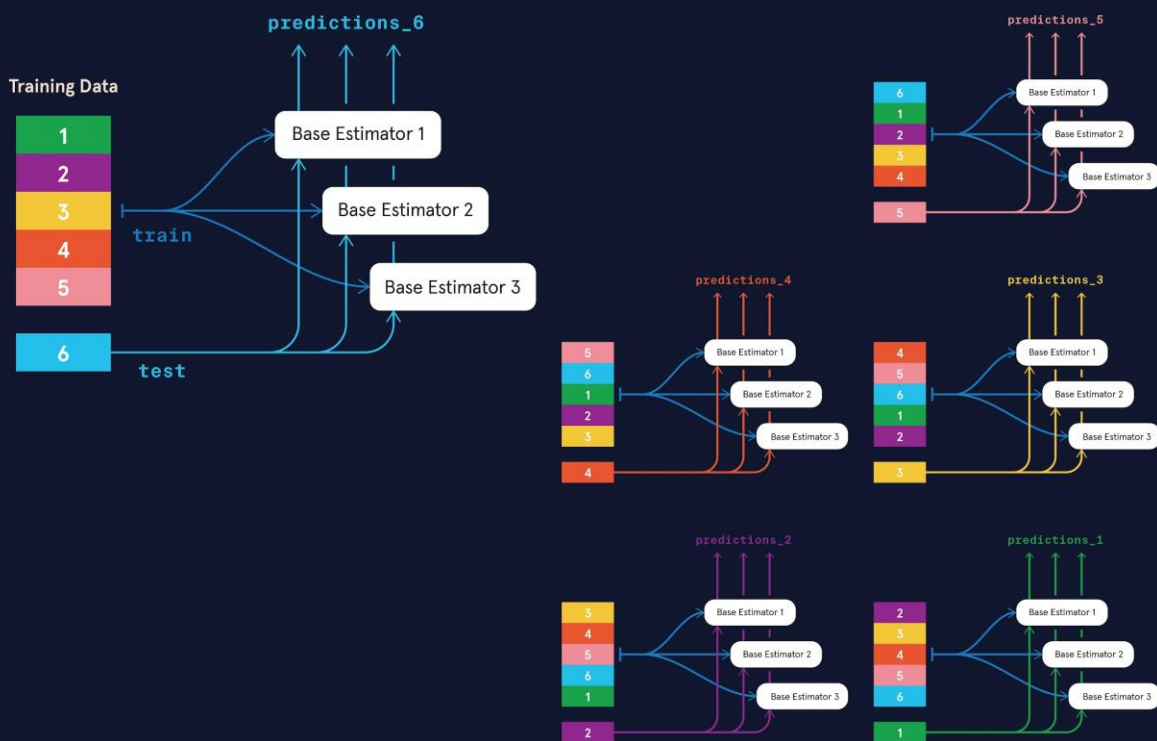
Consider the scenario in which we are handling a classification problem where we are exploring different models. When using a decision tree we find that our model has poor generalization performance as a result of overfitting. Additionally, when using a logistic regression classification model, we discover our model parameters are hard to tune due to highly-correlated features. We might discover that while each model has a unique advantage over the others, each may also have a distinct drawback such as poor generalization accuracy or the inability to predict a specific class. As it turns out, there's no rule stating we can't take the best of both worlds!

Stacking can be thought of as a democracy of machine learning models, where different models are trained and subsequently cast their vote through their predictions. A majority-rules approach can be used for determining the final model prediction if we weighted each estimator equally. In practice, each base estimator may need to be weighted differently, so we have a later-stage model to learn how to appropriately weigh the predictions of all the prior base estimators.

Let's talk about creating a stacking model and how to set up our training data for it.

Training Base Estimators

We can select from combinations of different base estimators, such as a logistic regression model in combination with a decision tree. We could additionally select models of the same learning algorithms, but with different parameters, such as multiple decision trees with varying depths. The number of estimators is arbitrary, so it's good practice to explore how different combinations behave. This introduces a problem, however. The estimators would be making predictions on data used in training. This puts our model at risk of overfitting. To avoid this, we use K-Fold Cross Validation as described next.



K-Fold Cross Validation

Consider 10 segments (or folds) and a stacking model that uses a logistic regression model and a decision tree model. Each estimator can be trained using data from 9 of the segments, and make predictions on the excluded 10th segment. We then append the predictions as new features to that 10th segment. Now 1/10th of the training data has two new features: one is the prediction made by the logistic regression model and the other is the prediction made by the decision tree model.

We want to do the same with the other 9 segments, so we rotate the excluded segment and repeat this process until all training data points are augmented with new features. The end result is a prediction made on each training sample, without having seen the sample during the training process.

Feature Augmentation

In our stacking setup, the base estimators need to be trained to make predictions on our training data. The prediction of each estimator will be appended to the corresponding data sample as a new feature. We thus augment the training data set with this additional information. The augmented training set is used by our later-stage stacking model to make the final prediction.



Training Data

So, in summary, say our training dataset has 10,000 samples, 10 features, and we select 3 base estimators. We would train each base estimator on the training set and make predictions on the training set. Each estimator would make a prediction on each sample, therefore each sample will have 3 predictions. These 3 predictions are appended to the pre-existing 10 features. This leaves us with 10,000 training samples with 13 features each.

Training the stacking model

With our augmented training set, we're ready to prepare the final model that will make the official prediction from our stacking setup.

This later-stage model is a learning algorithm that we select just like the base estimators. We may even reuse an algorithm from an earlier stage here. The purpose of this model is to learn the proper weighting of the earlier estimator given our training samples now include a data point from each estimator. Some estimators may perform better than others, so our overall model should account for this.

The only difference in the training process is that the model will be designed to accept samples of the augmented size rather than the given size. This means if the given data set has n features and m base estimators, this model will require $n + m$ features on each sample.

And that's it! Once the later-stage model is trained, we feed testing data samples into the base estimators, which will append their predictions to the data sample. This sample is then given to the later-stage model for our final prediction.

Example

In the associated code for this article, we'll be referencing a water quality dataset. The features of this dataset are ones that can be used to determine how safe a supply of water is to drink or its "potability" represented by a 1 for drinkable and a 0 for not drinkable. For simplicity, let's take a look at a subset of the data with three of its features:

Sample	pH	Conductivity	Turbidity	Potability
1	8.316766	363.266516	4.628771	1
2	9.092223	398.410813	4.075075	0
3	5.584087	280.467916	2.559708	1

For this example, we'll select a Logistic Regression model and a Decision Tree model as our base estimators. The table below shows how the training dataset expands with new features provided by the predictions of the trained base estimators.

Sample	Logistic Regression prediction	Decision Tree prediction	pH	Conductivity	Turbidity	Potability
1	0	1	8.316766	363.266516	4.628771	1
2	0	0	9.092223	398.410813	4.075075	0
3	1	1	5.584087	280.467916	2.559708	1

Implementing stacking with `scikit-learn`

```
import pandas as pd
df = pd.read_csv('water_potability')
print(df.columns, df.shape)
```

Output:

```
Index(['ph', 'Hardness', 'Solids', 'Chloramines', 'Sulfate', 'Conductivity',
       'Organic_carbon', 'Trihalomethanes', 'Turbidity', 'Potability'],
      dtype='object') (2011, 10)
```

We'll take a train-test split of our data to handle the training process.

```
X = water_potability.drop(['Potability'], axis=1)
y = water_potability['Potability']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
                                                    random_state=rand_state)
```

To assemble our ensemble, we'll make a dictionary of base estimators. This will be contained within the `level_0_estimators` dict. Also, our final estimator will be a Random Forest as represented with `level_1_estimator`. Notice also how we prepare to add new features to our training dataset (as columns) in `level_0_columns`.

```
level_0_estimators = dict()
level_0_estimators["logreg"] = LogisticRegression(random_state=rand_state)
level_0_estimators["forest"] = RandomForestClassifier(random_state=rand_state)

level_0_columns = [f"{name}_prediction" for name in level_0_estimators.keys()]

level_1_estimator = RandomForestClassifier(random_state=rand_state)
```

Handling our k-fold cross-validation is fairly straightforward using `sklearn.model_selection.StratifiedKFold` from the `scikit-learn` library. The kfold is then given to the instantiated `StackingClassifier`.

```
kfold = StratifiedKFold(n_splits=10, shuffle=True, random_state=rand_state)
stacking_clf = StackingClassifier(estimators=list(level_0_estimators.items()),
                                final_estimator=level_1_estimator,
                                passthrough=True, cv=kfold, stack_method="predict_proba")
```

Calling `fit_transform` on our classifier manages a lot of the heavy lifting. It will handle the training of our `level_0` base estimators along with the `level_1_estimator`, make cross-validated predictions on the training set, and augment the training set with predictions from each estimator. Let's see how the resulting training dataset looks:

```
df = pd.DataFrame(stacking_clf.fit_transform(X_train, y_train), columns=level_0_columns +
list(X_train.columns))
```

	logreg_prediction	forest_prediction	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity
0	0.417408	0.17	9.927024	208.490738	19666.992792	8.008618	340.237824	482.842435	11.360427	85.829113	4.051733
1	0.379019	0.26	8.769676	215.368742	13969.438863	7.548543	322.799070	369.016667	18.919188	54.755214	3.776718
2	0.409582	0.87	8.077261	125.302719	23931.282833	8.773162	317.693331	398.328789	15.279583	62.668356	4.279871
3	0.407893	0.73	9.739562	166.948864	13623.160063	7.235922	385.059134	369.591289	12.322604	68.505852	2.568080
4	0.399452	0.04	5.343075	211.662091	45166.912141	6.651801	279.767500	485.959717	19.682337	70.546862	4.240032

Finally, with our full model trained, we can make predictions. Let's compare how our Stacking classifier performed with how a lone linear model and a lone decision tree model perform!

```
y_val_pred = stacking_clf.predict(X_test)
stacking_accuracy = accuracy_score(y_test, y_val_pred)

vanilla_logistic_regression = LogisticRegression(random_state=rand_state).fit(X_train, y_train)
lr_accuracy = accuracy_score(y_test, vanilla_logistic_regression.predict(X_test))

vanilla_decision_tree = RandomForestClassifier(random_state=rand_state).fit(X_train, y_train)
dt_accuracy = accuracy_score(y_test, vanilla_decision_tree.predict(X_test))

print(f'Stacking accuracy: {stacking_accuracy:.4f}')
print(f'Logistic Regression accuracy: {lr_accuracy:.4f}')
print(f'Decision Tree accuracy: {dt_accuracy:.4f}')
```

Output:

```
Stacking accuracy: 0.6576
Logistic Regression accuracy: 0.5732
Decision Tree accuracy: 0.6526
```

Limitations of stacking

Stacking is very powerful in that we remove the occasionally difficult choice of which learning algorithm to use for our problem. Depending on the use case, this benefit does come with some limitations worth noting:

Additional Considerations

How many layers can a stacking model have?

Stacking offers some creativity and openendedness in how we want to build our model. A vanilla stacking model may have one tier of models to contribute to the final prediction. We could alternatively construct a multi-tier approach in which one level of models feeds into a later stage of models before making a final prediction.

As with other machine learning models, we also have many parameters to tune and select from. We can enhance model diversity by using different training algorithms, different training sets, different feature subsets, and different hyperparameters. In the next section, we'll use a k-fold cross-validation to preprocess the training data, where k=10. In reality, the value for k is arbitrary, and a different k folds may lead to performance improvement or decline.

Conclusion

We have covered in-depth how stacking can combine the predictive power of various machine learning models. This property distinguishes stacking from other ensemble methods like bagging or boosting, while not being mutually exclusive in application. Further, we discussed how a k-fold cross-validation setup of the training data can help our stacking model avoid overfitting and augment the data to leverage all of its predictive strengths. In a coded example, we see this in action using the `scikit-learn`

library where we assembled a stacking model with logistic regression estimators and decision tree estimators. We found that using these estimators in a combination yielded higher prediction accuracy compared to using them independently.

Machine Learning + Engineering: Bringing it together

We're now ready to put all the things we've learned thus far in the career path together to actually do some machine learning engineering. (ML+ E = MLE!)

Welcome

Welcome to Building Machine Learning Pipelines! We're now ready to combine your machine learning knowledge with software engineering skills to build machine learning pipelines with `scikit-learn`.

Why Learn Building Machine Learning Pipelines?

Machine learning applications are powered by robust end-to-end pipelines. Building these pipelines involves combining the ability to build and tune high-performing machine learning models as well as writing production-quality code that can fit well within a tech company's engineering workflow.

What Will We Cover in This Module?

In this module you'll learn about the different components of a machine learning workflow and how to turn your workflows into end-to-end pipelines. Along the way you'll learn some nifty `scikit-learn` tools that'll help modularize your machine learning code to make it reproducible and effective!

Prerequisites

This module requires prior knowledge of machine learning as well as basic software engineering in Python. Check out our [Intermediate Machine Learning Skill Path](#) and our [Software Engineering for Data Scientists Skill Path](#) to brush up on either!

Learning is Social

Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Machine Learning Workflows

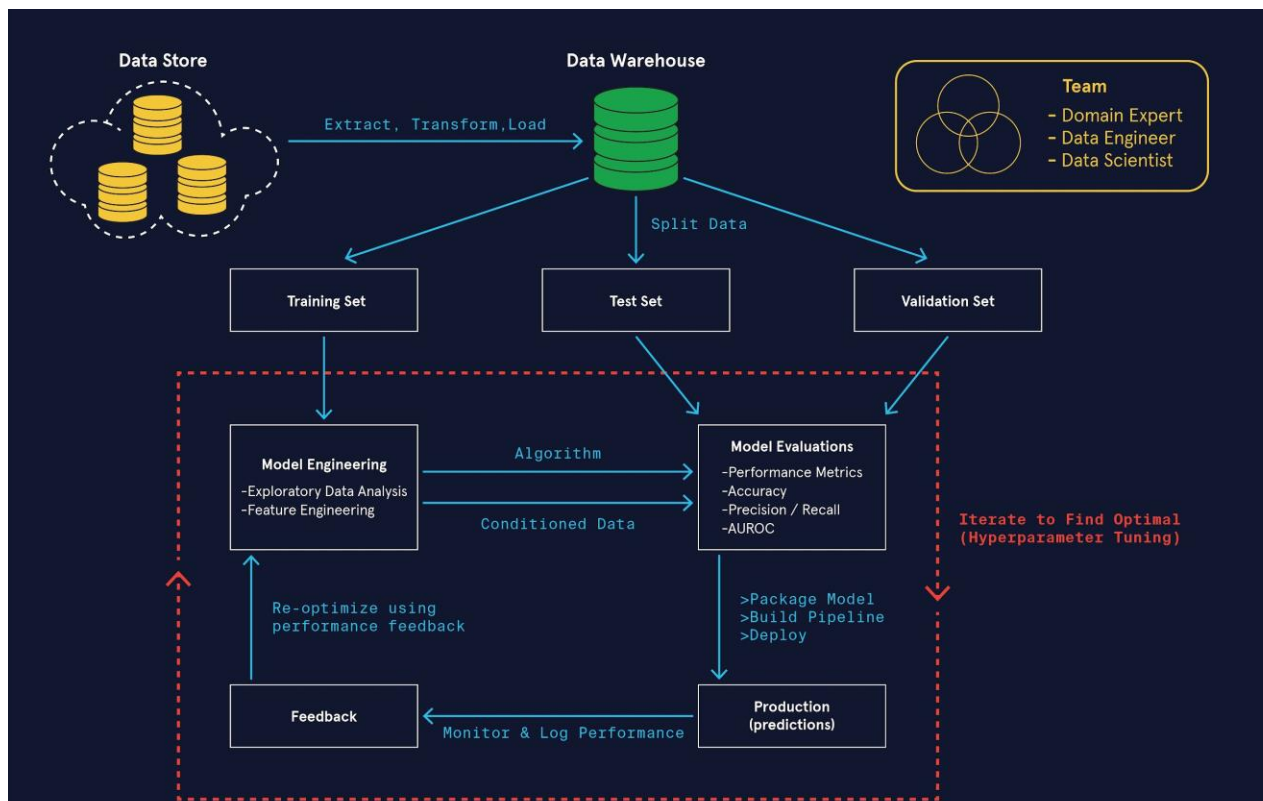
Learn about machine learning workflows!

Introduction

There are many steps in the process of creating, implementing and iterating over a machine learning model for a specific data-driven problem. While there is no single universal way of sequencing the different steps that go into a workflow, there are some general principles that are good to follow for optimal performance of a machine learning algorithm. This article will layout the steps in a ML workflow and how to implement them for any dataset.

A note to the learner: This article and module requires intermediate-level knowledge of machine learning. Check out our [The Intermediate Machine Learning Skill Path](#) if you'd like to brush up on that. Throughout the article we will also provide links to specific content items if you want to just review some of those concepts.

Machine Learning Workflow



A machine learning workflow has the following steps. Depending on the dataset, the question we're trying to answer and the tech stack we're working with, one or more of these steps can be omitted or combined with another. The steps are:

We're now going to go through each of these steps in detail.

1. Extract, Transform and Load (ETL) data

This process can look different depending on the data sources and the tech-stack you maybe working with at a specific company. It is often the case that data is stored in a SQL database with a cloud service provider like AWS, Digital Ocean, etc. Depending on the volume of data, an engineer would use a tool like PySpark to extract this data, transform it and load it into a local database. Some times this falls within the purview of a Machine Learning Engineer (MLE) but depending on the technical sophistication of the stack, there might be Data Engineer or a Pipeline Engineer performing this task instead.

2. Data Cleaning and Aggregation

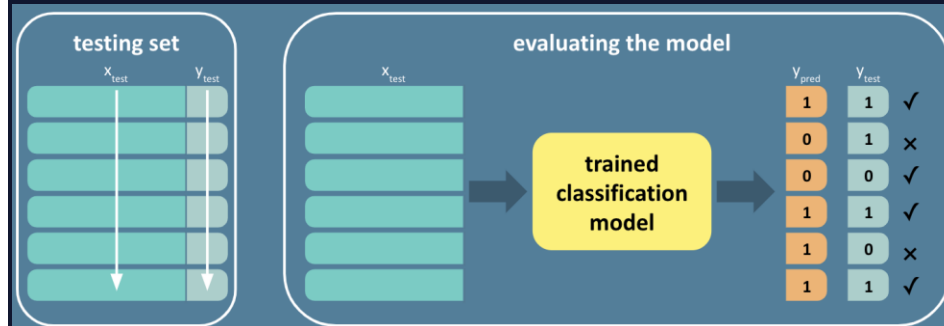
This step is often combined with the previous one. This can involve a range of tasks depending on the form and type of data as well as the problem that the machine learning pipeline is being designed to solve. Some examples include: dealing with null or missing entries, conforming timestamps to a standard, carrying out aggregations like grouping events based on timestamps by the hour or day, grouping IP's by location, etc. Since Spark is best suited to perform such tasks on big data, this task might very well be the "Transform" part of the aforementioned ETL step. Alternately, raw data might get handed over to a MLE who then does the same after the ETL step.

3. Train-Test-Validation Split

The next step before "editing" the data in any manner, is to perform the train-test-validation split. You are likely very familiar with the idea of training and test data & mash; `scikit-learn`'s `model_selection` module has a `train_test_split` function that's used to to this. If you have ever implemented a machine learning model using `scikit-learn`, you've likely written the following piece of code:

```
from sklearn.model_selection import train_test_split
# For feature matrix X and target variable y
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
random_state=42)
```

In the example above, we see that the test data makes up a third of the original dataset. The `random_state` argument is set to ensure reproducibility here — i.e., if we reused 42, it would split a certain dataset in the exact same manner. The training data is used to learn a model's parameters and the test data is used to test its performance. For models in production, there's a third portion of the dataset that's set aside known as a holdout or validation dataset used to tune hyperparameters and/or to perform model validation later on. If you would like to revisit the train-test-validation split, checkout this [article](#) on the same.



The reason we don't want to perform data manipulations before splitting the dataset into training, test and validation datasets is that we don't want data points from one of these to influence the other. Suppose we needed a machine learning model to predict housing prices and wanted to standardize a feature like the size of an apartment, the average of this value would look different between the entire dataset and each of the individual datasets. Scaling the entire column to the average value misses the unique information contained within the subpopulations and will make the model evaluation and validation process less objective.

4. Exploratory Data Analysis

Exploratory Data Analysis or EDA in the context of a machine learning workflow, is the step of inspecting, analyzing and altering your data to get it ready for machine learning modeling. Often this is the step where decisions on how to deal with outliers, transform are made. Our [EDA in Python course](#) covers this topic exhaustively but specifically, the articles on performing EDA before supervised ([classification](#) or [regression](#)) or [unsupervised](#) learning models are worth checking out if you'd like a refresher.

5. Feature Engineering

Feature engineer refers to an umbrella of methods to prep, select and reduce features in a machine learning problem. This can involve methods that overlap with EDA such as normalization, removing autocorrelations, discretization, etc. Feature engineering can also involve using machine learning algorithms like PCA to reduce dimensionality or methods that are implemented during the model fitting step like regularization. If you'd like to brush up on feature engineering methods, the [Feature Engineering Skill Path](#) is for you!

6. Model Selection and Implementation

Now we're ready to test out different machine learning models. The choice of the model depends on the attributes of the data one's working with as well as the type of question we're trying to answer. `scikit-learn` has a nifty [cheatsheet](#) to choose the right estimator.

7. Model Evaluation

We're now getting into the iterative part of the workflow. Whatever model is built, it must be evaluated on the test data. For classification problems, metrics like accuracy, precision, recall, F1 score and AUROC scores indicate how performant the model is and for regression problems, scores like RMSE and R-squared are some commonly used metrics. If you'd like to review these ideas, check out [this lesson](#) on Evaluation metrics. Machine learning engineers iterate over different types of models to figure out the most optimal model for the problem at hand.

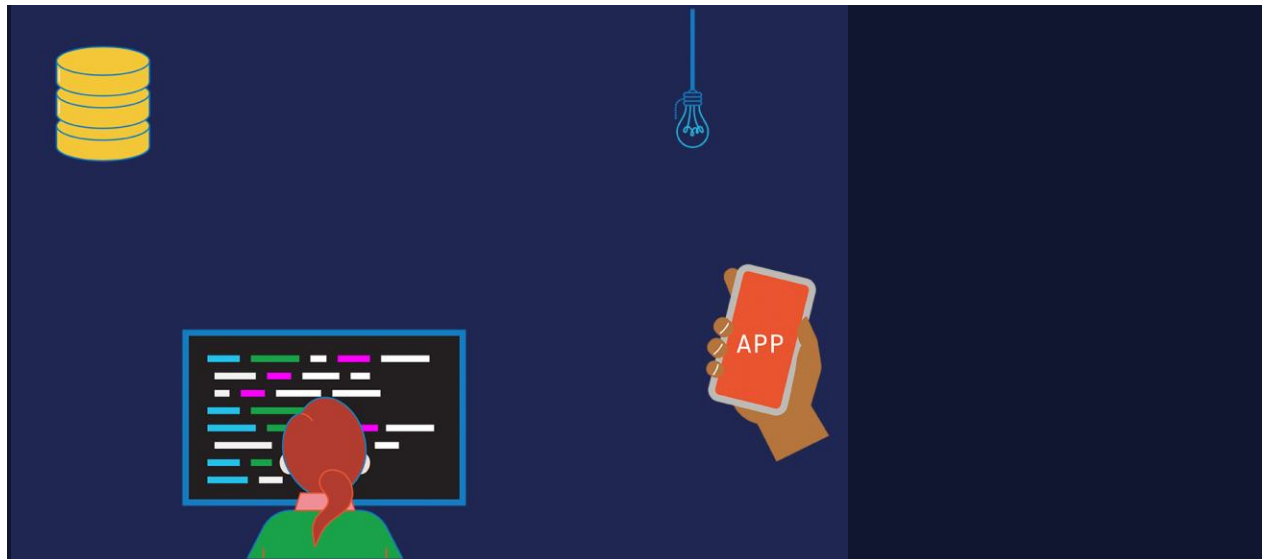
8. Hyperparameter Tuning

Once a model has been decided upon, it can be tuned for better performance. Hyperparameter tuning is essential in making sure that the model does not overfit or underfit the data. If you would like a refresher on what hyperparameters are and methods of tuning hyperparameters, the hyperparameter tuning [module](#) might be just for you!

This is key to how well the model is fitting known data and how well it's able to generalize to new data as well. Hence hyperparameter tuning might be done on the validation or holdout dataset.

9. Model Validation

Model validation is the process of making sure that the model is still performant on data that it hasn't seen at all — neither in the training phase nor in the test phase. This can be done either during the hyperparameter tuning step or after. Typically the same metrics used during the model evaluation phase needs to be used here as well so as to make a reasonable comparison with the former.



10. Build ML pipeline!

When a machine learning workflow is part of a production cycle, it is often the case that a model is tuned and updated based on incoming information. In other words the model that worked well on last month's data might not be applicable for this month. It is the job of a Machine Learning Engineer or a Pipeline Engineer to make sure that the model deployed into production is thus flexible and alterable without affecting the rest of the codebase. ML pipelines allow one to do the same!

A ML pipeline is a modular sequence of objects that codifies and automates a ML workflow to make it efficient, reproducible and generalizable.

Summary

We've gone through each of the steps within a machine learning workflow in detail. We're now ready to build ML workflows and turn them into pipelines!

Pipeline

Machine Learning Pipelines

From Workflows to Pipelines

In this lesson we're going to learn how to turn a

[machine learning](#)

Preview: Docs Loading link description

(ML) workflow to a pipeline using `scikit-learn`. A ML pipeline is a modular sequence of objects that codifies and automates a ML workflow to make it efficient, reproducible and generalizable. While the process of building pipelines is not singular, there are some tools that are universally used to do this. The most accessible of these is `scikit-learn`'s `Pipeline` object which allows us to chain together the different steps that go into a ML workflow.

Turning a workflow into a pipeline has many other advantages too. Pipelines provide consistency — the same steps will always be applied in the same order under the same conditions. They also are very concise and can streamline your code. The `Pipeline` object within `scikit-learn` has consistent methods to use the many other estimators and transformers we have already covered in our ML curriculum. It is usually the starting point for a Machine Learning Engineer before turning to more sophisticated tools for scaling pipelines (such as PySpark, etc) and we will delve deeper into it in this lesson

What can go into a pipeline? For any of the intermediate steps, it must have both the `.fit` and `.transform` methods. This includes preprocessing, imputation, feature selection and dimensionality reduction. The final step must have the `.fit` [method](#)

Preview: Docs Loading link description

. Examples of tasks we've seen already that could benefit from a pipeline include:

- scaling data then applying principal component analysis

- filling in missing values then fitting a regression model

- one-hot-encoding categorical

- [variables](#)

Preview: Docs Loading link description

- and scaling numerical variables

In the following exercises, we will walk through various chained functions and how to incorporate these into a `Pipeline`.

Machine Learning Pipelines

Data Cleaning (Numeric)

To introduce pipelines, let's look at a common set of data cleaning/EDA tasks — dealing with missing values and scaling numeric [variables](#)

Preview: Docs Loading link description

. We're going to convert an existing code base that performs these tasks to more concise code that uses `scikit-learn`'s `Pipeline` using the following steps.

First, to define a pipeline, we pass a list of tuples of the form `(name, transform/estimator)` into a `Pipeline` object. For example, if we wanted to perform imputation with a `SimpleImputer` first, and scale our numerical variables with a `StandardScaler` next, the code would look as follows:

```
from sklearn.pipeline import Pipeline
pipeline = Pipeline([("imputer", SimpleImputer()), ("scale", StandardScaler())])
```

Copy to Clipboard

Once a `Pipeline` object has been instantiated, the methods `.fit` and `.transform` can be called like we would with any data transformation object in `scikit-learn`.

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(X, y, random_state=0, test_size=0.25)
pipeline.fit(x_train)
pipeline.transform(x_test)
```

Copy to Clipboard

If the pipeline includes a

[machine learning](#)

Preview: Docs Loading link description

model as well, `.predict` can also be called down the line. Each step in the pipeline will be fit in the order provided. Further parameters can be passed to each step as well. For example, if we want to pass the

[parameter](#)

Preview: Docs Loading link description

with `_mean=False` to the `StandardScaler`, we'd use:

```
Pipeline([("imputer", SimpleImputer()), ("scale", StandardScaler(with_mean=False))])
```

Copy to Clipboard

In the

[code editor](#)

Preview: Docs Loading link description

, we've loaded a [dataset](#) from the UCI Machine Learning repository containing information about different attributes of [abalones](#), typically used to predict the age of the abalone. We've defined the predictor and target variables, identified the numerical and categorical columns, added some missing values to impute, and performed a train-test split.

Instructions

Checkpoint 1 Passed

1.

Examine the existing code that fills in missing values with the mean value (using a `SimpleImputer`) and then scales the data (using `StandardScaler`). Rewrite this code to create `pipeline`, a `Pipeline` object using the sample code given in the narrative.

Checkpoint 2 Passed

2.

Fit the pipeline to the numerical columns of the training data. Transform the test data (numeric columns only) using the fit pipeline and call this object `x_transform`.

To get the training/test data containing the numerical columns (`num_cols`) alone, replace `x_train` and `x_test` in `.fit()` & `transform()` methods with `x_train[num_cols]` and `x_test[num_cols]` respectively.

Checkpoint 3 Passed

3.

To see if `x_transform` and `x_test_fill_missing_scale` are equal, use the `np.array_equal()` function. The arrays will be the two arguments inside this function. Then save the results to the variable `array_diff`. If the arrays are equal then `np.array_equal()` returns `True`.

Print `array_diff` to see if this is indeed the case.

When using `np.array_equal(array_1, array_2)`, the two arrays must be the same shape and have the same elements to return `True`. See the [documentation](#) to learn more.

Checkpoint 4 Passed

4.

Define a new pipeline, `pipeline_median` with the imputer strategy parameter set to `median`.

Checkpoint 5 Passed

5.

To compare the results between the two pipelines, implement the following steps:

Fit the numerical training data with `pipeline_median` and transform the numerical test data to obtain a new array, `x_transform_median`.

Calculate `new_array_diff`, the sum of the absolute differences between the two arrays, `x_transform` and `x_transform_median`.

Print `new_array_diff` to confirm that the results between the pipelines are indeed different!

See example

```
abs(array_1 - array_2).sum()
```

Copy to Clipboard

SOLUTIONS:

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

## Loading the dataset
columns = ["sex", "length", "diam", "height", "whole", "shucked", "viscera", "shell", "age"]
df = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data", names=columns)
## Defining target and predictor variables
y = df.age
X = df.drop(columns=['age'])

## Numerical columns:
num_cols = X.select_dtypes(include=np.number).columns
## Categorical columns
cat_cols = X.select_dtypes(include=['object']).columns

## Create some missing values
for i in range(1000):
    X.loc[np.random.choice(X.index), np.random.choice(X.columns)] = np.nan

## Perform train-test split
x_train, x_test, y_train, y_test = train_test_split(X, y, random_state=0, test_size=0.25)

#####-----Imputation and Scaling: Code base to transform -----#####
## Numerical training data
x_train_num = x_train[num_cols]
# Filling in missing values with mean on numeric features only
x_train_fill_missing = x_train_num.fillna(x_train_num.mean())
## Fitting standard scaler on x_train_fill_missing
scale = StandardScaler().fit(x_train_fill_missing)
## Scaling data after filling in missing values
x_train_fill_missing_scale = scale.transform(x_train_fill_missing)
## Same steps as above, but on the test set:
x_test_fill_missing = x_test[num_cols].fillna(x_train_num.mean())
x_test_fill_missing_scale = scale.transform(x_test_fill_missing)
#####-----Imputation and Scaling: Code base to transform -----#####

#1. Rewrite using Pipelines!
pipeline = Pipeline([("imputer", SimpleImputer(strategy='mean')), ("scale", StandardScaler())])

#2. Fit pipeline on the test and compare results
pipeline.fit(x_train[num_cols])
x_transform = pipeline.transform(x_test[num_cols])

# 3. Verify pipeline transform test set is the same by using np.array_equal()
array_diff = np.array_equal(x_transform, x_test_fill_missing_scale)
print(array_diff)

#4. Change imputer strategy to median
pipeline_median = Pipeline([("imputer", SimpleImputer(strategy='median')),
                             ("scale", StandardScaler())])
pipeline_median.fit(x_train[num_cols])

# 5 Compare results between the two pipelines
```

```
x_transform_median = pipeline_median.transform(x_test[num_cols])
new_array_diff = abs(x_transform-x_transform_median).sum()
print(new_array_diff)
```

Machine Learning Pipelines

Data Cleaning (Categorical)

We're now going to implement a task similar to the previous exercise with `pipeline.Pipeline()`, but with categorical [variables](#)

Preview: Docs [Loading link description](#)

now. Specifically, we'll be dealing with missing values in categorical data and one-hot-encoding categorical variables. We will convert an existing codebase to a pipeline like in the previous exercise. The two steps in detail are:

`SimpleImputer()` will be used again to fill missing values in the pipeline, but this time, the strategy [parameter](#)

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis():` `js function tripleThis(x) { return x * 3; } tripleThis(6);`

will need to be updated to `most_frequent`.

`OneHotEncoder()` will be used as the second step in the pipeline. The default setting in `scikit-learn's` `OneHotEncoder()` is that a sparse [array](#)

Preview: Docs [Loading link description](#)

will be returned from this transform, so we will use `sparse='False'` to return a full array.

Instructions

Checkpoint 1 Passed

1.

Examine the existing code that fills in missing values with the mode of the categorical variable and then creates dummy variables (with `OneHotEncoder`). Create a `Pipeline()` object named `pipeline` that will do the same tasks by making:

The first step in the pipeline should be named "imputer", which is a `SimpleImputer()` that employs the `most_frequent` strategy

The second step in the pipeline should be named "ohe", which is a `OneHotEncoder()` with arguments `drop = 'first'` and `sparse = False`

Checkpoint 2 Passed

2.

First, fit this pipeline to the training data with categorical columns alone. You can index the categorical data with `x_train[cat_cols]` since the categorical columns were already saved as `cat_cols`.

Next, transform the test data (categorical columns only) and call the resulting array, `x_transform`.

Use `.fit()` to fit the pipeline and `.transform()` to transform the test data.

Checkpoint 3 Passed

3.

To see if `x_transform` and `x_test_fill_missing_ohe` are equal, use the `np.array_equal()` function. The arrays will be the two arguments inside this function. Then save the results to the variable `check_arrays`. If the arrays are equal then `np.array_equal()` returns `True`.

Print `check_arrays` to see if this is indeed the case.

When using `np.array_equal(array_1,array_2)`, the two arrays must be the same shape and have the same elements to return `True`. See the [documentation](#) to learn more.

SOLUTIONS:

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import KFold, train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

columns = ["sex", "length", "diam", "height", "whole", "shucked", "viscera", "shell", "age"]
```

```

df = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-
databases/abalone/abalone.data",names=columns)

y = df.age
X=df.drop(columns=['age'])
num_cols = X.select_dtypes(include=np.number).columns
cat_cols = X.select_dtypes(include=['object']).columns
#create some missing values
for i in range(1000):
    X.loc[np.random.choice(X.index),np.random.choice(X.columns)] = np.nan

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.25)
x_train_cat = x_train[cat_cols]
#fill missing values with mode on categorical features only
x_train_fill_missing = x_train_cat.fillna(x_train_cat.mode().values[0][0])
#apply one hot encoding on x_train_fill_missing
ohe = OneHotEncoder(sparse=False, drop='first').fit(x_train_fill_missing)
#transform data after filling in missing values
x_train_fill_missing_ohe = ohe.transform(x_train_fill_missing)

#Now want to do the same thing on the test set!
x_test_fill_missing = x_test[cat_cols].fillna(x_train_cat.mode().values[0][0])
x_test_fill_missing_ohe = ohe.transform(x_test_fill_missing)

#1. Rewrite using Pipelines!
pipeline = Pipeline([("imputer",SimpleImputer(strategy='most_frequent')),
("ohe",OneHotEncoder(sparse=False, drop='first'))])

#2. Fit the pipeline and transform the test data (categorical columns only!)
pipeline.fit(x_train[cat_cols])
x_transform = pipeline.transform(x_test[cat_cols])

#3. Check if the two arrays are the same using np.array_equal()
check_arrays = np.array_equal(x_transform,x_test_fill_missing_ohe)
print('Are the arrays equal?')
print(check_arrays)

```

Machine Learning Pipelines

Column Transformer

Often times, you may not want to simply apply every function to all columns. If our columns are of different types, we may only want to apply certain parts of the pipeline to a subset of columns. This is what we saw in the two previous exercises. One set of transformations are applied to numeric columns and another set to the categorical ones. We can use `scikit-learn`'s `ColumnTransformer` as one way of combining these processes together.

`ColumnTransformer` takes in a list of tuples of the form `(name, pipeline, columns)`:

```

example_column_transformer = ColumnTransformer(
    transformers=[ ("name_1", pipeline_1, columns_1),
                  ("name_2", pipeline_2, columns_2)])

```

Copy to Clipboard

The transformer can be anything with a `.fit` and `.transform` [method](#)

Preview: Docs Loading link description

like we used previously (like `SimpleImputer` or `StandardScaler`), but can also itself be a pipeline, as we will use in the exercise.

Instructions

Checkpoint 1 Passed

1.

Following what we did in the previous exercise, we're going to create two pipelines, one for the numerical preprocessing and another for the categorical preprocessing. First, create a numerical preprocessing pipeline, `num_vals` pipeline that has:

- a `SimpleImputer()` named `'imputer'` with imputing strategy set to `'mean'`
- a `StandardScaler()` named `'scale'`

Input arguments for a sample pipeline in `scikit-learn` can be given as follows:

```

pipeline = Pipeline([("imputer",SimpleImputer()), ("scale",StandardScaler())])

```

Copy to Clipboard

Checkpoint 2 Passed

2.

Now, create a pipeline for categorical preprocessing called `cat_vals`. This pipeline should have

a `SimpleImputer()` named `'imputer'` with an imputing strategy set to `'most_frequent'`

a `OneHotEncoder()` named `'ohe'`, with arguments `sparse = False` and `drop = 'first'`

Input arguments for a sample pipeline in scikit-learn can be given as follows:

```
pipeline = Pipeline([("imputer", SimpleImputer()), ("ohe", OneHotEncoder())])
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Similar to the sample code in the narrative, create a `ColumnTransformer` named `preprocess` that takes the two pipelines you've defined, `num_vals` and `cat_vals`, as transformers and then names them as `num_preprocess` and `cat_preprocess` respectively.

A sample `ColumnTransformer` initialization looks as follows:

```
example_column_transformer = ColumnTransformer(
    transformers=[ ("pipeline_name_1", pipeline_transformer_1, columns_1),
                  ("pipeline_name_2", pipeline_transformer_2, columns_2)])
```

Copy to Clipboard

For the numerical preprocessing part, the tuple would be: `('num_preprocess', num_vals, num_cols)`.

Checkpoint 4 Passed

4.

Fit the transformer on the training set, transform the test data and store the transformed test data as `x_transform`.

SOLUTIONS:

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import KFold, train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer

columns = ["sex", "length", "diam", "height", "whole", "shucked", "viscera", "shell", "age"]
df = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data", names=columns)

y = df.age
X = df.drop(columns=['age'])
num_cols = X.select_dtypes(include=np.number).columns
cat_cols = X.select_dtypes(include=['object']).columns
#create some missing values
for i in range(1000):
    X.loc[np.random.choice(X.index), np.random.choice(X.columns)] = np.nan

x_train, x_test, y_train, y_test = train_test_split(X, y, random_state=0, test_size=0.25)

#1. Create a pipeline `num_vals` to process numerical data
num_vals = Pipeline([("imputer", SimpleImputer()), ("scale", StandardScaler())])

#2. Create a pipeline `cat_vals` to process categorical data
cat_vals = Pipeline([("imputer", SimpleImputer(strategy = 'most_frequent')), ("ohe", OneHotEncoder(drop = 'first', sparse = False))])

#3. Create a column transformer, `preprocess` with the numerical and categorical pipelines
preprocess = ColumnTransformer(
    transformers=[
        ("num_preprocess", num_vals, num_cols),
        ("cat_preprocess", cat_vals, cat_cols)
    ]
)
```

```
#4. Fit the preprocess transformer to training data
preprocess.fit(x_train)
#Transform the test data
x_transform = preprocess.transform(x_test)
```

Machine Learning Pipelines

Adding a Model

Great! Now that we have all the preprocessing done and coded succinctly using `ColumnTransformer` and `Pipeline`, we can add a model. We will take the result at the end of the previous exercise, and now create a final pipeline with the `ColumnTransformer` as the first step, and a `LinearRegression()` model as the second step.

By adding a model to the final step, the last step no longer has a `.transform` [method](#)

Preview: Docs Loading link description

. This is the only step in a pipeline that can be a non-transformer. But now the final step also has a `.predict` method, which can be called on the entire pipeline! Additionally the `.score()` method, which estimates the default prediction score on any `scikit-learn` model can also be used to evaluate the performance of the pipeline.

Instructions

Checkpoint 1 Passed

1.

We've preloaded all the code from the previous exercise. Create a new pipeline named `pipeline` that has the column transformer `preprocess` as the first component and then a `LinearRegression()` model, `regr` as the second.

Recall that you can set the components as tuples like `("preprocessing", preprocess)`, etc.

Checkpoint 2 Passed

2.

Fit the pipeline on the training data and use the `.predict` method to obtain the predictions on the test data. Store these predictions in an array named `y_pred`.

Checkpoint 3 Passed

3.

The properties of the last component of a pipeline transfer over to the pipeline object. We've seen how when the last component is a model, then, characteristics of `scikit-learn` models such as `.fit()` and `.predict()` transfer over. Similarly, `.score()` which gives the default scoring metric of a model can also be accessed via the pipeline!

The default scoring metric for a Linear Regression model in `scikit-learn` is the R^2 score. We can now use the `.score()` method to evaluate the predictions made. Calculate this score and store it as `pipeline_score`.

We've also written some code that calculates the R^2 value using the usual method. Uncomment the lines pertaining to this.

Write a print statement to print the `pipeline_score` and press Run to check if both these methods give the same value.

The `.score()` method takes the test predictor and target data as its inputs. In this case this would be `(x_test, y_test)`.

SOLUTIONS:

```
import numpy as np
import pandas as pd

from sklearn import svm, datasets
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
```



```

from sklearn.metrics import r2_score

columns = ["sex", "length", "diam", "height", "whole", "shucked", "viscera", "shell", "age"]
df = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data", names=columns)

y = df.age
X=df.drop(columns=['age'])
num_cols = X.select_dtypes(include=np.number).columns
cat_cols = X.select_dtypes(include=['object']).columns
#create some missing values
for i in range(1000):
    X.loc[np.random.choice(X.index), np.random.choice(X.columns)] = np.nan

x_train, x_test, y_train, y_test = train_test_split(X,y, random_state=0, test_size=0.25)

cat_vals = Pipeline([("imputer", SimpleImputer(strategy='most_frequent')),
("ohe", OneHotEncoder(sparse=False, drop='first'))])

num_vals = Pipeline([("imputer", SimpleImputer(strategy='mean')), ("scale", StandardScaler())])

preprocess = ColumnTransformer(
    transformers=[
        ("cat_process", cat_vals, cat_cols),
        ("num_process", num_vals, num_cols)
    ]
)

#1. Create a pipeline with `preprocess` and a linear regression model, `regr`
pipeline = Pipeline([("preprocess", preprocess),
("regr", LinearRegression())])

#2. Fit the pipeline on the training data and predict on the test data

pipeline.fit(x_train, y_train)
y_pred = pipeline.predict(x_test)

#3. Calculate pipeline score and compare to estimator score
#Pipeline score
pipeline_score = pipeline.score(x_test, y_test)
print(pipeline_score)

#r-squared score
r2_score = r2_score(y_test, y_pred)
print(r2_score)

```

Machine Learning Pipelines

Hyperparameter Tuning

Great, we have a very condensed bit of code that does all our data cleaning, preprocessing, and modeling in a reusable fashion! What now? Well, we can tune some of the parameters of the model by applying a grid search over a range of hyperparameter values.

A linear regression model has very few hyperparameters and here we'll be using the hyperparameter that pertains to whether we include an intercept or not. As we've seen, the pipeline created in the previous exercise is an estimator and we can call the `.fit()` and `.predict()` methods on it. So in fact, the whole pipeline can be passed as an estimator for `GridSearchCV`. This will then refit the pipeline for each combination of

[parameter](#)

Preview: Docs Loading link description

values in the grid and each fold in the cross-validation split.

That's a lot – but the code is again very short. One last thing to keep in mind while referencing hyperparameters in a pipeline is the following: any hyperparameter can be called using `pipeline_step_name + '__' + hyperparameter`. For example, `regr__fit_intercept` corresponds to a pipeline step named “regr” and the hyperparameter “fit_intercept”. Let's see how this all works, shall we?

Note: If you'd like a refresher on how hyperparameters can be tuned check out our [lesson](#) on the same.

Instructions

Checkpoint 1 Passed

1.

We've preloaded all the code from the previous exercise once again. Using the the previously built `pipeline` as an input define a `GridSearchCV` object named `gs` with the following arguments:

Set `scoring` to `'neg_mean_squared_error'`

Set number of cross-validation folds, `cv` to 5

Checkpoint 2 Passed

2.

Fit the grid search model `gs` to the training data. We can now call the `GridSearchCV` [attributes](#) like usual.

Calculate the best score among the parameters in the grid and store it as `best_score`.

Calculate optimal parameters from the grid search and store it as `best_params`.

Print both to examine if it's best to fit an intercept to the regression model here or not!

The `best_score_` and `best_params_` methods get us the best score and parameters respectively.

SOLUTIONS:

```
import numpy as np
import pandas as pd

from sklearn import svm, datasets
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn import metrics

columns = ["sex", "length", "diam", "height", "whole", "shucked", "viscera", "shell", "age"]
df = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data", names=columns)

y = df.age
X=df.drop(columns=['age'])
num_cols = X.select_dtypes(include=np.number).columns
cat_cols = X.select_dtypes(include=['object']).columns
#create some missing values
for i in range(1000):
    X.loc[np.random.choice(X.index), np.random.choice(X.columns)] = np.nan

x_train, x_test, y_train, y_test = train_test_split(X,y, random state=0, test size=0.25)

cat_vals = Pipeline([("imputer", SimpleImputer(strategy='most_frequent')),
                     ("ohe", OneHotEncoder(sparse=False, drop='first'))])
num_vals = Pipeline([("imputer", SimpleImputer(strategy='mean')), ("scale", StandardScaler())])

preprocess = ColumnTransformer(
    transformers=[
        ("cat_process", cat_vals, cat_cols),
        ("num_process", num_vals, num_cols)
    ]
)

#Create a pipeline with preproces and a linear regression model
pipeline = Pipeline([("preprocess", preprocess),
                     ("regr", LinearRegression())])

#Very simple parameter grid, with and without the intercept
param_grid = {
    "regr__fit_intercept": [True, False]
}

#-----
#1. Grid search using previous pipeline
gs = GridSearchCV(pipeline, param_grid, scoring='neg_mean_squared_error', cv=5)

#2. fit grid using training data and print best score
gs.fit(x_train, y_train)
```

```
best_score = gs.best_score_  
best_params = gs.best_params_  
print(best_score, best_params)
```

Machine Learning Pipelines

Final Pipeline

Way to go! Now that we are getting the hang of pipelines, we're going to take things up a notch. We will now be searching over different types of models, each having their own sets of hyperparameters! In the original `pipeline`, we defined `regr` to be an instance of `LinearRegression()`. Then in defining the

[parameter](#)

Preview: Docs Loading link description

grid to search over, we used the

[dictionary](#)

Preview: Docs Loading link description

`{"regr__fit_intercept": [True, False]}` to define the values of the `fit_intercept` term. We can equivalently do this by passing both the estimator AND parameters in a single dictionary as

```
{'regr': [LinearRegression()], 'regr__fit_intercept': [True, False]}
```

Copy to Clipboard

We can add more models to it as follows. Suppose we wanted to add a Ridge regression model and also perform hyperparameter tuning using `GridSearchCV` to find the best regularization parameter `alpha`, we would add it to previous dictionary to create an [array](#)

Preview: Docs Stores elements of various data types in an ordered collection.

of dictionaries as follows:

```
search_space = [{'regr': [LinearRegression()], 'regr__fit_intercept': [True, False]},  
                 {'regr': [Ridge()], 'regr__alpha': [0, 0.1, 1, 10, 100]}
```

Copy to Clipboard

Note: If you'd like a refresher on regularization using hyperparameter tuning in regression models, check out our [article](#) and [lesson](#) on the same.

The goal of this process is to find the best estimator for our dataset and problem in the most efficient manner possible. The pipeline module allows us to do exactly that! In a couple of lines of code, we're able to preprocess the data and search an entire model *and* hyperparameter space. The final step is to access the pipeline elements to draw out the information about which estimator and hyperparameter set gets us the best score. We do this by using the `.next_steps`

[method](#)

Preview: Docs Loading link description

by using the strings we've used in the dictionary. For instance, the regression model can be accessed using the

[string](#)

Preview: Docs Loading link description

`'regr'` from the dictionary as follows:

Get the best estimator using `GridSearchCV`'s `.best_estimator_` method

Use `.named_steps['regr'].get_params()` on the best estimator to get its hyperparameters!

Instructions

Checkpoint 1 Passed

1.

Let's try this! First, update the `search_space` array of dictionaries from the narrative to add a Lasso Regression model as the third dictionary as follows:

Use `'regr'` for all three regression models

Use the key `'regr__alpha'` to refer to the regularization parameter `alpha` for both Ridge and Lasso

Set the array of `alpha`'s to search over the array `[0.01, 0.1, 1, 10, 100]` for both Ridge and Lasso

Checkpoint 2 Passed

2.

Like in the previous exercise, define a `GridSearchCV` object named `gs` with `pipeline` as its first argument followed by the following arguments:

Set `param_grid` to `search_space`

Set `scoring` to `'neg_mean_squared_error'`


```

# 1. Update the `search_space` array from the narrative to add a Lasso Regression model as the third dictionary.
search_space = [{'regr': [LinearRegression()], 'regr_fit_intercept': [True, False]},
                 {'regr': [Ridge()],
                  'regr__alpha': [0.01, 0.1, 1, 10, 100]},
                 {'regr': [Lasso()],
                  'regr__alpha': [0.01, 0.1, 1, 10, 100]}]

# 2. Initialize a grid search on `search_space`
gs = GridSearchCV(pipeline, search_space, scoring='neg_mean_squared_error', cv=5)

#3. Find the best pipeline, regression model and its hyperparameters

## i. Fit to training data
gs.fit(x_train, y_train)

## ii. Find the best pipeline
best_pipeline = gs.best_estimator_

## iii. Find the best regression model
best_regression_model = best_pipeline.named_steps['regr']
print('The best regression model is:')
print(best_regression_model)

## iv. Find the hyperparameters of the best regression model
best_model_hyperparameters = best_regression_model.get_params()
print('The hyperparameters of the regression model are:')
print(best_model_hyperparameters)

#4. Access the hyperparameters of the categorical preprocessing step
cat_preprocess_hyperparameters =
best_pipeline.named_steps['preprocess'].named_transformers['cat_preprocess'].named_steps['imputer'].get_params()
print('The hyperparameters of the imputer are:')
print(cat_preprocess_hyperparameters)

```

Machine Learning Pipelines

Writing Custom Classes & Summary

While scikit-learn contains many existing transformers and classes that can be used in pipelines, you may need at some point to create your own. This is simpler than you may think, as a step in the pipeline needs to have only a few methods implemented. If it is an intermediate step, it will need fit and transform methods. We will implement all of this in the exercise below!

Here are some of the major takeaways on building pipelines in `scikit-learn`:

- Pipelines help make concise, reproducible, code by combining steps of transformers and/or a final estimator.

- Intermediate steps of a pipeline must have both the `.fit()` and `.transform()` methods. This includes preprocessing, imputation, feature selection, dimension reduction.

- The final step of a pipeline must have the `.fit()` [method](#)

- Preview: Docs Loading link description

- this can include a transformer or an estimator/model.

If the pipeline is meant to only transform your data by combining preprocessing and data cleaning steps, then each step in the pipeline will be a transformer. If your pipeline will also include a model (a final estimation or prediction step), then the last step must be an estimator.

Once the steps of a pipeline are defined, it can be used like an other transformer/estimator by calling fit, transform, and/or predict methods. Similarly, it can be used in place of an estimator in a hyperparameter grid search.

Instructions

Checkpoint 1 Passed

1.

Examine the code written for the class `MyImputer`. This replicates the `SimpleImputer` using the mean strategy. Notice both `fit` and `transform` methods are defined. Use this new class as the first step in `new_pipeline` and second step `StandardScaler`.

Checkpoint 2 Passed

2.

Fit the new pipeline on the training data, numeric columns only. This will be identical to the pipeline created in exercise 2. Verify this by performing the following steps:

Transform the test set `x_test[num_cols]` using this and write it to a new variable `x_transform`.

To compare `x_transform` and `x_test_fill_missing_scale`, use `np.array_equal()` to a variable `check_arrays` and print it.

To compare two arrays, you can use `np.array_equal()`. If the two arrays are the same shape and have the same elements, it returns `True`. See the [documentation](#) to learn more.

SOLUTIONS:

```
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import KFold, train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.base import BaseEstimator, TransformerMixin

columns = ["sex", "length", "diam", "height", "whole", "shucked", "viscera", "shell", "age"]
df = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-
databases/abalone/abalone.data", names=columns)
y = df.age
X = df.drop(columns=['age'])
num_cols = X.select_dtypes(include=np.number).columns
cat_cols = X.select_dtypes(include=['object']).columns

for i in range(1000):
    X.loc[np.random.choice(X.index), np.random.choice(X.columns)] = np.nan
x_train, x_test, y_train, y_test = train_test_split(X, y, random_state=0, test_size=0.25)
x_train_num = x_train[num_cols]
#fill missing values with mean on numeric features only
x_train_fill_missing = x_train_num.fillna(x_train_num.mean())
#fit standard scaler on x_train_fill_missing
scale = StandardScaler().fit(x_train_fill_missing)
#scale data after filling in missing values
x_train_fill_missing_scale = scale.transform(x_train_fill_missing)
x_test_fill_missing = x_test[num_cols].fillna(x_train_num.mean())
x_test_fill_missing_scale = scale.transform(x_test_fill_missing)

class MyImputer(BaseEstimator, TransformerMixin):
    def __init__(self):
        return None

    def fit(self, X, y = None):
        self.means = np.mean(X, axis=0) # calculate the mean of each column
        return self

    def transform(self, X, y = None):
        #transform method fills in missing values with means using pandas
        return X.fillna(self.means)

#1. Create new pipeline using the custom class MyImputer as the first step and standard scaler on the
second
new_pipeline = Pipeline([("imputer", MyImputer()), ("scale", StandardScaler())])

#2. 1. Fit new pipeline on the training data with num_cols only
new_pipeline.fit(x_train[num_cols])
x_transform = new_pipeline.transform(x_test[num_cols])

#2 2. Verify that the results of the transform are the same on test set
check_arrays = np.array_equal(x_transform, x_test_fill_missing_scale)
print(check_arrays)
```

What is a Portfolio Project?

This project is a little different from other Codecademy projects you've encountered. In this project, you will bring together what you have learned in previous lessons to build a project in your own development environment and publish it to the web!



Create your own files
outside of the
Codecademy platform



Write your own code
with less guided
instructions



Use common project
management processes to
track your progress

What is a Portfolio Project?

This project is a little different from other Codecademy projects you've encountered. In this project, you will bring together what you have learned in previous lessons to build a project in your own development environment and publish it to the web!



Create your own files
outside of the
Codecademy platform



Write your own code
with less guided
instructions



Use common project
management processes to
track your progress

How Do Portfolio Projects Work?

We'll provide you with high-level tasks to guide your project to completion, but you will be responsible for deciding how to implement them in your code.

There are many possible ways to correctly fulfill all of these requirements, and you should expect to use the internet, Codecademy, and other resources when you encounter a problem that you cannot easily solve.

Note that there are hints that can assist you, but they will only provide one potential implementation. Do not worry if your program looks different from ours!



Project Prompt

Machine Learning Career Path Portfolio Project

Bring together all of your machine learning engineering skills into a portfolio project. Implement an end-to-end workflow and turn it into a machine learning pipeline that is performant, reproducible and productionizable!

Click [Begin](#) to get started!

Project Tasks

Keep track of your progress by dragging each task from "To Do" to "In Progress" to "Done" as you work on them. You can also click on a task to see more information about it.

To Do

⋮ **Communicate your Findings**
⋮ Make your visualizations and docume...

⋮ **Conclusion and Next Steps**
⋮ Being a machine learning engineer is ...

In Progress

Done

8 / 10 done

⋮ ✓ **Set up your Git Repository**
⋮ Create a new Git repository for th...

⋮ ✓ **Project Scoping** ↗
⋮ Properly scoping your project will...

⋮ ✓ **Select ML-Solvable Problem**
⋮ First, choose a topic of interest; t...

⋮ ✓ **Select a dataset**
⋮ If you didn't already have a datas...

⋮ ✓ **Extract, Transform and Load (ETL)**
⋮ You likely have multiple datasets t...

⋮ ✓ **Feature Engineering**
⋮ Perform feature engineering on t...

⋮ ✓ **Implement Machine Learning Workflow**

⋮ ✓ **Build ML Pipeline**
⋮ Once you have a fully functional a...

Machine Learning/AI Engineer Career Path: Next Steps

You've finished the Machine Learning/AI Engineer Career Path – what will you learn next?

Congratulations on completing the Machine Learning Engineer Career Path! There are a lot of places you can go with your new skills: You could work as a Machine Learning Engineer or Data Scientist, in AI or Machine Learning Research, or create hobby projects. As you know, there is always more to learn.

So, now that you have finished this learning journey, you may be asking, "What's next?". We're working on bringing you more content in this Career Path, delving into more advanced Machine Learning content as well as MLOps and systems design. So keep an eye out for those notifications! In the mean time, there are many other resources both within and outside our platform that we'd like to direct you to!

Other Areas

While you've completed your Machine Learning/AI Engineer Career Path, there is still a lot more to learn about:

Deep Learning, a subfield of ML methods with neural networks, from our [Build Deep Learning Models with TensorFlow Course](#)

Artificial Intelligence basics from a Computer Science perspective, from our [Artificial Intelligence \(AI\) with Minimax](#)

Data Science, from our [Data Scientist: Machine Learning Career Path](#)

Big Data, from our upcoming [Big Data with PySpark](#) course

AI/ML Events

Events give you a chance to connect with other data scientists and learn about the latest topics in the field. There are a lot of events out there including [PyData](#), Association for Advancement of Artificial Intelligence Conference, [AAAI Conference](#), the Neural Information Processing Systems Conference, [NeurIPS](#), and the list goes on! You can also join or create a [Codecademy meetup](#) to connect with other new MLE's.

Once again, congratulations! We're excited to see what you accomplish next.

SOLUTIONS:

SOLUTIONS:

SOLUTIONS:

